AI & Agents — Complete Guide

LLMs + Agents: Beginner → Senior in 7 Days

Generated on: February 23, 2026

Table of Contents

1. Day 1: Modern LLM Foundations
2. Day 1A: Tokens & Embeddings
3. Day 1B: Transformer & Attention
4. Day 1C: Training & Inference
5. Day 1 Appendix: Advanced Techniques
6. Day 2: Advanced Prompt Engineering
7. Day 3: Single-Agent Systems (ReAct)
8. Day 4: Planner/Executor Architecture
9. Day 5: RAG + Vector Memory
10. Day 6: Multi-Agent Collaboration
11. Day 7: Final Project
12. Appendix: Real-World Agents
13. Q&A Reference

Day 1 — Modern LLM Foundations

> 🎯 **Goal**: Understand how LLMs work from tokens to training — the foundation for everything in this course.

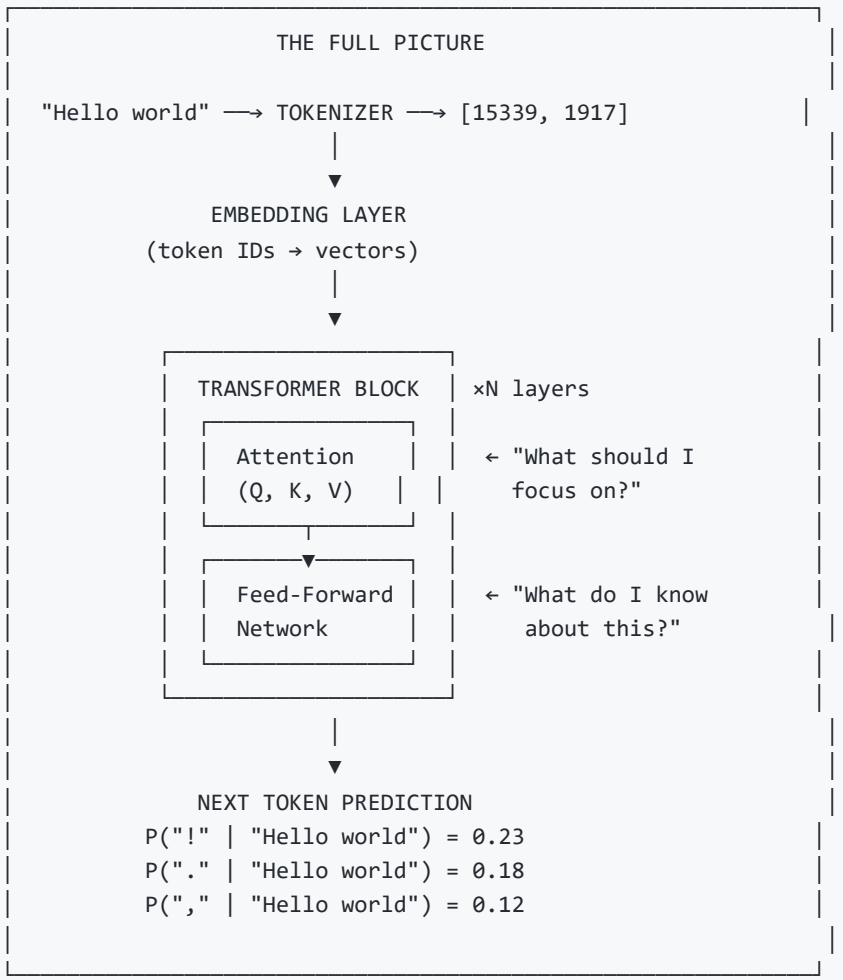
> ⏱️ **Time**: 3.5–4.5h total (across 3 parts)

> 📦 **Deliverable**: Tokenization explorer + sampling visualizer + ability to explain the full Transformer pipeline

---

Overview

Before you can build agents, you need to understand the engine they run on. This day covers the complete LLM pipeline — from raw text to trained model to generated output.



Parts

This day is split into three focused parts:

Part	Topic	Time	What You'll Learn
Part A	Tokens & Embeddings	~1.5h	What tokens are, how BPE works, why embeddings capture meaning
Part B	Transformer & Attention	~2.5h	Self-attention (Q/K/V), multi-head attention, FFN, residual connections
Part C	Training & Inference	~2h	Positional encoding, pre-training/SFT/RLHF pipeline, sampling strategies

Appendix	Advanced Techniques	Optional	MoE, GQA, Flash Attention, LoRA, quantization, scaling laws

Recommended Order

```
Part A (foundation) → Part B (architecture) → Part C (training & generation)
```

Each part is self-contained with its own exercises, but concepts build on each other.

Key Questions You'll Be Able to Answer

After completing Day 1, you should be able to answer:

1. **Tokenization**: Why isn't a token always a word? What is BPE?
2. **Embeddings**: How do computers represent meaning as numbers?
3. **Attention**: What are Q, K, V and why do we need all three?
4. **Architecture**: What does each component of a Transformer block do?
5. **Training**: What's the difference between pre-training, SFT, and RLHF?
6. **Inference**: How does temperature affect generation? What is KV-cache?

▶ 📝 **Answer Key (click to expand)**

✓ Day 1 Deliverable Checklist

- ☐ Explored tokenization with tiktoken
- ☐ Can draw the Transformer architecture from memory
- ☐ Built a sampling strategy visualizer
- ☐ Can explain: tokens, embeddings, Q/K/V, attention, FFN
- ☐ Can explain: pre-training vs SFT vs RLHF
- ☐ Can explain: temperature, top-k, top-p, KV-cache

Day 1A — Tokens, Tokenization & Embeddings

> 🎯 **Goal**: Understand how LLMs represent language — from raw text to the numerical vectors that make reasoning possible.

> ⏱️ **Time**: 1h theory + 0.5h practice

> 📦 **Deliverable**: Python notebook demonstrating tokenization with tiktoken + embedding similarity calculations

---

Table of Contents

1. Why This Matters
2. Tokens & Tokenization
3. Embeddings — The Missing Foundation
4. Putting It Together
5. Practice Exercises

Why This Matters

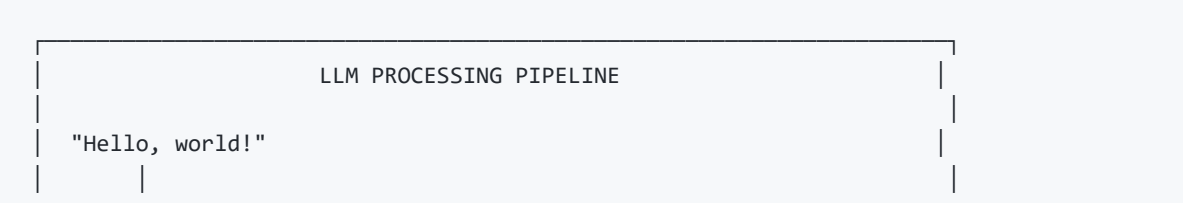
LLMs are **next-token predictors**, not magic. Every time GPT-5 or Claude Opus writes a sentence, it is doing one thing: predicting the most likely next token given everything that came before. That's it.

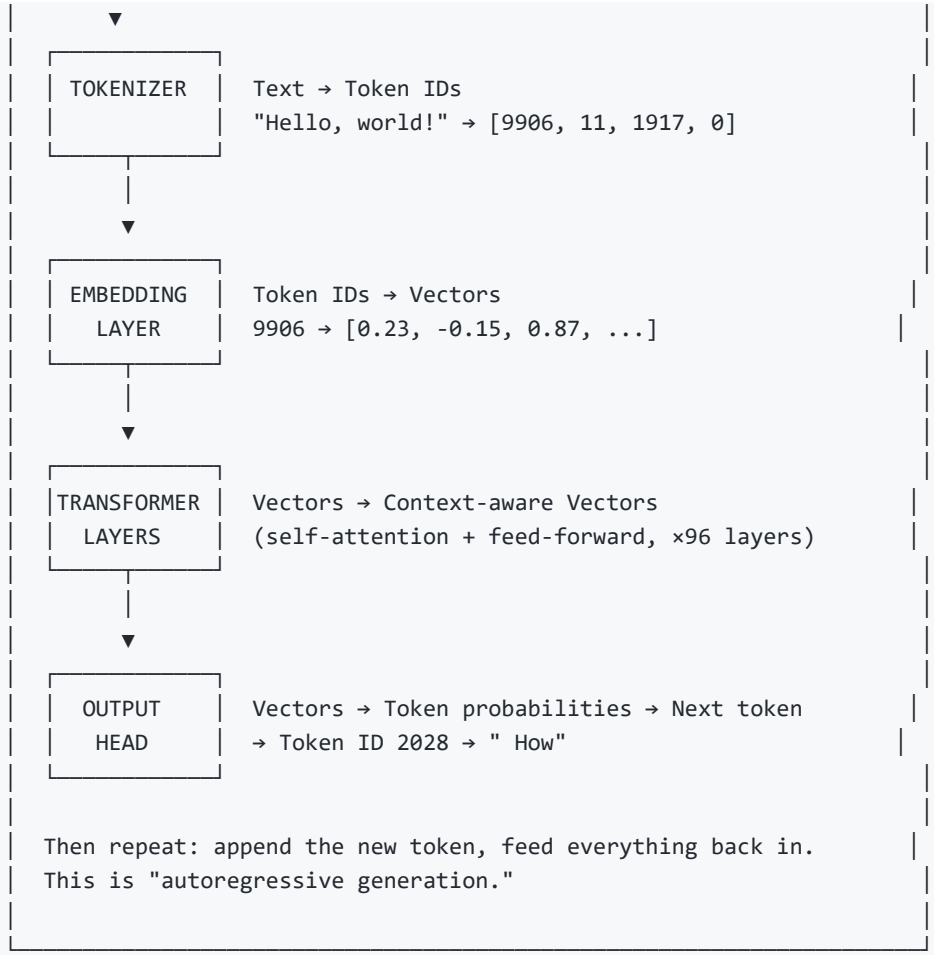
Why should you care about the internals? Because when you build agents, **you will hit problems** that only make sense if you understand the machinery underneath:

- **Token limits**: Why does your agent crash with long documents? Tokens.
- **Cost estimation**: API pricing is per-token, not per-word.
- **Prompt engineering**: Some prompts work better because of how tokenization splits the text.
- **Retrieval (RAG)**: Embeddings power the "find similar documents" step.
- **Debugging**: When the model produces garbage, understanding tokens helps you see why.

The Full Pipeline

Every time you send text to an LLM, this is what actually happens:





Today we focus on the first two boxes: **Tokenizer** and **Embedding Layer**. Parts B and C will cover the Transformer itself.

Tokens & Tokenization

What Is a Token?

A token is **NOT a character** and **NOT a word**. It is a **subword unit** — a chunk of text that the model treats as a single atomic symbol.

Why not just use characters? Too granular — the model would need to "figure out" spelling before it could reason about meaning. Why not whole words? Too many unique words in human language (think: "unhappiness", "unhappy", "happiness", "happily" — do we need separate entries for all of them?).

Subword tokenization is the sweet spot: common words get their own token ("the", "hello"), while rare words get split into reusable pieces ("un" + "happi" + "ness").



Why Tokenization Exists

The core problem: human language is **infinite** (you can always write a new sentence), but a neural network needs a **finite** set of input symbols. Tokenization is the bridge — it maps infinite text to a fixed vocabulary of ~100k tokens.

BPE: Byte Pair Encoding

The most common tokenization algorithm (used by the GPT series, Claude, and most modern LLMs) is **Byte Pair Encoding**. Here's how it works, step by step:

Starting point: Every individual byte (character) is a token.

Algorithm: Repeatedly find the most frequent pair of adjacent tokens and merge them into a new token. Stop when you reach your desired vocabulary size.

```
BPE Walkthrough — Training on: "low lower lowest"

Step 0 (character-level):
  l o w _ l o w e r _ l o w e s t
  (where _ = end-of-word marker)

Step 1: Most frequent pair = (l, o) → merge into "lo"
  lo w _ lo w e r _ lo w e s t

Step 2: Most frequent pair = (lo, w) → merge into "low"
  low _ low e r _ low e s t

Step 3: Most frequent pair = (e, r) appears once, (e, s) appears once,
         (low, _) appears once... pick (low, e) → merge into "lowe"
  low _ lowe r _ lowe s t

...and so on until vocabulary size is reached.

Result: The tokenizer learns that "low" is a reusable unit!
```

Token ≠ Word: The Pitfalls

This is where things get surprising. Tokenization creates **counter-intuitive splits** that directly affect model behavior:

1. Leading whitespace becomes part of the token

```
# " world" (with leading space) is ONE token
# "world" (without space) is a DIFFERENT token
# This is why formatting matters in prompts!
```

2. Long words get split into multiple tokens

```
"tokenization"  →  ["token", "ization"]       (2 tokens)
"embedding"     →  ["embed", "ding"]           (2 tokens)
"transformer"   →  ["trans", "former"]         (2 tokens)
"antidisestablishmentarianism" → 5+ tokens
```

3. Numbers are tokenized weirdly

```
"123"    →  ["123"]                   (1 token — small number, common)
"123456" →  ["123", "456"]           (2 tokens — split at 3 digits)
"98765"  →  ["987", "65"]            (2 tokens — inconsistent split!)

This is why LLMs are bad at arithmetic — the digits are jumbled across tokens!
```

4. Multilingual text is expensive

```
English: "Hello"        → 1 token
Chinese: "你好"          → 2-3 tokens (each character may be a token)
Arabic:  "5-3 →         "مرحبا tokens
Korean:  "안녕하세요"      → 3-5 tokens

Same meaning, wildly different token counts. English dominates the training
data, so English gets more efficient tokenization.
```

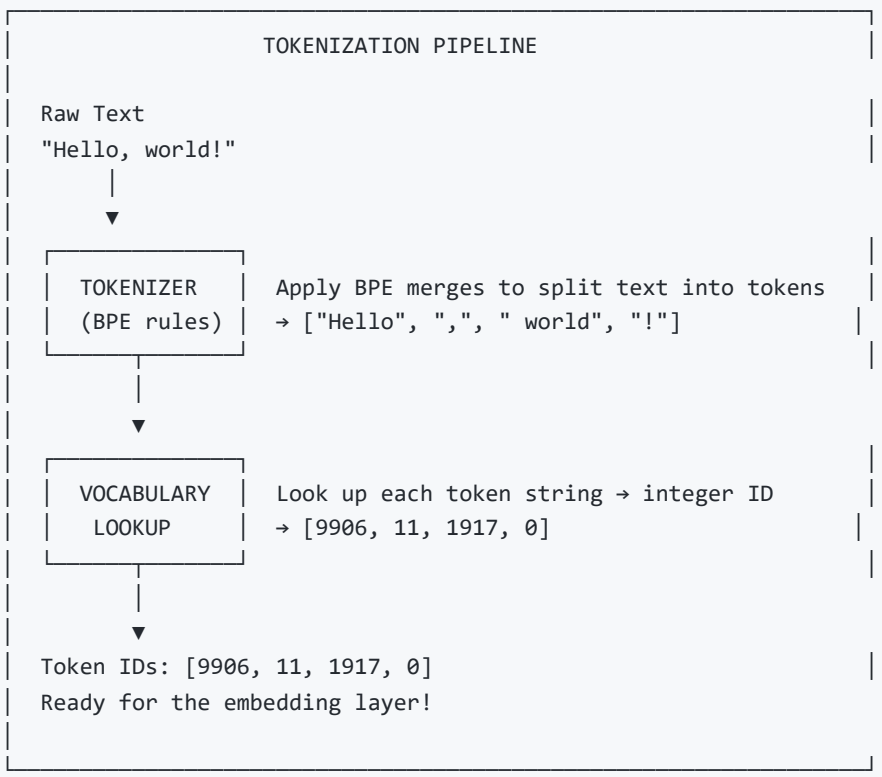
Vocabulary Sizes

```
┌──────────────────────────┬──────────────────┬──────────────────────────────┐
│ Model                    │ Vocab Size       │ Tokenizer                    │
├──────────────────────────┼──────────────────┼──────────────────────────────┤
│ GPT-5 / GPT-5.1 / 5.2    │ ~200,000         │ o200k_base                   │
│ GPT-4o                   │ ~200,064         │ o200k_base                   │
│ GPT-4                    │ ~100,256         │ cl100k_base                  │
```

```
| Claude (Opus 4.6,        | ~100,000+        | BPE (proprietary)          |
|  Sonnet 4.5)             |                  |                            |
| Gemini 3 Pro             | ~256,000         | SentencePiece              |
| LLaMA 3 / 4              | ~128,256         | tiktoken-based BPE         |
| LLaMA 2                  | ~32,000          | SentencePiece BPE          |
| GPT-2                    | ~50,257          | BPE (original)             |
| BERT                     | ~30,522          | WordPiece                  |
└──────────────────────────┴──────────────────┴────────────────────────────┘

Larger vocabulary = fewer tokens per text = faster inference
  but also = larger embedding matrix = more memory
```

The Tokenization Pipeline



Code Example: Tokenization with tiktoken

```python
import tiktoken

# Load the tokenizer (o200k_base, used by GPT-4o and GPT-5 series)
enc = tiktoken.encoding_for_model("gpt-4")

# Encode text into token IDs
text = "Hello, world!"
tokens = enc.encode(text)
print(f"Token IDs: {tokens}")
# Token IDs: [9906, 11, 1917, 0]
```

```
# Decode each token individually to see what it represents
token_strings = [enc.decode([t]) for t in tokens]
print(f"Token strings: {token_strings}")
# Token strings: ['Hello', ',', ' world', '!']

# Count tokens (critical for API cost estimation)
print(f"Token count: {len(tokens)}")
# Token count: 4

# Demonstrate the pitfalls
examples = {
    "tokenization":  enc.encode("tokenization"),
    "123456":        enc.encode("123456"),
    "Hello":         enc.encode("Hello"),
    " Hello":        enc.encode(" Hello"),   # note the leading space
}

for text, toks in examples.items():
    decoded = [enc.decode([t]) for t in toks]
    print(f"  '{text}' → {len(toks)} tokens: {decoded}")

# Output:
#   'tokenization' → 2 tokens: ['token', 'ization']
#   '123456' → 2 tokens: ['123', '456']
#   'Hello' → 1 tokens: ['Hello']
#   ' Hello' → 1 tokens: [' Hello']    ← space is part of the token!
```

Embeddings — The Missing Foundation

Why We Need Embeddings

We now have token IDs — integers like 9906 for "Hello" and 1917 for " world". But there's a problem: **these numbers are arbitrary**. The fact that 9906 comes before 9907 means nothing about meaning. "Hello" (9906) is not more similar to token 9907 than to token 42.

We need a representation where **meaning is captured numerically** — where similar concepts have similar numbers.

That's what embeddings do.

Analogy: GPS Coordinates for Meaning

Think of embeddings as **GPS coordinates, but for meaning instead of geography**.

```
GPS maps physical locations to coordinates:
  New York  → (40.7, -74.0)
```

```
  London    → (51.5, -0.1)
  Paris     → (48.9,  2.3)

  London and Paris are geographically close → their coordinates are close.
  New York is far from both → its coordinates are far.

Embeddings map MEANING to coordinates:
  "king"    → [0.82, -0.31, 0.55, 0.12, ...]    (768 dimensions)
  "queen"   → [0.79, -0.28, 0.53, 0.14, ...]    (nearby!)
  "banana"  → [-0.45, 0.67, -0.22, 0.88, ...]   (far away)

  "king" and "queen" mean related things → their vectors are close.
  "banana" is semantically unrelated → its vector is distant.
```

How Embedding Space Works

Embeddings live in **high-dimensional space** — typically 768 to 4096+ dimensions. We can't visualize 768 dimensions, but the math works the same as in 2D or 3D:

```
Key properties of embedding space:

  1. DIMENSIONS: Each of the 768+ dimensions captures some aspect
     of meaning. No single dimension means "is-a-verb" or
     "is-positive" — it's distributed across many dimensions.

  2. DISTANCE = SIMILARITY: Words with similar meanings are
     nearby in this space (measured by cosine similarity).

  3. DIRECTIONS HAVE MEANING: The direction from "man" to "woman"
     captures the concept of gender. The direction from "walk" to
     "walked" captures past tense.
```

King − Man + Woman = Queen

This famous example demonstrates that embedding space captures **relational structure**, not just similarity. Here's why it works:

```
┌──────────────────────────────────────────────────────────────┐
│              VECTOR ARITHMETIC IN EMBEDDING SPACE            │
│                                                              │
│  The direction from "man" → "woman" encodes a gender shift.  │
│  The direction from "king" → ??? should be the same shift.   │
│  So king − man + woman = ... queen!                          │
│                                                              │
│  Vector math:                                                │
│    king  = [0.82, -0.31, 0.55, 0.12]                         │
│  − man   = [0.60, -0.20, 0.48, 0.35]                         │
│  + woman = [0.58, -0.17, 0.46, 0.38]                         │
│  ──────────────────────────────────                          │
│  result  = [0.80, -0.28, 0.53, 0.15]                         │
```

```
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  Nearest neighbor of result → "queen" [0.79, -0.28, 0.53, 0.14]  │
│                                                                  │
│                                                                  │
│  Simplified 2D visualization:                                    │
│                                                                  │
│       queen ♛ ·                                                  │
│               ↑  gender                                          │
│       king  ♚ ·  direction                                       │
│                                                                  │
│                     woman ♀ ·                                    │
│                             ↑  gender                            │
│                     man   ♂ ·  direction                         │
│                                                                  │
│  Same offset (gender direction) applied to "king" lands on       │
│  "queen". The space has LEARNED these relationships.             │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Why does this matter for agents? Because this same principle powers:

- **RAG**: Finding relevant documents by vector similarity
- **Classification**: Grouping similar inputs together
- **Tool selection**: Matching user intent to the right tool

Token Embeddings vs Contextual Embeddings

This is a critical distinction that many people miss:

```
┌──────────────────────────────────────────────────────────────────┐
│        TOKEN EMBEDDINGS vs CONTEXTUAL EMBEDDINGS                 │
│                                                                  │
│  TOKEN EMBEDDING (before Transformer):                           │
│  ─────────────────────────────────────                           │
│  A static lookup table. The word "bank" ALWAYS gets the same     │
│  vector, regardless of context.                                  │
│                                                                  │
│    "river bank"  → bank → [0.23, -0.15, 0.87, ...]               │
│    "bank account" → bank → [0.23, -0.15, 0.87, ...]  ← SAME!     │
│                                                                  │
│  Problem: "bank" means completely different things in these      │
│  sentences, but gets the same representation.                    │
│                                                                  │
│                                                                  │
│  CONTEXTUAL EMBEDDING (after Transformer):                       │
│  ──────────────────────────────────────────                      │
│  After passing through attention layers, each token's vector     │
│  has been enriched with information from ALL other tokens.       │
│                                                                  │
│    "river bank"   → bank → [0.85, -0.40, 0.12, ...]              │
│    "bank account" → bank → [-0.33, 0.72, 0.55, ...]  ← DIFF!     │
│                                                                  │
```

```
│  Now the representation captures the MEANING IN CONTEXT.          │
│                                                                   │
│  This is exactly WHY we need the Transformer — to transform       │
│  static token embeddings into context-aware representations.      │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

Embedding Space Visualization

If we squash the 768+ dimensions down to 2D (using techniques like t-SNE or UMAP), we can see that **semantically related words cluster together**:

```
┌──────────────────────────────────────────────────────────────────┐
│          2D PROJECTION OF EMBEDDING SPACE                        │
│                                                                   │
│                                                                   │
│        cat ·   · dog                                             │
│     kitten ·      · puppy            · red                       │
│      tiger ·         · wolf          · blue                      │
│                                      · green                     │
│                                      · yellow                    │
│                                                                   │
│                                                                   │
│                  · happy                                          │
│                  · joyful     · Paris                             │
│                  · glad       · London                            │
│                               · Tokyo                             │
│       · sad                   · Berlin                            │
│       · gloomy                                                   │
│       · melancholy                                               │
│                                                                   │
│                                                                   │
│  Animals cluster together.  Colors cluster together.             │
│  Emotions cluster together. Cities cluster together.             │
│  Antonyms (happy/sad) are related but separated.                 │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

Putting It Together

The Embedding Matrix

In practice, the embedding layer is a **giant lookup table** — a matrix of shape `vocabulary_size × embedding_dimension`:

```
┌──────────────────────────────────────────────────────────────────┐
│                  THE EMBEDDING MATRIX                            │
```

```
│                                                                  │
│  Vocabulary size: 100,256 tokens                                 │
│  Embedding dim:   4,096–16,384 (varies by model)                 │
│                                                                  │
│  Matrix shape: 100,256 × 4,096                                   │
│  That's ~410 million parameters JUST for the embedding layer!    │
│                                                                  │
│         dim 0   dim 1   dim 2   dim 3   ...   dim 4095           │
│        ┌───────┬───────┬───────┬───────┬─────┬────────┐          │
│  ID 0  │  0.12 │ -0.34 │  0.56 │  0.01 │ ... │  0.23  │ "!"      │
│  ID 1  │ -0.05 │  0.22 │  0.18 │ -0.67 │ ... │  0.44  │ ...      │
│  ...   │  ...  │  ...  │  ...  │  ...  │ ... │  ...   │ ...      │
│  9906  │  0.82 │ -0.31 │  0.55 │  0.12 │ ... │ -0.09  │ "Hello"  │
│  ...   │  ...  │  ...  │  ...  │  ...  │ ... │  ...   │ ...      │
│        └───────┴───────┴───────┴───────┴─────┴────────┘          │
│                                                                  │
│  To embed token 9906 ("Hello"):                                  │
│    → Just look up row 9906 → [0.82, -0.31, 0.55, 0.12, ...]      │
│                                                                  │
│  That's it! Each token ID is an INDEX into this matrix.          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

These Embeddings Are Learned

A crucial point: nobody hand-designs these vectors. The embedding matrix is **initialized randomly** and then **updated during training** alongside all the other model parameters. Through billions of training examples, the model discovers that certain arrangements of numbers (certain regions of embedding space) are useful for predicting the next token.

The king/queen relationship, the clustering of animals, the separation of topics — all of this **emerges from training**, not from human engineering.

What Happens After Embedding?

Once each token has its embedding vector, two more things are added before the Transformer layers:

1. **Positional information** — the embedding alone doesn't know if "bank" is the 1st or 50th word. Position encodings are added so the model knows word order. (Covered in Day 1C)
2. **The vectors enter the Transformer** — where self-attention creates the contextual embeddings we discussed above. (Covered in Day 1B)

Practice Exercises

Exercise 1: Token Exploration

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

# 1. Tokenize these sentences and compare token counts
sentences = [
    "The quick brown fox jumps over the lazy dog.",
    "Supercalifragilisticexpialidocious is a long word.",
    "こんにちは世界",                          # Japanese "Hello World"
    "2 + 2 = 4",
    "2222222 + 3333333 = 5555555",
    "def fibonacci(n): return n if n <= 1 else fibonacci(n-1) + fibonacci(n-2)",
]

for s in sentences:
    tokens = enc.encode(s)
    decoded = [enc.decode([t]) for t in tokens]
    print(f"\n'{s}'")
    print(f"  Tokens ({len(tokens)}): {decoded}")
```

Exercise 2: Cost Estimation

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

def estimate_cost(text, model="gpt-4", is_input=True):
    """Estimate the API cost of a text string."""
    tokens = enc.encode(text)
    n = len(tokens)

    # Approximate pricing (USD per 1M tokens, check provider for current rates)
    rates = {
        "gpt-5.2":       {"input": 2.00,  "output": 8.00},
        "gpt-5.1":       {"input": 1.00,  "output": 4.00},
        "gpt-4o":        {"input": 2.50,  "output": 10.00},
        "claude-opus":   {"input": 15.00, "output": 75.00},
        "claude-sonnet": {"input": 3.00,  "output": 15.00},
    }

    rate = rates[model]["input" if is_input else "output"]
    cost = (n / 1_000_000) * rate

    print(f"Text: '{text[:50]}...'")
    print(f"  Tokens: {n}")
    print(f"  Estimated cost ({model}, {'input' if is_input else 'output'}): ${cost:
    return cost

# Try it with a system prompt
system_prompt = """You are an expert code reviewer. Analyze the following code
for bugs, security vulnerabilities, and performance issues. For each issue found,
```

```
    provide the line number, severity (high/medium/low), and a suggested fix."""

    estimate_cost(system_prompt, "gpt-5.1")
    estimate_cost(system_prompt, "claude-sonnet")
```

Exercise 3: Tokenization Gotchas Quiz

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

# Predict the token count BEFORE running each one:
quiz = [
    ("Hello", "?"),           # Guess: ___
    (" Hello", "?"),          # Guess: ___ (note: leading space)
    ("hello", "?"),           # Guess: ___ (note: lowercase)
    ("HELLO", "?"),           # Guess: ___
    ("   ", "?"),             # Guess: ___ (three spaces)
    ("100", "?"),             # Guess: ___
    ("1000000", "?"),         # Guess: ___
    ("🎉", "?"),               # Guess: ___
]

print("Tokenization Quiz — were your guesses right?\n")
for text, _ in quiz:
    tokens = enc.encode(text)
    decoded = [enc.decode([t]) for t in tokens]
    print(f"  '{text}' → {len(tokens)} token(s): {decoded}")
```

✓ Day 1A Deliverable Checklist

- ☐ Can explain: What is a token and why not use characters or words?
- ☐ Can explain: How does BPE build a vocabulary from text?
- ☐ Can explain: Why are LLMs bad at arithmetic? (token splits)
- ☐ Can explain: What is an embedding and why are token IDs insufficient?
- ☐ Can explain: Token embedding vs contextual embedding — what's the difference?
- ☐ Ran tiktoken code and explored tokenization pitfalls
- ☐ Estimated API cost for a sample prompt
- ☐ Can explain: What does "king − man + woman = queen" demonstrate?

▶ 📝 **Answer Key (click to expand)**

Day 1B — The Transformer & Self-Attention

> 🎯 **Goal**: Understand the Transformer architecture and self-attention mechanism deeply enough to reason about LLM behavior.

> ⏱️ **Time**: 1.5h theory + 1h practice

> 📦 **Deliverable**: Hand-traced attention computation for a 3-token sequence, annotated Transformer block diagram

---

Table of Contents

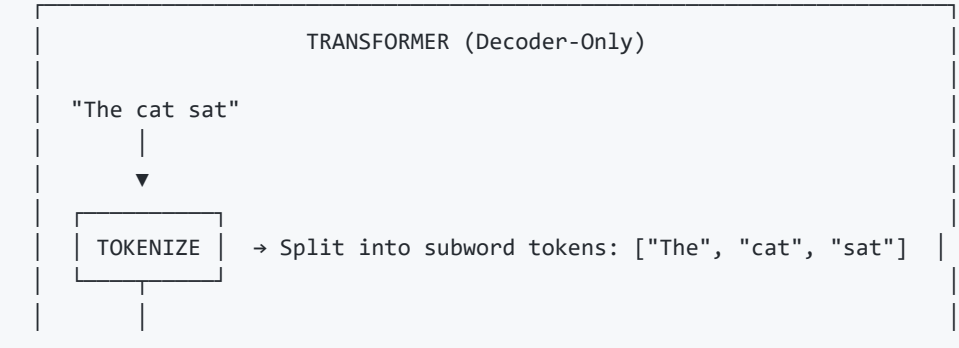
1. The Transformer Architecture (Overview)
2. Self-Attention — The Core Innovation
3. Multi-Head Attention
4. Inside the Transformer Block
5. Practice Exercises

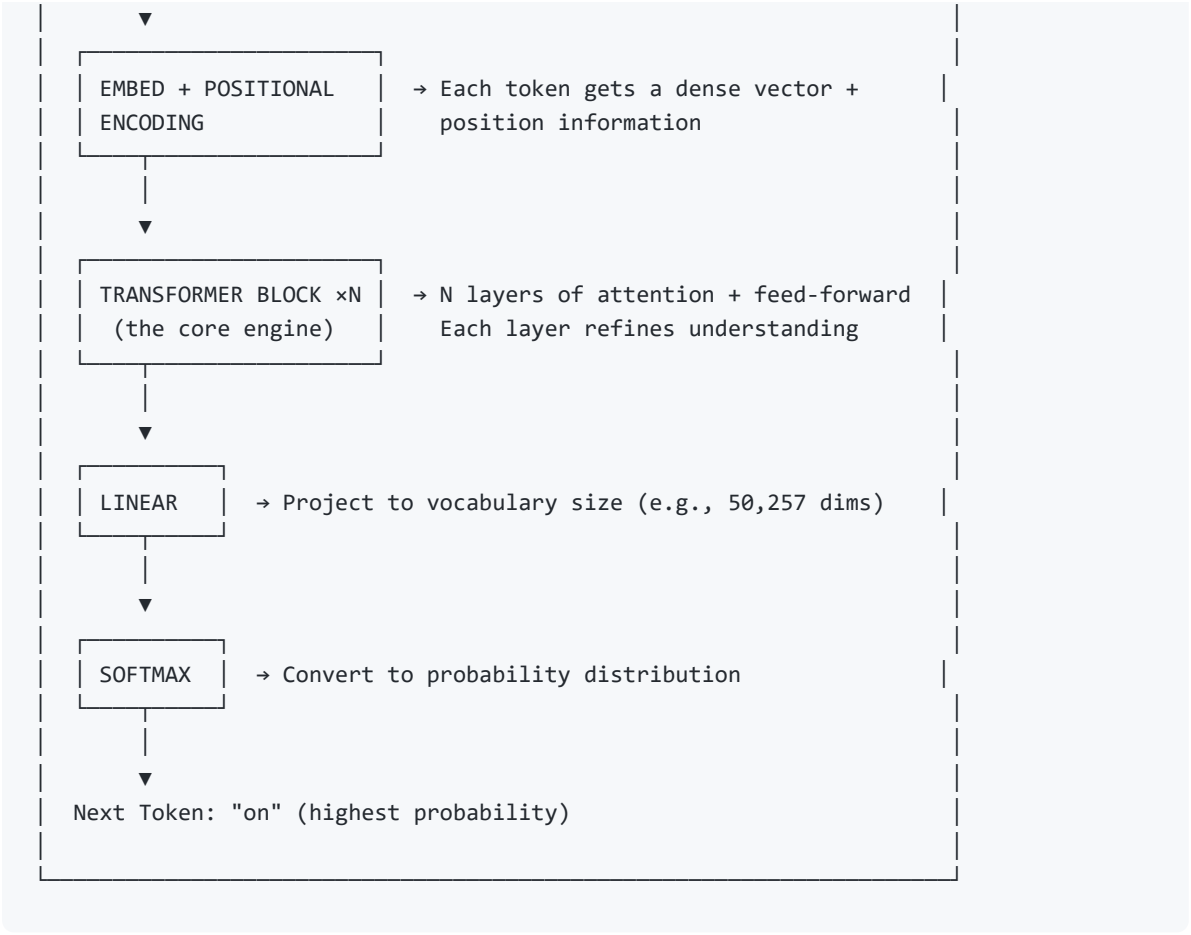
The Transformer Architecture (Overview)

Before 2017, sequence models meant RNNs and LSTMs — architectures that processed tokens **one at a time**, left to right. They were slow (no parallelism) and struggled with long-range dependencies (by the time you reach token 500, you've forgotten token 1).

The 2017 paper **"Attention Is All You Need"** (Vaswani et al.) changed everything. The Transformer processes **all tokens simultaneously** using attention, enabling massive parallelism on GPUs and capturing long-range relationships trivially.

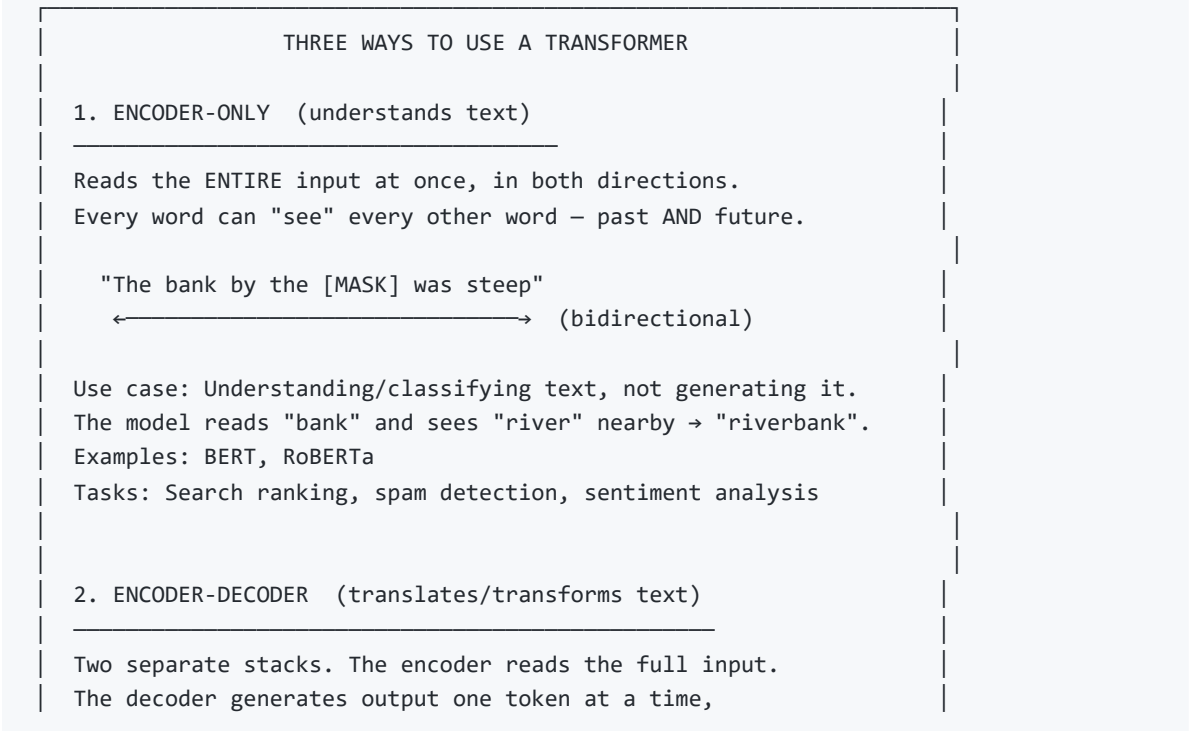
Big-Picture Pipeline





Three Transformer Variants

Not all Transformers are the same. The original paper had both an encoder and decoder, but the field has diverged into three families. Think of it like different reading strategies:



```
│  while looking back at the encoder's understanding.                │
│                                                                    │
│    Encoder: "The cat sat on the mat"  →  [understanding]           │
│                                                 ↓                  │
│    Decoder: "Le" → "chat" → "s'est" → "assis" → ...                │
│                                                                    │
│  Use case: When input and output are different "languages."        │
│  Examples: T5, BART                                                │
│  Tasks: Translation, summarization, format conversion              │
│                                                                    │
│                                                                    │
│  3. DECODER-ONLY  (generates text)  ← THIS IS WHAT ChatGPT IS      │
│  ──────────────────────────────────────────────────────────────    │
│  Reads left-to-right ONLY. Each token can only see tokens          │
│  that came BEFORE it. Predicts the next token, one at a time.      │
│                                                                    │
│    "The capital of France is" → ???                                │
│     ←──────────────────────→    ↑                                  │
│     can see these tokens         predicts "Paris"                  │
│                                                                    │
│  Use case: Everything — chat, code, reasoning, translation.        │
│  It turns out next-token prediction is all you need at scale.      │
│  Examples: GPT-5, LLaMA 4, Claude Opus, Gemini 3                   │
│  Tasks: All of the above (via prompting)                           │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

Summary:

	Encoder-Only	Encoder-Decoder	Decoder-Only
Sees	All tokens (bidirectional)	All (enc), past only (dec)	Only past tokens (causal)
Generates?	No (classifies)	Yes (seq→seq)	Yes (open-ended)
Analogy	Reading an essay and grading it	Reading a book, then writing a book report	Writing a story one word at a time
Status (2026)	Niche	Some use cases	Dominant

Why Decoder-Only Won

Modern LLMs (GPT-5, Claude Opus 4.6, LLaMA 4, Gemini 3) are **all decoder-only**. Why?

1. **Simplicity** — One architecture, one training objective (predict next token)
2. **Scaling** — Decoder-only scales more predictably with compute
3. **Emergent abilities** — At sufficient scale, next-token prediction produces reasoning, coding, translation — all from one objective

4. **Flexibility** — Classification, generation, translation, coding — a single decoder-only model can do it all via prompting

The "N×" Stacking — Why Depth Matters

Each Transformer layer refines the model's internal representation. Early layers capture surface-level patterns (syntax, word associations). Later layers capture abstract reasoning and long-range dependencies.

```
Layer  1–10:  "This word is a noun, that word is a verb"
Layer 11–30:  "This sentence is about finance, not rivers"
Layer 31–60:  "The argument in paragraph 2 contradicts paragraph 5"
Layer 61–96:  "The user wants a Python function that sorts by date"

Model sizes:
  GPT-3:           96 layers,  175B parameters
  LLaMA-7B:        32 layers,    7B parameters
  LLaMA-70B:       80 layers,   70B parameters
  GPT-4:         ~120 layers,   ~1.8T parameters (rumored MoE)
  Claude Opus 4.6: undisclosed, estimated 500B+ parameters
  GPT-5.1/5.2:    undisclosed, likely MoE architecture
```

More layers = deeper understanding, but with diminishing returns and higher cost.

Self-Attention — The Core Innovation

The Intuition: Why We Need Attention

Consider the word "**bank**":

- "I sat by the river **bank**" → bank = edge of a river
- "I deposited money at the **bank**" → bank = financial institution

A simple word embedding gives "bank" **one fixed vector** regardless of context. But we need **different representations** depending on surrounding words. How does the model decide which meaning to use?

It looks at the other tokens in the sentence. That's attention.

```
"I deposited money at the bank"

Without attention:  "bank" = [0.5, 0.3, -0.2, 0.8]  (always the same)

With attention:
  "bank" sees "money", "deposited" → activates financial meaning
```

```
  "bank" = [0.1, 0.9, -0.4, 0.2]  (context-aware: financial)

"I sat by the river bank"

With attention:
  "bank" sees "river", "sat" → activates geographical meaning
  "bank" = [0.8, 0.1, 0.6, -0.3]  (context-aware: riverside)
```

Attention creates **context-dependent embeddings** — every token's representation is shaped by the tokens around it.

The Q, K, V Framework — A Library Analogy

This is the most important concept in this guide. Let's build the intuition carefully.

Imagine you walk into a library looking for information about a topic.

```
┌─────────────────────────────────────────────────────────────┐
│                    THE LIBRARY ANALOGY                      │
│                                                             │
│  YOU (a token) walk into the library with a QUESTION:       │
│                                                             │
│    Query (Q): "I need information about rivers"             │
│                                                             │
│  Each book on the shelf has a LABEL on its spine:           │
│                                                             │
│    Key (K):  "Geography"  "Finance"  "Rivers of Europe"     │
│                                                             │
│  You compare your question to each label:                   │
│                                                             │
│    Q · K₁ = "rivers" vs "Geography"       → moderate match  │
│    Q · K₂ = "rivers" vs "Finance"         → low match       │
│    Q · K₃ = "rivers" vs "Rivers of Europe"→ high match!     │
│                                                             │
│  Then you READ the content of the most relevant books:      │
│                                                             │
│    Value (V): The actual detailed content inside each book  │
│                                                             │
│  Your output = weighted combination of book contents,       │
│  weighted by how relevant each book's label was to your query│
└─────────────────────────────────────────────────────────────┘
```

Every token in the sequence plays ALL three roles simultaneously:

```
Token "bank" is:
  - A VISITOR asking questions about other tokens  (Query)
  - A BOOK being searched by other tokens          (Key)
  - A CONTENT PROVIDER giving info to other tokens (Value)
```

```
All tokens query all other tokens at the same time.
This is what makes it "self"-attention — the sequence attends to itself.
```

Why Three Separate Projections (Q, K, V)?

You might wonder: **Why not just compare the raw embeddings directly?**

Because the **question you ask**, the **label you present**, and the **content you provide** are fundamentally different things:

```
Consider the token "bank":

  As a Query (what it's LOOKING FOR):
    "I need context — am I financial or geographical?"

  As a Key (what it ADVERTISES to others):
    "I'm a noun that could be a place or institution"

  As a Value (what it CONTRIBUTES when attended to):
    "Here's my detailed semantic content about institutions/places"

These are three different roles! A single vector can't optimally
serve all three purposes. Learned projection matrices (W_Q, W_K, W_V)
let the model create SPECIALIZED representations for each role.
```

Step-by-Step Numerical Walkthrough

Let's trace attention for **3 tokens: "The", "cat", "sat"** using tiny 4-dimensional vectors.

Step 1: Start with embeddings

```
After embedding + positional encoding, each token has a vector:

  "The" = [1.0,  0.5, -0.5,  0.2]
  "cat" = [0.3,  0.8,  0.6, -0.1]
  "sat" = [0.7, -0.2,  0.4,  0.9]
```

Step 2: Project to Q, K, V using learned weight matrices

In practice, W_Q, W_K, W_V are learned during training. Here we use simple matrices for illustration (projecting 4-dim → 3-dim for clarity):

```
          ┌              ┐
          │ 1   0   1    │
W_Q   =   │ 0   1   0    │     (4×3 matrix)
          │ 1   0  -1    │
          │ 0   1   1    │
          └              ┘
```

```
          └              ┘

Q_the = [1.0, 0.5, -0.5, 0.2] × W_Q = [0.5,  1.2,  1.2]
Q_cat = [0.3, 0.8,  0.6, -0.1] × W_Q = [0.9,  0.7, -0.1]
Q_sat = [0.7, -0.2, 0.4,  0.9] × W_Q = [1.1,  0.7,  0.2]

Similarly for K and V (using different W_K, W_V matrices):

K_the = [0.8,  0.4,  0.6]      V_the = [0.3,  0.7,  0.5]
K_cat = [0.5,  0.9,  0.3]      V_cat = [0.8,  0.2,  0.6]
K_sat = [0.6,  0.1,  0.8]      V_sat = [0.4,  0.9,  0.3]
```

Step 3: Compute attention scores (Q × Kᵀ)

Each query asks: "How relevant is each key to me?"

```
Score = Q × Kᵀ  (matrix multiplication)

                    K_the   K_cat   K_sat
                    [0.8]   [0.5]   [0.6]
                    [0.4]   [0.9]   [0.1]
                    [0.6]   [0.3]   [0.8]

Q_the [0.5, 1.2, 1.2]  →  [1.60,   1.69,   1.26]
Q_cat [0.9, 0.7, -0.1]  →  [1.00,   1.05,   0.53]
Q_sat [1.1, 0.7, 0.2]   →  [1.28,   1.22,   0.89]

Score Matrix:
              The    cat    sat
    The  [   1.60   1.69   1.26  ]
    cat  [   1.00   1.05   0.53  ]
    sat  [   1.28   1.22   0.89  ]
```

Step 4: Scale by √d_k

```
WHY SCALE? Without scaling, when d_k is large (e.g., 64), the dot
products become very large. Large values pushed into softmax produce
extremely peaked distributions (one value ≈ 1.0, rest ≈ 0.0).
This kills the gradient — the model can't learn!

Scaling by √d_k keeps values in a moderate range where softmax
produces useful (non-saturated) gradients.

d_k = 3 (our key dimension), so √d_k = √3 ≈ 1.73

Scaled scores = Score / 1.73:

              The    cat    sat
    The  [   0.92   0.98   0.73  ]
    cat  [   0.58   0.61   0.31  ]
    sat  [   0.74   0.71   0.51  ]
```

Step 5: Apply softmax (row-wise)

```
WHY SOFTMAX? We need scores to become probabilities (positive, sum to 1).
Softmax is differentiable (critical for backpropagation) and amplifies
differences — higher scores get disproportionately more weight.

softmax(x_i) = e^(x_i) / Σ(e^(x_j))

Row "The": e^0.92=2.51, e^0.98=2.66, e^0.73=2.08  → sum=7.25
  → [2.51/7.25, 2.66/7.25, 2.08/7.25] = [0.346, 0.367, 0.287]

Row "cat": e^0.58=1.79, e^0.61=1.84, e^0.31=1.36  → sum=4.99
  → [1.79/4.99, 1.84/4.99, 1.36/4.99] = [0.359, 0.369, 0.272]

Row "sat": e^0.74=2.10, e^0.71=2.03, e^0.51=1.67  → sum=5.80
  → [2.10/5.80, 2.03/5.80, 1.67/5.80] = [0.362, 0.350, 0.288]

Attention Weights:
              The    cat    sat
    The  [  0.346  0.367  0.287  ]   ← "The" attends most to "cat"
    cat  [  0.359  0.369  0.272  ]   ← "cat" attends most to itself
    sat  [  0.362  0.350  0.288  ]   ← "sat" attends most to "The"
```

Step 6: Multiply attention weights by V

```
Output = Attention_Weights × V

                     V_the    V_cat    V_sat
                    [0.3]    [0.8]    [0.4]
                    [0.7]    [0.2]    [0.9]
                    [0.5]    [0.6]    [0.3]

Out_the = 0.346×[0.3,0.7,0.5] + 0.367×[0.8,0.2,0.6] + 0.287×[0.4,0.9,0.3]
        = [0.104,0.242,0.173] + [0.294,0.073,0.220] + [0.115,0.258,0.086]
        = [0.513, 0.573, 0.479]

Out_cat = 0.359×[0.3,0.7,0.5] + 0.369×[0.8,0.2,0.6] + 0.272×[0.4,0.9,0.3]
        = [0.108,0.251,0.180] + [0.295,0.074,0.221] + [0.109,0.245,0.082]
        = [0.512, 0.570, 0.483]

Out_sat = 0.362×[0.3,0.7,0.5] + 0.350×[0.8,0.2,0.6] + 0.288×[0.4,0.9,0.3]
        = [0.109,0.253,0.181] + [0.280,0.070,0.210] + [0.115,0.259,0.086]
        = [0.504, 0.582, 0.477]
```

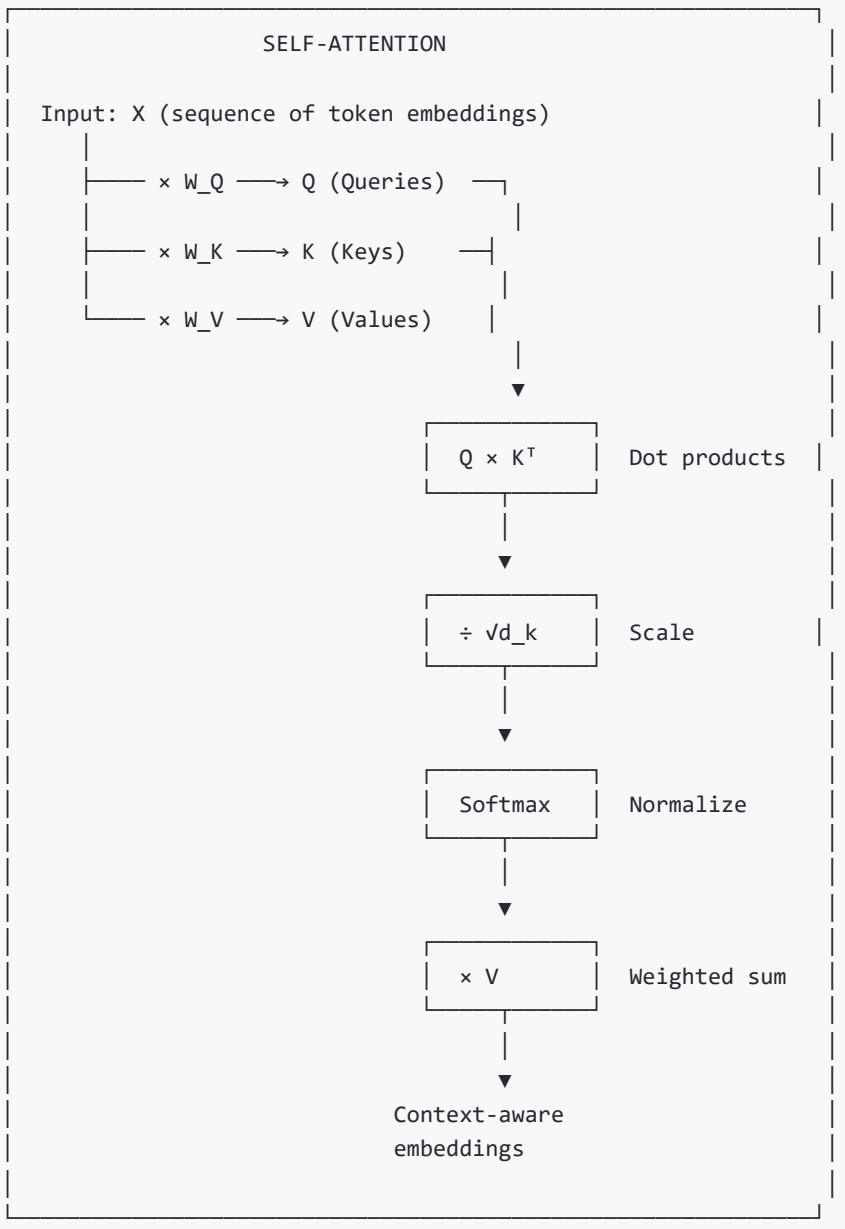
Step 7: What changed?

```
Before attention (raw embeddings — context-blind):
  "The" = [1.0,  0.5, -0.5,  0.2]
  "cat" = [0.3,  0.8,  0.6, -0.1]
  "sat" = [0.7, -0.2,  0.4,  0.9]
```

```
After attention (context-aware representations):
  "The" = [0.513, 0.573, 0.479]
  "cat" = [0.512, 0.570, 0.483]
  "sat" = [0.504, 0.582, 0.477]

Each token's output is now a BLEND of information from all tokens,
weighted by relevance. "The" now carries information about "cat"
and "sat". No token is an island — every representation is
shaped by its context.
```

Attention Computation Flow



Formula: $\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$

Multi-Head Attention

Why Multiple Heads?

A single attention head learns **one type of relationship** between tokens. But language has **many simultaneous relationships**:

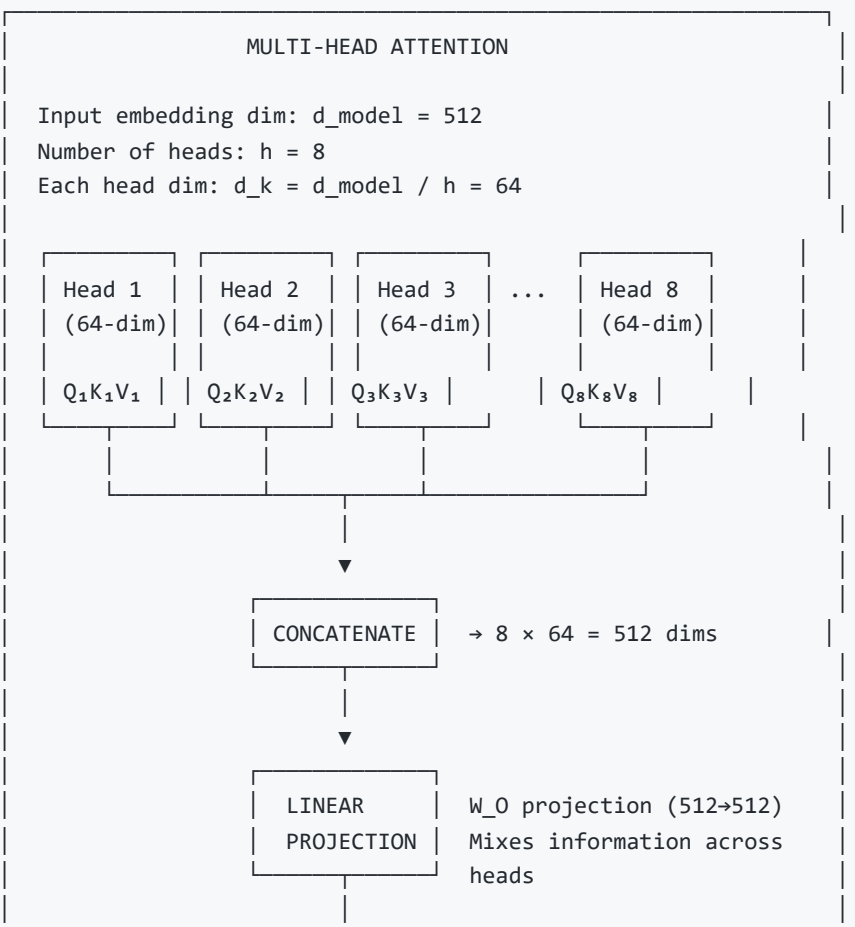
```
"Alice told Bob that she would meet him at the bank near the river"

Relationships a model needs to track simultaneously:

  Head 1 (Syntax):     "Alice" ↔ "told"  (subject-verb)
  Head 2 (Coreference): "she" → "Alice"    (pronoun resolution)
  Head 3 (Coreference): "him" → "Bob"      (pronoun resolution)
  Head 4 (Semantics):   "bank" ← "river"   (word sense: riverside)
  Head 5 (Position):    "at" ← "bank"      (prepositional attachment)
  Head 6 (Discourse):   "told" ← "meet"    (reported speech link)

One attention head can't capture all of these at once.
Multiple heads let the model attend to different relationship
types in parallel.
```

How Multi-Head Attention Works



```
│                          ▼                                    │
│                  Output (512-dim)                              │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

Typical Head Counts

```
Model          d_model    Heads    d_k (per head)
─────────────  ─────────  ──────   ──────────────
BERT-base        768       12          64
BERT-large      1024       16          64
GPT-2            768       12          64
LLaMA-7B        4096       32         128
LLaMA-70B       8192       64         128
GPT-3           12288       96         128
GPT-5.x        varies     GQA      varies (MoE)
```

Masked (Causal) Attention

For **text generation**, a critical constraint: when predicting the next token, the model **cannot look at future tokens**. You can't use the answer to predict the answer!

This is enforced with a **causal mask** — a triangular matrix that blocks attention to future positions:

```
Tokens:    "The"  "cat"  "sat"  "on"

Mask Matrix (1 = can attend, 0 = blocked):

            The   cat   sat    on
    The  [   1     0     0     0  ]   ← "The" can only see itself
    cat  [   1     1     0     0  ]   ← "cat" sees "The" + itself
    sat  [   1     1     1     0  ]   ← "sat" sees "The", "cat", itself
    on   [   1     1     1     1  ]   ← "on" sees everything before it

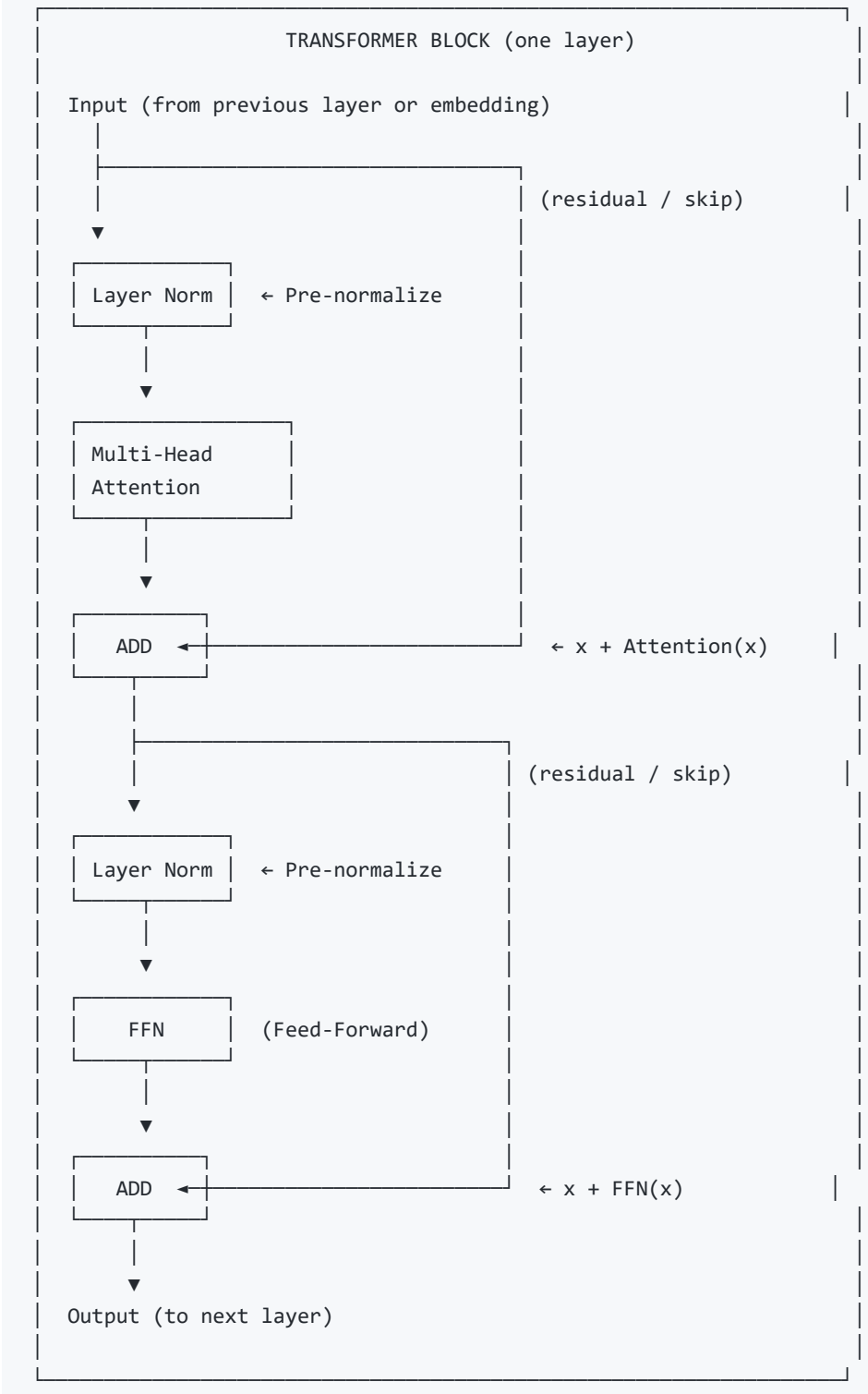
Implementation: set masked positions to -∞ BEFORE softmax.
softmax(-∞) = 0, so future tokens get zero attention weight.

This is why it's called "causal" — each token can only be
caused by (depend on) past tokens, never future ones.
This is what makes decoder-only models autoregressive.
```

Inside the Transformer Block

Each of the N transformer layers is a **block** with the same internal structure. Understanding this block is understanding the entire model.

Full Transformer Block



Layer Normalization

Why do we need it? As data flows through 32–96+ layers, values can explode (grow huge) or vanish (shrink to zero). Without normalization, deep networks become untrainable.

```
Problem without LayerNorm:

  Layer 1 output:  [0.5,  0.3, -0.2]        ← reasonable
  Layer 10 output: [15.2, -8.7,  22.1]      ← growing
  Layer 50 output: [4521, -2893, 7104]       ← exploded!

With LayerNorm after each sub-layer:

  Layer 1 output:  [0.5,  0.3, -0.2]  → normalize → [0.87, 0.22, -1.09]
  Layer 10 output: [0.8, -0.4,  0.6]  → normalize → [1.07, -1.34,  0.27]
  Layer 50 output: [0.3,  0.7, -0.1]  → normalize → [-0.51, 1.38, -0.87]
                                          ↑ stays bounded!
```

What it does:

1. Compute mean and standard deviation of the vector
2. Normalize: (x - mean) / std → mean=0, std=1
3. Scale and shift with **learned** parameters γ and β

```
LayerNorm(x) = γ × (x - μ) / σ + β

μ = mean of x,  σ = std of x
γ, β = learned parameters (the model decides the best scale/shift)
```

Pre-norm vs Post-norm:

```
Original paper (2017) — Post-norm:
  x → Attention → Add → LayerNorm → FFN → Add → LayerNorm

Modern models (GPT-5, LLaMA 4, Claude) — Pre-norm:
  x → LayerNorm → Attention → Add → LayerNorm → FFN → Add

Pre-norm trains more stably. The residual path stays "clean"
(un-normalized), which helps gradients flow through deep networks.
```

Feed-Forward Network (FFN) — The Knowledge Store

Why do we need FFN when we already have attention?

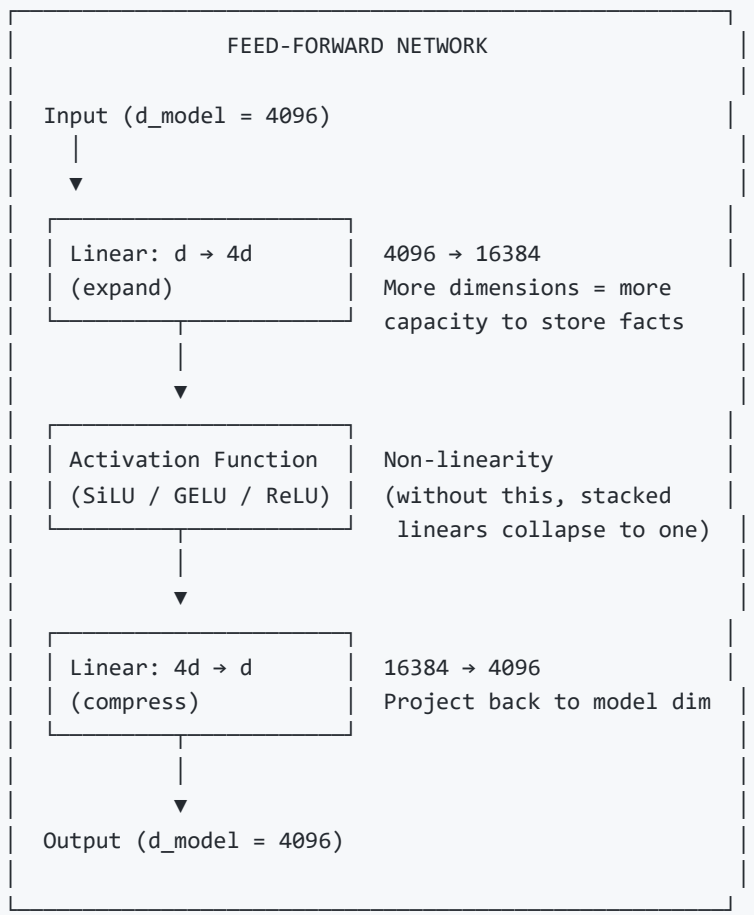
Attention decides **WHAT information to gather** from context. FFN decides **WHAT TO DO with that information**. They have complementary roles:

```
Analogy:
  Attention = Reading the right books in the library
  FFN       = Thinking about what you read and forming conclusions

Without FFN: The model can look at relevant context but can't
```

```
             reason about it or apply stored knowledge.
Without Attention: The model can think but has no way to
                   gather relevant context.
```

FFN Structure:



Why the 4× expansion? More parameters in the hidden layer = more capacity to **store factual knowledge**. Research shows that specific neurons in the FFN activate for specific facts:

```
Neuron 3847 in layer 22:  activates for "Paris is the capital of France"
Neuron 1204 in layer 35:  activates for Python syntax patterns
Neuron 8891 in layer 48:  activates for date formatting conventions

The FFN layers collectively form a massive key-value memory store.
The 4× expansion gives the model enough "slots" to store millions
of learned associations.
```

Activation functions — brief comparison:

```
┌────────────┬──────────────────────────┬──────────────────────────┐
│ Function   │ Formula                  │ Used In                  │
├────────────┼──────────────────────────┼──────────────────────────┤
```

```
| ReLU     | max(0, x)                | Original Transformer  |
| GELU     | x × Φ(x) (Gaussian CDF) | BERT, GPT-2, GPT-3    |
| SiLU     | x × σ(x) (x × sigmoid)  | LLaMA, Mistral,       |
|          |                          | GPT-5, Claude         |
├──────────┼──────────────────────────┼──────────────────────┤
| Trend    | ReLU → GELU → SiLU       | Smoother = better     |
|          | (progressively smoother) | gradient flow         |
└──────────┴──────────────────────────┴──────────────────────┘
```

Residual (Skip) Connections

The `+` in `output = layer(input) + input` is the **residual connection** — and it's what makes deep Transformers possible.

Why are residual connections essential?

Without them, a 96-layer network would be untrainable. Gradients (the learning signal) must flow backwards through every layer during training. In deep networks without residuals, gradients **vanish** — they shrink exponentially until early layers receive essentially zero learning signal.

```
Without residual connections (96 layers):

  Gradient at layer 96: 1.0
  Gradient at layer 50: 0.0001
  Gradient at layer  1: 0.0000000000001  ← effectively zero, can't learn

With residual connections:

  Gradient at layer 96: 1.0
  Gradient at layer 50: 0.95    ← gradient flows through skip connection
  Gradient at layer  1: 0.82   ← still strong, can learn!
```

The highway analogy:

```
┌─────────────────────────────────────────────────┐
│              RESIDUAL CONNECTION                │
│                                                 │
│  Input ──────────────────────────┐              │
│    │                             │ HIGHWAY      │
│    │    ┌──────────────────┐     │ (skip)       │
│    └──→ │  Attention or FFN │     │              │
│         │  (scenic route)   │     │              │
│         └──────────┬────────┘     │              │
│                    │              │              │
│                    ▼              │              │
│                  [ + ] ←──────────┘              │
│                    │                            │
│                    ▼                            │
│                 Output                          │
└─────────────────────────────────────────────────┘
```

```
│                                                          │
│  output = layer(input) + input                           │
│                                                          │
│  Each layer only needs to learn the DELTA (improvement)  │
│  not the entire representation from scratch.             │
│                                                          │
│  If a layer has nothing useful to add, it can learn      │
│  to output ≈ 0, and the input passes through unchanged.  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

This is why Transformers can be stacked so deep — each layer learns a **small refinement** on top of what's already there, rather than rebuilding the representation from scratch.

Practice Exercises

Exercise 1: Trace Attention by Hand

Take the sentence "**I love cats**" with these embeddings (3-dim for simplicity):

```
"I"    = [1.0, 0.0, 0.5]
"love" = [0.2, 1.0, 0.3]
"cats" = [0.8, 0.5, 1.0]
```

Using these weight matrices:

```
        ┌         ┐          ┌         ┐          ┌         ┐
W_Q =   │ 1  0    │  W_K =   │ 0  1    │  W_V =   │ 1  1    │
        │ 0  1    │          │ 1  0    │          │ 0  1    │
        │ 1  1    │          │ 1  1    │          │ 1  0    │
        └         ┘          └         ┘          └         ┘
```

1. Compute Q, K, V for each token
2. Compute the score matrix $Q \times K^T$
3. Scale by $\sqrt{d_k}$ (d_k = 2)
4. Apply softmax row-wise
5. Compute the final output

Exercise 2: Draw the Causal Mask

For a 5-token sequence `["The", "dog", "runs", "very", "fast"]`:

1. Draw the 5×5 causal mask matrix

2. Show what the attention score matrix looks like **before** and **after** applying the mask
3. Explain why position 3 ("very") cannot attend to position 5 ("fast")

Exercise 3: Annotate a Transformer Block

Draw (on paper or digitally) a complete Transformer block diagram and annotate each component with:

1. Its purpose (one sentence)
2. Its input/output dimensions for LLaMA-7B (d_model=4096, heads=32, d_ff=11008)
3. Why it's necessary (what breaks if you remove it?)

✓ Day 1B Checklist

- [ ] Can explain what self-attention does and why it's needed (context-dependent embeddings)
- [ ] Can trace Q, K, V computation through a small example by hand
- [ ] Understand why Q, K, V are separate projections (different roles)
- [ ] Know why we scale by √d_k (prevent softmax saturation)
- [ ] Can explain multi-head attention (parallel relationship detection)
- [ ] Understand the causal mask and why it's needed for generation
- [ ] Can explain all components of a Transformer block (LayerNorm, FFN, residuals)
- [ ] Know the role of FFN vs attention (knowledge store vs context gathering)
- [ ] Can explain why residual connections enable deep networks

▶ 📝 **Answer Key (click to expand)**

Day 1C — Training Pipeline & Inference

> 🎯 **Goal**: Understand how LLMs are trained (pre-training → SFT → RLHF) and how they generate text at inference time.

> ⏱️ **Time**: 1h theory + 1h practice

> 📦 **Deliverable**: A sampling strategy visualizer + tokenization explorer + clear mental model of the 3-phase training pipeline

---

Table of Contents

1. Positional Encoding
2. The Training Pipeline
3. Inference — How Generation Actually Works
4. Practice Exercises
5. Deliverable Checklist

Positional Encoding

Why Position Matters

Transformers process all tokens in **PARALLEL** — every token attends to every other token simultaneously. This is what makes them fast (unlike RNNs, which go left-to-right, one step at a time). But it creates a critical problem:

```
Without positional info, the model sees:

  "dog bites man"  ↔  "man bites dog"

as the SAME input! The set of tokens is identical.
The model has NO concept of order. None. Zero.

It's like dumping a bag of Scrabble tiles on the table —
you know WHICH letters exist, but not what order they go in.
```

We need to **inject** position information into the embeddings.

Sinusoidal Encoding (Original Transformer Paper)

The clock analogy: Think of telling time. The seconds hand spins fast, the minutes hand slower, the hours hand slowest. Each "frequency" captures position at a different granularity. Sinusoidal encoding does the same thing — each dimension of the embedding oscillates at a different frequency.

```
Formula:

  PE(pos, 2i)     = sin(pos / 10000^(2i/d))
  PE(pos, 2i+1)   = cos(pos / 10000^(2i/d))

Where:
  pos = position in the sequence (0, 1, 2, ...)
```

```
  i   = dimension index
  d   = embedding dimension (e.g., 512)
```

Why sin AND cos? Because the model can learn to compute **RELATIVE** positions via linear transformation. `PE(pos+k)` can be expressed as a linear function of `PE(pos)` — this means the model can figure out "how far apart are these two tokens?" just from the encodings.

```
Intuition — it's like binary counting, but smooth:

  Position 0:  [0.00, 1.00, 0.00, 1.00, 0.00, 1.00, ...]
  Position 1:  [0.84, 0.54, 0.02, 1.00, 0.00, 1.00, ...]
  Position 2:  [0.91, -0.4, 0.03, 1.00, 0.00, 1.00, ...]
                 ↑ changes fast       ↑ changes slowly

  Low dimensions  = fast oscillation  (like seconds hand)
  High dimensions = slow oscillation  (like hours hand)
```

Learned Position Embeddings (GPT-2 Style)

Just learn a separate embedding vector for each position. Position 0 gets one vector, position 1 gets another, etc.

```
Simple lookup table:

  position_embeddings = Embedding(max_positions, d_model)

  Position 0 → [0.12, -0.34, 0.56, ...]  ← learned during training
  Position 1 → [0.08, -0.21, 0.43, ...]  ← learned during training
  ...
  Position 2047 → [...]                   ← last supported position
  Position 2048 → ???                     ← CANNOT extrapolate!
```

Pro: Simple, works well within training length. **Con**: Can't handle sequences longer than seen in training. Position 2048 is a hard wall.

RoPE (Rotary Position Embeddings)

Used in **LLaMA 3/4**, **Mistral**, **GPT-NeoX**, **Gemini**, and most modern models. This is the current state of the art.

Key insight: Instead of ADDING position info to embeddings, **ROTATE** the embedding vectors. Position becomes a rotation angle.

```
Traditional:  embedding = token_embedding + position_embedding
RoPE:         embedding = Rotate(token_embedding, angle=f(position))

Think of it as spinning a vector in 2D space:

  Position 0: →        (no rotation)
  Position 1: ↗        (rotated slightly)
  Position 2: ↑        (rotated more)
  Position 3: ↖        (rotated even more)
```

Why this is brilliant: When you compute Q·K (the attention score), the rotation naturally encodes the **DISTANCE** between tokens. Two tokens at positions 5 and 7 will have a rotation difference of 2, regardless of their absolute positions. Closer tokens → smaller rotation difference → naturally captures locality.

Advantage over alternatives: RoPE naturally captures relative position AND can extrapolate to longer sequences than seen in training (with techniques like NTK-aware scaling or YaRN).

Comparison: Positional Encoding Methods

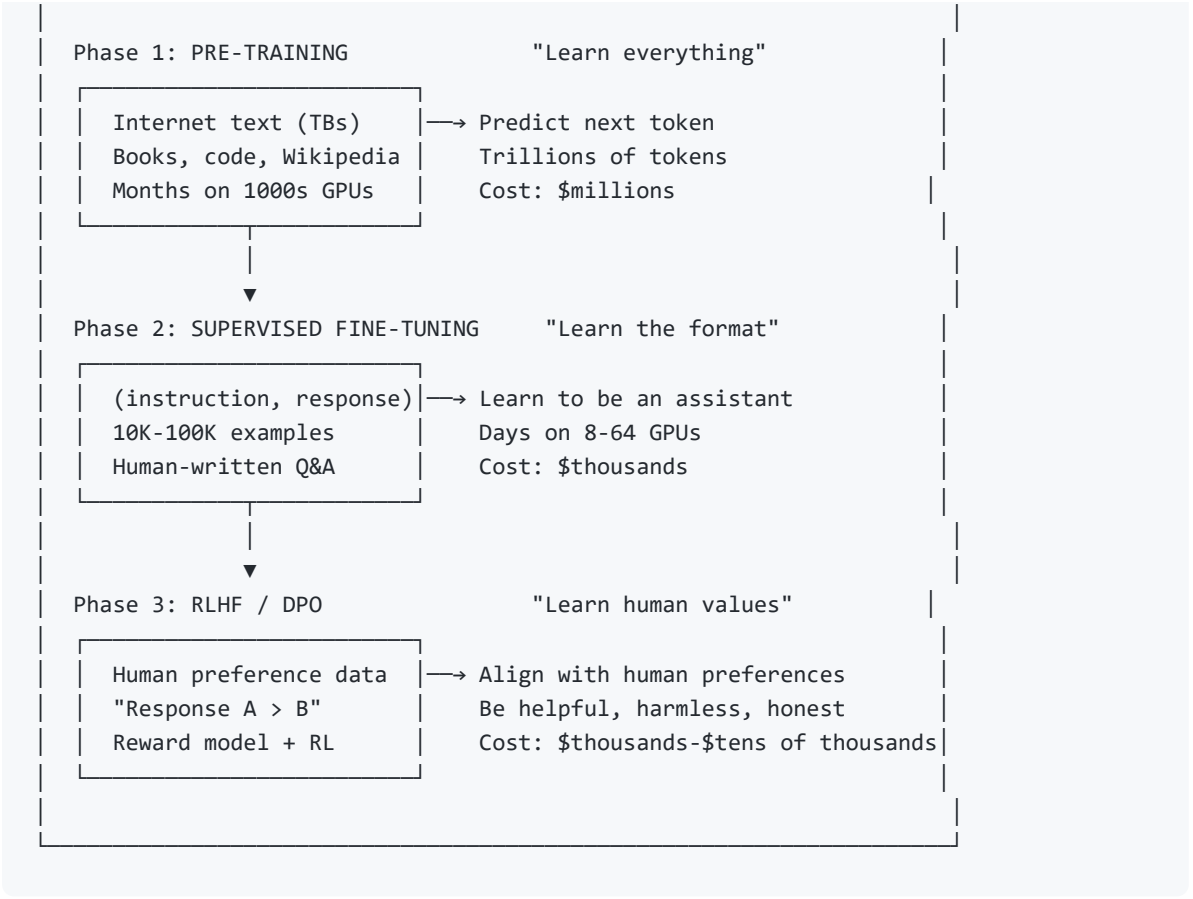
	Sinusoidal	Learned	RoPE
Extrapolation beyond training	Theoretically yes, but weak	Cannot extrapolate	Yes (with scaling tricks)
Relative position awareness	Via linear transform	No (absolute only)	Built-in (Q·K encodes distance)
Trainable?	No (fixed)	Yes	No (but can tune frequency)
Modern usage	Original Transformer	GPT-2, BERT	LLaMA 4, GPT-5, Claude, Gemini

---

The Training Pipeline

Modern LLMs go through **three distinct phases**, each serving a fundamentally different purpose. Think of it as: learn language → learn format → learn values.

```
┌──────────────────────────────────────────────────────────────────┐
│                  THE 3-PHASE TRAINING PIPELINE                   │
```



Phase 1: Pre-training — Learning Language

The model's job is deceptively simple: **predict the next token**. Over and over, trillions of times.

```
Training example:

  Input:  "The capital of France is"
  Target: "Paris"

  The model sees TRILLIONS of such examples scraped from the internet:
  books, Wikipedia, code, forums, news — everything.
```

Cross-Entropy Loss — Intuition

The model outputs a probability distribution over ALL tokens in the vocabulary (e.g., 100,000 probabilities). The correct next token should have HIGH probability.

Cross-entropy measures: "How surprised is the model by the correct answer?"

```
Example: "The capital of France is ___"

  Model's prediction (probability distribution over 100K tokens):
```

```
  Token         Probability     -log(p)
  _____         ___________     ________
  "Paris"         0.95          0.05   ← low loss (good! not surprised)
  "the"           0.02          3.91
  "Lyon"          0.01          4.61
  "Berlin"        0.005         5.30
  ... (99,996 other tokens share the remaining 0.015)

  Loss = -log(P("Paris")) = -log(0.95) = 0.05  ← Great!

  ________________________________________________

  Bad model's prediction:

  "Paris"         0.01          4.61   ← high loss (bad! very surprised)
  "the"           0.30          1.20
  "banana"        0.15          1.90

  Loss = -log(P("Paris")) = -log(0.01) = 4.61  ← Terrible!
```

Key insight: The log function penalizes **confident wrong answers** far more than uncertain ones. If the model says P("Paris") = 0.001 with high confidence in something else, the loss EXPLODES. This forces the model to be well-calibrated.

What Emerges at Different Scales

```
┌──────────────────────┬──────────────────────────────────────────┐
│ Scale                │ Capabilities that emerge                 │
├──────────────────────┼──────────────────────────────────────────┤
│ Small (1B params)    │ Grammar, basic facts, simple completion  │
│ Medium (7–13B)       │ Reasoning, coding basics, following intent │
│ Large (70–405B)      │ Complex reasoning, few-shot learning,    │
│                      │ nuanced understanding                    │
│ Frontier (MoE /      │ Emergent abilities: chain-of-thought,    │
│  undisclosed scale)  │ in-context learning, self-correction,    │
│                      │ agentic behavior (GPT-5, Claude Opus 4.6) │
└──────────────────────┴──────────────────────────────────────────┘

Training data scale:
  GPT-3:      300 billion tokens
  LLaMA-2:    2 trillion tokens
  LLaMA-3:    15 trillion tokens
  LLaMA-4:    ~30+ trillion tokens (estimated)
  GPT-5.x:    undisclosed, likely 20T+ tokens
  Modern:     10T–50T+ tokens
```

Cost: Millions of dollars, thousands of GPUs, weeks to months of continuous training. Pre-training is the most expensive phase by far.

Phase 2: Supervised Fine-Tuning (SFT) — Learning the Format

The problem: After pre-training, the model is a **COMPLETOR**, not an **ASSISTANT**. It has absorbed immense knowledge, but it doesn't know it should *answer questions*.

```
What pre-training produces:

  User:  "What is 2+2?"
  Model: "This is a common math question often asked in elementary
          school. Teachers typically introduce addition in the first
          grade, and students learn that..."
          ↑ Continues the text, doesn't ANSWER

What we want:

  User:  "What is 2+2?"
  Model: "4"
          ↑ Direct answer
```

Solution: Fine-tune on curated (instruction, response) pairs.

```
SFT training data looks like:

  {"instruction": "What is the capital of Japan?",
   "response": "The capital of Japan is Tokyo."}

  {"instruction": "Write a Python function to reverse a string.",
   "response": "def reverse_string(s):\n    return s[::-1]"}

  {"instruction": "Summarize this article: ...",
   "response": "The article discusses..."}

  Dataset size: 10K-100K examples (MUCH smaller than pre-training)
```

Critical insight: The model's knowledge doesn't really change during SFT — it already KNEW the answers from pre-training. SFT teaches it the **FORMAT** of being helpful: start with an answer, be concise, follow instructions, use the right tone. It's like a brilliant expert who finally learns presentation skills.

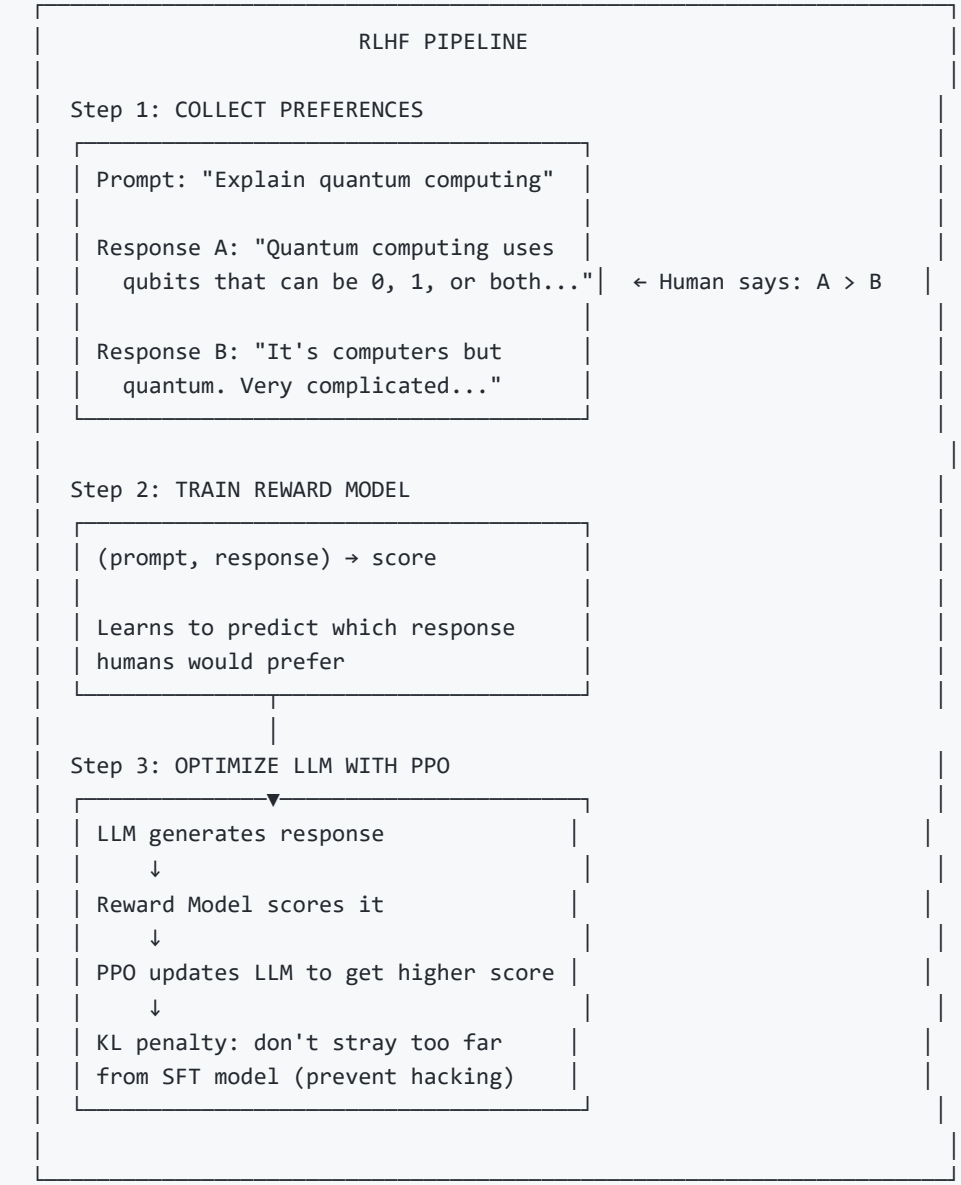
Phase 3: RLHF / DPO — Learning Human Values

Why SFT isn't enough: The model follows instructions now but may still be:

- **Harmful**: generates dangerous or offensive content
- **Unhelpful**: verbose, off-topic, hedging too much
- **Inconsistent**: great at some topics, terrible at others

We need to align the model with **human preferences** — not just "what is correct" but "what do humans actually want?"

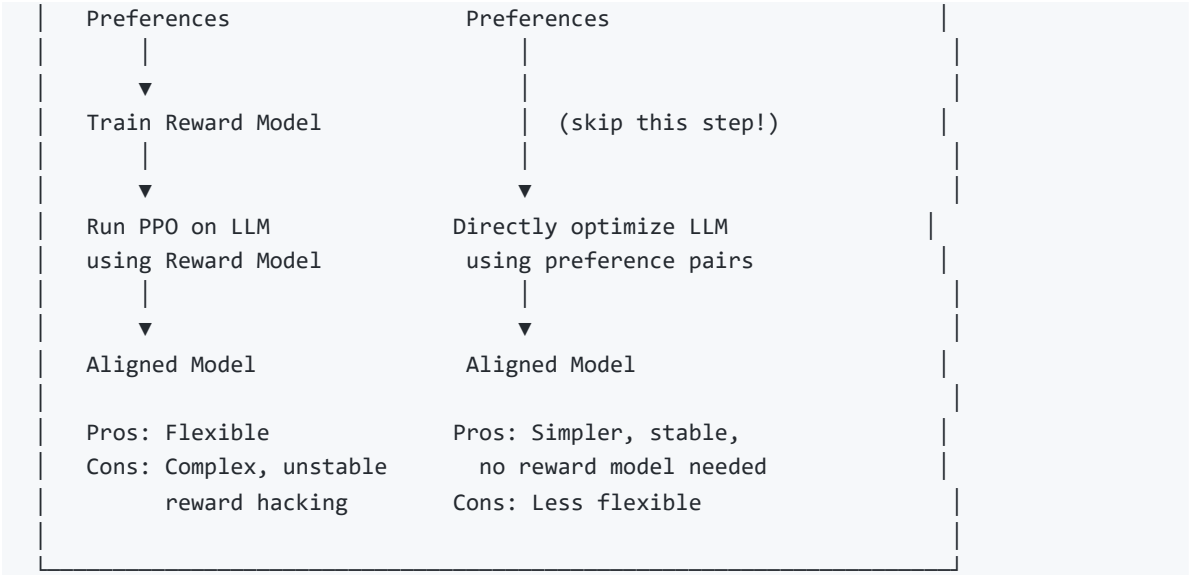
RLHF (Reinforcement Learning from Human Feedback)



KL divergence penalty: Without this, the model learns to "hack" the reward model — generating nonsensical outputs that happen to score high. The KL penalty says: "You can improve, but stay close to the SFT model."

DPO (Direct Preference Optimization) — The Simpler Alternative



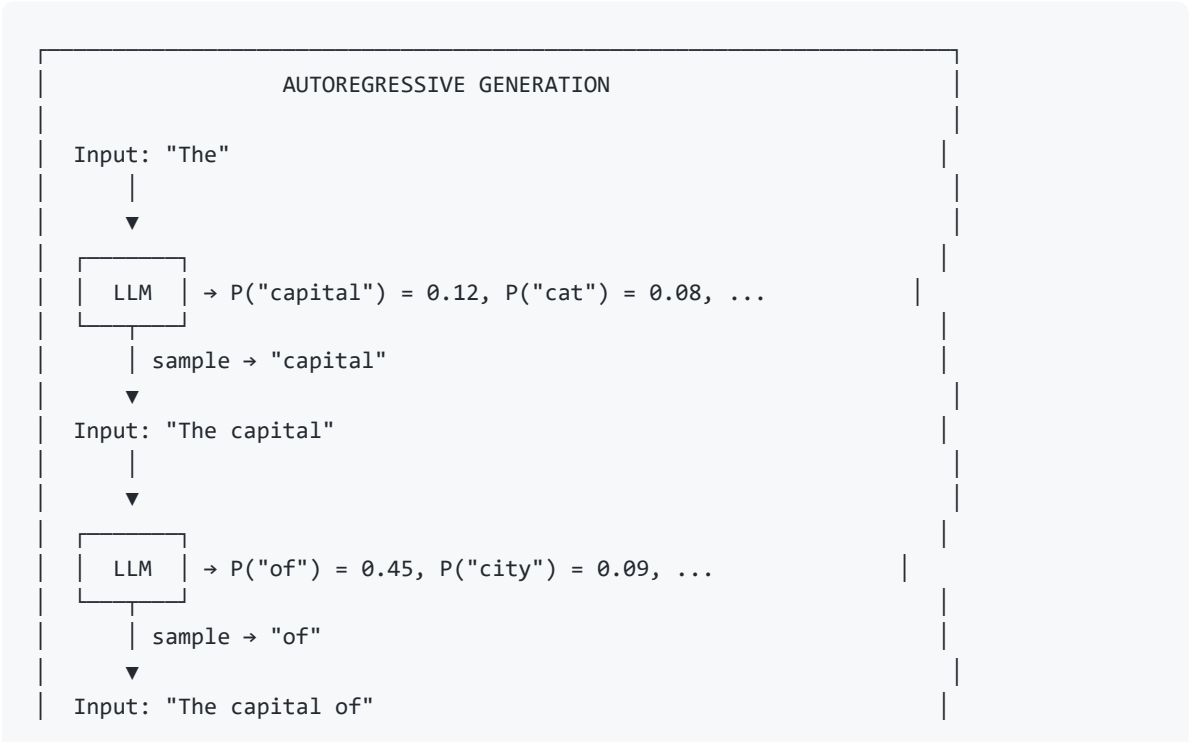


DPO skips the reward model entirely. It directly uses the preference pairs (chosen response vs. rejected response) to optimize the LLM. Simpler, more stable, increasingly popular. Used by Meta (LLaMA-3/4), Anthropic (Claude Sonnet/Opus), and OpenAI (GPT-5 series).

Inference — How Generation Actually Works

Autoregressive Generation

LLMs generate text **one token at a time**. Each generated token is fed back as input for the next step. This is called **autoregressive** generation.



```
│      │                                                                  │
│      ▼                                                                  │
│   ┌──────┐                                                              │
│   │ LLM  │ → P("France") = 0.31, P("the") = 0.15, ...                   │
│   └──────┘                                                              │
│      │ sample → "France"                                                │
│      ▼                                                                  │
│   Output so far: "The capital of France"                                │
│   ... continues until <EOS> token or max length ...                    │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

Sampling Strategies

When the model outputs a probability distribution, HOW do we pick the next token? This choice dramatically affects output quality and creativity.

Temperature

Temperature controls randomness. It scales the logits (raw scores) before applying softmax.

```
Distribution for next token after "The weather today is":

  T=0.1 (nearly deterministic)     T=1.0 (normal)      T=2.0 (very random)

  "sunny"  ██████████ 0.92         "sunny"  ████ 0.40   "sunny"  ██ 0.22
  "nice"   ▌ 0.05                  "nice"   ███ 0.25    "nice"   ██ 0.18
  "warm"   ▒ 0.02                  "warm"   ▌ 0.15      "warm"   ▌ 0.15
  "cold"   ▒ 0.01                  "cold"   ▌ 0.10      "cold"   ▌ 0.14
  "great"  ▒ 0.00                  "great"  ▌ 0.05      "great"  ▌ 0.11
  others   ▒ 0.00                  others   ▒ 0.05      others   ▌ 0.20

  ↑ Almost always "sunny"          ↑ Balanced            ↑ Anything goes
  Use for: factual Q&A             Use for: general      Use for: creative
                                                          writing, brainstorm
```

- **T → 0**: Always pick the highest probability token (greedy/deterministic)
- **T = 1**: Sample from the actual learned distribution
- **T > 1**: Flatten the distribution, making unlikely tokens more probable

Top-k Sampling

Only consider the **k highest-probability** tokens, then sample among them. Everything else gets probability 0.

```
Full distribution (10 tokens shown):
```

```
  "sunny"   0.40
  "nice"    0.25
  "warm"    0.15    ← Top-3 boundary
  ─────────────
  "cold"    0.10    ← zeroed out
  "great"   0.05    ← zeroed out
  "awful"   0.03    ← zeroed out
  ...

After top-k (k=3), renormalized:

  "sunny"   0.50   (0.40 / 0.80)
  "nice"    0.31   (0.25 / 0.80)
  "warm"    0.19   (0.15 / 0.80)
  others    0.00

k=1  → greedy decoding (always pick the best)
k=50 → moderate diversity (GPT-2 default)
```

Top-p (Nucleus Sampling)

Pick the **smallest set of tokens** whose cumulative probability ≥ p. The set size ADAPTS to the distribution.

```
Example 1: Confident distribution (p=0.9)

  "Paris"   0.85  ← cumulative: 0.85
  "Lyon"    0.06  ← cumulative: 0.91 ≥ 0.9 → STOP
  "London"  0.04  ← zeroed
  ...
  → Only 2 tokens considered

Example 2: Uncertain distribution (p=0.9)

  "the"     0.15  ← cumulative: 0.15
  "a"       0.12  ← cumulative: 0.27
  "its"     0.10  ← cumulative: 0.37
  "his"     0.09  ← cumulative: 0.46
  "their"   0.08  ← cumulative: 0.54
  "my"      0.07  ← cumulative: 0.61
  "her"     0.07  ← cumulative: 0.68
  "our"     0.06  ← cumulative: 0.74
  "your"    0.05  ← cumulative: 0.79
  "some"    0.05  ← cumulative: 0.84
  "this"    0.04  ← cumulative: 0.88
  "that"    0.03  ← cumulative: 0.91 ≥ 0.9 → STOP
  → 12 tokens considered!
```

Why top-p is generally preferred over top-k: Top-p ADAPTS. When the model is confident, it considers few tokens. When uncertain, it considers many. Top-k always considers exactly k

tokens regardless of how the probability is distributed — which can be too few (missing valid options) or too many (including garbage).

KV-Cache — Critical for Performance

Problem: At generation step N, the naive approach recomputes attention for ALL N tokens. That's $O(N^2)$ total work — quadratic!

```
Naive approach (no cache):

  Step 1: Compute Q, K, V for ["The"]                    → 1 token
  Step 2: Compute Q, K, V for ["The", "capital"]          → 2 tokens
  Step 3: Compute Q, K, V for ["The", "capital", "of"]    → 3 tokens
  Step 4: Compute Q, K, V for all 4 tokens                → 4 tokens
  ...
  Step N: Compute Q, K, V for all N tokens                → N tokens

  Total work: 1 + 2 + 3 + ... + N = N(N+1)/2 = O(N²)  ← BAD!
```

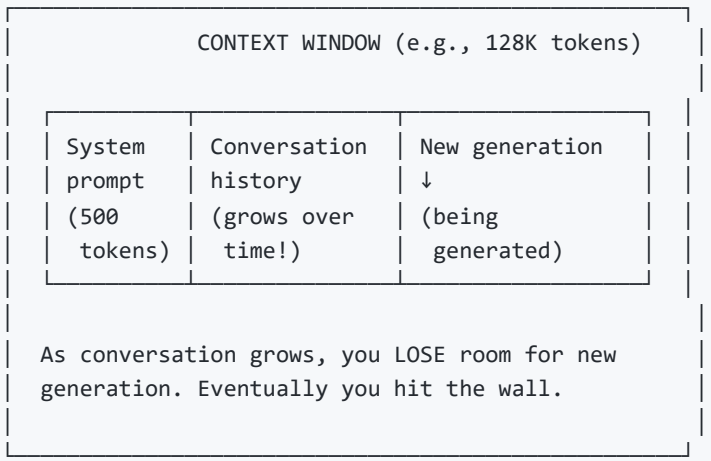
Solution: Cache the K and V vectors from previous steps. Only compute Q for the NEW token.

```
┌──────────────────────────────────────────────────────────────────┐
│                        KV-CACHE                                  │
│                                                                  │
│  Step 1: Compute K₁, V₁ for "The"        → cache [K₁, V₁]        │
│  Step 2: Compute K₂, V₂ for "capital"    → cache [K₁K₂, V₁V₂]    │
│  Step 3: Compute K₃, V₃ for "of"         → cache [K₁K₂K₃,V₁V₂V₃]  │
│  Step 4: Compute ONLY Q₄ for "France"                            │
│          Attend Q₄ against cached [K₁K₂K₃K₄, V₁V₂V₃V₄]           │
│                                                                  │
│  At each step: O(1) new computation (just the new token)         │
│  Total work:   O(N) instead of O(N²)  ← MUCH better!             │
│                                                                  │
│  ┌─────┬─────┬─────┬─────┬─────┐                                  │
│  │ K₁  │ K₂  │ K₃  │ K₄  │ ... │  ← KV-cache grows linearly     │
│  │ V₁  │ V₂  │ V₃  │ V₄  │ ... │     with sequence length       │
│  └─────┴─────┴─────┴─────┴─────┘                                  │
│                                                                  │
│  Trade-off: Faster generation but uses lots of GPU MEMORY        │
│  This is why context window limits exist!                        │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Memory cost: For a 70B+ parameter model with 128K context, the KV-cache alone can use **tens of gigabytes** of GPU memory. This is the primary bottleneck for serving LLMs. Frontier models like GPT-5.2 and Claude Opus 4.6 use techniques like GQA (Grouped-Query Attention), sliding window attention, and quantized KV-cache to manage this.

Context Window and Agents

The **context window** is the maximum number of tokens the model can "see" at once — prompt + generated output combined.



Why this matters for agents:

- Agent conversations can be LONG (tool calls, observations, reasoning steps)
- Each tool call + result eats into the context window
- Need strategies to manage context: summarization, sliding window, RAG (preview of Day 5)
- This is why context window management is a **core agent engineering skill**

Practice Exercises

Exercise 1: Tokenization Explorer

```
"""
Exercise 1: Tokenization Explorer
Explore how different tokenizers handle various inputs.
Find surprising edge cases in tokenization.

Requirements: pip install tiktoken
No API key needed.
"""

import tiktoken


def explore_tokenization():
    """Compare tokenization across different model encodings."""
```

```
# Load tokenizers for different models
enc_gpt4 = tiktoken.encoding_for_model("gpt-4o")
enc_gpt35 = tiktoken.encoding_for_model("gpt-4")

print("=" * 60)
print("TOKENIZATION EXPLORER")
print("=" * 60)

# --- Test 1: Basic text ---
test_texts = [
    "Hello, world!",
    "The capital of France is Paris.",
    "def fibonacci(n): return n if n <= 1 else fibonacci(n-1) + fibonacci(n-2)",
    "    " * 10,  # lots of whitespace
    "🎉🎊🎈🎁",  # emojis
    "こんにちは世界",  # Japanese
    "Привет мир",  # Russian
    "2 + 2 = 4",
    "HTTP/1.1 200 OK",
    "aaaaaaaaaa",  # repeated characters
    " a  a   a    a",  # varying whitespace
    "don't won't can't",  # contractions
]

print("\n--- Token counts (GPT-4o / o200k encoding) ---")
print(f"{'Text':<50} {'Tokens':<8} {'Chars/Token':<12}")
print("-" * 70)

for text in test_texts:
    tokens = enc_gpt4.encode(text)
    ratio = len(text) / len(tokens) if tokens else 0
    display = text[:47] + "..." if len(text) > 50 else text
    print(f"{display:<50} {len(tokens):<8} {ratio:<12.2f}")

# --- Test 2: See actual tokens ---
print("\n--- Token breakdown for selected texts ---")
interesting = [
    "Hello, world!",
    "    " * 4,
    "don't won't can't",
    "def fibonacci(n):",
]

for text in interesting:
    tokens = enc_gpt4.encode(text)
    decoded = [enc_gpt4.decode([t]) for t in tokens]
    print(f"\nText: {repr(text)}")
    print(f"  Tokens ({len(tokens)}): {decoded}")
    print(f"  Token IDs: {tokens}")

# --- Test 3: Find edge cases ---
print("\n--- Surprising edge cases ---")
edge_cases = {
    "Numbers": ("42", "420", "4200", "42000", "4200000"),
    "Spacing": (" hello", "  hello", "   hello", "    hello"),
    "Similar words": ("running", "run", "ran", "runner", "runs"),
```

```
        "Programming": ("print()", 'print("hello")', "System.out.println()"),
    }

    for category, texts in edge_cases.items():
        print(f"\n{category}:")
        for text in texts:
            tokens = enc_gpt4.encode(text)
            decoded = [enc_gpt4.decode([t]) for t in tokens]
            print(f"  {text:<30} → {len(tokens)} tokens: {decoded}")

    # --- Test 4: Vocabulary size ---
    print(f"\n--- Vocabulary info ---")
    print(f"GPT-4o / GPT-5 vocabulary size: {enc_gpt4.n_vocab:,} tokens")

    # --- Test 5: Cost estimation ---
    print("\n--- Cost estimation ---")
    sample_texts = {
        "A tweet (280 chars)": "x" * 280,
        "A short email (500 words)": " ".join(["word"] * 500),
        "A page of code (50 lines)": "\n".join(
            ["    result = process(item)  # do something"] * 50
        ),
        "A novel chapter (~5000 words)": " ".join(["word"] * 5000),
    }

    for desc, text in sample_texts.items():
        tokens = enc_gpt4.encode(text)
        cost_in = len(tokens) * 0.000001  # rough GPT-5.1 input pricing ($1/M tokens
        print(f"  {desc:<35} → {len(tokens):>6} tokens (~${cost_in:.4f} GPT-5.1 inpu


if __name__ == "__main__":
    explore_tokenization()
```

Exercise 2: Sampling Strategy Visualizer

```
"""
Exercise 2: Sampling Strategy Visualizer
Visualize how temperature, top-k, and top-p change a probability distribution.
Uses ASCII art — no matplotlib needed, no API key needed.

Requirements: None (pure Python)
"""

import math
import random


def softmax(logits, temperature=1.0):
    """Apply softmax with temperature scaling."""
    if temperature == 0:
```

```
        # Greedy: all probability on the max
        result = [0.0] * len(logits)
        result[logits.index(max(logits))] = 1.0
        return result
    scaled = [x / temperature for x in logits]
    max_val = max(scaled)
    exps = [math.exp(x - max_val) for x in scaled]
    total = sum(exps)
    return [e / total for e in exps]


def apply_top_k(probs, k):
    """Zero out everything except the top-k probabilities, then renormalize."""
    indexed = sorted(enumerate(probs), key=lambda x: -x[1])
    result = [0.0] * len(probs)
    total = 0.0
    for i, (idx, p) in enumerate(indexed):
        if i < k:
            result[idx] = p
            total += p
    if total > 0:
        result = [p / total for p in result]
    return result


def apply_top_p(probs, p_threshold):
    """Keep smallest set of tokens with cumulative probability >= p."""
    indexed = sorted(enumerate(probs), key=lambda x: -x[1])
    result = [0.0] * len(probs)
    cumulative = 0.0
    for idx, p in indexed:
        result[idx] = p
        cumulative += p
        if cumulative >= p_threshold:
            break
    total = sum(result)
    if total > 0:
        result = [p / total for p in result]
    return result


def ascii_bar(prob, max_width=30):
    """Create an ASCII bar for a probability."""
    filled = int(prob * max_width)
    return "" * filled + "" * (max_width - filled)


def display_distribution(probs, labels, title=""):
    """Display a probability distribution as ASCII bars."""
    print(f"\n  {title}")
    print(f"  {'Token':<12} {'Prob':>6}  {'Distribution':<30}")
    print(f"  {'-'*12} {'-'*6}  {'-'*30}")
    for label, prob in zip(labels, probs):
        if prob > 0.001:
            bar = ascii_bar(prob)
            print(f"  {label:<12} {prob:>5.1%}  {bar}")
```

```
    tokens_shown = sum(1 for p in probs if p > 0.001)
    tokens_hidden = sum(1 for p in probs if 0 < p <= 0.001)
    if tokens_hidden > 0:
        print(f"  ... and {tokens_hidden} tokens with < 0.1% each")


def main():
    print("=" * 65)
    print("  SAMPLING STRATEGY VISUALIZER")
    print("=" * 65)

    # Simulate logits for next token after "The weather today is"
    tokens = [
        "sunny", "nice", "warm", "cold", "great",
        "terrible", "rainy", "hot", "perfect", "awful",
        "beautiful", "cloudy", "windy", "mild", "freezing",
    ]
    # Raw logits (before softmax) — "sunny" is the most likely
    logits = [
        4.5, 3.2, 2.8, 2.1, 1.9,
        1.5, 1.3, 1.1, 0.9, 0.5,
        0.3, 0.1, -0.2, -0.5, -1.0,
    ]

    # --- Demo 1: Temperature ---
    print("\n" + "—" * 65)
    print("  TEMPERATURE EFFECT")
    print("  Context: 'The weather today is ___'")
    print("—" * 65)

    for temp in [0.1, 0.5, 1.0, 2.0]:
        probs = softmax(logits, temperature=temp)
        label = {
            0.1: "T=0.1 (nearly deterministic)",
            0.5: "T=0.5 (focused)",
            1.0: "T=1.0 (normal)",
            2.0: "T=2.0 (creative/random)",
        }[temp]
        display_distribution(probs, tokens, label)

    # --- Demo 2: Top-k ---
    print("\n" + "—" * 65)
    print("  TOP-K SAMPLING (at T=1.0)")
    print("—" * 65)

    base_probs = softmax(logits, temperature=1.0)
    for k in [1, 3, 5, 10]:
        filtered = apply_top_k(base_probs, k)
        active = sum(1 for p in filtered if p > 0)
        display_distribution(
            filtered, tokens, f"Top-k (k={k}) — {active} tokens active"
        )

    # --- Demo 3: Top-p ---
    print("\n" + "—" * 65)
    print("  TOP-P (NUCLEUS) SAMPLING (at T=1.0)")
```

```
print("−" * 65)

for p in [0.5, 0.7, 0.9, 0.95]:
    filtered = apply_top_p(base_probs, p)
    active = sum(1 for pr in filtered if pr > 0)
    display_distribution(
        filtered, tokens, f"Top-p (p={p}) — {active} tokens in nucleus"
    )

# --- Demo 4: Simulate generation ---
print("\n" + "−" * 65)
print("  SIMULATED GENERATION (10 samples each)")
print("−" * 65)

configs = [
    ("Greedy (T=0)",       {"temperature": 0.0}),
    ("Low temp (T=0.3)",   {"temperature": 0.3}),
    ("Normal (T=1.0)",     {"temperature": 1.0}),
    ("Creative (T=1.5)",   {"temperature": 1.5}),
    ("Top-k=3, T=1.0",    {"temperature": 1.0, "top_k": 3}),
    ("Top-p=0.9, T=1.0",  {"temperature": 1.0, "top_p": 0.9}),
]

for name, config in configs:
    samples = []
    for _ in range(10):
        temp = config.get("temperature", 1.0)
        probs = softmax(logits, temperature=temp) if temp > 0 else softmax(logit
        if "top_k" in config:
            probs = apply_top_k(probs, config["top_k"])
        if "top_p" in config:
            probs = apply_top_p(probs, config["top_p"])

        # Sample from distribution
        r = random.random()
        cumulative = 0.0
        chosen = tokens[-1]
        for token, prob in zip(tokens, probs):
            cumulative += prob
            if r <= cumulative:
                chosen = token
                break
        samples.append(chosen)

    unique = len(set(samples))
    print(f"\n  {name}:")
    print(f"    Samples: {', '.join(samples)}")
    print(f"    Unique tokens: {unique}/10")

print("\n" + "=" * 65)
print("  KEY TAKEAWAYS:")
print("  • Temperature controls the 'sharpness' of the distribution")
print("  • Top-k fixes the number of candidates (rigid)")
print("  • Top-p adapts the candidates to the distribution (flexible)")
print("  • For factual tasks: low temperature (0.0-0.3)")
print("  • For creative tasks: higher temperature + top-p (0.7-1.0)")
```

```
    print("=" * 65)


if __name__ == "__main__":
    main()
```

Exercise 3: Token Probability Analyzer (Optional — Needs API Key)

```
"""
Exercise 3: Token Probability Analyzer
Use OpenAI API to see where the model is confident vs uncertain.

Requirements: pip install openai
Needs: OPENAI_API_KEY environment variable
"""

import os

try:
    from openai import OpenAI
except ImportError:
    print("Run: pip install openai")
    exit(1)


def analyze_logprobs(prompt: str, model: str = "gpt-5.1-codex-mini"):
    """Generate text and analyze per-token probabilities."""
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        print("Set OPENAI_API_KEY environment variable first.")
        print("Export: export OPENAI_API_KEY='sk-...'")
        return

    client = OpenAI(api_key=api_key)

    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=100,
        logprobs=True,
        top_logprobs=5,
        temperature=1.0,
    )

    choice = response.choices[0]
    content = choice.message.content
    logprobs_data = choice.logprobs.content

    print(f"\nPrompt: {prompt}")
    print(f"Response: {content}")
    print(f"\n{'Token':<20} {'Prob':>8} {'Confidence':<15} {'Alternatives'}")
```

```
    print("-" * 80)

    import math

    for token_info in logprobs_data:
        token = token_info.token
        logprob = token_info.logprob
        prob = math.exp(logprob)

        # Confidence indicator
        if prob > 0.9:
            confidence = "████████ SURE"
        elif prob > 0.7:
            confidence = "██████▒▒ high"
        elif prob > 0.4:
            confidence = "████▒▒▒▒ med"
        elif prob > 0.1:
            confidence = "██▒▒▒▒▒▒ low"
        else:
            confidence = "▒▒▒▒▒▒▒▒ guess"

        # Top alternatives
        alts = []
        for alt in token_info.top_logprobs[:3]:
            alt_prob = math.exp(alt.logprob)
            if alt.token != token:
                alts.append(f"{repr(alt.token)}:{alt_prob:.1%}")

        alt_str = ", ".join(alts[:3]) if alts else "(no alternatives)"
        print(f"{repr(token):<20} {prob:>7.1%} {confidence:<15} {alt_str}")


def main():
    print("=" * 60)
    print("TOKEN PROBABILITY ANALYZER")
    print("=" * 60)

    prompts = [
        "The capital of France is",
        "Write a haiku about programming",
        "What is 17 * 23?",
        "The meaning of life is",
    ]

    for prompt in prompts:
        print("\n" + "=" * 60)
        analyze_logprobs(prompt)


if __name__ == "__main__":
    main()
```

✓ Day 1 Deliverable Checklist

```
- [ ] Explored tokenization with tiktoken (found 3+ surprising edge cases)
- [ ] Can draw the full Transformer architecture from memory
- [ ] Built a sampling strategy visualizer
- [ ] Can explain: What is a token and why isn't it always a word?
- [ ] Can explain: What are Q, K, V and why do we need all three?
- [ ] Can explain: What is the difference between pre-training, SFT, and RLHF?
- [ ] Can explain: How does temperature affect generation?
- [ ] Can explain: Why does the KV-cache exist?
- [ ] Can explain: What is the feed-forward network's role vs attention's role?
- [ ] Can explain: Why do modern LLMs use decoder-only architecture?
```

▶ 📝 **Answer Key (click to expand)**

Day 1 Appendix — Advanced Techniques & Tools

> 🎯 **Goal**: Deep-dive into the training tricks, architecture innovations, and inference optimizations that make modern LLMs work at scale.

> ⏱️ **Time**: Optional deep-dive — 2–3h (reference material, revisit as needed)

> 📦 **Deliverable**: A mental model of how every major LLM optimization works, why it exists, and when to use it

---

Table of Contents

1. Training Essentials
   - Cross-Entropy Loss
   - Adam / AdamW Optimizer
   - Learning Rate Schedule
   - Mixed-Precision Training (BF16/FP16)
2. Modern Architecture Tricks
   - Mixture of Experts (MoE)
   - Grouped-Query Attention (GQA)
   - Flash Attention
   - RMSNorm vs LayerNorm

3. Fine-Tuning Methods
    - LoRA (Low-Rank Adaptation)
    - QLoRA
    - Decision Table: When to Use What
4. Inference Speedups
    - Quantization (INT8 / INT4)
    - Speculative Decoding
    - PagedAttention / vLLM
    - KV-Cache Compression
5. Evaluation & Scaling
    - Perplexity
    - Key Benchmarks
    - Chinchilla Scaling Law
    - Inference-Time Compute Scaling

Training Essentials

Cross-Entropy Loss

Intuition: "How surprised is the model by the correct answer?"

When a model sees the sentence "The cat sat on the ___", it produces a probability for every word in its vocabulary. Cross-entropy loss measures how much probability it assigned to the **right** answer. If the model is confident and correct → low loss. If the model is confident and **wrong** → very high loss.

```
The formula:

  L = -log(P(correct_token))

Breaking it down:

  P(correct_token)  →  The probability the model assigned to the right answer.
                       This is a number between 0 and 1.

  -log(...)         →  Converts probability into "surprise."
                       High probability → low surprise → low loss.
                       Low probability  → high surprise → high loss.

  The minus sign (-) →  log of a number < 1 is negative, so we negate
                       to get a positive loss value.
```

Example — seeing the numbers:

```
Suppose the correct next token is "mat":

  Model A (good):    P("mat") = 0.95  →  L = -log(0.95) = 0.05   ✓ Low loss!
  Model B (unsure):  P("mat") = 0.30  →  L = -log(0.30) = 1.20   ⚠ Medium loss
  Model C (wrong):   P("mat") = 0.01  →  L = -log(0.01) = 4.60   ✗ Terrible!

Notice: the loss doesn't just go up linearly — it EXPLODES when the model
is confidently wrong. That's exactly what we want. A model that says
"I'm 99% sure it's 'banana'" when the answer is "mat" gets punished HARD.
```

```
Probability distribution over vocabulary:

  P  │
  0.6│       ██
  0.5│       ██
  0.4│       ██
  0.3│       ██  ██
  0.2│    ██ ██  ██
  0.1│ ██ ██ ██  ██ ██
  0.0└──┴──┴──┴──┴──┴──┴──
     "the" "a" "mat" "cat" "dog" ...
               ▲
               │
          correct token
          (we want THIS bar to be tall)
```

Why cross-entropy and not just "% correct"? Two reasons:

1. **Differentiable** — "% correct" is either 0 or 1, so gradients can't flow through it. Cross-entropy gives smooth gradients for optimization.
2. **Penalizes confident mistakes** — A model that puts 50% on the right answer and 50% on one wrong answer gets a much lower loss than a model that puts 1% on the right answer and 99% on a wrong answer. "% correct" would score both as "correct" with the same 1.

Adam / AdamW Optimizer

Analogy: Imagine a ball rolling downhill trying to find the lowest valley (the best model weights).

- **Plain SGD** = a ball on ice. It slides around wildly, overshoots valleys, gets stuck oscillating.
- **SGD + Momentum** = a ball with some weight. It builds up speed in consistent directions and pushes through small bumps.
- **Adam** = a ball with friction AND adaptive wheel size. It moves faster on gentle, consistent slopes and automatically slows down on steep, noisy terrain.

What Adam actually does — two key ideas:

```
1. MOMENTUM (m) — "Where have I been heading?"
   Keeps a running average of recent gradients.
   If gradients consistently point left, momentum builds up → bigger step left.
   If gradients flip-flop randomly, momentum stays small → cautious steps.

2. ADAPTIVE LEARNING RATE (v) — "How bumpy is this terrain?"
   Keeps a running average of squared gradients.
   For parameters with large, volatile gradients → v is big → step shrinks.
   For parameters with small, consistent gradients → v is small → step grows.
```

The update formula (explained):

```
  θ = θ − lr × m̂ / (√v̂ + ε)

  θ        →  The model parameter we're updating (one weight)
  lr       →  Learning rate — the base step size (typically 1e-4 to 3e-4 for LLMs)
  m̂        →  Smoothed gradient direction (momentum term, bias-corrected)
  √v̂       →  Smoothed gradient magnitude (how bumpy this parameter's landscape is)
  ε        →  Tiny number (~1e-8) to prevent division by zero

  In plain English: "Take a step proportional to the smoothed gradient,
  but shrink it if this parameter has been volatile."
```

AdamW — the modern variant:

```
Standard Adam:   θ = θ − lr × (m̂/(√v̂+ε) + λ×θ)    ← weight decay mixed in
AdamW:           θ = θ − lr × m̂/(√v̂+ε) − lr×λ×θ    ← weight decay SEPARATED

Why it matters: In standard Adam, the adaptive learning rate accidentally
scales the weight decay too. AdamW fixes this by decoupling them.

Result: better generalization, more predictable regularization.
AdamW is the default optimizer for virtually all modern LLM training
(GPT-5, Claude Opus 4.6, LLaMA 4, Gemini 3).
```

Key hyperparameters:

- `lr`: 1e-4 to 3e-4 (typical for LLM pre-training)
- β_1: 0.9 (momentum decay — how much to remember past gradients)
- β_2: 0.999 (adaptive LR decay — how much to remember past magnitudes)
- `weight_decay (λ)`: 0.01 to 0.1 (prevents weights from growing too large)

Learning Rate Schedule

Analogy: Warming up a car engine on a cold morning.

1. **Warmup** — Start the engine gently. If you floor the gas immediately (high LR from step 0), the engine stalls (random initial weights + high LR = chaotic updates the model never recovers from).
2. **Peak** — Cruise at full speed. The model has found the general landscape and can take confident steps.
3. **Decay** — Ease into the destination. As you approach a good solution, slow down so you don't overshoot it.

```
Learning Rate over Training Steps:

    LR
    |       /‾‾‾‾‾\
    |      /       \
    |     /         \
    |    /           \
    |   /             \  cosine decay
    |  /               \
    | /                 \
    └──┬────┬────────────┬──→ steps
       0  warmup        end
          phase

    Phase 1: Linear warmup     (0 → peak LR over ~1-5% of training)
    Phase 2: Cosine decay       (peak LR → ~10% of peak LR)
```

Why warmup is critical:

```
Without warmup:
  Step 0: Weights are random. Gradients are HUGE and noisy.
  High LR × huge gradient = MASSIVE weight update.
  The model jumps to a terrible region of weight space.
  It never recovers → training diverges (loss goes to infinity).

With warmup:
  Step 0: LR is tiny (e.g., 1e-7). Even with noisy gradients,
  the updates are small. The model gently finds its footing.
  Gradually ramp up to full LR (e.g., 3e-4) by step 2000.
  By then, gradients are stable → full speed ahead.
```

Typical values for LLM training:

- Warmup: 1–5% of total training steps (e.g., 2000 steps out of 100K)
- Peak LR: 3e-4 (common for models up to ~7B)
- Final LR: 10% of peak (3e-5) — some trainers go to 0
- Schedule: Cosine decay is the most popular (smooth, no sudden jumps)

Mixed-Precision Training (BF16/FP16)

Analogy: Using pencil sketches (BF16) for draft work and ink (FP32) for the final drawing. Most of the computational work doesn't need full precision — you can sketch faster and still get the same picture.

What the precision formats look like:

```
┌──────────────────────────────────────────────────────────────────┐
│                    NUMBER FORMATS COMPARED                       │
│                                                                  │
│  FP32 (32 bits):  [1 sign][8 exponent][23 mantissa]              │
│     → Full precision. The "source of truth."                     │
│     → Range: ±3.4 × 10³⁸, ~7 decimal digits precision            │
│                                                                  │
│  FP16 (16 bits):  [1 sign][5 exponent][10 mantissa]              │
│     → Half the bits → half the memory, ~2× faster matmuls        │
│     → Range: ±65504 only! Can overflow during training.          │
│     → Needs "loss scaling" to avoid underflow in gradients.      │
│                                                                  │
│  BF16 (16 bits):  [1 sign][8 exponent][7 mantissa]               │
│     → Same exponent range as FP32 (no overflow!)                 │
│     → Less decimal precision, but that's fine for neural nets.   │
│     → No loss scaling needed. Simpler, more stable.              │
│     → Preferred by: GPT-5, Claude Opus 4.6, LLaMA 4, Gemini 3    │
└──────────────────────────────────────────────────────────────────┘
```

What runs in which precision:

```
┌──────────────────────────────────┬──────────────┐
│ Operation                        │ Precision    │
├──────────────────────────────────┼──────────────┤
│ Master copy of weights           │ FP32         │
│ Forward pass (matrix multiplies) │ BF16         │
│ Backward pass (gradient compute) │ BF16         │
│ Weight update (optimizer step)   │ FP32         │
│ Loss computation                 │ FP32         │
└──────────────────────────────────┴──────────────┘

Why keep master weights in FP32:
  Small gradient updates (e.g., 0.00001) would vanish in BF16
  (it only has ~3 decimal digits of precision).
  FP32 accumulates these tiny updates correctly over time.
```

Performance gains:

- **Speed**: ~2× faster matrix multiplications on modern GPUs (A100, H100)
- **Memory**: ~50% reduction in activation memory

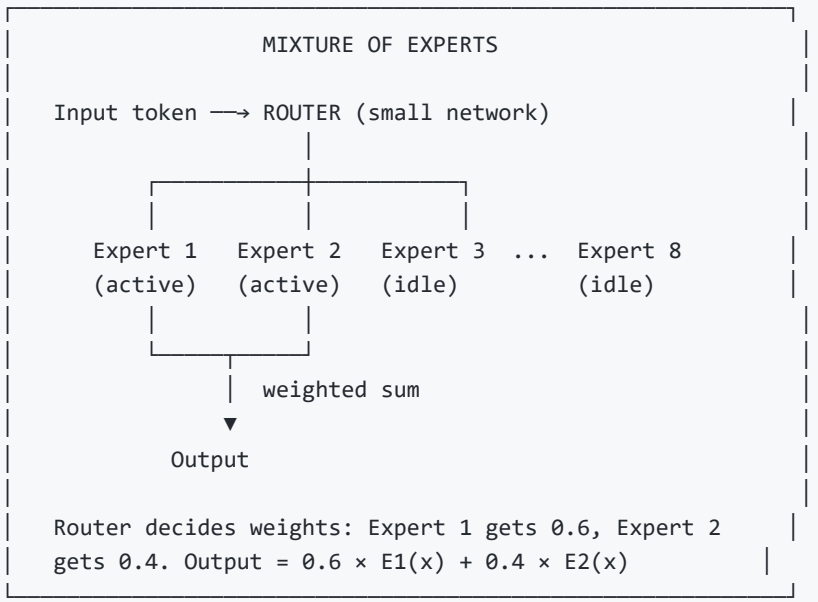
- **BF16 vs FP16**: BF16 is strictly preferred on modern hardware. Same speed, no overflow risk, no loss scaling hassle.

Modern Architecture Tricks

Mixture of Experts (MoE)

Analogy: Imagine a large company with 8 specialist departments — tax, legal, engineering, marketing, HR, sales, R&D, and support. When a request comes in, the **receptionist** (router) reads it and sends it to only the **2 most relevant departments**. The other 6 departments don't even see it.

Why it matters: A model with 8 experts has 8× the total parameters (knowledge capacity) but only activates 2 experts per token (compute cost). You get the **knowledge** of a massive model with the **speed** of a much smaller one.



Real-world MoE models:

Model	Total Params	Active/Token	Experts
GPT-4 (rumored)	~1.8T	~220B	8 experts
GPT-5 / 5.1 / 5.2	Likely MoE	Likely MoE	Undisclosed
Mixtral 8×7B	47B	~13B	8 experts
LLaMA 4 Maverick	~400B	Subset	17 experts
DeepSeek-V2	236B	~21B	160 experts

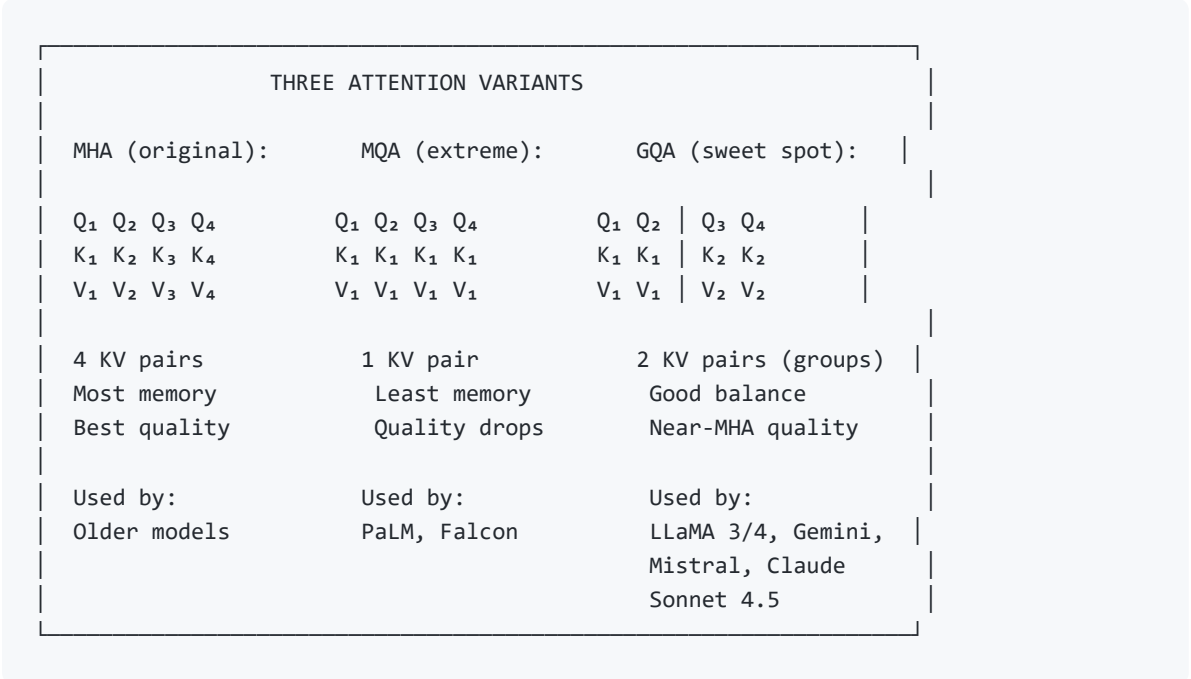
Key insight: MoE models are **big in storage** but **fast in inference**. A 47B-parameter Mixtral model runs at roughly the speed of a 13B dense model — because only 13B parameters are active per token.

The router balancing problem: If the router keeps sending all tokens to the same 2 experts, the other 6 learn nothing. Training adds an **auxiliary load-balancing loss** that penalizes uneven expert usage, ensuring all experts develop useful specializations.

Grouped-Query Attention (GQA)

The problem: In standard Multi-Head Attention (MHA), every head has its own Query (Q), Key (K), and Value (V) projections. During inference, the K and V values for every past token are stored in the **KV-cache** so they don't need to be recomputed. With 64 attention heads, that means 64 separate K and 64 separate V matrices per token — consuming enormous GPU memory.

The solution: Share K and V across **groups** of query heads. Instead of 64 unique KV pairs, you might have only 8. Each group of 8 query heads shares one K and one V.



Memory savings: With 64 query heads and 8 KV groups (GQA-8), you reduce the KV-cache by **8×**. For a 70B model serving thousands of concurrent users, this is the difference between needing 8 GPUs and needing 2.

When to use:

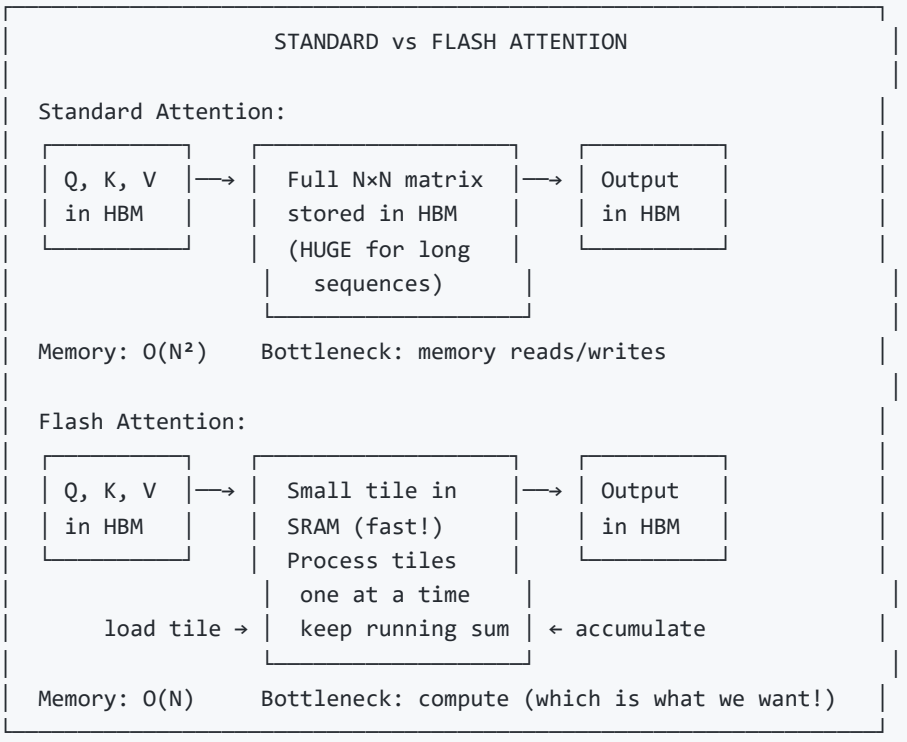
- MHA: Research, small models, when quality is the only priority
- MQA: Extreme latency requirements (e.g., real-time gaming NPCs)

- GQA: Production LLMs (the standard for virtually all modern models)

Flash Attention

The problem: Standard attention computes the full N×N attention matrix and stores it in GPU HBM (High-Bandwidth Memory). For a sequence of 100K tokens, that matrix has 10 **billion** entries. The GPU spends most of its time moving data between memory levels — not actually computing.

Analogy: Imagine processing a massive spreadsheet. The standard way: load the ENTIRE spreadsheet into RAM, compute every cell, then write results back. Flash Attention: process small tiles of the spreadsheet one at a time, keeping running totals in fast on-chip SRAM. Same exact answer, but you never load the whole spreadsheet at once.



Key insight: Flash Attention is **NOT an approximation**. It computes the **exact same result** as standard attention, just with smarter memory access patterns. The math is identical — it's purely an engineering optimization.

Why it matters:

- **2–4× faster** attention computation
- **O(N) memory** instead of O(N^2) — enables context windows of 100K+ tokens
- Used in: virtually every modern LLM training and inference stack
- Flash Attention 2/3: further optimizations for even better hardware utilization

RMSNorm vs LayerNorm

Both are normalization techniques that stabilize training by keeping activations in a reasonable range. Think of them as "volume control" for neural network signals.

LayerNorm (original, used in GPT-2/3, BERT):

```
norm(x) = γ × (x − μ) / σ + β

Step 1: Subtract the mean (μ) — center the values around zero
Step 2: Divide by standard deviation (σ) — normalize the spread
Step 3: Scale by γ (learned) and shift by β (learned) — let the
        model decide the final range
```

RMSNorm (modern, used in LLaMA 3/4, Mistral, GPT-NeoX, Gemini 3):

```
norm(x) = γ × x / RMS(x)

Where RMS(x) = √(mean(x²))

Key difference: SKIPS the mean subtraction step entirely.
Just divides by the root-mean-square of the values.
No β (shift) parameter either — simpler.
```

Why RMSNorm wins:

- **~10–15% faster**: Skipping mean computation and the shift parameter saves time per layer, and transformers have many layers.
- **Works equally well in practice**: Empirically, the mean subtraction doesn't help much for transformer language models.
- **Simpler**: Fewer parameters, fewer operations, easier to implement efficiently on hardware.

When to use: If you're training a new model, use RMSNorm. LayerNorm is only relevant for understanding older architectures.

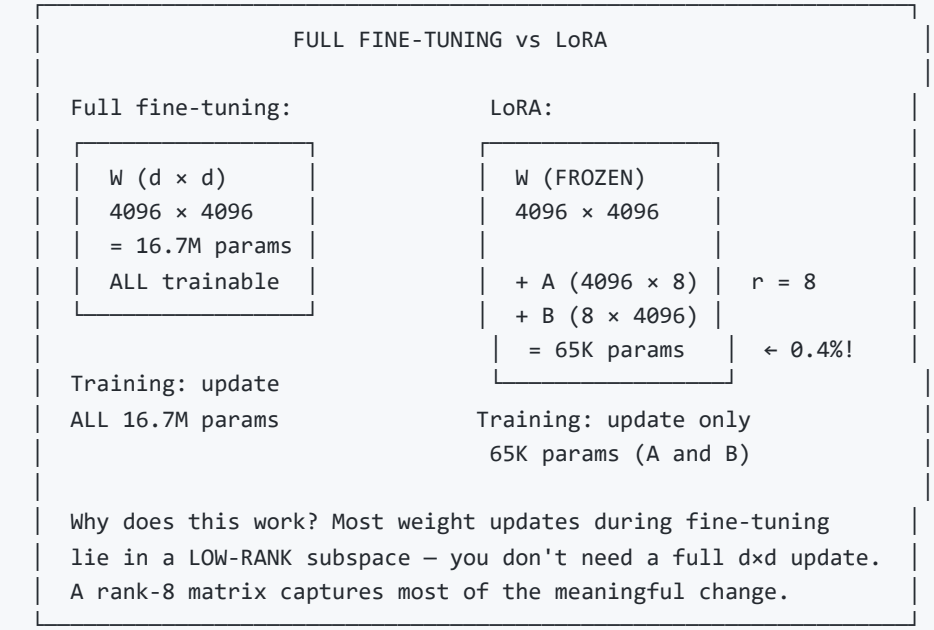
Fine-Tuning Methods

LoRA (Low-Rank Adaptation)

Analogy: Instead of rewriting an entire textbook for a new course (full fine-tuning, updating all weights), you add **sticky notes** with corrections and additions to specific pages. The original

textbook stays completely unchanged. When you read a page, you see the original content + your sticky notes combined.

What it does: Freezes the original weight matrix W (the "textbook"). Adds a tiny trainable decomposition on top: W' = W + A × B, where A and B are much smaller matrices.



The math (simplified):

```
Original:   output = W × input         (frozen, not updated)
LoRA adds:  output = W × input + A × B × input

W is (d × d)  = 4096 × 4096 = 16,777,216 params (FROZEN)
A is (d × r)  = 4096 × 8    =      32,768 params (trainable)
B is (r × d)  = 8 × 4096    =      32,768 params (trainable)

Total trainable: 65,536 params = 0.39% of W

r = "rank" — controls capacity. Think of it as the number of
"sticky note dimensions." Higher rank = more capacity = more params.
```

Typical configuration:

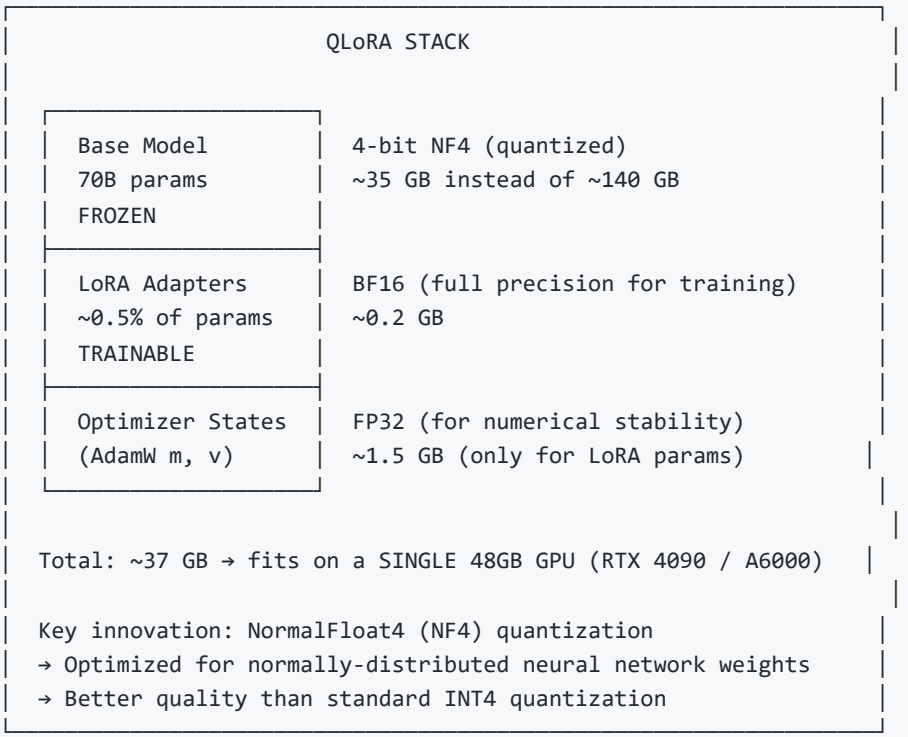
- **Rank**: r = 8 to r = 64. Start with r = 16, increase if quality is insufficient.
- **Which layers**: Usually attention projection matrices (Q, V). For more capacity, apply to all linear layers (Q, K, V, O, and FFN).
- **Alpha**: A scaling factor (usually α = 2×r). Controls how much the LoRA update influences the output.

When to use LoRA: Style changes, format adaptation, moderate domain shift. NOT for teaching the model fundamentally new knowledge (use full fine-tuning for that).

QLoRA

The problem: Even with LoRA, you still need to load the **full base model** in memory for the forward pass. A 70B parameter model in FP16 requires ~140 GB of GPU memory — that's 2× A100 80GB GPUs just to load the weights.

The solution: Quantize the base model to **4-bit** (NF4) for storage and forward pass. Keep the LoRA adapter weights in BF16 for training precision.



When to use QLoRA: When you want LoRA fine-tuning but don't have access to multi-GPU setups. Quality is slightly below standard LoRA but the accessibility trade-off is excellent.

Decision Table: When to Use What

	Full Fine-Tune	LoRA	QLoRA	Prompt Engineering
Training data	10K–1M+	1K–100K	1K–100K	0 (just prompts)
GPU memory	8×A100	1–2×A100	1×RTX4090	None

```
| Training time     | Days–weeks | Hours     | Hours     | Minutes          |
| Quality ceiling   | Best       | Near-best | Good      | Depends on task  |
| Model changes     | All weights| Adapters  | Adapters  | No weights       |
| Cost              | $$$$$      | $$$       | $$        | $                |
| When to use       | New domain,| Style /   | Budget-   | Try this FIRST!  |
|                   | specialized| format    | limited   | Often enough.    |
|                   | knowledge  | changes   | hardware  |                  |
└──────────────────┴────────────┴───────────┴───────────┴──────────────────┘

Rule of thumb:
  1. Start with prompt engineering (free, instant)
  2. If not enough, try LoRA (hours, one GPU)
  3. If tight on hardware, use QLoRA (same quality, less memory)
  4. Full fine-tune only when you need deep domain knowledge changes
```

Inference Speedups

Quantization (INT8 / INT4)

Analogy: JPEG compression for model weights. You lose a tiny bit of precision, but save massive amounts of memory and run faster. Just like a JPEG photo looks 99% as good as the raw file at 10% the size.

How it works: Convert FP16 weights (16 bits per number) to INT8 (8 bits) or INT4 (4 bits). Fewer bits per weight = less memory = faster memory transfers = faster inference.

```
┌──────────┬────────────┬──────────┬──────────────────┬─────────────────┐
| Precision| Bits/param| Memory   | Speed            | Quality Loss |
├──────────┼────────────┼──────────┼──────────────────┼─────────────────┤
| FP16     | 16        | 1× (ref) | 1× (ref)         | None          |
| INT8     | 8         | 0.5×     | ~1.5× faster     | Minimal       |
| INT4     | 4         | 0.25×    | ~2× faster       | Small         |
| 2-bit    | 2         | 0.125×   | ~3× faster       | Noticeable    |
└──────────┴────────────┴──────────┴──────────────────┴─────────────────┘

70B model at different precisions:
  FP16: ~140 GB → needs 2× A100 80GB
  INT8:  ~70 GB → fits 1× A100 80GB
  INT4:  ~35 GB → fits 1× RTX 4090 (48GB)
  2-bit: ~18 GB → fits 1× RTX 4080 (16GB)... but quality degrades
```

Popular quantization methods:

```
┌──────────────────┬──────────────────────────────────────────────┐
| Method           | How It Works                                  |
├──────────────────┼──────────────────────────────────────────────┤
| GPTQ             | Post-training, layer-by-layer. Fast to apply.  |
```

```
|                  | Good quality. Very popular for local models.    |
|------------------|-------------------------------------------------|
| AWQ              | Activation-aware. Keeps important weights at    |
|                  | higher precision. Better quality than GPTQ.     |
|------------------|-------------------------------------------------|
| bitsandbytes     | Easy integration with HuggingFace. Just add     |
|                  | load_in_4bit=True. Great for quick experiments. |
|------------------|-------------------------------------------------|
| GGUF (llama.cpp) | CPU-friendly format. Run models on laptops      |
|                  | without a GPU. Various quant levels (Q4, Q5).   |
└──────────────────┴─────────────────────────────────────────────────┘
```

When to use: Almost always in production. INT8 has negligible quality loss. INT4 is the sweet spot for running large models on consumer hardware.

Speculative Decoding

Analogy: An intern writes a draft email (fast but imperfect). The senior manager reviews the draft and accepts or revises each sentence (slow but accurate). Net result: faster than the senior writing from scratch, because reviewing is faster than writing.

How it works:

```
┌──────────────────────────────────────────────────────────┐
│              SPECULATIVE DECODING                        │
│                                                          │
│  Step 1: Draft model (small, fast) generates K tokens:   │
│                                                          │
│    "The capital of" → "France" "is" "Paris" "."          │
│                       (generated quickly, cheaply)       │
│                                                          │
│  Step 2: Target model (large, accurate) verifies ALL     │
│  K tokens in ONE forward pass (parallel verification):   │
│                                                          │
│    "France" ✓   "is" ✓   "Paris" ✓   "." ✓               │
│                                                          │
│  Result: 4 tokens generated at nearly the cost of        │
│  1 forward pass of the large model!                      │
│                                                          │
│  ──────────────────────────────────────────────────      │
│                                                          │
│  What if the draft is wrong at token 3?                  │
│                                                          │
│    "France" ✓   "is" ✓   "Lyon" ✗   "." (discarded)      │
│                                                          │
│  Accept tokens 1–2, resample token 3 from the target     │
│  model's distribution, discard the rest. Try again.      │
│                                                          │
│  ──────────────────────────────────────────────────      │
│                                                          │
```

```
┌──────────────────────────────────────────────────────────┐
│  CRITICAL PROPERTY: The output distribution is           │
│  IDENTICAL to running the large model alone.             │
│  Speculative decoding NEVER changes the output           │
│  quality — it only speeds things up.                     │
└──────────────────────────────────────────────────────────┘
```

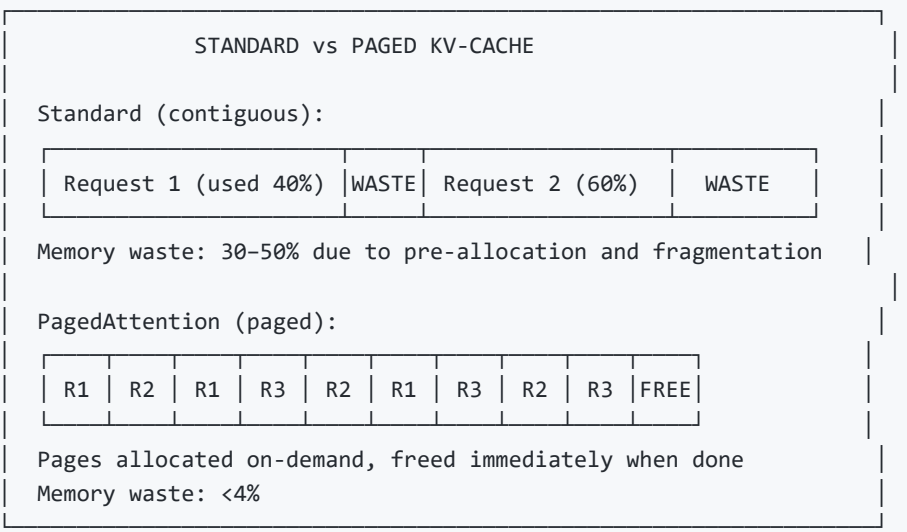
Speedup: Typically **2–3×** depending on the draft model's acceptance rate. If the draft model agrees with the target model 80% of the time, most draft tokens get accepted and you skip expensive sequential generation.

When to use: Production inference of large models (GPT-5, Claude Opus 4.6) where latency matters. The draft model is usually a smaller variant from the same family (e.g., a 1B draft for a 70B target).

PagedAttention / vLLM

The problem: During inference, every active request maintains a **KV-cache** — the stored keys and values for all past tokens. In standard serving, memory for each request is allocated in one big contiguous block. With many concurrent users, GPU memory becomes fragmented — like a parking lot where every car has a reserved row of 20 spots, but most cars only use 5.

The solution: Borrowed from operating systems. Just like virtual memory breaks RAM into small **pages** that can be stored anywhere, PagedAttention breaks the KV-cache into small pages that can be scattered across GPU memory.



Result: 2–4× more concurrent requests on the same hardware, near-zero memory waste.

Used by: vLLM, TensorRT-LLM, SGLang — most production LLM serving stacks. If you're deploying an LLM to serve users, you're almost certainly using PagedAttention under the hood.

KV-Cache Compression

As context windows grow to 100K–1M tokens, the KV-cache becomes the dominant memory consumer. Several techniques compress it:

Sliding Window Attention:

```
Only keep the last W tokens in cache (e.g., W = 4096).
Older tokens are evicted.

┌──────────────────────────────────────────────────┐
│  Token stream: [1] [2] [3] ... [4000] ... [8000]│
│                                                  │
│  Cache keeps: [3905] [3906] ... [8000]           │
│  Evicted:     [1] [2] ... [3904]                 │
└──────────────────────────────────────────────────┘

✓ Simple, constant memory
✗ Loses long-range context entirely
Used by: Mistral (combined with full attention in alternating layers)
```

H2O (Heavy Hitter Oracle):

```
Keep tokens that consistently get HIGH attention scores.
Evict tokens that nobody pays attention to.

Intuition: In most text, a few tokens are "anchor" tokens that
everything else attends to (e.g., the subject of the paragraph,
key nouns). Keep those, evict filler tokens.

✓ Retains important context
✓ Adaptive — different texts keep different tokens
✗ Slightly more complex than sliding window
```

StreamingLLM:

```
Keep the first few "attention sink" tokens + recent window.

┌──────────────────────────────────────────────────┐
│  [SINK₁][SINK₂][SINK₃][SINK₄] ... [recent W]  │
│   ↑ first 4 tokens          ↑ last W tokens     │
│   (attention sinks)         (sliding window)    │
└──────────────────────────────────────────────────┘

Discovery: The first few tokens in ANY sequence absorb
disproportionate attention, regardless of content. Removing
them breaks generation. Keeping them + recent window enables
infinite-length generation with constant memory.
```

```
  ✓ Enables infinite-length generation
  ✓ Constant memory usage
  ✗ Still loses middle context
```

Evaluation & Scaling

Perplexity

Intuition: "On average, how many words is the model choosing between at each step?"

A perplexity of 10 means the model is, on average, as uncertain as if it were choosing between 10 equally likely options at every token. Lower is better — it means the model is more confident and usually more accurate.

```
The formula:

  PPL = e^(average cross-entropy loss)

  This is just the exponential of the average loss we computed earlier.
  It converts the abstract loss number into something interpretable:
  "equivalent number of choices."

Examples:
  GPT-5 on English text   ≈ 5–10 PPL    (choosing between ~5–10 options)
  Small model on English   ≈ 20–50 PPL   (much more uncertain)
  Random model (50K vocab) ≈ 50,000 PPL  (no idea, guessing uniformly)
  Perfect model            = 1 PPL       (knows every token in advance)
```

Important caveats:

- Perplexity is only comparable when using the **same tokenizer and dataset**. A model with a 32K vocabulary and one with a 100K vocabulary can't be directly compared.
- Low perplexity ≠ "good at tasks." A model can predict text well but still fail at reasoning. Perplexity measures language modeling, not intelligence.

Key Benchmarks

```
┌──────────────┬──────────────────────────────┬──────────────────────────┐
│ Benchmark    │ What It Tests                │ Example Question         │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ MMLU         │ 57-subject knowledge         │ "The Krebs cycle..."     │
│              │ (history, law, physics, etc.) │                          │
├──────────────┼──────────────────────────────┼──────────────────────────┤
```

HumanEval	Code generation (Python) functional correctness	Write fibonacci()
GSM8K	Grade-school math reasoning (multi-step word problems)	"If Alice has 3 apples and buys 5..."
ARC	Science reasoning (grade-school level)	"What causes tides?"
GPQA	PhD-level science questions (expert-validated)	Quantum mechanics Q
SWE-bench	Real GitHub bug fixing (end-to-end software eng.)	Fix this actual bug in a real repository
MATH	Competition-level math (AMC, AIME difficulty)	Prove / solve a competition problem

Caveat — benchmark contamination: A real concern. Models may have seen benchmark questions during pre-training (they're on the internet). This inflates scores. Always look at **held-out** benchmarks like SWE-bench (generated from real GitHub issues after training cutoff dates) for more reliable evaluation.

Chinchilla Scaling Law

Key insight (DeepMind, 2022): For a given compute budget, you should train a **smaller model on more data**, not a bigger model on less data.

```
┌──────────────────────────────────────────────────────────────┐
│                    CHINCHILLA SCALING LAW                    │
│                                                              │
│  The optimal ratio: ~20 tokens per parameter                 │
│                                                              │
│  ┌──────────────┬──────────────┬──────────────────────────┐  │
│  │ Model Size   │ Optimal Data │ Actual Training Data     │  │
│  ├──────────────┼──────────────┼──────────────────────────┤  │
│  │ 7B params    │ 140B tokens  │ LLaMA-2: 2T tokens       │  │
│  │              │              │ (14× "optimal"!)         │  │
│  │ 70B params   │ 1.4T tokens  │ LLaMA-2: 2T tokens       │  │
│  │              │              │ (1.4× "optimal")         │  │
│  │ 175B params  │ 3.5T tokens  │ GPT-3: 300B tokens       │  │
│  │              │              │ (0.09× — undertrained!)  │  │
│  └──────────────┴──────────────┴──────────────────────────┘  │
│                                                              │
│  The twist: "Chinchilla-optimal" minimizes TRAINING cost.    │
│  But for INFERENCE cost, over-training smaller models is better. │
│                                                              │
│  Why? You pay for training ONCE, but inference MILLIONS of times.│
│  A 7B model trained on 2T tokens (14× over-trained) is:      │
```

```
│   - More expensive to train than "optimal"                          │
│   - But MUCH cheaper to serve (7B vs 70B per inference call)        │
│   - And nearly as capable as the "optimal" 70B trained on 1.4T      │
│                                                                      │
│  Modern strategy: train small models on as much data as possible.   │
│  LLaMA 4, Mistral, Phi-3 all follow this over-training approach.    │
└──────────────────────────────────────────────────────────────────────┘
```

Inference-Time Compute Scaling

The new frontier: Instead of making models bigger (more parameters), make them **think longer** at inference time. Spend more compute per question, not per training run.

```
┌──────────────────────────────────────────────────────────────────────┐
│              INFERENCE-TIME COMPUTE SCALING                          │
│                                                                      │
│  Traditional scaling (pre-2024):                                     │
│    "Want better answers? Train a bigger model."                      │
│    7B → 70B → 700B → ???                                             │
│    Problem: diminishing returns, massive serving costs               │
│                                                                      │
│  Inference-time scaling (2024+):                                     │
│    "Want better answers? Let the same model think longer."           │
│    Same model → more reasoning steps → better answers                │
│                                                                      │
│  Techniques:                                                         │
│  ┌──────────────────────┬──────────────────────────────────────────┐ │
│  │ Chain-of-Thought     │ Model reasons step by step               │ │
│  │                      │ "Let me think through this..."           │ │
│  ├──────────────────────┼──────────────────────────────────────────┤ │
│  │ Best-of-N            │ Generate N responses, score them,        │ │
│  │                      │ pick the best one                        │ │
│  ├──────────────────────┼──────────────────────────────────────────┤ │
│  │ Self-Consistency     │ Generate multiple reasoning paths,       │ │
│  │                      │ take the majority vote on answers        │ │
│  ├──────────────────────┼──────────────────────────────────────────┤ │
│  │ Tree Search          │ Explore multiple solution branches       │ │
│  │                      │ systematically (like chess engines)      │ │
│  ├──────────────────────┼──────────────────────────────────────────┤ │
│  │ Iterative Refinement │ Model critiques and improves its         │ │
│  │                      │ own output over multiple passes          │ │
│  └──────────────────────┴──────────────────────────────────────────┘ │
│                                                                      │
│  Why this matters for agents:                                        │
│                                                                      │
│    Agent loops ARE inference-time compute scaling!                   │
│                                                                      │
│    think → act → observe → think → act → observe → ...               │
│                                                                      │
│    Each loop iteration spends more compute on the problem.           │
│    The agent keeps refining its approach based on feedback.          │
```

```
│    This is exactly "think longer, get better answers."           │
│                                                                  │
│  Models built for this: GPT-5.1/5.2 (reasoning modes),          │
│  Claude Opus 4.6 (extended thinking), Gemini 3 (deep research)  │
└──────────────────────────────────────────────────────────────────┘
```

The key takeaway: The future of AI performance is not just "bigger models" — it's smarter inference. Agents, chain-of-thought, and search-based decoding are all ways to trade inference compute for better answers. This is why understanding agent architectures (Days 3–7 of this curriculum) is so important.

Day 2 — Advanced Prompt Engineering

> 🎯 **Goal**: Build mastery over prompt structures and behaviors.

> ⏱️ **Time**: 1.5–2h theory + 2–3h practice

> 📦 **Deliverable**: A reusable advanced prompt library with validators

---

Table of Contents

1. Why Prompt Engineering Matters
2. Zero-Shot Prompting
3. Few-Shot Prompting
4. Chain-of-Thought (CoT)
5. Structured Output Prompting
6. Function Calling
7. Prompt Failure Modes
8. Advanced Techniques
9. Practice Exercises

Why Prompt Engineering Matters

The same model can give wildly different results depending on how you prompt it. Prompt engineering is **the** skill that separates someone who "uses ChatGPT" from someone who **builds with LLMs**.

```
Same model, same question, different prompts:

Prompt 1 (bad): "Is this code secure?"
Response: "Yes, it looks fine." ← Useless

Prompt 2 (good): "You are a senior security engineer. Analyze the following
code for OWASP Top 10 vulnerabilities. For each vulnerability found, provide:
1. The vulnerability type (e.g., SQL Injection)
2. The exact line(s) affected
3. Why it's vulnerable
4. A fixed version of the code
If no vulnerabilities are found, explain what security measures are in place."
Response: Detailed, actionable security analysis ← Valuable
```

Zero-Shot Prompting

Zero-shot means giving the model a task **without any examples**. You rely entirely on the model's pre-trained knowledge.

When to Use

- Simple, well-defined tasks
- Tasks the model has seen plenty of during training
- When you need speed/simplicity

Anatomy of a Good Zero-Shot Prompt

```
┌─────────────────────────────────────────────────────┐
│                   ZERO-SHOT PROMPT                  │
│                                                     │
│  1. ROLE/PERSONA                                    │
│     "You are a senior backend engineer..."          │
│                                                     │
│  2. TASK DESCRIPTION                                │
│     "Analyze the following API endpoint for..."     │
│                                                     │
│  3. INPUT DATA                                      │
│     "```python\ndef get_user(id): ...\n```"         │
│                                                     │
│  4. OUTPUT FORMAT SPECIFICATION                     │
│     "Respond with a JSON object containing..."      │
│                                                     │
│  5. CONSTRAINTS                                     │
│     "Do not include explanations outside the JSON." │
│                                                     │
└─────────────────────────────────────────────────────┘
```

Examples

```
# Classification (zero-shot)
SENTIMENT_PROMPT = """You are a sentiment analysis system.

Classify the following text into exactly one category:
- POSITIVE
- NEGATIVE
- NEUTRAL

Text: "{text}"

Respond with only the category name, nothing else."""

# Extraction (zero-shot)
ENTITY_PROMPT = """Extract all named entities from the following text.

Text: "{text}"

Respond as a JSON array of objects with these fields:
- "entity": the extracted text
- "type": one of PERSON, ORGANIZATION, LOCATION, DATE, MONEY

Example output format:
[{{"entity": "John", "type": "PERSON"}}]"""

# Transformation (zero-shot)
SQL_PROMPT = """You are a SQL expert. Convert the following natural language
query into a PostgreSQL query.

Database schema:
- users(id, name, email, created_at)
- orders(id, user_id, amount, status, created_at)
- products(id, name, price, category)
- order_items(order_id, product_id, quantity)

Natural language query: "{query}"

Respond with only the SQL query, no explanations."""
```

Few-Shot Prompting

Few-shot means providing **examples** of the desired input→output behavior. The model learns the pattern from your examples and applies it to new inputs.

Why Few-Shot Works

```
The model performs "in-context learning" — it doesn't update its weights,
but it recognizes the pattern in your examples and continues it.

Think of it as:
  "I showed you 3 examples of how I want this done.
   Now do the same thing for this new input."
```

How to Choose Good Examples

```
Rules for selecting few-shot examples:

1. DIVERSE: Cover different cases, not just the easy ones
   Bad:  3 examples of positive sentiment
   Good: 1 positive, 1 negative, 1 neutral

2. REPRESENTATIVE: Match the distribution of real inputs
   Bad:  All short sentences when real data has long paragraphs
   Good: Mix of lengths matching your actual data

3. EDGE CASES: Include at least one tricky example
   Bad:  All straightforward cases
   Good: Include an ambiguous case showing the desired behavior

4. CONSISTENT FORMAT: Output format must be identical across all examples
   Bad:  Example 1 uses JSON, Example 2 uses plain text
   Good: All examples use the exact same JSON structure

5. ORDERED: Put the hardest/most relevant example last (recency bias)
```

Examples

```
  CLASSIFICATION_FEW_SHOT = """Classify customer support tickets into categories.

  Examples:

  Ticket: "I can't log into my account, it says password incorrect"
  Category: AUTHENTICATION

  Ticket: "My payment was charged twice for order #12345"
  Category: BILLING

  Ticket: "The app crashes when I try to upload a photo larger than 10MB"
  Category: BUG_REPORT

  Ticket: "Can you add dark mode to the mobile app?"
  Category: FEATURE_REQUEST

  Ticket: "I want to cancel my subscription but I can't find the option"
  Category: ACCOUNT_MANAGEMENT
```

```
Now classify this ticket:
Ticket: "{ticket_text}"
Category:"""

CODE_REVIEW_FEW_SHOT = """Review code and provide feedback in JSON format.

Example 1:
Code: `user_input = request.args.get('query'); db.execute(f"SELECT * FROM users WHER
Review: {{"severity": "CRITICAL", "issue": "SQL Injection", "fix": "Use parameterize

Example 2:
Code: `password = 'admin123'`
Review: {{"severity": "HIGH", "issue": "Hardcoded credential", "fix": "Use environme

Example 3:
Code: `items = [x for x in range(1000000)]`
Review: {{"severity": "MEDIUM", "issue": "Memory inefficiency", "fix": "Use generato

Now review this code:
Code: `{code}`
Review:"""
```

How Many Examples?

```
┌──────────┬──────────────────────────────────────────────┐
│ Examples │ When to Use                                  │
├──────────┼──────────────────────────────────────────────┤
│ 0        │ Simple tasks, well-defined output format     │
│ 1-2      │ Format clarification, style matching         │
│ 3-5      │ Complex classification, nuanced behavior     │
│ 5-10     │ Very specific patterns, custom DSLs          │
│ 10+      │ Rarely needed; consider fine-tuning instead  │
└──────────┴──────────────────────────────────────────────┘

Warning: More examples = more tokens = higher cost + slower response
  Each example might use 50-200 tokens
  5 examples × 100 tokens = 500 tokens of context consumed
```

Chain-of-Thought (CoT)

Chain-of-Thought prompting asks the model to **show its reasoning steps** before giving a final answer. This dramatically improves performance on complex tasks.

Why CoT Works

```
Without CoT:
  Q: "Roger has 5 tennis balls. He buys 2 more cans of 3.
      How many tennis balls does he have?"
  A: "11" ← The model jumps to the answer (sometimes wrong)

With CoT:
  Q: "Roger has 5 tennis balls. He buys 2 more cans of 3.
      How many tennis balls does he have? Let's think step by step."
  A: "Roger starts with 5 balls.
      He buys 2 cans, each with 3 balls.
      2 cans × 3 balls = 6 new balls.
      5 + 6 = 11 total balls.
      The answer is 11." ← The model reasons through it

Why it helps:
  1. Forces sequential reasoning (not jumping to conclusions)
  2. Each step provides context for the next step
  3. Errors are visible and correctable
  4. The model effectively "shows its work"
```

Types of CoT

```
1. ZERO-SHOT CoT: Just add "Let's think step by step"
   Simple but effective. Works for most reasoning tasks.

2. FEW-SHOT CoT: Provide examples with reasoning chains
   More reliable, controls the reasoning format.

3. SELF-CONSISTENCY CoT: Generate multiple reasoning paths,
   take the majority answer.
   Prompt → Generate 5 different CoTs → Vote on final answer
   More expensive but more accurate.

4. TREE-OF-THOUGHT: Explore multiple reasoning branches,
   evaluate each, backtrack if needed.
   Best for complex planning/puzzle tasks.
```

Implementation

```python
# Zero-shot CoT
REASONING_PROMPT = """Solve the following problem. Think step by step,
showing your reasoning clearly before giving the final answer.

Problem: {problem}

Solution:
Let me work through this step by step.

Step 1:"""
```

```
# Few-shot CoT
MATH_COT_PROMPT = """Solve math word problems by reasoning step by step.

Problem: A store sells notebooks for $3 each. If you buy 5 or more,
you get a 20% discount. How much do 7 notebooks cost?

Solution:
Step 1: Calculate the base price. 7 notebooks × $3 = $21
Step 2: Check if discount applies. 7 ≥ 5, so yes, 20% discount.
Step 3: Calculate discount. 20% of $21 = $4.20
Step 4: Subtract discount. $21 - $4.20 = $16.80
Answer: $16.80

Problem: A train travels at 60 mph for 2 hours, then at 80 mph for
1.5 hours. What is the total distance?

Solution:
Step 1: Distance at 60 mph. 60 × 2 = 120 miles
Step 2: Distance at 80 mph. 80 × 1.5 = 120 miles
Step 3: Total distance. 120 + 120 = 240 miles
Answer: 240 miles

Problem: {problem}

Solution:"""

# Self-consistency (multiple samples, take majority)
import openai
from collections import Counter

def solve_with_self_consistency(problem, n_samples=5):
    """Generate multiple reasoning paths and vote on the answer."""
    answers = []

    for _ in range(n_samples):
        response = openai.chat.completions.create(
            model="gpt-4",
            messages=[{
                "role": "user",
                "content": REASONING_PROMPT.format(problem=problem)
            }],
            temperature=0.7,  # Some randomness for diverse paths
            max_tokens=500
        )

        text = response.choices[0].message.content
        # Extract final answer (last line or after "Answer:")
        answer = extract_answer(text)
        answers.append(answer)

    # Majority vote
    vote = Counter(answers).most_common(1)[0]
    return vote[0], vote[1] / n_samples  # answer, confidence

def extract_answer(text):
```

```
    """Extract the final answer from a CoT response."""
    lines = text.strip().split("\n")
    for line in reversed(lines):
        if "answer" in line.lower() or "=" in line:
            # Extract number or final value
            import re
            numbers = re.findall(r'[\d,]+\.?\d*', line)
            if numbers:
                return numbers[-1]
    return lines[-1].strip()
```

Structured Output Prompting

For production systems, you need the model to output **valid, parseable data** — not free-form text.

JSON Output

```
# Basic JSON enforcement
JSON_PROMPT = """Analyze the following customer review and extract structured data.

Review: "{review}"

Respond with ONLY a valid JSON object (no markdown, no explanation) matching
this exact schema:
{{
  "sentiment": "positive" | "negative" | "neutral",
  "rating_estimate": 1-5,
  "topics": ["string"],
  "key_phrases": ["string"],
  "requires_response": true | false,
  "urgency": "low" | "medium" | "high"
}}"""

# Using system message for format enforcement
SYSTEM_MSG = """You are a JSON-only API. You NEVER output anything except
valid JSON. No markdown code blocks, no explanations, no preamble.
Your entire response must be parseable by JSON.parse()."""
```

Schema-Enforced Output

```
# Provide the exact JSON Schema for validation
SCHEMA_PROMPT = """Extract entities from the text and return as JSON.

JSON Schema (your output MUST conform to this):
```

```
{{
  "type": "object",
  "required": ["entities", "summary"],
  "properties": {{
    "entities": {{
      "type": "array",
      "items": {{
        "type": "object",
        "required": ["name", "type", "confidence"],
        "properties": {{
          "name": {{"type": "string"}},
          "type": {{"type": "string", "enum": ["PERSON", "ORG", "LOCATION"]}},
          "confidence": {{"type": "number", "minimum": 0, "maximum": 1}}
        }}
      }}
    }},
    "summary": {{"type": "string", "maxLength": 200}}
  }}
}}

Text: "{text}"

JSON output:"""
```

DSL (Domain-Specific Language) Output

```
# Sometimes JSON is overkill — define a simple DSL
TASK_DSL_PROMPT = """Convert the user request into a task specification
using this format:

ACTION: <verb>
TARGET: <what to act on>
PARAMS:
  - key: value
  - key: value
CONDITIONS:
  - condition description
OUTPUT: <expected result type>

User request: "{request}"

Task specification:"""

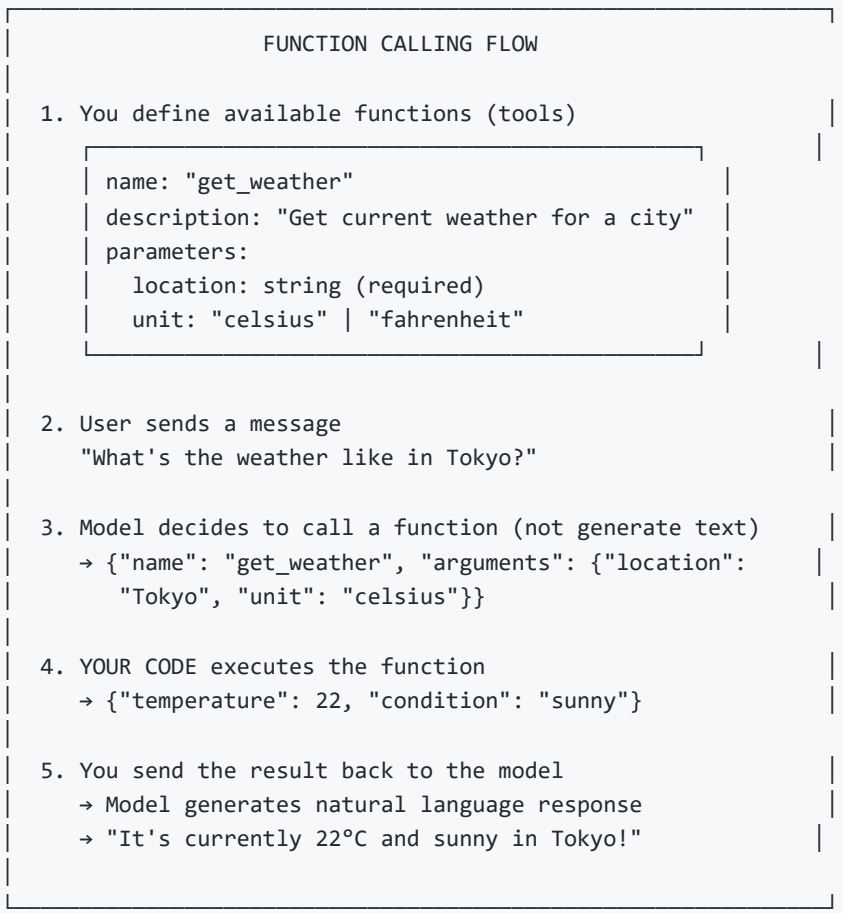
# Example output:
# ACTION: SEARCH
# TARGET: database.users
# PARAMS:
#   - field: email
#   - pattern: *@company.com
#   - limit: 100
# CONDITIONS:
#   - created_at > 2024-01-01
```

```
#   - status = active
# OUTPUT: list of user records
```

Function Calling

Function calling lets the model decide **when and how to call external tools**. This is the foundation of tool-using agents (Day 3).

How Function Calling Works



Implementation with OpenAI

```
import openai
import json

# Step 1: Define your tools
tools = [
    {
```

```
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the current weather in a given location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "City name, e.g., 'San Francisco, CA'"
                    },
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "Temperature unit"
                    }
                },
                "required": ["location"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "search_database",
            "description": "Search the product database",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "Search query"},
                    "category": {
                        "type": "string",
                        "enum": ["electronics", "clothing", "food"],
                        "description": "Product category filter"
                    },
                    "max_results": {
                        "type": "integer",
                        "description": "Maximum results to return",
                        "default": 5
                    }
                },
                "required": ["query"]
            }
        }
    }
]

# Step 2: Your actual function implementations
def get_weather(location, unit="celsius"):
    """Simulate a weather API call."""
    # In production, call a real weather API
    return {"temperature": 22, "condition": "sunny", "location": location}

def search_database(query, category=None, max_results=5):
    """Simulate a database search."""
    return [{"name": f"Product matching '{query}'", "price": 29.99}]
```

```python
# Map function names to implementations
FUNCTION_MAP = {
    "get_weather": get_weather,
    "search_database": search_database,
}

# Step 3: The conversation loop
def chat_with_tools(user_message):
    messages = [{"role": "user", "content": user_message}]

    # First call — model may decide to use a tool
    response = openai.chat.completions.create(
        model="gpt-4",
        messages=messages,
        tools=tools,
        tool_choice="auto"  # Let model decide
    )

    msg = response.choices[0].message

    # Check if model wants to call a function
    if msg.tool_calls:
        messages.append(msg)  # Add assistant's tool call message

        for tool_call in msg.tool_calls:
            fn_name = tool_call.function.name
            fn_args = json.loads(tool_call.function.arguments)

            print(f"  → Calling {fn_name}({fn_args})")

            # Execute the function
            result = FUNCTION_MAP[fn_name](**fn_args)

            # Add function result to messages
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": json.dumps(result)
            })

        # Second call — model generates final response using function results
        response = openai.chat.completions.create(
            model="gpt-4",
            messages=messages,
            tools=tools
        )
        return response.choices[0].message.content

    # No tool call — direct response
    return msg.content

# Test it
print(chat_with_tools("What's the weather in Tokyo and Paris?"))
# → Calling get_weather({"location": "Tokyo"})
```

```
  # → Calling get_weather({"location": "Paris"})
  # → "In Tokyo it's 22°C and sunny. In Paris it's..."
```

Parallel Function Calls

Modern models can request **multiple function calls in a single response**:

```
  # Model's response might contain multiple tool_calls:
  # tool_calls = [
  #     {"id": "call_1", "function": {"name": "get_weather", "arguments": '{"location"
  #     {"id": "call_2", "function": {"name": "get_weather", "arguments": '{"location"
  # ]
  #
  # You execute BOTH, then send BOTH results back
  # This is more efficient than sequential calls
```

Prompt Failure Modes

Understanding how prompts fail is critical for building reliable systems.

Failure Mode 1: Hallucinations

```
What it is:
  The model generates plausible-sounding but factually incorrect information.

Example:
  Q: "What paper introduced the RLHF technique?"
  A: "RLHF was introduced in 'Training Language Models to Follow
      Instructions with Human Feedback' by Ouyang et al. (2022),
      published at NeurIPS 2022."

  Reality: The paper exists and is correct, BUT sometimes models
  fabricate paper titles, author names, or publication venues.

Mitigation strategies:
  1. Ground responses in provided context (RAG — Day 5)
  2. Ask model to cite specific sources from given documents
  3. Add "If you don't know, say 'I don't know'" to prompt
  4. Use lower temperature (0.0-0.3) for factual tasks
  5. Verify with a second model call or external check
```

Failure Mode 2: Invalid JSON

```
What it is:
  The model outputs text that looks like JSON but isn't valid.

Common issues:
  - Trailing commas: {"a": 1, "b": 2,}  ← invalid
  - Single quotes: {'key': 'value'}  ← invalid JSON
  - Unquoted keys: {key: "value"}  ← invalid
  - Comments: {"key": "value" /* comment */}  ← invalid
  - Markdown wrapping: ```json\n{...}\n```  ← extra text
  - Truncation: {"key": "very long value that gets cu  ← cut off

Mitigation (Validate → Retry → Fix pattern):
```

```
  import json
  import re

  def extract_json(text):
      """Extract JSON from model output, handling common issues."""
      # Strip markdown code blocks
      text = re.sub(r'^```(?:json)?\s*\n?', '', text.strip())
      text = re.sub(r'\n?```\s*$', '', text.strip())

      # Try parsing directly
      try:
          return json.loads(text)
      except json.JSONDecodeError:
          pass

      # Try finding JSON object in text
      match = re.search(r'\{.*\}', text, re.DOTALL)
      if match:
          try:
              return json.loads(match.group())
          except json.JSONDecodeError:
              pass

      # Try finding JSON array
      match = re.search(r'\[.*\]', text, re.DOTALL)
      if match:
          try:
              return json.loads(match.group())
          except json.JSONDecodeError:
              pass

      return None

  def get_valid_json(prompt, model="gpt-4", max_retries=3):
      """Repeatedly prompt until valid JSON is obtained."""
      import openai

      messages = [
          {"role": "system", "content": "Respond with valid JSON only. "
           "No markdown, no explanations, no text outside the JSON."},
```

```
            {"role": "user", "content": prompt}
        ]

        for attempt in range(max_retries):
            response = openai.chat.completions.create(
                model=model,
                messages=messages,
                temperature=0.0 if attempt > 0 else 0.3  # Lower temp on retry
            )

            text = response.choices[0].message.content
            result = extract_json(text)

            if result is not None:
                return result

            # Add error context for retry
            messages.append({"role": "assistant", "content": text})
            messages.append({
                "role": "user",
                "content": f"That was not valid JSON. Error: {get_json_error(text)}. "
                           f"Please respond with ONLY valid JSON, no other text."
            })
            print(f"  Retry {attempt + 1}/{max_retries}: Invalid JSON, retrying...")

        raise ValueError(f"Failed to get valid JSON after {max_retries} attempts")

    def get_json_error(text):
        """Get a helpful error message for invalid JSON."""
        try:
            json.loads(text)
            return "Unknown"
        except json.JSONDecodeError as e:
            return str(e)
```

Failure Mode 3: Mode Collapse / Refusal

```
What it is:
  The model refuses to answer or gives overly generic responses.

Examples:
  - "I can't help with that" (overly cautious alignment)
  - "As an AI language model, I..." (meta-responses)
  - Repeating the question back instead of answering

Mitigation:
  1. Be specific about what you want — vague prompts get vague answers
  2. Use a system prompt that establishes expertise
  3. Frame tasks professionally (not as circumventing safety)
  4. Break complex requests into smaller, clearer steps
```

Failure Mode 4: Instruction Following Failures

```
What it is:
  The model ignores part of your instructions.

Common cases:
  - Includes explanations when you said "JSON only"
  - Uses wrong format despite explicit schema
  - Adds fields not in the schema
  - Ignores length constraints

Why it happens:
  - Prompt is too long (instructions get "lost")
  - Conflicting instructions
  - Model's training biases override your instructions
  - Important instructions buried in the middle (lost-in-the-middle)

Mitigation:
  1. Put the MOST important instructions at the START and END
  2. Use clear delimiters (###, ---) between sections
  3. Repeat critical constraints
  4. Use structured prompts (numbered lists)
  5. Post-process outputs to enforce format

Position matters (lost-in-the-middle effect):
   ┌──────────────────────────────────────────┐
   │ High recall                              │
   │ ███████                          ███████ │
   │ ███████                          ███████ │
   │ ███████      Low recall          ███████ │
   │ ███████     ▒▒▒▒▒▒▒▒▒▒           ███████ │
   │ ███████     ▒▒▒▒▒▒▒▒▒▒           ███████ │
   │ ███████     ▒▒▒▒▒▒▒▒▒▒           ███████ │
   │                                          │
   │ Beginning    Middle                End   │
   │ of prompt    of prompt           of prompt │
   └──────────────────────────────────────────┘

  Put critical instructions at the beginning and end!
```

Advanced Techniques

System Messages

```
  # System messages set the behavior for the entire conversation
  # They have the strongest influence on model behavior

  SYSTEM_MESSAGES = {
      "json_api": (
```

```
            "You are a JSON-only API. Every response must be valid JSON. "
            "Never include markdown, explanations, or any text outside JSON. "
            "If you cannot complete the request, respond with: "
            '{"error": "description of the problem"}'
        ),

        "code_reviewer": (
            "You are a senior software engineer performing code reviews. "
            "Focus only on bugs, security issues, and performance problems. "
            "Do not comment on style or formatting. "
            "Rate each issue as CRITICAL, HIGH, MEDIUM, or LOW severity."
        ),

        "sql_expert": (
            "You are a PostgreSQL expert. Generate only SQL queries. "
            "Always use parameterized queries (never string interpolation). "
            "Include helpful comments for complex queries. "
            "Consider query performance and suggest indexes when relevant."
        )
    }
```

Prompt Chaining

```
Break complex tasks into sequential prompts, where each prompt
uses the output of the previous one:

┌─────────────────────────────────────────────────────────┐
│                    PROMPT CHAINING                      │
│                                                         │
│  Prompt 1: "Identify the key topics in this text"       │
│      │                                                  │
│      ▼ Output: ["AI safety", "regulation", "ethics"]    │
│                                                         │
│  Prompt 2: "For each topic, find supporting quotes"     │
│      │                                                  │
│      ▼ Output: {"AI safety": ["quote1"], ...}           │
│                                                         │
│  Prompt 3: "Write a summary based on these findings"    │
│      │                                                  │
│      ▼ Output: "The article discusses three main..."    │
│                                                         │
└─────────────────────────────────────────────────────────┘

Why chain? Each step is simpler and more reliable than one
mega-prompt trying to do everything at once.
```

```
import openai

def chain_prompts(text):
    """Multi-step analysis using prompt chaining."""
```

```
    # Step 1: Extract topics
    topics_response = openai.chat.completions.create(
        model="gpt-4",
        messages=[{
            "role": "user",
            "content": f"List the 3 main topics in this text as a JSON array "
                       f"of strings. Only output the JSON array.\n\n{text}"
        }],
        temperature=0.0
    )
    topics = json.loads(topics_response.choices[0].message.content)

    # Step 2: Analyze each topic
    analyses = {}
    for topic in topics:
        analysis_response = openai.chat.completions.create(
            model="gpt-4",
            messages=[{
                "role": "user",
                "content": f"From this text, extract key points about '{topic}'. "
                           f"Return as JSON: {{\"points\": [\"...\"], "
                           f"\"sentiment\": \"positive|negative|neutral\"}}\n\n{text
            }],
            temperature=0.0
        )
        analyses[topic] = json.loads(analysis_response.choices[0].message.content)

    # Step 3: Synthesize
    synthesis_response = openai.chat.completions.create(
        model="gpt-4",
        messages=[{
            "role": "user",
            "content": f"Given this analysis:\n{json.dumps(analyses, indent=2)}\n\n"
                       f"Write a 3-paragraph executive summary."
        }],
        temperature=0.3
    )

    return synthesis_response.choices[0].message.content
```

Meta-Prompting

```
# Use an LLM to generate/optimize prompts for you

META_PROMPT = """You are a prompt engineering expert. Given a task description,
generate an optimized prompt that will produce the best results from GPT-4.

Your prompt should include:
1. A clear system role
2. Precise task description
3. Output format specification
```

```
4. Edge case handling
5. At least 2 few-shot examples

Task: {task_description}

Generate the optimized prompt:"""

# Example: Generate a prompt for code documentation
task = "Generate API documentation from Python function signatures"
optimized_prompt = call_llm(META_PROMPT.format(task_description=task))
# → Returns a carefully crafted prompt with examples and format specs
```

Practice Exercises

Exercise 1: Build a Prompt Library

```
"""
Reusable prompt library with template rendering and validation.
"""
import json
import re
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class PromptTemplate:
    """A reusable prompt template with variable substitution."""
    name: str
    system_message: str
    user_template: str
    output_format: str  # "json", "text", "dsv"
    json_schema: Optional[dict] = None
    examples: list = field(default_factory=list)
    temperature: float = 0.3
    max_tokens: int = 1000

    def render(self, **kwargs) -> tuple:
        """Render the prompt with variables. Returns (system, user) messages."""
        user_content = self.user_template.format(**kwargs)

        # Add examples if few-shot
        if self.examples:
            examples_text = "\n\nExamples:\n"
            for i, ex in enumerate(self.examples, 1):
                examples_text += f"\nExample {i}:\n"
                examples_text += f"Input: {ex['input']}\n"
                examples_text += f"Output: {ex['output']}\n"
            user_content = examples_text + "\nNow process this:\n" + user_content
```

```
        return self.system_message, user_content

    def validate_output(self, output: str) -> tuple:
        """Validate and parse the model's output. Returns (parsed, errors)."""
        if self.output_format == "json":
            return self._validate_json(output)
        elif self.output_format == "text":
            return output.strip(), []
        return output, []

    def _validate_json(self, output: str) -> tuple:
        """Parse and optionally validate JSON against schema."""
        # Clean markdown
        cleaned = re.sub(r'^```(?:json)?\s*\n?', '', output.strip())
        cleaned = re.sub(r'\n?```\s*$', '', cleaned.strip())

        try:
            parsed = json.loads(cleaned)
        except json.JSONDecodeError as e:
            return None, [f"Invalid JSON: {e}"]

        errors = []
        if self.json_schema:
            errors = self._check_schema(parsed, self.json_schema)

        return parsed, errors

    def _check_schema(self, data, schema, path="root") -> list:
        """Simple JSON schema validation."""
        errors = []

        if schema.get("type") == "object" and isinstance(data, dict):
            for req in schema.get("required", []):
                if req not in data:
                    errors.append(f"Missing required field '{req}' at {path}")

            props = schema.get("properties", {})
            for key, val_schema in props.items():
                if key in data:
                    errors.extend(
                        self._check_schema(data[key], val_schema, f"{path}.{key}")
                    )

        elif schema.get("type") == "array" and isinstance(data, list):
            item_schema = schema.get("items", {})
            for i, item in enumerate(data):
                errors.extend(
                    self._check_schema(item, item_schema, f"{path}[{i}]")
                )

        elif schema.get("type") == "string" and not isinstance(data, str):
            errors.append(f"Expected string at {path}, got {type(data).__name__}")

        elif schema.get("type") == "number" and not isinstance(data, (int, float)):
            errors.append(f"Expected number at {path}, got {type(data).__name__}")
```

```
        if "enum" in schema and data not in schema["enum"]:
            errors.append(f"Value '{data}' at {path} not in {schema['enum']}")

        return errors


# ── PROMPT LIBRARY ──────────────────────────────────────────

PROMPTS = {}

# Sentiment Analysis
PROMPTS["sentiment"] = PromptTemplate(
    name="sentiment",
    system_message="You are a sentiment analysis API. Output valid JSON only.",
    user_template='Analyze the sentiment of this text:\n\n"{text}"',
    output_format="json",
    json_schema={
        "type": "object",
        "required": ["sentiment", "confidence", "reasoning"],
        "properties": {
            "sentiment": {"type": "string", "enum": ["positive", "negative", "neutra
            "confidence": {"type": "number"},
            "reasoning": {"type": "string"}
        }
    },
    examples=[
        {
            "input": "This product is amazing, best purchase ever!",
            "output": '{"sentiment": "positive", "confidence": 0.95, '
                      '"reasoning": "Strong positive language: amazing, best ever"}'
        },
        {
            "input": "The delivery was late and the item was broken.",
            "output": '{"sentiment": "negative", "confidence": 0.9, '
                      '"reasoning": "Multiple negative events: late delivery, broken
        }
    ],
    temperature=0.0
)

# Code Review
PROMPTS["code_review"] = PromptTemplate(
    name="code_review",
    system_message=(
        "You are a senior engineer performing code review. "
        "Focus on bugs, security issues, and performance. "
        "Output valid JSON only."
    ),
    user_template='Review this code:\n\n```\n{code}\n```',
    output_format="json",
    json_schema={
        "type": "object",
        "required": ["issues", "overall_quality"],
        "properties": {
            "issues": {
                "type": "array",
```

```
                "items": {
                    "type": "object",
                    "required": ["severity", "description", "fix"],
                    "properties": {
                        "severity": {"type": "string",
                                     "enum": ["CRITICAL", "HIGH", "MEDIUM", "LOW"]},
                        "description": {"type": "string"},
                        "line": {"type": "number"},
                        "fix": {"type": "string"}
                    }
                }
            },
            "overall_quality": {"type": "string",
                                "enum": ["excellent", "good", "needs_work", "poor"]}
        }
    },
    temperature=0.0
)

# SQL Generator
PROMPTS["sql_generator"] = PromptTemplate(
    name="sql_generator",
    system_message=(
        "You are a PostgreSQL expert. Generate only the SQL query. "
        "Use parameterized queries with $1, $2, etc. for user inputs. "
        "No explanations."
    ),
    user_template=(
        "Database schema:\n{schema}\n\n"
        "Generate a query for: {question}"
    ),
    output_format="text",
    examples=[
        {
            "input": "Find users who signed up this month",
            "output": "SELECT * FROM users WHERE created_at >= date_trunc('month', C
        }
    ],
    temperature=0.0
)


# ── USAGE EXAMPLE ──────────────────────────────────────────

def demo():
    """Demonstrate the prompt library."""
    # Render a prompt
    prompt = PROMPTS["sentiment"]
    system_msg, user_msg = prompt.render(
        text="The new update is terrible, it broke everything!"
    )
    print(f"System: {system_msg}")
    print(f"User: {user_msg}")

    # Simulate model output and validate
    model_output = '{"sentiment": "negative", "confidence": 0.92, "reasoning": "Stro
```

```
    parsed, errors = prompt.validate_output(model_output)
    print(f"\nParsed: {parsed}")
    print(f"Errors: {errors}")  # Should be empty

    # Test with bad output
    bad_output = '{"sentiment": "angry", "confidence": "high"}'
    parsed, errors = prompt.validate_output(bad_output)
    print(f"\nBad output errors: {errors}")
    # → ["Value 'angry' at root.sentiment not in ['positive', 'negative', 'neutral']
    #    "Expected number at root.confidence, got str"]

if __name__ == "__main__":
    demo()
```

Exercise 2: JSON Output with Validate → Retry → Fix

```
"""
Robust JSON extraction with automatic retry and self-correction.
"""
import json
import re
import time
from typing import Optional

class RobustJSONExtractor:
    """Extract valid JSON from LLM outputs with retry logic."""

    def __init__(self, model="gpt-4", max_retries=3):
        self.model = model
        self.max_retries = max_retries
        self.stats = {"attempts": 0, "successes": 0, "retries": 0}

    def extract(self, prompt: str, json_schema: Optional[dict] = None) -> dict:
        """
        Send prompt to LLM and extract valid JSON with retries.

        Flow:
        ┌──────────────────────────────────────────┐
        │  Send prompt to LLM                      │
        │         │                                │
        │         ▼                                │
        │  Parse response as JSON                  │
        │         │                                │
        │    ┌────┴────┐                           │
        │    │ Valid?  │                           │
        │    └────┬────┘                           │
        │    Yes  │  No                            │
        │    │    │                                │
        │    │    ▼                                │
        │    │  Validate against schema            │
        │    │    │                                │
        │    │  ┌─┴──┐                             │
        │    │  │
```

```
|    |  |Pass?|                              |
|    |  └──┬──┘                              |
|    | Yes|  No                              |
|    |   |   |                               |
|    |   |   ▼                               |
|    |   |  Send error + retry prompt        |
|    |   |   |                               |
|    |   |   └──── (up to max_retries) ─┐    |
|    |   |                              |    |
|    ▼   ▼                              |    |
|  Return parsed JSON                   |    |
|                                       └────┘
└──────────────────────────────────────────┘
"""
import openai

self.stats["attempts"] += 1

messages = [
    {
        "role": "system",
        "content": (
            "You are a JSON-only API. Your ENTIRE response must be "
            "a single valid JSON object. No markdown code fences, "
            "no explanations, no text before or after the JSON."
        )
    },
    {"role": "user", "content": prompt}
]

for attempt in range(self.max_retries + 1):
    response = openai.chat.completions.create(
        model=self.model,
        messages=messages,
        temperature=max(0.0, 0.3 - attempt * 0.1),  # Lower temp each retry
        max_tokens=2000
    )

    raw_output = response.choices[0].message.content

    # Try to parse JSON
    parsed = self._try_parse(raw_output)

    if parsed is None:
        error_msg = self._get_parse_error(raw_output)
        if attempt < self.max_retries:
            self.stats["retries"] += 1
            messages.append({"role": "assistant", "content": raw_output})
            messages.append({
                "role": "user",
                "content": (
                    f"ERROR: Your response was not valid JSON.\n"
                    f"Parse error: {error_msg}\n\n"
                    f"Please respond with ONLY a valid JSON object. "
                    f"Do not wrap in markdown. Do not add explanations."
                )
```

```
                    })
                    continue
                raise ValueError(f"Failed to parse JSON after {self.max_retries} ret

            # Validate against schema
            if json_schema:
                schema_errors = self._validate_schema(parsed, json_schema)
                if schema_errors:
                    if attempt < self.max_retries:
                        self.stats["retries"] += 1
                        messages.append({"role": "assistant", "content": raw_output}
                        messages.append({
                            "role": "user",
                            "content": (
                                f"The JSON is valid but doesn't match the required s
                                f"Schema errors:\n"
                                + "\n".join(f"  - {e}" for e in schema_errors)
                                + f"\n\nPlease fix these issues and respond with cor
                            )
                        })
                        continue
                    raise ValueError(f"Schema validation failed: {schema_errors}")

            self.stats["successes"] += 1
            return parsed

    def _try_parse(self, text: str) -> Optional[dict]:
        """Attempt to parse JSON from text, with cleanup."""
        # Direct parse
        text = text.strip()
        try:
            return json.loads(text)
        except json.JSONDecodeError:
            pass

        # Remove markdown fences
        cleaned = re.sub(r'^```(?:json)?\s*\n?', '', text)
        cleaned = re.sub(r'\n?```\s*$', '', cleaned).strip()
        try:
            return json.loads(cleaned)
        except json.JSONDecodeError:
            pass

        # Find JSON object
        match = re.search(r'\{[\s\S]*\}', text)
        if match:
            try:
                return json.loads(match.group())
            except json.JSONDecodeError:
                pass

        return None

    def _get_parse_error(self, text: str) -> str:
        """Get a helpful error message."""
        try:
```

```
                json.loads(text)
                return "Unknown error"
            except json.JSONDecodeError as e:
                return f"Line {e.lineno}, Col {e.colno}: {e.msg}"

        def _validate_schema(self, data, schema, path="$") -> list:
            """Validate data against a JSON schema (simplified)."""
            errors = []
            expected_type = schema.get("type")

            type_map = {
                "object": dict, "array": list, "string": str,
                "number": (int, float), "integer": int, "boolean": bool
            }

            if expected_type and expected_type in type_map:
                if not isinstance(data, type_map[expected_type]):
                    errors.append(
                        f"{path}: expected {expected_type}, got {type(data).__name__}"
                    )
                    return errors

            if expected_type == "object" and isinstance(data, dict):
                for req in schema.get("required", []):
                    if req not in data:
                        errors.append(f"{path}: missing required field '{req}'")
                for key, prop_schema in schema.get("properties", {}).items():
                    if key in data:
                        errors.extend(self._validate_schema(data[key], prop_schema, f"{p

            if expected_type == "array" and isinstance(data, list):
                for i, item in enumerate(data):
                    if "items" in schema:
                        errors.extend(self._validate_schema(item, schema["items"], f"{pa

            if "enum" in schema and data not in schema["enum"]:
                errors.append(f"{path}: '{data}' not in allowed values {schema['enum']}")

            return errors


    # ── USAGE ──────────────────────────────────────────────────────────

    def demo_extractor():
        """Demonstrate robust JSON extraction."""
        extractor = RobustJSONExtractor(max_retries=3)

        schema = {
            "type": "object",
            "required": ["summary", "key_points", "sentiment"],
            "properties": {
                "summary": {"type": "string"},
                "key_points": {
                    "type": "array",
                    "items": {"type": "string"}
                },
```

```
                "sentiment": {
                    "type": "string",
                    "enum": ["positive", "negative", "neutral", "mixed"]
                }
            }
        }

        result = extractor.extract(
            prompt="Analyze this review: 'Great product but shipping was slow'",
            json_schema=schema
        )

        print(f"Result: {json.dumps(result, indent=2)}")
        print(f"Stats: {extractor.stats}")

    if __name__ == "__main__":
        demo_extractor()
```

✓ Day 2 Deliverable Checklist

- [ ] Built a `PromptTemplate` class with variable substitution
- [ ] Created at least 3 reusable prompts (sentiment, code review, SQL)
- [ ] Implemented JSON validation against schemas
- [ ] Built a Validate → Retry → Fix mechanism
- [ ] Can explain: What is chain-of-thought and why does it help?
- [ ] Can explain: What are the main prompt failure modes?
- [ ] Can explain: Why does instruction position matter?
- [ ] Can explain: When to use few-shot vs zero-shot?

Day 3 — Single-Agent Systems (ReAct)

> 🎯 **Goal**: Build a reliable, tool-using Agent.

> ⏱️ **Time**: 1–1.5h theory + 3h practice

> 📦 **Deliverable**: A functioning single-agent ReAct system with tool execution

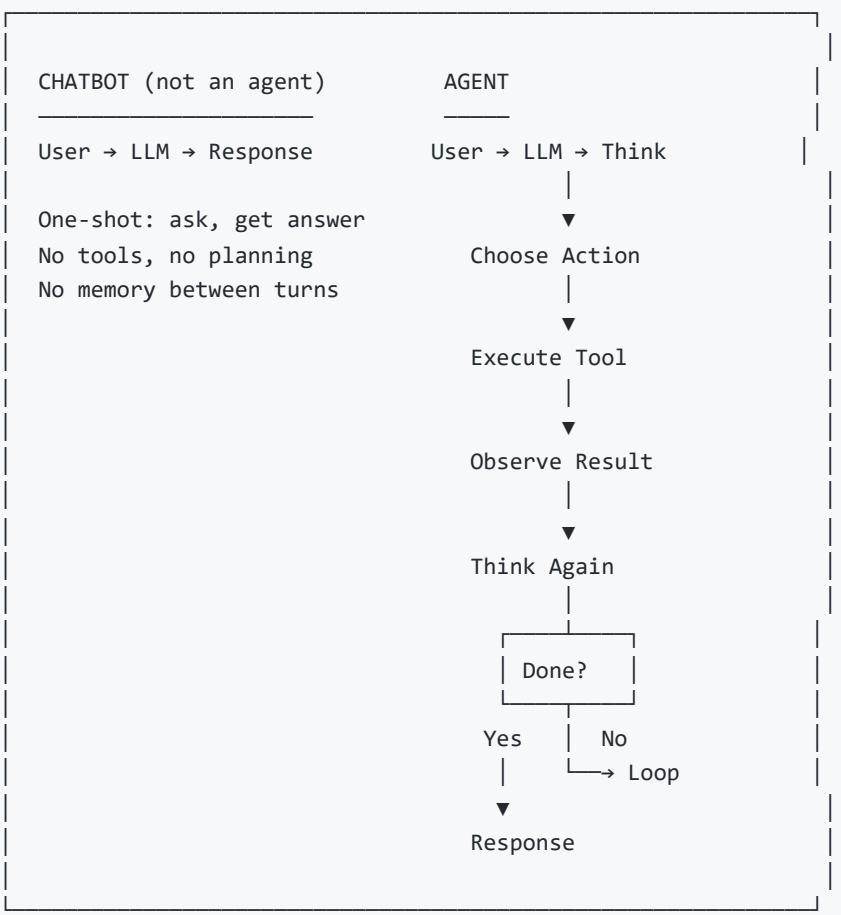
---

Table of Contents

1. What Is an Agent?
2. The ReAct Pattern
3. Agent Memory Types
4. Why Agents Fail
5. Practice: Build a ReAct Agent

What Is an Agent?

An Agent is an LLM that can **decide what actions to take**, **execute those actions**, and **use the results to inform its next steps** — autonomously.



Key Differences

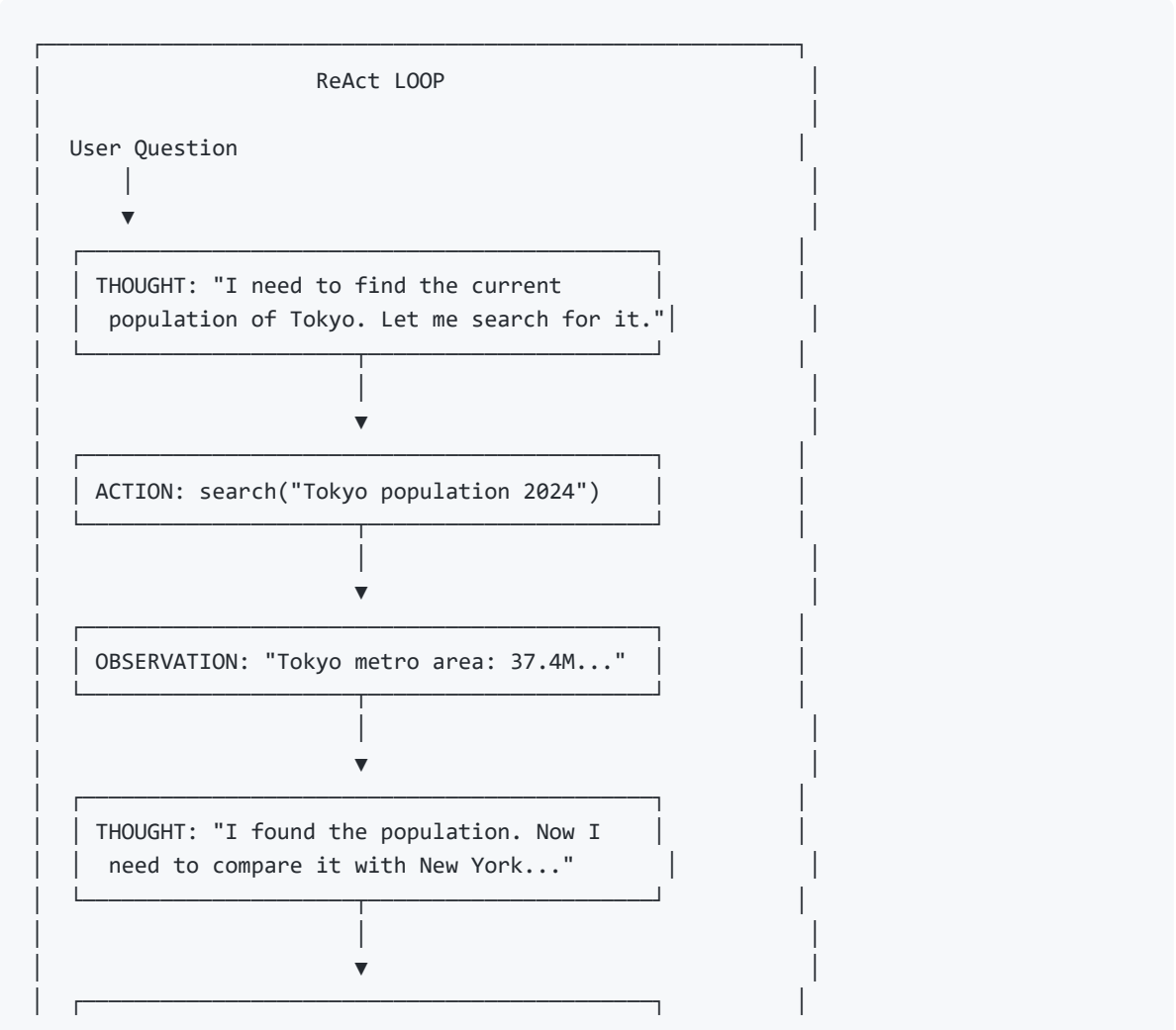
Feature	Chatbot	Agent
Tools	None	Calculator, APIs, databases, code execution

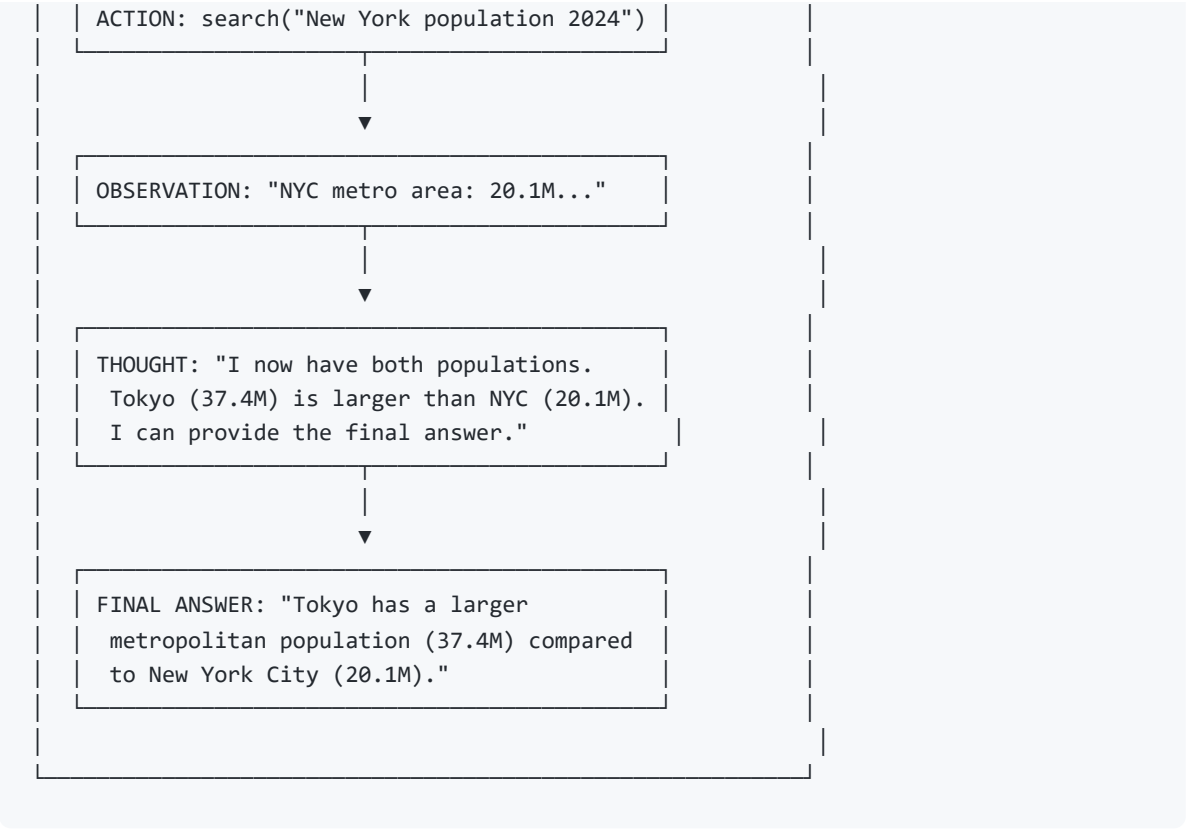
Planning	None	Multi-step task decomposition
Memory	Conversation history only	Short-term + long-term + episodic
Autonomy	Responds to each message	Decides what to do next
Loops	Single turn	Multiple think→act→observe cycles
Error handling	None	Self-correction, retries

The ReAct Pattern

ReAct (Reason + Act) is the most important agent pattern. Published by Yao et al. (2022), it interleaves **reasoning** (thinking about what to do) with **acting** (using tools) and **observing** (processing results).

The ReAct Loop





Why ReAct Works

```
Without ReAct (pure reasoning):
  Q: "What's 47 × 83 + 156?"
  A: "4,057" ← The LLM does mental math (often wrong for large numbers)

With ReAct (reason + act):
  Thought: "I need to calculate 47 × 83 first, then add 156.
            Let me use the calculator tool."
  Action: calculator("47 * 83")
  Observation: 3901
  Thought: "Now I need to add 156."
  Action: calculator("3901 + 156")
  Observation: 4057
  Answer: "47 × 83 + 156 = 4,057"

The key insight: Let the LLM REASON about what to do,
but let TOOLS do the actual work.
```

ReAct vs Other Patterns

```
┌──────────────────────────────────────────────────────────┐
│                    AGENT PATTERNS                        │
│                                                          │
│  1. REASON-ONLY (Chain-of-Thought)                       │
│     Think → Think → Think → Answer                       │
│     Problem: Can't access real-time data or compute      │
```

```
│                                                          │
│  2. ACT-ONLY (Tool chaining)                             │
│     Tool → Tool → Tool → Answer                          │
│     Problem: No reasoning about which tool to use        │
│                                                          │
│  3. ReAct (Reason + Act)  ← BEST                         │
│     Think → Act → Observe → Think → Act → Observe → Ans  │
│     Reasoning guides tool use, observations guide next   │
│     reasoning step                                       │
│                                                          │
│  4. PLAN-then-EXECUTE (Day 4)                            │
│     Plan all steps → Execute sequentially → Review       │
│     Better for complex multi-step tasks                  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Agent Memory Types

Agents need memory to operate effectively. There are three types:

1. Short-Term Memory (Scratchpad)

```
What: The current conversation / reasoning trace
Where: In the LLM's context window
Lifespan: Current task only
Size: Limited by context window (4K–128K tokens)

Example:
  The agent's scratchpad during a ReAct loop:
  ┌──────────────────────────────────────────┐
  │ Thought 1: I need to find the weather    │
  │ Action 1: get_weather("Tokyo")           │
  │ Observation 1: 22°C, sunny               │
  │ Thought 2: Now I need the forecast...    │  ← This IS short-term memory
  │ Action 2: get_forecast("Tokyo", days=3)  │
  │ Observation 2: [22, 25, 20]              │
  │ Thought 3: I have all the information... │
  └──────────────────────────────────────────┘

Trade-off: Easy to implement but context window is finite.
  Llama 3 8B: 8K context → ~20 ReAct steps max
  GPT-4: 128K context → ~300 ReAct steps max
  But cost increases linearly with context length!
```

2. Long-Term Memory (Vector Store)

```
What: Persistent knowledge the agent can search
Where: External vector database (ChromaDB, Pinecone, FAISS)
```

```
Lifespan: Persists across sessions
Size: Unlimited (GBs of documents)

How it works:
  1. Documents → split into chunks → embed → store in vector DB
  2. Agent queries: "What did we discuss about authentication?"
  3. Vector DB returns most similar chunks
  4. Agent uses retrieved chunks as context

  ┌──────────────────────────────────────────────────────┐
  │                  LONG-TERM MEMORY                    │
  │                                                      │
  │  Agent query: "authentication best practices"        │
  │       │                                              │
  │       ▼                                              │
  │  Embed query → [0.23, 0.87, -0.12, ...]              │
  │       │                                              │
  │       ▼                                              │
  │  ┌──────────────────────────────────────┐            │
  │  │           VECTOR DATABASE            │            │
  │  │                                      │            │
  │  │  Doc 1: "Use JWT tokens for..."  0.92  ← Similar  │
  │  │  Doc 2: "OAuth 2.0 flow: ..."   0.87  ← Similar  │
  │  │  Doc 3: "Database indexes..."   0.23  ← Not      │
  │  │  Doc 4: "Rate limiting with..." 0.31  ← Not      │
  │  └──────────────────────────────────────┘            │
  │                                                      │
  │  Return top-K similar documents → inject into context │
  └──────────────────────────────────────────────────────┘

We'll build this fully on Day 5 (RAG).
```

3. Episodic Memory

```
What: Memory of past actions, decisions, and outcomes
Where: Structured log / database
Lifespan: Persists across sessions
Size: Grows over time

Purpose: Learn from past mistakes and successes

Example:
  ┌──────────────────────────────────────────────────────┐
  │               EPISODIC MEMORY LOG                    │
  │                                                      │
  │  Episode 1 (2024-01-15):                             │
  │    Task: "Deploy to production"                      │
  │    Actions: [run_tests, build, deploy]               │
  │    Outcome: FAILED — tests passed but deploy timed out│
  │    Lesson: "Increase deploy timeout to 300s"         │
  │                                                      │
  │  Episode 2 (2024-01-16):                             │
  │    Task: "Deploy to production"                      │
  │    Actions: [run_tests, build, deploy(timeout=300)]  │
```

```
   │    Outcome: SUCCESS                                      │
   │    Lesson: "300s timeout works for production deploy"    │
   │                                                          │
   │  When agent gets "Deploy to production" again:           │
   │  → Retrieves episodes → Uses 300s timeout from start     │
   └──────────────────────────────────────────────────────────┘
```

Memory Comparison

Type	Location	Lifespan	Retrieval	Use Case
Short-term	Context window	Current task	Automatic (in prompt)	Reasoning steps, current state
Long-term	Vector DB	Permanent	Similarity search	Knowledge base, documents
Episodic	Database/log	Permanent	Task-based lookup	Learning from past actions

---

Why Agents Fail

Understanding failure modes is critical for building reliable agents.

Failure 1: Infinite Loops

```
What happens:
  Thought: "I need to search for X"
  Action: search("X")
  Observation: "No results found"
  Thought: "Let me try searching for X"     ← SAME action again!
  Action: search("X")
  Observation: "No results found"
  Thought: "Let me search for X"            ← Stuck in a loop!
  ... forever ...

Why it happens:
  - The model doesn't recognize it already tried this
  - No mechanism to try alternative approaches
  - Same failed action feels "right" to the model

Prevention:
  1. Track action history → reject duplicate actions
  2. Set a maximum number of steps (e.g., max_steps=10)
  3. After N failures, force the agent to try a different approach
  4. Include action history in the prompt
```

Failure 2: Hallucinated Tools

```
What happens:
  Thought: "Let me use the email_sender tool"
  Action: email_sender("user@example.com", "Hello!")

  Problem: email_sender doesn't exist! The model invented it.

Why it happens:
  - The model has seen many tools in training data
  - It "remembers" tools that aren't available
  - Similar to how models hallucinate facts

Prevention:
  1. Explicitly list available tools in the system prompt
  2. Validate tool names before execution
  3. Return clear error: "Tool 'email_sender' not available.
     Available tools: calculator, search, get_weather"
  4. Let the agent recover from the error
```

Failure 3: Malformed Tool Arguments

```
What happens:
  Action: calculator("forty-seven times eighty-three")

  Problem: Calculator expects "47 * 83", not English words

Why it happens:
  - Model doesn't perfectly follow argument schemas
  - Especially with complex or nested arguments

Prevention:
  1. Provide clear argument specifications with examples
  2. Validate arguments before execution
  3. Use JSON schemas for tool arguments
  4. On error, show the agent the schema and ask it to retry
```

Failure 4: Premature Termination

```
What happens:
  User: "Find the 3 cheapest flights from NYC to Tokyo next month"
  Thought: "Let me search for flights"
  Action: search_flights("NYC", "Tokyo", "2024-02-01")
  Observation: [Flight A: $800, Flight B: $950, Flight C: $1100]
  Answer: "The cheapest flight is $800 on Flight A."

  Problem: Only reported 1 flight, user asked for 3!

Why it happens:
  - Model "thinks" it's done when it has a partial answer
```

```
   - Doesn't check its answer against the original question

Prevention:
  1. Include the original question in every reasoning step
  2. Add a validation step: "Does my answer fully address the question?"
  3. Use a separate critic prompt to check completeness
```

Anti-Pattern Summary

```
┌──────────────────┬──────────────────────────────────────────┐
│ Anti-Pattern     │ Solution                                 │
├──────────────────┼──────────────────────────────────────────┤
│ Infinite loops   │ Max steps + duplicate detection + fallback │
│ Tool hallucin.   │ Strict tool registry + validation        │
│ Bad arguments    │ Schema validation + retry with error msg │
│ Early stopping   │ Answer validation + critic check         │
│ Context overflow │ Summarize old steps + truncation strategy │
│ Wrong tool pick  │ Better tool descriptions + examples      │
└──────────────────┴──────────────────────────────────────────┘
```

Practice: Build a ReAct Agent

Complete ReAct Agent Implementation

```python
"""
ReAct Agent — A complete implementation with tool use,
validation, and loop prevention.

This agent:
  1. Creates a multi-step plan
  2. Selects and uses tools
  3. Validates its reasoning
  4. Avoids action loops
  5. Handles errors gracefully
"""
import json
import re
import math
import urllib.request
import urllib.parse
from dataclasses import dataclass, field
from typing import Callable


# ═══════════════════════════════════════════════════════════
# SECTION 1: TOOL DEFINITIONS
# ═══════════════════════════════════════════════════════════
```

```python
@dataclass
class Tool:
    """A tool the agent can use."""
    name: str
    description: str
    parameters: dict  # JSON schema for parameters
    function: Callable

    def to_prompt_description(self) -> str:
        """Format tool info for the agent's system prompt."""
        params_str = json.dumps(self.parameters, indent=2)
        return (
            f"Tool: {self.name}\n"
            f"Description: {self.description}\n"
            f"Parameters: {params_str}\n"
        )


def calculator(expression: str) -> str:
    """Safely evaluate a mathematical expression."""
    # Allow only safe math operations
    allowed_chars = set("0123456789+-*/().% ")
    allowed_funcs = {"sqrt": math.sqrt, "pow": pow, "abs": abs,
                     "round": round, "min": min, "max": max}

    if not all(c in allowed_chars or c.isalpha() for c in expression):
        return f"Error: Invalid characters in expression: {expression}"

    try:
        # Replace common math functions
        expr = expression
        for name, func in allowed_funcs.items():
            expr = expr.replace(name, f"__{name}__")

        # Evaluate safely
        result = eval(expr, {"__builtins__": {}},
                      {f"__{k}__": v for k, v in allowed_funcs.items()})
        return str(result)
    except Exception as e:
        return f"Error: Could not evaluate '{expression}': {e}"


def get_weather(city: str) -> str:
    """Get weather for a city (simulated for practice)."""
    # In production, call a real API like OpenWeatherMap
    weather_data = {
        "tokyo": {"temp": 22, "condition": "Sunny", "humidity": 45},
        "new york": {"temp": 15, "condition": "Cloudy", "humidity": 65},
        "london": {"temp": 12, "condition": "Rainy", "humidity": 80},
        "paris": {"temp": 18, "condition": "Partly Cloudy", "humidity": 55},
        "sydney": {"temp": 28, "condition": "Sunny", "humidity": 40},
    }

    city_lower = city.lower().strip()
    if city_lower in weather_data:
```

```
            data = weather_data[city_lower]
            return json.dumps({
                "city": city,
                "temperature_celsius": data["temp"],
                "condition": data["condition"],
                "humidity_percent": data["humidity"]
            })

        return json.dumps({"error": f"Weather data not available for '{city}'. "
                           f"Available cities: {', '.join(weather_data.keys())}"})


    def search(query: str) -> str:
        """Search for information (simulated knowledge base)."""
        # In production, use a real search API (Google, Bing, etc.)
        knowledge = {
            "population tokyo": "Tokyo metropolitan area has a population of approximate
            "population new york": "New York City metropolitan area has a population of
            "python creator": "Python was created by Guido van Rossum and first released
            "speed of light": "The speed of light in vacuum is approximately 299,792,458
            "largest ocean": "The Pacific Ocean is the largest ocean, covering approxima
            "bitcoin creator": "Bitcoin was created by an anonymous person or group usin
        }

        query_lower = query.lower().strip()

        # Find best match
        best_match = None
        best_score = 0
        for key, value in knowledge.items():
            # Simple keyword matching
            query_words = set(query_lower.split())
            key_words = set(key.split())
            overlap = len(query_words & key_words)
            if overlap > best_score:
                best_score = overlap
                best_match = value

        if best_match and best_score > 0:
            return best_match

        return f"No results found for '{query}'. Try different search terms."


    def string_utility(operation: str, text: str) -> str:
        """Perform string operations."""
        operations = {
            "length": lambda t: str(len(t)),
            "upper": lambda t: t.upper(),
            "lower": lambda t: t.lower(),
            "reverse": lambda t: t[::-1],
            "word_count": lambda t: str(len(t.split())),
        }

        if operation not in operations:
            return f"Unknown operation '{operation}'. Available: {', '.join(operations.k
```

```
    return operations[operation](text)


# Define tools with their schemas
TOOLS = {
    "calculator": Tool(
        name="calculator",
        description="Evaluate mathematical expressions. Supports +, -, *, /, %, sqrt
        parameters={
            "type": "object",
            "required": ["expression"],
            "properties": {
                "expression": {
                    "type": "string",
                    "description": "Math expression to evaluate, e.g., '47 * 83 + 15
                }
            }
        },
        function=lambda args: calculator(args["expression"])
    ),
    "search": Tool(
        name="search",
        description="Search for factual information. Use for questions about people,
        parameters={
            "type": "object",
            "required": ["query"],
            "properties": {
                "query": {
                    "type": "string",
                    "description": "Search query, e.g., 'population of Tokyo'"
                }
            }
        },
        function=lambda args: search(args["query"])
    ),
    "get_weather": Tool(
        name="get_weather",
        description="Get current weather for a city. Returns temperature, condition,
        parameters={
            "type": "object",
            "required": ["city"],
            "properties": {
                "city": {
                    "type": "string",
                    "description": "City name, e.g., 'Tokyo', 'New York'"
                }
            }
        },
        function=lambda args: get_weather(args["city"])
    ),
    "string_utility": Tool(
        name="string_utility",
        description="Perform string operations: length, upper, lower, reverse, word_
        parameters={
            "type": "object",
```

```
            "required": ["operation", "text"],
            "properties": {
                "operation": {
                    "type": "string",
                    "enum": ["length", "upper", "lower", "reverse", "word_count"],
                    "description": "The string operation to perform"
                },
                "text": {
                    "type": "string",
                    "description": "The text to operate on"
                }
            }
        },
        function=lambda args: string_utility(args["operation"], args["text"])
    ),
}


# ═══════════════════════════════════════════════════════════════════
# SECTION 2: THE ReAct AGENT
# ═══════════════════════════════════════════════════════════════════

class ReActAgent:
    """
    A ReAct agent that reasons and acts in an interleaved loop.

    Architecture:
    ┌─────────────────────────────────────────────┐
    │              ReAct Agent                    │
    │                                             │
    │   ┌─────────┐   ┌─────────┐   ┌─────────┐   │
    │   │ LLM     │   │ Tool    │   │ Memory  │   │
    │   │ (reason)│   │ Registry│   │ (scratch-│  │
    │   │         │   │         │   │  pad)   │   │
    │   └────┬────┘   └────┬────┘   └────┬────┘   │
    │        │             │             │        │
    │        └─────────────┼─────────────┘        │
    │                      │                      │
    │              Agent Loop                     │
    │         Think → Act → Observe               │
    │                      │                      │
    │              Max Steps Guard                │
    │              Loop Detection                 │
    └─────────────────────────────────────────────┘
    """

    def __init__(self, tools: dict, model: str = "gpt-4", max_steps: int = 10):
        self.tools = tools
        self.model = model
        self.max_steps = max_steps
        self.scratchpad = []  # Short-term memory
        self.action_history = []  # For loop detection

    def _build_system_prompt(self) -> str:
        """Build the system prompt with tool descriptions."""
        tools_desc = "\n".join(
```

```
            tool.to_prompt_description() for tool in self.tools.values()
        )

        return f"""You are a helpful AI agent that solves tasks by reasoning step-by

## Available Tools
{tools_desc}

## How to Respond

You MUST respond in one of two formats:

### Format 1: Use a tool
```

Thought: Action: Action Input:

```
### Format 2: Give the final answer (when you have enough information)
```

Thought: <your reasoning about why you're done> Final Answer: <your complete answer to the user's question>

```
## Rules
1. Always start with a Thought explaining your reasoning
2. Use tools when you need external information or computation
3. NEVER make up information — use tools to get facts
4. If a tool returns an error, try a different approach
5. Do NOT repeat the same action with the same inputs
6. When you have enough information, provide the Final Answer
7. The Final Answer should fully address the user's original question"""

    def _parse_response(self, text: str) -> dict:
        """Parse the agent's response into structured components."""
        result = {"thought": "", "action": None, "action_input": None,
                  "final_answer": None}

        # Extract Thought
        thought_match = re.search(r'Thought:\s*(.*?)(?=\n(?:Action:|Final Answer:)|$)',
                                  text, re.DOTALL)
        if thought_match:
            result["thought"] = thought_match.group(1).strip()

        # Check for Final Answer
        final_match = re.search(r'Final Answer:\s*(.*)', text, re.DOTALL)
        if final_match:
            result["final_answer"] = final_match.group(1).strip()
            return result

        # Extract Action and Action Input
        action_match = re.search(r'Action:\s*(\w+)', text)
```

```
        if action_match:
            result["action"] = action_match.group(1).strip()

        input_match = re.search(r'Action Input:\s*(.*?)(?=\n\n|$)', text, re.DOTALL)
        if input_match:
            raw_input = input_match.group(1).strip()
            try:
                result["action_input"] = json.loads(raw_input)
            except json.JSONDecodeError:
                # Try to interpret as simple string argument
                result["action_input"] = {"query": raw_input}

        return result

    def _execute_tool(self, action: str, action_input: dict) -> str:
        """Execute a tool and return the result."""
        if action not in self.tools:
            available = ", ".join(self.tools.keys())
            return (f"Error: Tool '{action}' not found. "
                    f"Available tools: {available}")

        try:
            tool = self.tools[action]
            result = tool.function(action_input)
            return result
        except Exception as e:
            return f"Error executing {action}: {str(e)}"

    def _check_loop(self, action: str, action_input: dict) -> bool:
        """Check if this action has been tried before with same inputs."""
        action_key = f"{action}:{json.dumps(action_input, sort_keys=True)}"

        if action_key in self.action_history:
            return True  # Loop detected!

        self.action_history.append(action_key)
        return False

    def run(self, question: str, verbose: bool = True) -> str:
        """
        Run the ReAct loop to answer a question.

        Args:
            question: The user's question
            verbose: Whether to print the reasoning trace

        Returns:
            The agent's final answer
        """
        self.scratchpad = []
        self.action_history = []

        system_prompt = self._build_system_prompt()

        messages = [
            {"role": "system", "content": system_prompt},
```

```
            {"role": "user", "content": question}
        ]

        if verbose:
            print(f"\n{'='*60}")
            print(f"Question: {question}")
            print(f"{'='*60}")

        for step in range(1, self.max_steps + 1):
            if verbose:
                print(f"\n--- Step {step}/{self.max_steps} ---")

            # Call LLM
            response = self._call_llm(messages)

            if verbose:
                print(response)

            # Parse response
            parsed = self._parse_response(response)

            # Check for final answer
            if parsed["final_answer"]:
                if verbose:
                    print(f"\n{'='*60}")
                    print(f"Final Answer: {parsed['final_answer']}")
                    print(f"Steps taken: {step}")
                    print(f"{'='*60}")
                return parsed["final_answer"]

            # Validate action
            if not parsed["action"]:
                messages.append({"role": "assistant", "content": response})
                messages.append({
                    "role": "user",
                    "content": "Please respond with either an Action to use a tool, "
                               "or a Final Answer if you're done."
                })
                continue

            # Check for loops
            if self._check_loop(parsed["action"], parsed["action_input"] or {}):
                messages.append({"role": "assistant", "content": response})
                messages.append({
                    "role": "user",
                    "content": f"You already tried {parsed['action']} with these "
                               f"same inputs. Please try a DIFFERENT approach."
                })
                if verbose:
                    print("⚠ Loop detected! Forcing different approach.")
                continue

            # Execute tool
            observation = self._execute_tool(
                parsed["action"],
                parsed["action_input"] or {}
```

```
            )

            if verbose:
                print(f"Observation: {observation}")

            # Add to conversation
            self.scratchpad.append({
                "step": step,
                "thought": parsed["thought"],
                "action": parsed["action"],
                "action_input": parsed["action_input"],
                "observation": observation
            })

            messages.append({"role": "assistant", "content": response})
            messages.append({
                "role": "user",
                "content": f"Observation: {observation}"
            })

        # Max steps reached
        return ("I was unable to fully answer the question within the "
                f"maximum {self.max_steps} steps. Here's what I found so far: "
                + self._summarize_scratchpad())

    def _call_llm(self, messages: list) -> str:
        """Call the LLM. Override this for different providers."""
        import openai

        response = openai.chat.completions.create(
            model=self.model,
            messages=messages,
            temperature=0.1,
            max_tokens=500,
            stop=["Observation:"]  # Stop before observation (we provide it)
        )
        return response.choices[0].message.content.strip()

    def _summarize_scratchpad(self) -> str:
        """Summarize what the agent found so far."""
        if not self.scratchpad:
            return "No information gathered."

        summary = []
        for entry in self.scratchpad:
            summary.append(f"- Used {entry['action']}: {entry['observation'][:100]}")
        return "\n".join(summary)


# ============================================================
# SECTION 3: STANDALONE VERSION (No API key needed)
# ============================================================

class StandaloneReActAgent:
    """
    A ReAct agent that works WITHOUT an LLM API.
```

```
    Uses rule-based reasoning for practice/learning.

    This demonstrates the ReAct pattern clearly without
    requiring any API keys or external services.
    """

    def __init__(self, tools: dict, max_steps: int = 10):
        self.tools = tools
        self.max_steps = max_steps
        self.scratchpad = []
        self.action_history = []

    def run(self, question: str, plan: list = None) -> str:
        """
        Run the agent with a predefined plan.

        Args:
            question: User's question
            plan: List of (action, action_input) tuples
                  If None, agent tries to create one from the question
        """
        self.scratchpad = []
        self.action_history = []

        print(f"\n{'='*60}")
        print(f"Question: {question}")
        print(f"{'='*60}")

        if plan is None:
            plan = self._simple_planner(question)

        print(f"\nPlan: {len(plan)} steps")
        for i, (action, inputs) in enumerate(plan, 1):
            print(f"  {i}. {action}({json.dumps(inputs)})")

        # Execute plan
        results = []
        for step, (action, action_input) in enumerate(plan, 1):
            print(f"\n--- Step {step} ---")
            print(f"Thought: Executing step {step} of the plan")
            print(f"Action: {action}")
            print(f"Action Input: {json.dumps(action_input)}")

            # Check for loops
            action_key = f"{action}:{json.dumps(action_input, sort_keys=True)}"
            if action_key in self.action_history:
                print("⚠ Loop detected! Skipping duplicate action.")
                continue
            self.action_history.append(action_key)

            # Execute
            if action not in self.tools:
                observation = f"Error: Tool '{action}' not found"
            else:
                try:
                    observation = self.tools[action].function(action_input)
```

```
                except Exception as e:
                    observation = f"Error: {e}"

            print(f"Observation: {observation}")
            results.append({"action": action, "result": observation})

            self.scratchpad.append({
                "step": step,
                "action": action,
                "input": action_input,
                "observation": observation
            })

        # Compile final answer
        answer = self._compile_answer(question, results)
        print(f"\n{'='*60}")
        print(f"Final Answer: {answer}")
        print(f"{'='*60}")
        return answer

    def _simple_planner(self, question: str) -> list:
        """Create a basic plan from keywords in the question."""
        plan = []
        q = question.lower()

        # Detect calculation needs
        if any(op in q for op in ["+", "-", "*", "/", "calculate", "compute", "sum"]):
            # Extract math expression
            expr = re.findall(r'[\d\s+\-*/().]+', question)
            if expr:
                plan.append(("calculator", {"expression": expr[0].strip()}))

        # Detect weather needs
        if "weather" in q:
            cities = ["tokyo", "new york", "london", "paris", "sydney"]
            for city in cities:
                if city in q:
                    plan.append(("get_weather", {"city": city}))

        # Detect search needs
        if any(w in q for w in ["who", "what", "population", "creator", "largest"]):
            plan.append(("search", {"query": question}))

        if not plan:
            plan.append(("search", {"query": question}))

        return plan

    def _compile_answer(self, question: str, results: list) -> str:
        """Compile results into a final answer."""
        parts = []
        for r in results:
            if "Error" not in r["result"]:
                parts.append(r["result"])

        if parts:
```

```
            return " | ".join(parts)
        return "Could not find an answer to your question."


# ============================================================
# SECTION 4: RUN THE AGENT
# ============================================================

def demo_standalone():
    """Demo the standalone agent (no API key needed)."""
    agent = StandaloneReActAgent(tools=TOOLS)

    # Test 1: Math calculation
    agent.run(
        "What is 47 * 83 + 156?",
        plan=[
            ("calculator", {"expression": "47 * 83"}),
            ("calculator", {"expression": "3901 + 156"}),
        ]
    )

    print("\n" + "="*60 + "\n")

    # Test 2: Weather query
    agent.run(
        "What's the weather in Tokyo?",
        plan=[
            ("get_weather", {"city": "Tokyo"}),
        ]
    )

    print("\n" + "="*60 + "\n")

    # Test 3: Multi-tool task
    agent.run(
        "Compare the weather in Tokyo and London, and calculate "
        "the temperature difference.",
        plan=[
            ("get_weather", {"city": "Tokyo"}),
            ("get_weather", {"city": "London"}),
            ("calculator", {"expression": "22 - 12"}),
        ]
    )

    print("\n" + "="*60 + "\n")

    # Test 4: Search + calculation
    agent.run(
        "What is the population of Tokyo?",
        plan=[
            ("search", {"query": "population tokyo"}),
        ]
    )

    print("\n" + "="*60 + "\n")
```

```
    # Test 5: Auto-plan (no predefined plan)
    agent.run("What's the weather in Paris?")


def demo_with_llm():
    """Demo with a real LLM (requires OPENAI_API_KEY)."""
    agent = ReActAgent(tools=TOOLS, model="gpt-4", max_steps=10)

    # Test complex multi-step reasoning
    answer = agent.run(
        "What is the weather in Tokyo? If the temperature is above 20°C, "
        "calculate how many degrees above 20 it is, then tell me the result."
    )
    print(f"\nResult: {answer}")


if __name__ == "__main__":
    print("Running standalone demo (no API key needed)...")
    print("=" * 60)
    demo_standalone()

    # Uncomment to run with real LLM:
    # demo_with_llm()
```

Exercise: Extend the Agent

Try these modifications to deepen your understanding:

```
1. ADD A NEW TOOL
   Create a "unit_converter" tool that converts between:
   - Celsius ↔ Fahrenheit
   - Kilometers ↔ Miles
   - Kilograms ↔ Pounds
   Add it to the TOOLS dict with proper schema.

2. ADD EPISODIC MEMORY
   Create a simple log file that records:
   - Each task the agent received
   - What tools it used
   - Whether it succeeded
   Load this log at startup so the agent can learn from past runs.

3. ADD A TIMEOUT
   If a tool takes more than 5 seconds, return an error
   and let the agent try a different approach.

4. ADD CONTEXT WINDOW MANAGEMENT
   When the scratchpad gets too long (e.g., > 2000 tokens),
   summarize earlier steps into a compact format.
```

✓ Day 3 Deliverable Checklist

- [ ] Built a complete ReAct agent with at least 3 tools
- [ ] Agent can plan multi-step reasoning
- [ ] Agent detects and avoids action loops
- [ ] Agent validates tool names and arguments
- [ ] Agent handles tool errors gracefully
- [ ] Tested with at least 3 different types of questions
- [ ] Can explain: What is the ReAct pattern and why is it effective?
- [ ] Can explain: What are the three types of agent memory?
- [ ] Can explain: What are common agent failure modes and how to prevent them?

Day 4 — Planner/Executor Architecture

> 🎯 **Goal**: Build a proper planning agent capable of decomposing tasks.

> ⏱️ **Time**: 1–2h theory + 2–3h practice

> 📦 **Deliverable**: Working Planner/Executor system with a Critic loop

---

Table of Contents

1. Why Planning Matters
2. Task Decomposition
3. Planner → Executor Pipeline
4. Self-Correction: The Critic Loop
5. Guardrails
6. Practice: Build the Full System

Why Planning Matters

Day 3's ReAct agent decides **one step at a time**. This works for simple tasks but fails for complex ones:

```
ReAct (step-by-step):
  "Build a web scraper that extracts prices from 3 e-commerce sites"

  Step 1: Hmm, let me start by scraping Amazon...
  Step 2: OK, got Amazon data. Now let me try eBay...
  Step 3: eBay has a different format... let me figure it out...
  Step 4: Done with eBay. Now what was the third site?  ← Lost track!
  Step 5: Oh wait, I should have extracted PRICES, not just HTML...

  Problem: No upfront plan → wasted steps, forgotten requirements

Planner/Executor (plan-first):
  PLANNER generates:
    Step 1: Identify 3 target e-commerce sites
    Step 2: For each site, determine the HTML structure for prices
    Step 3: Write extraction logic for each site
    Step 4: Run extractors and collect prices
    Step 5: Format results as comparison table
    Step 6: Validate all 3 sites returned data

  EXECUTOR runs each step.
  CRITIC validates after each step.

  Result: Organized, complete, no wasted effort.
```

When to Use Planning vs ReAct

Task Type	Use ReAct	Use Planner/Exec
Simple Q&A	✓	
Single tool lookup	✓	
2-3 step tasks	✓	
Multi-step research		✓
Tasks with dependencies		✓
Code generation + test		✓
Document analysis		✓
Complex data pipelines		✓

Task Decomposition

Task decomposition is the art of breaking a complex task into smaller, manageable subtasks.

Decomposition Strategies

```
Strategy 1: SEQUENTIAL DECOMPOSITION
  Break into ordered steps where each depends on the previous.

  Task: "Analyze our competitors' pricing"
  Decomposition:
    1. Identify top 5 competitors  (independent)
    2. Collect pricing data         (depends on 1)
    3. Normalize pricing formats    (depends on 2)
    4. Create comparison matrix     (depends on 3)
    5. Write analysis report        (depends on 4)

Strategy 2: PARALLEL DECOMPOSITION
  Break into independent subtasks that can run simultaneously.

  Task: "Analyze weather in 3 cities"
  Decomposition:
    1a. Get weather for Tokyo     (independent)
    1b. Get weather for London    (independent)
    1c. Get weather for New York  (independent)
    2. Compare results            (depends on 1a, 1b, 1c)

Strategy 3: HIERARCHICAL DECOMPOSITION
  Break into categories, then sub-steps within each.

  Task: "Create a user registration system"
  Decomposition:
    1. Backend
       1.1 Design database schema
       1.2 Create API endpoints
       1.3 Add input validation
    2. Frontend
       2.1 Design registration form
       2.2 Add client-side validation
       2.3 Connect to API
    3. Testing
       3.1 Unit tests for API
       3.2 Integration tests
       3.3 E2E tests
```

The Decomposition Prompt

```
  PLANNER_PROMPT = """You are an expert task planner. Given a complex task,
  break it down into clear, actionable steps.

  ## Rules for Planning
  1. Each step must be concrete and actionable (not vague)
  2. Steps should be in dependency order
  3. Each step should be completable by a single tool or action
  4. Include validation steps to verify correctness
  5. Mark dependencies explicitly

  ## Available Tools
```

```
{tools_description}

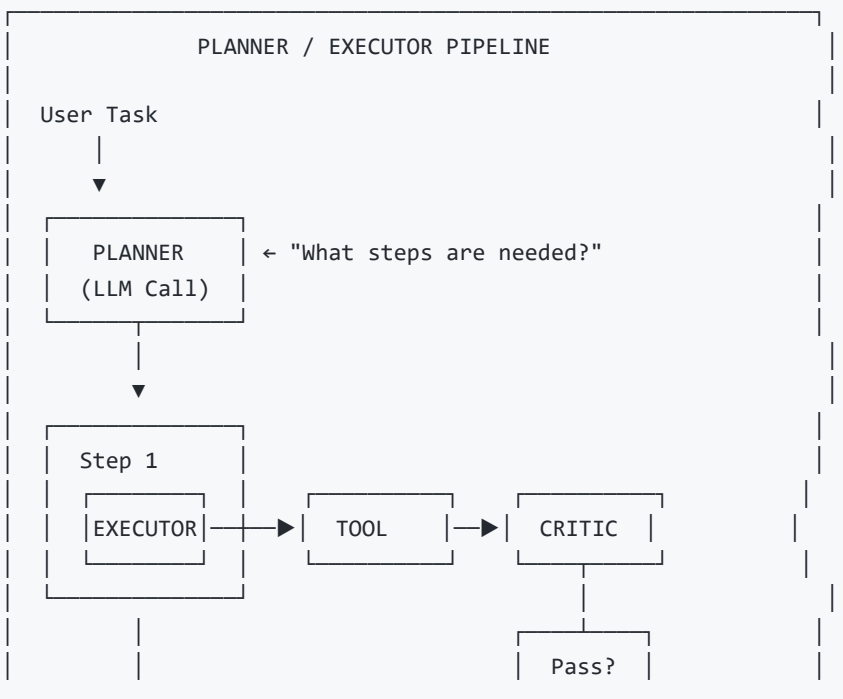
## Output Format
Respond with a JSON object:
{{
  "task_understanding": "Brief restatement of the task",
  "approach": "High-level approach in 1-2 sentences",
  "steps": [
    {{
      "id": 1,
      "description": "What to do",
      "tool": "tool_name or 'reasoning'",
      "inputs": {{}},
      "depends_on": [],
      "expected_output": "What success looks like",
      "validation": "How to verify this step succeeded"
    }}
  ],
  "success_criteria": "How to know the entire task is complete"
}}

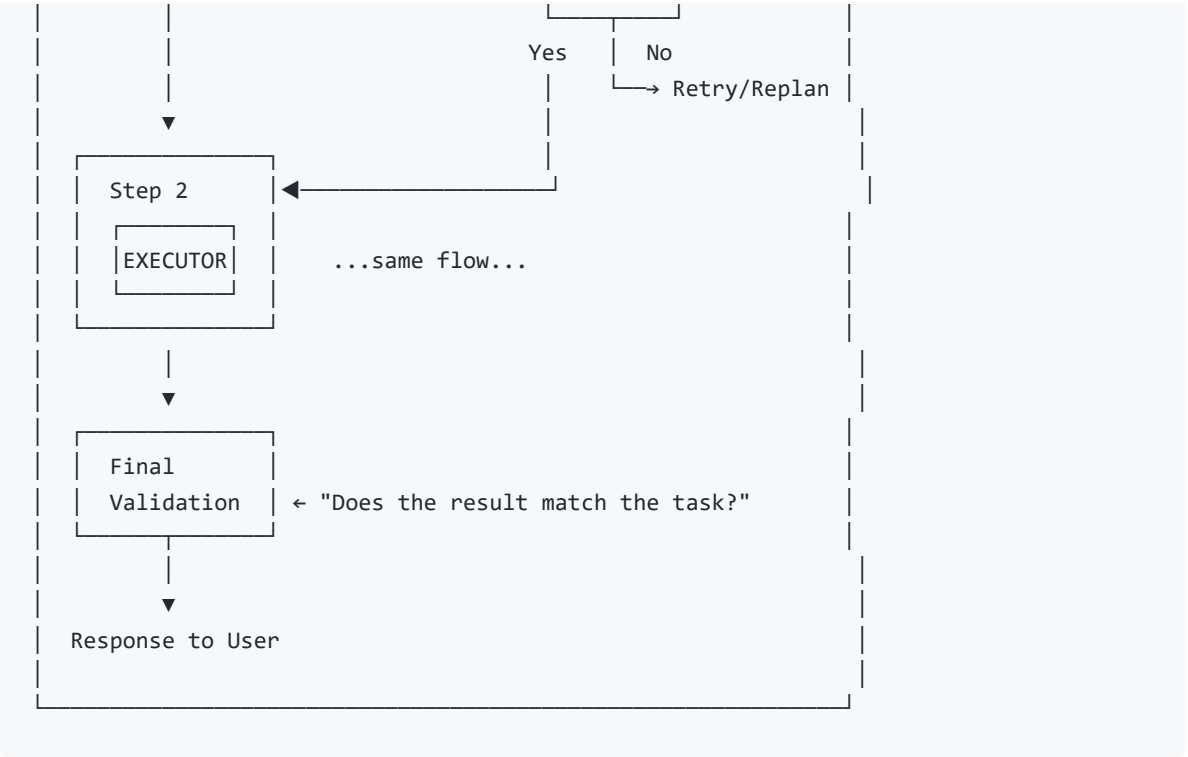
## Task
{task}

Plan:"""
```

Planner → Executor Pipeline

The Planner/Executor architecture separates **thinking** from **doing**:





Component Responsibilities

```
PLANNER:
  Input:  User's task + available tools
  Output: Ordered list of steps with dependencies
  LLM:    Needs strong reasoning (GPT-4, Claude 3)
  Temp:   Low (0.0-0.2) for consistent plans

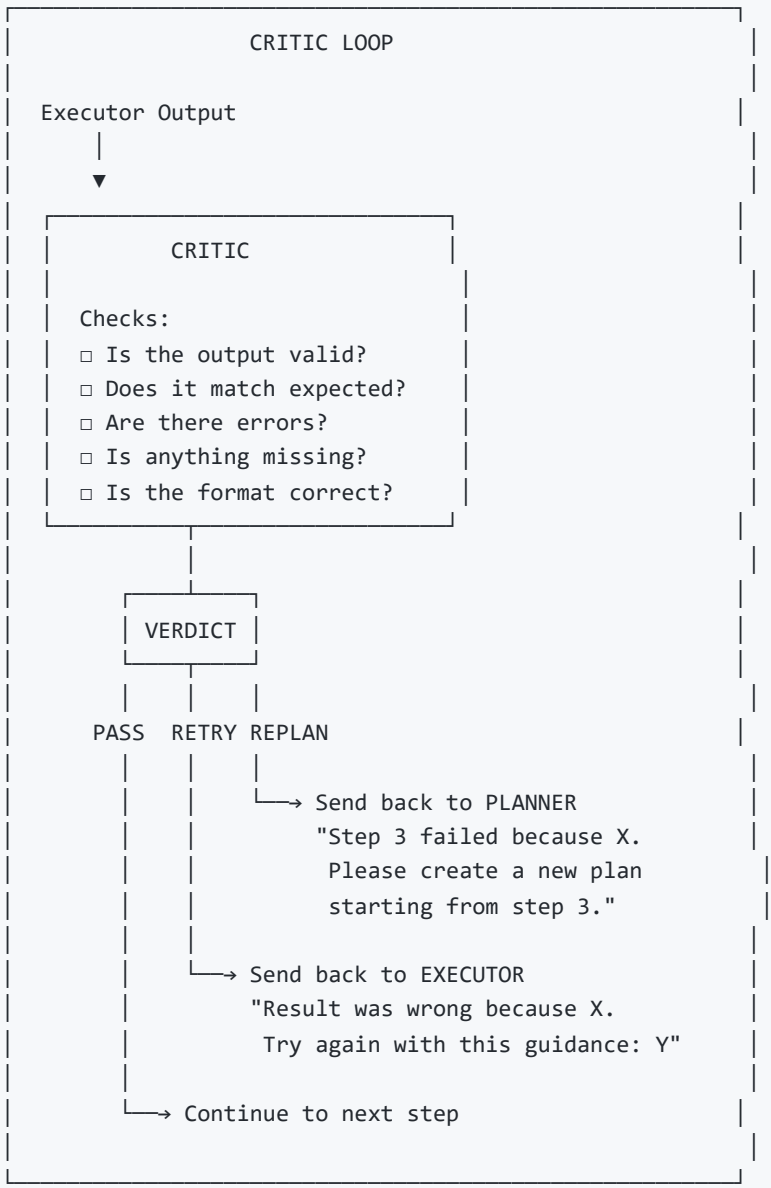
EXECUTOR:
  Input:  Single step from the plan + available tools
  Output: Step result + status (success/failure)
  LLM:    Can use lighter model (GPT-3.5, Llama 8B)
  Temp:   Low (0.0) for reliable execution

CRITIC:
  Input:  Step description + expected output + actual output
  Output: Pass/Fail + reasoning + suggestions
  LLM:    Needs good judgment (GPT-4, Claude 3)
  Temp:   Low (0.0) for consistent evaluation
```

Self-Correction: The Critic Loop

The Critic is what makes the system **reliable**. Without it, errors cascade through the pipeline.

The Critic Loop



Critic Prompt

```
CRITIC_PROMPT = """You are a quality assurance critic. Your job is to evaluate
whether a step was executed correctly.

## Step Details
- Description: {step_description}
- Expected Output: {expected_output}
- Validation Criteria: {validation_criteria}

## Actual Output
{actual_output}

## Evaluate

Respond with a JSON object:
```

```
  {{
    "verdict": "PASS" | "RETRY" | "REPLAN",
    "reasoning": "Why you made this decision",
    "issues": ["List of specific issues found"],
    "suggestions": ["How to fix or improve"],
    "confidence": 0.0-1.0
  }}

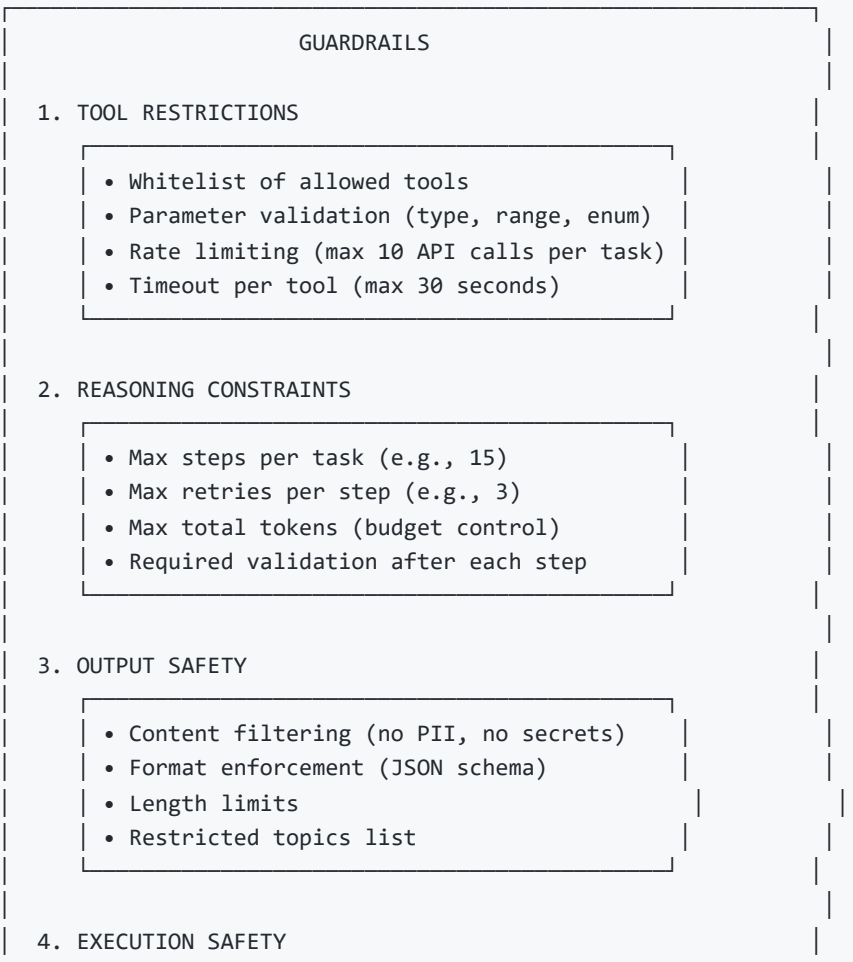
  Rules:
  - PASS: Output is correct and complete
  - RETRY: Output has minor issues that can be fixed by re-executing the step
  - REPLAN: Output is fundamentally wrong; the plan needs to change

  Be strict but fair. Minor formatting issues = PASS with suggestions.
  Missing data or wrong answers = RETRY or REPLAN."""
```

Guardrails

Guardrails prevent the agent from doing harmful or unintended things.

Types of Guardrails



```
│     ┌──────────────────────────────────────────────┐          │
│     │ • Sandboxed code execution                   │          │
│     │ • Read-only file access                      │          │
│     │ • No network calls without approval          │          │
│     │ • Audit logging of all actions               │          │
│     └──────────────────────────────────────────────┘          │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

Implementing Guardrails

```python
from dataclasses import dataclass, field
from typing import Optional
import time

@dataclass
class GuardrailConfig:
    """Configuration for agent guardrails."""
    max_steps: int = 15
    max_retries_per_step: int = 3
    max_total_retries: int = 10
    tool_timeout_seconds: float = 30.0
    max_tool_calls: int = 50
    allowed_tools: Optional[list] = None  # None = all tools allowed
    blocked_tools: list = field(default_factory=list)
    max_output_length: int = 5000
    require_critic: bool = True


class GuardrailEnforcer:
    """Enforces guardrails during agent execution."""

    def __init__(self, config: GuardrailConfig):
        self.config = config
        self.step_count = 0
        self.retry_counts = {}  # step_id → retry count
        self.total_retries = 0
        self.tool_call_count = 0
        self.audit_log = []

    def check_step_limit(self) -> tuple:
        """Check if we've exceeded the step limit."""
        self.step_count += 1
        if self.step_count > self.config.max_steps:
            return False, f"Maximum steps ({self.config.max_steps}) exceeded"
        return True, ""

    def check_retry_limit(self, step_id: str) -> tuple:
        """Check if we've exceeded retries for a step."""
        self.retry_counts[step_id] = self.retry_counts.get(step_id, 0) + 1
        self.total_retries += 1

        if self.retry_counts[step_id] > self.config.max_retries_per_step:
```

```
            return False, f"Max retries ({self.config.max_retries_per_step}) for ste

        if self.total_retries > self.config.max_total_retries:
            return False, f"Max total retries ({self.config.max_total_retries}) exce

        return True, ""

    def check_tool_allowed(self, tool_name: str) -> tuple:
        """Check if a tool is allowed."""
        if tool_name in self.config.blocked_tools:
            return False, f"Tool '{tool_name}' is blocked"

        if self.config.allowed_tools and tool_name not in self.config.allowed_tools:
            return False, f"Tool '{tool_name}' not in allowed list"

        self.tool_call_count += 1
        if self.tool_call_count > self.config.max_tool_calls:
            return False, f"Max tool calls ({self.config.max_tool_calls}) exceeded"

        return True, ""

    def execute_with_timeout(self, func, *args, **kwargs):
        """Execute a function with a timeout."""
        import threading

        result = [None]
        error = [None]

        def target():
            try:
                result[0] = func(*args, **kwargs)
            except Exception as e:
                error[0] = e

        thread = threading.Thread(target=target)
        thread.start()
        thread.join(timeout=self.config.tool_timeout_seconds)

        if thread.is_alive():
            return None, f"Tool timed out after {self.config.tool_timeout_seconds}s"

        if error[0]:
            return None, f"Tool error: {error[0]}"

        return result[0], ""

    def validate_output(self, output: str) -> tuple:
        """Validate output against guardrails."""
        if len(output) > self.config.max_output_length:
            return False, f"Output too long ({len(output)} > {self.config.max_output
        return True, ""

    def log_action(self, action: str, details: dict):
        """Log an action for audit purposes."""
        self.audit_log.append({
            "timestamp": time.time(),
```

```
            "action": action,
            "details": details
        })

    def get_summary(self) -> dict:
        """Get a summary of guardrail enforcement."""
        return {
            "total_steps": self.step_count,
            "total_retries": self.total_retries,
            "tool_calls": self.tool_call_count,
            "retry_counts": dict(self.retry_counts),
            "audit_log_size": len(self.audit_log)
        }
```

Practice: Build the Full System

Complete Planner/Executor/Critic Implementation

```
"""
Planner/Executor/Critic Agent System

Architecture:
   ┌─────────┐     ┌──────────┐     ┌──────────┐
   │ PLANNER │────▶│ EXECUTOR │────▶│  CRITIC  │
   │         │     │          │     │          │
   │ Breaks  │     │ Runs     │     │ Validates│
   │ task into│     │ each     │     │ each     │
   │ steps   │     │ step     │     │ result   │
   └─────────┘     └──────────┘     └──────┬───┘
        ▲                                  │
        │          ┌──────────┐            │
        └──────────│ REPLAN   │◀───────────┘
                   │ if needed│     (on failure)
                   └──────────┘
"""
import json
import re
from dataclasses import dataclass
from typing import Optional


# ═══════════════════════════════════════════════════════════════
# DATA STRUCTURES
# ═══════════════════════════════════════════════════════════════

@dataclass
class PlanStep:
    """A single step in a plan."""
    id: int
```

```
    description: str
    tool: str  # tool name or "reasoning"
    inputs: dict
    depends_on: list
    expected_output: str
    validation: str
    status: str = "pending"  # pending, executing, passed, failed, skipped
    result: Optional[str] = None
    retry_count: int = 0


@dataclass
class CriticVerdict:
    """Critic's evaluation of a step."""
    verdict: str  # PASS, RETRY, REPLAN
    reasoning: str
    issues: list
    suggestions: list
    confidence: float


@dataclass
class Plan:
    """A complete execution plan."""
    task: str
    approach: str
    steps: list  # List[PlanStep]
    success_criteria: str
    version: int = 1


# ═══════════════════════════════════════════════════════════════
# TOOLS (reuse from Day 3 + additions)
# ═══════════════════════════════════════════════════════════════

import math

def calculator(expression: str) -> str:
    try:
        allowed = {"sqrt": math.sqrt, "pow": pow, "abs": abs,
                   "round": round, "min": min, "max": max}
        result = eval(expression, {"__builtins__": {}}, allowed)
        return str(result)
    except Exception as e:
        return f"Error: {e}"

def search(query: str) -> str:
    knowledge = {
        "python web frameworks": "Popular Python web frameworks: Django (full-featur
        "rest api best practices": "REST API best practices: Use nouns for resources
        "database comparison": "PostgreSQL: Best for complex queries, ACID complianc
        "authentication methods": "Authentication methods: JWT (stateless, scalable)
    }

    query_lower = query.lower()
    for key, value in knowledge.items():
```

```
            if any(word in query_lower for word in key.split()):
                return value

    return f"No results for '{query}'. Try: {', '.join(knowledge.keys())}"

def code_generator(spec: str) -> str:
    """Generate code based on a specification (simulated)."""
    if "hello world" in spec.lower():
        return 'def hello():\n    print("Hello, World!")\n\nhello()'
    if "fibonacci" in spec.lower():
        return ('def fibonacci(n):\n'
                '    if n <= 1:\n'
                '        return n\n'
                '    a, b = 0, 1\n'
                '    for _ in range(2, n + 1):\n'
                '        a, b = b, a + b\n'
                '    return b')
    if "api endpoint" in spec.lower() or "fastapi" in spec.lower():
        return ('from fastapi import FastAPI, HTTPException\n'
                'from pydantic import BaseModel\n\n'
                'app = FastAPI()\n\n'
                'class Item(BaseModel):\n'
                '    name: str\n'
                '    price: float\n\n'
                'items_db = []\n\n'
                '@app.post("/items")\n'
                'async def create_item(item: Item):\n'
                '    items_db.append(item)\n'
                '    return {"id": len(items_db), "item": item}\n\n'
                '@app.get("/items")\n'
                'async def list_items():\n'
                '    return items_db')
    return f"# Generated code for: {spec}\n# TODO: Implement"

def code_reviewer(code: str) -> str:
    """Review code for issues (simulated)."""
    issues = []

    if "eval(" in code:
        issues.append("CRITICAL: Use of eval() — potential code injection")
    if "password" in code.lower() and "=" in code:
        issues.append("HIGH: Possible hardcoded password")
    if "except:" in code or "except Exception:" in code:
        issues.append("MEDIUM: Broad exception handling — catch specific exceptions"
    if "# TODO" in code:
        issues.append("LOW: TODO comments found — incomplete implementation")
    if not issues:
        issues.append("No critical issues found. Code looks reasonable.")

    return json.dumps({"issues": issues, "count": len(issues)})


TOOLS = {
    "calculator": {"fn": calculator, "desc": "Evaluate math expressions"},
    "search": {"fn": search, "desc": "Search for information"},
    "code_generator": {"fn": code_generator, "desc": "Generate code from a specifica
```

```
    "code_reviewer": {"fn": code_reviewer, "desc": "Review code for issues"},
}


# ============================================================
# THE PLANNER
# ============================================================

class Planner:
    """
    Creates execution plans from task descriptions.

    In production, this would use an LLM. For learning,
    we use rule-based planning so you can run without API keys.
    """

    def create_plan(self, task: str) -> Plan:
        """Create a plan for the given task."""
        task_lower = task.lower()

        # Route to specialized planners based on task type
        if any(w in task_lower for w in ["build", "create", "implement", "write"]):
            return self._plan_build_task(task)
        elif any(w in task_lower for w in ["analyze", "compare", "evaluate"]):
            return self._plan_analysis_task(task)
        elif any(w in task_lower for w in ["calculate", "compute", "math"]):
            return self._plan_calculation_task(task)
        else:
            return self._plan_research_task(task)

    def replan(self, original_plan: Plan, failed_step: int,
               error: str) -> Plan:
        """Create a new plan when a step fails."""
        # Keep completed steps, redo from failed step
        completed = [s for s in original_plan.steps if s.status == "passed"]

        new_steps = list(completed)
        step_id = len(completed) + 1

        # Add recovery step
        new_steps.append(PlanStep(
            id=step_id,
            description=f"Recover from failure: {error}",
            tool="reasoning",
            inputs={"context": error},
            depends_on=[s.id for s in completed],
            expected_output="Alternative approach identified",
            validation="Recovery plan is actionable"
        ))

        # Add remaining steps with adjusted IDs
        for old_step in original_plan.steps:
            if old_step.status == "pending":
                step_id += 1
                new_steps.append(PlanStep(
                    id=step_id,
```

```
                    description=old_step.description,
                    tool=old_step.tool,
                    inputs=old_step.inputs,
                    depends_on=[step_id - 1],
                    expected_output=old_step.expected_output,
                    validation=old_step.validation
                ))

        return Plan(
            task=original_plan.task,
            approach=f"Revised plan after failure at step {failed_step}: {error}",
            steps=new_steps,
            success_criteria=original_plan.success_criteria,
            version=original_plan.version + 1
        )

    def _plan_build_task(self, task: str) -> Plan:
        """Plan for building/creating something."""
        return Plan(
            task=task,
            approach="Research → Design → Implement → Review → Validate",
            steps=[
                PlanStep(
                    id=1,
                    description="Research best practices and technologies",
                    tool="search",
                    inputs={"query": task},
                    depends_on=[],
                    expected_output="Relevant information about technologies and app
                    validation="Research results are relevant to the task"
                ),
                PlanStep(
                    id=2,
                    description="Generate the code based on requirements and researc
                    tool="code_generator",
                    inputs={"spec": task},
                    depends_on=[1],
                    expected_output="Working code that implements the requirements",
                    validation="Code is syntactically valid and addresses requiremen
                ),
                PlanStep(
                    id=3,
                    description="Review the generated code for issues",
                    tool="code_reviewer",
                    inputs={},  # Will be filled with step 2's output
                    depends_on=[2],
                    expected_output="Code review with no critical issues",
                    validation="No CRITICAL or HIGH severity issues"
                ),
                PlanStep(
                    id=4,
                    description="Compile final result with code and documentation",
                    tool="reasoning",
                    inputs={},
                    depends_on=[3],
                    expected_output="Complete deliverable with code and explanation"
```

```
                    validation="All parts of the task are addressed"
                ),
            ],
            success_criteria="Code is generated, reviewed, and has no critical issue
        )

    def _plan_analysis_task(self, task: str) -> Plan:
        """Plan for analysis/comparison tasks."""
        return Plan(
            task=task,
            approach="Gather data → Analyze → Compare → Summarize",
            steps=[
                PlanStep(
                    id=1,
                    description="Gather information about the subject",
                    tool="search",
                    inputs={"query": task},
                    depends_on=[],
                    expected_output="Comprehensive information gathered",
                    validation="Data is relevant and sufficient"
                ),
                PlanStep(
                    id=2,
                    description="Analyze and organize the findings",
                    tool="reasoning",
                    inputs={},
                    depends_on=[1],
                    expected_output="Organized analysis with key points",
                    validation="Analysis is logical and well-structured"
                ),
                PlanStep(
                    id=3,
                    description="Create final summary with conclusions",
                    tool="reasoning",
                    inputs={},
                    depends_on=[2],
                    expected_output="Clear summary answering the original question",
                    validation="Summary addresses all aspects of the task"
                ),
            ],
            success_criteria="Complete analysis with clear conclusions"
        )

    def _plan_calculation_task(self, task: str) -> Plan:
        """Plan for calculation tasks."""
        # Extract math expressions
        expressions = re.findall(r'[\d\s+\-*/().%]+', task)
        steps = []

        for i, expr in enumerate(expressions, 1):
            expr = expr.strip()
            if expr and any(c.isdigit() for c in expr):
                steps.append(PlanStep(
                    id=i,
                    description=f"Calculate: {expr}",
                    tool="calculator",
```

```
                    inputs={"expression": expr},
                    depends_on=[i-1] if i > 1 else [],
                    expected_output=f"Numeric result of {expr}",
                    validation="Result is a valid number"
                ))

        if not steps:
            steps.append(PlanStep(
                id=1,
                description="Parse and calculate the expression",
                tool="calculator",
                inputs={"expression": task},
                depends_on=[],
                expected_output="Numeric result",
                validation="Result is a valid number"
            ))

        return Plan(
            task=task,
            approach="Parse expressions → Calculate → Verify",
            steps=steps,
            success_criteria="All calculations completed with verified results"
        )

    def _plan_research_task(self, task: str) -> Plan:
        """Default plan for research tasks."""
        return Plan(
            task=task,
            approach="Search → Analyze → Answer",
            steps=[
                PlanStep(
                    id=1,
                    description=f"Search for information about: {task}",
                    tool="search",
                    inputs={"query": task},
                    depends_on=[],
                    expected_output="Relevant information found",
                    validation="Results are relevant to the question"
                ),
                PlanStep(
                    id=2,
                    description="Formulate answer from search results",
                    tool="reasoning",
                    inputs={},
                    depends_on=[1],
                    expected_output="Clear, complete answer",
                    validation="Answer addresses the original question"
                ),
            ],
            success_criteria="Question is fully answered with supporting information
        )


# ═══════════════════════════════════════════════════════════
# THE EXECUTOR
# ═══════════════════════════════════════════════════════════
```

```python
class Executor:
    """
    Executes individual steps from a plan.
    """

    def __init__(self, tools: dict):
        self.tools = tools
        self.context = {}  # Results from previous steps

    def execute_step(self, step: PlanStep) -> str:
        """Execute a single step and return the result."""
        step.status = "executing"

        if step.tool == "reasoning":
            # Reasoning steps compile context from previous steps
            return self._reason(step)

        if step.tool not in self.tools:
            return f"Error: Tool '{step.tool}' not available"

        # Prepare inputs (may reference previous step results)
        inputs = self._resolve_inputs(step)

        # Execute the tool
        try:
            tool_fn = self.tools[step.tool]["fn"]

            # Handle different tool signatures
            if step.tool == "code_reviewer" and not inputs.get("code"):
                # Use code from previous code_generator step
                prev_code = self._find_previous_code()
                if prev_code:
                    inputs["code"] = prev_code

            # Call the tool with the first relevant parameter
            if step.tool == "calculator":
                result = tool_fn(inputs.get("expression", str(inputs)))
            elif step.tool == "search":
                result = tool_fn(inputs.get("query", str(inputs)))
            elif step.tool == "code_generator":
                result = tool_fn(inputs.get("spec", str(inputs)))
            elif step.tool == "code_reviewer":
                result = tool_fn(inputs.get("code", str(inputs)))
            else:
                result = tool_fn(**inputs)

            self.context[step.id] = result
            return result

        except Exception as e:
            return f"Error executing {step.tool}: {e}"

    def _reason(self, step: PlanStep) -> str:
        """Perform a reasoning step using accumulated context."""
        context_summary = "\n".join(
```

```
                f"Step {sid}: {result[:200]}"
                for sid, result in self.context.items()
            )
            return f"Based on gathered information:\n{context_summary}\n\nConclusion for

        def _resolve_inputs(self, step: PlanStep) -> dict:
            """Resolve step inputs, potentially using previous results."""
            inputs = dict(step.inputs)

            # If inputs reference previous steps, substitute
            for key, value in inputs.items():
                if isinstance(value, str) and value.startswith("$step_"):
                    step_id = int(value.replace("$step_", ""))
                    if step_id in self.context:
                        inputs[key] = self.context[step_id]

            return inputs

        def _find_previous_code(self) -> Optional[str]:
            """Find code generated by a previous step."""
            for result in reversed(self.context.values()):
                if isinstance(result, str) and ("def " in result or "class " in result):
                    return result
            return None


    # ═══════════════════════════════════════════════════════════════
    # THE CRITIC
    # ═══════════════════════════════════════════════════════════════

    class Critic:
        """
        Evaluates step results for correctness and quality.
        """

        def evaluate(self, step: PlanStep, result: str) -> CriticVerdict:
            """Evaluate a step's result."""
            issues = []
            suggestions = []

            # Check for errors
            if result.startswith("Error:"):
                return CriticVerdict(
                    verdict="RETRY",
                    reasoning=f"Step returned an error: {result}",
                    issues=[result],
                    suggestions=["Fix the error and retry"],
                    confidence=0.9
                )

            # Check for empty results
            if not result or result.strip() == "":
                return CriticVerdict(
                    verdict="RETRY",
                    reasoning="Step produced empty output",
                    issues=["Empty result"],
```

```
            suggestions=["Verify inputs and retry"],
            confidence=0.8
        )

    # Check for "no results" from search
    if "no results" in result.lower() or "not found" in result.lower():
        return CriticVerdict(
            verdict="RETRY",
            reasoning="Search returned no results",
            issues=["No information found"],
            suggestions=["Try different search terms",
                         "Broaden the search query"],
            confidence=0.7
        )

    # Check code review results
    if step.tool == "code_reviewer":
        try:
            review = json.loads(result)
            critical = [i for i in review.get("issues", [])
                        if "CRITICAL" in i or "HIGH" in i]
            if critical:
                return CriticVerdict(
                    verdict="RETRY",
                    reasoning="Code has critical issues that need fixing",
                    issues=critical,
                    suggestions=["Fix the critical issues before proceeding"],
                    confidence=0.85
                )
        except json.JSONDecodeError:
            pass

    # Check calculator results
    if step.tool == "calculator":
        try:
            float(result)  # Verify it's a valid number
        except ValueError:
            issues.append(f"Calculator returned non-numeric result: {result}")
            return CriticVerdict(
                verdict="RETRY",
                reasoning="Calculator output is not a valid number",
                issues=issues,
                suggestions=["Check the expression syntax"],
                confidence=0.9
            )

    # General quality checks
    if len(result) < 10 and step.tool != "calculator":
        suggestions.append("Result seems brief — consider if more detail is need

    # Default: PASS
    return CriticVerdict(
        verdict="PASS",
        reasoning="Step produced valid, relevant output",
        issues=issues,
        suggestions=suggestions,
```

```
            confidence=0.8
        )


# ═══════════════════════════════════════════════════════════════
# THE ORCHESTRATOR
# ═══════════════════════════════════════════════════════════════

class PlannerExecutorSystem:
    """
    Orchestrates the Planner, Executor, and Critic.

    ┌─────────────────────────────────────────────────┐
    │              ORCHESTRATOR                       │
    │                                                 │
    │  Task ──▶ PLANNER ──▶ Plan                      │
    │                        │                        │
    │              ┌─────────┘                        │
    │              │                                  │
    │              ▼                                  │
    │         For each step:                          │
    │              │                                  │
    │              ├──▶ EXECUTOR ──▶ Result           │
    │              │                    │             │
    │              │              CRITIC ──▶ Verdict  │
    │              │                    │             │
    │              │    PASS ◀──────────┤             │
    │              │    RETRY ◀─────────┤             │
    │              │    REPLAN ◀────────┘             │
    │              │                                  │
    │              └──▶ Next step or Replan           │
    │                                                 │
    │  All steps done ──▶ Final Result                │
    └─────────────────────────────────────────────────┘
    """

    def __init__(self, tools: dict, max_retries: int = 3,
                 max_replans: int = 2):
        self.planner = Planner()
        self.executor = Executor(tools)
        self.critic = Critic()
        self.max_retries = max_retries
        self.max_replans = max_replans

    def run(self, task: str, verbose: bool = True) -> dict:
        """
        Run the full Planner/Executor/Critic pipeline.

        Returns:
            dict with keys: task, plan, results, final_answer, stats
        """
        replan_count = 0

        # Step 1: Create plan
        plan = self.planner.create_plan(task)
```

```
        if verbose:
            self._print_plan(plan)

        # Step 2: Execute plan with critic validation
        while replan_count <= self.max_replans:
            success, results = self._execute_plan(plan, verbose)

            if success:
                # Step 3: Compile final answer
                final = self._compile_final_answer(task, plan, results)

                stats = {
                    "total_steps": len(plan.steps),
                    "replans": replan_count,
                    "total_retries": sum(s.retry_count for s in plan.steps),
                    "plan_version": plan.version
                }

                if verbose:
                    self._print_final(final, stats)

                return {
                    "task": task,
                    "plan": plan,
                    "results": results,
                    "final_answer": final,
                    "stats": stats
                }

            # Replan needed
            replan_count += 1
            if replan_count <= self.max_replans:
                failed_step = next(
                    (s for s in plan.steps if s.status == "failed"), None
                )
                if failed_step:
                    if verbose:
                        print(f"\n⚠ REPLANNING (attempt {replan_count}/{self.max_re
                        print(f"  Failed step: {failed_step.description}")

                    plan = self.planner.replan(
                        plan, failed_step.id,
                        failed_step.result or "Unknown error"
                    )
                    if verbose:
                        self._print_plan(plan)

        return {
            "task": task,
            "plan": plan,
            "results": {},
            "final_answer": "Failed to complete task after maximum replanning attemp
            "stats": {"replans": replan_count}
        }

    def _execute_plan(self, plan: Plan, verbose: bool) -> tuple:
```

```
        """Execute all steps in a plan. Returns (success, results)."""
        results = {}

        for step in plan.steps:
            if step.status in ("passed", "skipped"):
                continue

            # Check dependencies
            deps_met = all(
                any(s.id == dep and s.status == "passed"
                    for s in plan.steps)
                for dep in step.depends_on
            )

            if not deps_met and step.depends_on:
                step.status = "skipped"
                if verbose:
                    print(f"\n⏭️ Skipping step {step.id}: dependencies not met")
                continue

            # Execute with retry loop
            for attempt in range(self.max_retries + 1):
                if verbose:
                    retry_msg = f" (retry {attempt})" if attempt > 0 else ""
                    print(f"\n{'─'*50}")
                    print(f"📋 Step {step.id}{retry_msg}: {step.description}")
                    print(f"   Tool: {step.tool}")

                # Execute
                result = self.executor.execute_step(step)

                if verbose:
                    result_preview = result[:200] + "..." if len(result) > 200 else
                    print(f"   Result: {result_preview}")

                # Critic evaluation
                verdict = self.critic.evaluate(step, result)

                if verbose:
                    emoji = {"PASS": "✅", "RETRY": "🔄", "REPLAN": "❌"}
                    print(f"   Critic: {emoji.get(verdict.verdict, '?')} {verdict.ve
                    print(f"   Reason: {verdict.reasoning}")
                    if verdict.issues:
                        print(f"   Issues: {verdict.issues}")

                if verdict.verdict == "PASS":
                    step.status = "passed"
                    step.result = result
                    results[step.id] = result
                    break

                elif verdict.verdict == "RETRY":
                    step.retry_count += 1
                    if attempt == self.max_retries:
                        step.status = "failed"
                        step.result = result
```

```
                        if verbose:
                            print(f"    ❌ Max retries reached for step {step.id}")
                        return False, results

                elif verdict.verdict == "REPLAN":
                    step.status = "failed"
                    step.result = result
                    return False, results

        return True, results

    def _compile_final_answer(self, task: str, plan: Plan,
                              results: dict) -> str:
        """Compile step results into a final answer."""
        parts = [f"Task: {task}\n"]

        for step in plan.steps:
            if step.status == "passed" and step.id in results:
                parts.append(f"• {step.description}:")
                result_text = results[step.id]
                if len(result_text) > 300:
                    result_text = result_text[:300] + "..."
                parts.append(f"  {result_text}\n")

        return "\n".join(parts)

    def _print_plan(self, plan: Plan):
        """Pretty-print a plan."""
        print(f"\n{'='*60}")
        print(f"📝 PLAN (v{plan.version})")
        print(f"Task: {plan.task}")
        print(f"Approach: {plan.approach}")
        print(f"{'─'*60}")
        for step in plan.steps:
            deps = f" (after: {step.depends_on})" if step.depends_on else ""
            status = {"pending": "⬜", "passed": "✅", "failed": "❌",
                      "skipped": "⏭", "executing": "🔄"}.get(step.status, "?")
            print(f"  {status} Step {step.id}: {step.description}")
            print(f"       Tool: {step.tool}{deps}")
        print(f"Success criteria: {plan.success_criteria}")
        print(f"{'='*60}")

    def _print_final(self, final: str, stats: dict):
        """Pretty-print the final result."""
        print(f"\n{'='*60}")
        print(f"🎯 FINAL RESULT")
        print(f"{'─'*60}")
        print(final)
        print(f"{'─'*60}")
        print(f"Stats: {json.dumps(stats, indent=2)}")
        print(f"{'='*60}")


# ═══════════════════════════════════════════════════════════════
# DEMO
# ═══════════════════════════════════════════════════════════════
```

```
def demo():
    """Run demo tasks through the Planner/Executor/Critic system."""
    system = PlannerExecutorSystem(tools=TOOLS)

    print("\n" + "="*60)
    print(" DEMO 1: Build Task")
    print("="*60)
    system.run("Build a FastAPI endpoint for a simple CRUD API")

    print("\n\n" + "="*60)
    print(" DEMO 2: Research Task")
    print("="*60)
    system.run("What are the best Python web frameworks and why?")

    print("\n\n" + "="*60)
    print(" DEMO 3: Calculation Task")
    print("="*60)
    system.run("Calculate 25 * 47 + 330 / 11")

    print("\n\n" + "="*60)
    print(" DEMO 4: Analysis Task")
    print("="*60)
    system.run("Analyze and compare database options for a new project")


if __name__ == "__main__":
    demo()
```

Exercise: Extend the System

```
1. ADD LLM-BASED PLANNER
   Replace the rule-based Planner with an LLM call using
   the PLANNER_PROMPT from the theory section.
   Parse the JSON plan into PlanStep objects.

2. ADD PARALLEL EXECUTION
   Modify the executor to detect steps with no dependencies
   on each other and execute them in parallel (threading).

3. ADD PLAN VISUALIZATION
   Create a function that outputs the plan as an ASCII
   dependency graph:
     Step 1 ──▶ Step 2 ──▶ Step 4
                Step 3 ──▶ Step 4

4. ADD PERSISTENT EXECUTION LOG
   Save each execution to a JSON file with:
   - Task, plan, results, timing
   - Load previous executions to improve future planning
```

✓ Day 4 Deliverable Checklist

- [ ] Built a Planner that decomposes tasks into steps
- [ ] Built an Executor that runs each step using tools
- [ ] Built a Critic that validates step results
- [ ] System handles PASS / RETRY / REPLAN verdicts
- [ ] Guardrails prevent infinite loops and runaway execution
- [ ] Tested with at least 3 different task types
- [ ] Can explain: Why is plan-then-execute better than step-by-step for complex tasks?
- [ ] Can explain: What does the Critic check and why is it important?
- [ ] Can explain: What are guardrails and why are they necessary?

Day 5 — RAG + Vector Memory for Agents

> 🎯 **Goal**: Integrate long-term knowledge into the agent.

> ⏱️ **Time**: 1.5h theory + 3h practice

> 📦 **Deliverable**: Agent with working RAG + dynamic, long-term memory

---

Table of Contents

1. Why RAG?
2. Vector Retrieval Fundamentals
3. Chunking Strategies
4. Embedding Models
5. Hybrid Search
6. Context Window Management
7. Practice: Build a RAG Pipeline

Why RAG?

RAG = Retrieval-Augmented Generation. Instead of relying on the LLM's training data (which is frozen at a point in time), RAG lets the model **look up** information from external documents

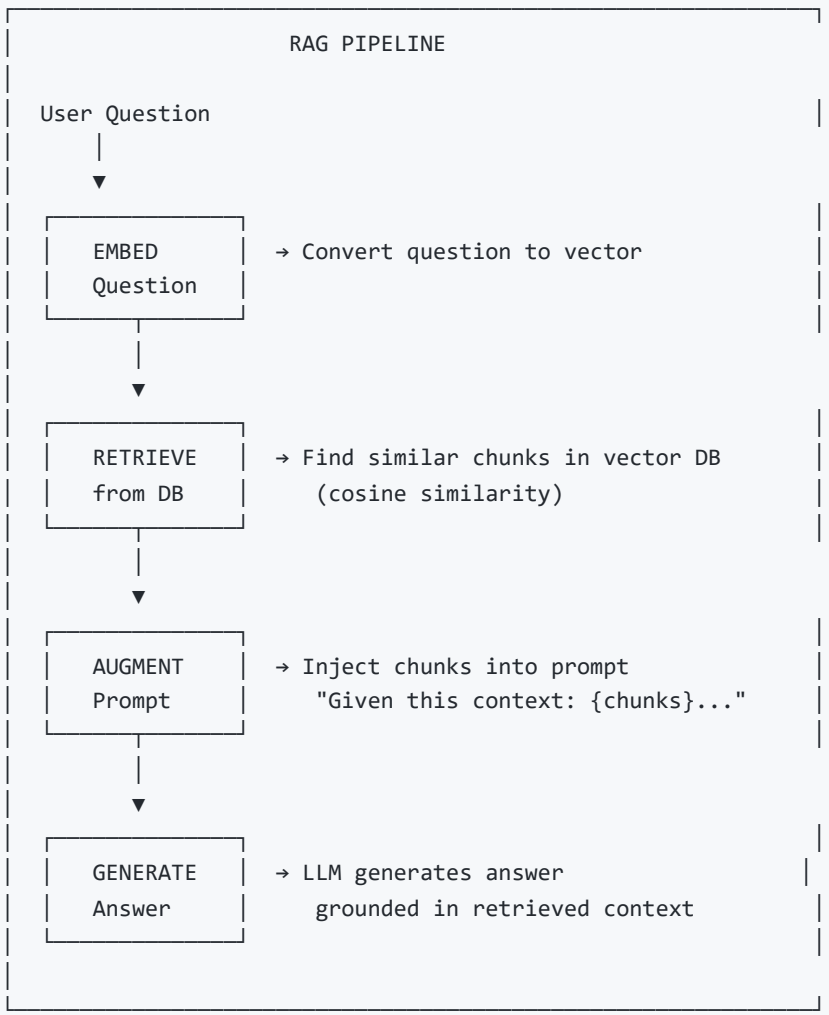
before answering.

```
Without RAG:
  Q: "What was our Q4 2024 revenue?"
  A: "I don't have access to your company's financial data."
      (or worse, hallucinated: "Your Q4 revenue was $12.5M")

With RAG:
  Q: "What was our Q4 2024 revenue?"

  1. RETRIEVE: Search company documents → find Q4 report
  2. AUGMENT:  Inject relevant chunks into the prompt
  3. GENERATE: LLM answers using the retrieved context

  A: "According to the Q4 2024 financial report, revenue was
      $18.3M, up 15% from Q3." ← Grounded in real data
```



RAG vs Fine-Tuning

	RAG	Fine-Tuning

```
├──────────────────┼──────────────────────┼──────────────────────┤
│ Data freshness   │ Real-time updates    │ Frozen at training   │
│ Cost             │ Low (just embeddings)│ High (GPU training)  │
│ Setup time       │ Hours                │ Days-weeks           │
│ Accuracy         │ High (grounded)      │ Variable             │
│ Hallucinations   │ Lower (has source)   │ Can still hallucinate│
│ Data volume      │ Any amount           │ Needs curated dataset│
│ Use case         │ Knowledge retrieval  │ Behavior change      │
│ Updatable        │ Add/remove docs live │ Retrain required     │
└──────────────────┴──────────────────────┴──────────────────────┘

When to use RAG:
  ✓ Company documents, knowledge bases
  ✓ Data that changes frequently
  ✓ When you need source citations
  ✓ Multi-domain knowledge

When to use Fine-Tuning:
  ✓ Specific output style/format
  ✓ Domain-specific reasoning (medical, legal)
  ✓ When RAG context window is insufficient
```

Vector Retrieval Fundamentals

What Are Embeddings?

Embeddings convert text into **dense numerical vectors** that capture semantic meaning. Similar texts → similar vectors.

```
Text: "The cat sat on the mat"     → [0.23, -0.15, 0.87, 0.42, ...]
Text: "A kitten rested on the rug" → [0.21, -0.13, 0.85, 0.39, ...]
                                      ↑ Very similar! (synonyms)

Text: "Stock prices rose sharply"   → [-0.45, 0.72, -0.11, 0.33, ...]
                                      ↑ Very different (different topic)

Key properties:
  - Dimension: typically 384–1536 floats
  - Semantic similarity → vector proximity
  - Language-agnostic (similar concepts in different languages map closely)
```

Cosine Similarity

The standard measure for comparing embeddings:

```
                    A · B          Σ(Ai × Bi)
cos(θ) = ──────────────────── = ──────────────────
```

```
            ||A|| × ||B||      √Σ(Ai²) × √Σ(Bi²)

Values:
   1.0 = identical meaning
   0.0 = completely unrelated
  -1.0 = opposite meaning (rare in practice)

Example:
  A = [0.23, -0.15, 0.87]   ("cat on mat")
  B = [0.21, -0.13, 0.85]   ("kitten on rug")

  A·B = (0.23×0.21) + (-0.15×-0.13) + (0.87×0.85) = 0.0483 + 0.0195 + 0.7395 = 0.8073
  ||A|| = √(0.0529 + 0.0225 + 0.7569) = √0.8323 = 0.9123
  ||B|| = √(0.0441 + 0.0169 + 0.7225) = √0.7835 = 0.8852

  cos(θ) = 0.8073 / (0.9123 × 0.8852) = 0.8073 / 0.8076 = 0.9996

  Very similar! ✓
```

Approximate Nearest Neighbor (ANN) Algorithms

Exact nearest-neighbor search over millions of vectors is too slow (O(n)). ANN algorithms trade slight accuracy for massive speed gains:

```
┌──────────────────┬──────────────────────────────────────────────┐
│ Algorithm        │ How It Works                                 │
├──────────────────┼──────────────────────────────────────────────┤
│ HNSW             │ Hierarchical graph — traverse layers from    │
│ (Hierarchical    │ coarse to fine. Most popular for in-memory.  │
│ Navigable        │ Used by: ChromaDB, Qdrant, pgvector          │
│ Small World)     │ Speed: ~O(log n)                             │
├──────────────────┼──────────────────────────────────────────────┤
│ IVF              │ Clusters vectors into cells. Search only     │
│ (Inverted        │ nearby cells. Good for very large datasets.  │
│ File Index)      │ Used by: FAISS                               │
│                  │ Speed: ~O(√n)                                │
├──────────────────┼──────────────────────────────────────────────┤
│ Product          │ Compresses vectors by splitting into         │
│ Quantization     │ sub-vectors and quantizing. Saves memory.    │
│ (PQ)             │ Often combined with IVF.                     │
│                  │ Used by: FAISS (IVF-PQ)                      │
└──────────────────┴──────────────────────────────────────────────┘

For most use cases (< 10M vectors), HNSW is the best choice.
```

Chunking Strategies

Before embedding documents, you must split them into **chunks** — smaller pieces that fit the embedding model's context window and provide focused, retrievable units.

Why Chunking Matters

```
Problem: Your document is 50 pages long.
  - Can't embed the whole thing (embedding models have limits: 512-8K tokens)
  - Even if you could, one giant vector wouldn't be specific enough
  - Retrieving the whole doc wastes context window on irrelevant parts

Solution: Split into chunks, embed each separately.
  50-page doc → 200 chunks → 200 vectors
  Query matches the 3 most relevant chunks → focused, specific context
```

Strategy 1: Fixed-Size Chunking

```
Split text into chunks of N tokens/characters with overlap.

Parameters:
  chunk_size: 500 tokens
  overlap: 50 tokens (10% overlap between consecutive chunks)

Document: "AAAA BBBB CCCC DDDD EEEE FFFF GGGG HHHH IIII JJJJ"

Chunk 1: "AAAA BBBB CCCC DDDD EEEE"
Chunk 2: "DDDD EEEE FFFF GGGG HHHH"   ← overlaps with chunk 1
Chunk 3: "GGGG HHHH IIII JJJJ"         ← overlaps with chunk 2

Why overlap?
  Without overlap, a sentence split across two chunks is lost.
  "The deadline is December 31" → "The deadline is" | "December 31"
  Neither chunk has the full information!
  With overlap, the full sentence appears in at least one chunk.

Pros: Simple, consistent chunk sizes
Cons: May split mid-sentence, mid-paragraph, mid-thought
```

Strategy 2: Recursive Character Splitting

```
Split by natural boundaries, recursively trying smaller separators.

Separator hierarchy:
  1. "\n\n" (paragraph boundaries)
  2. "\n" (line boundaries)
  3. ". " (sentence boundaries)
  4. " " (word boundaries)
  5. "" (character-level, last resort)

Algorithm:
  1. Try splitting by "\n\n"
  2. If any chunk > max_size, split that chunk by "\n"
  3. If still > max_size, split by ". "
  4. Continue down the hierarchy
```

```
Result: Chunks respect natural text boundaries
  Paragraph-level chunks when possible
  Sentence-level when paragraphs are too long

Pros: Preserves semantic units
Cons: Variable chunk sizes
```

Strategy 3: Semantic Chunking

```
Split based on topic/meaning changes, not character boundaries.

Algorithm:
  1. Split into sentences
  2. Embed each sentence
  3. Compare adjacent sentence embeddings
  4. When similarity drops below threshold → new chunk

Sentences:  S1    S2    S3    S4    S5    S6    S7    S8
Similarity:    0.9   0.85  0.82  0.3   0.88  0.91  0.4
                                 ↑                   ↑
                              Topic change!       Topic change!

Chunks: [S1,S2,S3,S4] | [S5,S6,S7] | [S8]

Pros: Best semantic coherence
Cons: Requires embedding each sentence (slow, expensive)
      Variable chunk sizes
```

Chunking Comparison

```
┌──────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│ Strategy     │ Quality      │ Speed        │ Consistency  │ Best For     │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Fixed-size   │ Medium       │ Very fast    │ Uniform size │ Large vol    │
│ Recursive    │ Good         │ Fast         │ Variable     │ General      │
│ Semantic     │ Best         │ Slow         │ Variable     │ Quality      │
└──────────────┴──────────────┴──────────────┴──────────────┴──────────────┘

Recommended defaults:
  chunk_size: 500 tokens (~2000 characters)
  overlap: 50-100 tokens (10-20%)
  strategy: Recursive (best balance)
```

Embedding Models

Popular Embedding Models

```
┌──────────────────────────┬──────┬──────┬──────────────────────────┐
│ Model                    │ Dims │ Max  │ Notes                    │
│                          │      │ Tok  │                          │
├──────────────────────────┼──────┼──────┼──────────────────────────┤
│ text-embedding-3-small   │ 1536 │ 8191 │ OpenAI, cheap, fast      │
│ text-embedding-3-large   │ 3072 │ 8191 │ OpenAI, best quality     │
│ text-embedding-ada-002   │ 1536 │ 8191 │ OpenAI, legacy           │
│ all-MiniLM-L6-v2         │ 384  │ 256  │ Free, local, fast        │
│ all-mpnet-base-v2        │ 768  │ 512  │ Free, local, better      │
│ bge-large-en-v1.5        │ 1024 │ 512  │ Free, local, top tier    │
│ nomic-embed-text-v1.5    │ 768  │ 8192 │ Free, long context       │
│ Cohere embed-english-v3  │ 1024 │ 512  │ API, good quality        │
└──────────────────────────┴──────┴──────┴──────────────────────────┘

Choosing a model:
  - Need free/local?  → all-MiniLM-L6-v2 (fastest) or bge-large (best)
  - Need API?         → text-embedding-3-small (best value)
  - Need max quality? → text-embedding-3-large
  - Need long chunks? → nomic-embed-text-v1.5
```

Embedding Code Example

```
# Option A: OpenAI embeddings
import openai

def embed_openai(texts: list[str], model="text-embedding-3-small") -> list:
    """Embed texts using OpenAI API."""
    response = openai.embeddings.create(
        model=model,
        input=texts
    )
    return [item.embedding for item in response.data]

# Option B: Local embeddings (free, no API key)
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

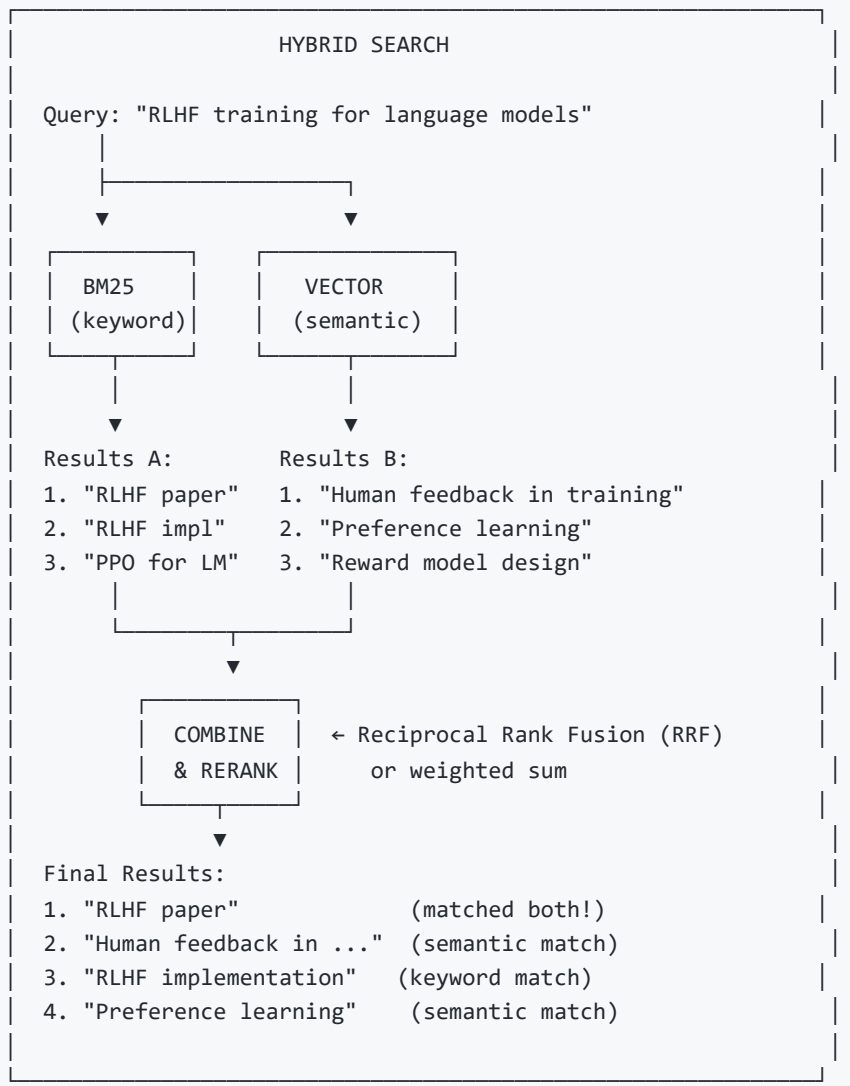
def embed_local(texts: list[str]) -> list:
    """Embed texts using local model."""
    embeddings = model.encode(texts, normalize_embeddings=True)
    return embeddings.tolist()

# Compare:
text = "How does vector search work?"
# OpenAI: 1536-dim vector, ~$0.00002 per text
# Local:  384-dim vector, free, ~10ms per text
```

Hybrid Search

Pure vector search misses exact keyword matches. Pure keyword search misses semantic similarity. **Hybrid search** combines both.

BM25 + Vector Search



BM25 Explained

```
BM25 is the classic information retrieval algorithm (used by Elasticsearch,
Lucene, etc.). It scores documents based on term frequency and rarity.

Score(D, Q) = Σ IDF(qi) × (f(qi, D) × (k1 + 1)) / (f(qi, D) + k1 × (1 - b + b × |D|/avgd

Where:
  IDF(qi) = log((N - n(qi) + 0.5) / (n(qi) + 0.5))  ← How rare is this word?
  f(qi, D) = frequency of word qi in document D
  |D| = document length
```

```
  avgdl = average document length
  k1 = 1.2 (tuning parameter, controls term frequency saturation)
  b = 0.75 (tuning parameter, controls length normalization)

In plain English:
  "Score higher if the document contains rare query words,
   but don't over-reward having the same word many times,
   and penalize very long documents slightly."

BM25 is great at:
  ✓ Exact keyword matches ("RLHF", "Kubernetes", error codes)
  ✓ Rare/specific terms (product names, acronyms)
  ✓ Fast and well-understood

BM25 is bad at:
  ✗ Synonyms ("car" won't match "automobile")
  ✗ Paraphrases ("how to cook" won't match "recipe preparation")
  ✗ Conceptual similarity
```

Reciprocal Rank Fusion (RRF)

```
Combines rankings from multiple search methods:

RRF_score(d) = Σ 1 / (k + rank_i(d))

Where k = 60 (constant, prevents outlier dominance)

Example:
  Document "RLHF paper":
    BM25 rank: 1,  Vector rank: 4
    RRF = 1/(60+1) + 1/(60+4) = 0.0164 + 0.0156 = 0.0320

  Document "Preference learning":
    BM25 rank: 8,  Vector rank: 2
    RRF = 1/(60+8) + 1/(60+2) = 0.0147 + 0.0161 = 0.0308

  "RLHF paper" scores higher because it ranks well in BOTH methods.
```

Context Window Management

The retrieved chunks must fit in the LLM's context window along with the system prompt, user query, and room for the response.

```
┌─────────────────────────────────────────────────────────┐
│              CONTEXT WINDOW BUDGET                      │
│                                                         │
│  Total context: 8,192 tokens (Llama 3 8B example)       │
│                                                         │
```



Strategies for Managing Context

```
1. LIMIT CHUNKS: Retrieve only top-K (e.g., K=3-5)
   More chunks = more context but also more noise

2. RERANK: Retrieve top-20, rerank with a cross-encoder,
   keep top-5. Better quality than just top-5.

3. SUMMARIZE: If chunks are long, summarize them before
   injecting. Trades detail for fitting more information.

4. MAP-REDUCE: For very large retrievals:
   - Map: Ask LLM to extract relevant info from each chunk
   - Reduce: Combine extractions into final answer

5. SLIDING WINDOW: For conversation history:
   - Keep last N turns
   - Summarize older turns into a compact paragraph
```

Practice: Build a RAG Pipeline

Complete RAG Implementation

```
"""
RAG Pipeline — Complete implementation with chunking,
embedding, retrieval, and grounding evaluation.

Architecture:
  Documents → Chunker → Embedder → Vector Store
                                        |
  User Query → Embedder → Similarity Search → Top-K Chunks
                                                  |
                                            Prompt Builder → LLM → Answer
                                                  |
                                            Grounding Evaluator → Score
"""
import json
import math
import re
import hashlib
from dataclasses import dataclass, field
from typing import Optional


# ═══════════════════════════════════════════════════════════
# SECTION 1: CHUNKING
# ═══════════════════════════════════════════════════════════

@dataclass
class Chunk:
    """A chunk of text with metadata."""
    id: str
    text: str
    source: str  # Document name/path
    index: int   # Position in document
    metadata: dict = field(default_factory=dict)
    embedding: list = field(default_factory=list)


class RecursiveChunker:
    """
    Splits text into chunks using recursive character splitting.

    Tries to split at natural boundaries:
    1. Double newlines (paragraphs)
    2. Single newlines
    3. Sentences (periods)
    4. Words (spaces)
    """

    def __init__(self, chunk_size: int = 500, chunk_overlap: int = 50):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.separators = ["\n\n", "\n", ". ", " ", ""]

    def chunk_text(self, text: str, source: str = "unknown") -> list:
        """Split text into overlapping chunks."""
```

```
        raw_chunks = self._split_recursive(text, self.separators)

        # Merge small chunks and add overlap
        merged = self._merge_with_overlap(raw_chunks)

        chunks = []
        for i, chunk_text in enumerate(merged):
            chunk_id = hashlib.md5(
                f"{source}:{i}:{chunk_text[:50]}".encode()
            ).hexdigest()[:12]

            chunks.append(Chunk(
                id=chunk_id,
                text=chunk_text.strip(),
                source=source,
                index=i,
                metadata={
                    "char_count": len(chunk_text),
                    "word_count": len(chunk_text.split())
                }
            ))

        return chunks

    def _split_recursive(self, text: str, separators: list) -> list:
        """Recursively split text using separator hierarchy."""
        if not separators:
            return [text]

        sep = separators[0]
        remaining_seps = separators[1:]

        if sep == "":
            # Character-level split as last resort
            return [text[i:i+self.chunk_size]
                    for i in range(0, len(text), self.chunk_size)]

        parts = text.split(sep)

        result = []
        current = ""

        for part in parts:
            candidate = current + sep + part if current else part

            if len(candidate) <= self.chunk_size:
                current = candidate
            else:
                if current:
                    result.append(current)

                if len(part) > self.chunk_size:
                    # This part is too big — split with next separator
                    sub_parts = self._split_recursive(part, remaining_seps)
                    result.extend(sub_parts)
                    current = ""
```

```
                else:
                    current = part

        if current:
            result.append(current)

        return result

    def _merge_with_overlap(self, chunks: list) -> list:
        """Add overlap between consecutive chunks."""
        if len(chunks) <= 1:
            return chunks

        result = []
        for i, chunk in enumerate(chunks):
            if i > 0 and self.chunk_overlap > 0:
                # Get overlap from end of previous chunk
                prev = chunks[i-1]
                overlap_text = prev[-self.chunk_overlap:]
                # Find word boundary
                space_idx = overlap_text.find(' ')
                if space_idx > 0:
                    overlap_text = overlap_text[space_idx+1:]
                chunk = overlap_text + " " + chunk

            result.append(chunk)

        return result


# ═══════════════════════════════════════════════════════════════
# SECTION 2: VECTOR STORE (In-Memory)
# ═══════════════════════════════════════════════════════════════

class SimpleVectorStore:
    """
    A minimal in-memory vector store.

    Uses brute-force cosine similarity (fine for < 100K vectors).
    In production, use ChromaDB, Pinecone, FAISS, or pgvector.
    """

    def __init__(self):
        self.chunks: list[Chunk] = []

    def add(self, chunks: list):
        """Add chunks with embeddings to the store."""
        for chunk in chunks:
            if not chunk.embedding:
                raise ValueError(f"Chunk {chunk.id} has no embedding")
            self.chunks.append(chunk)

    def search(self, query_embedding: list, top_k: int = 5) -> list:
        """Find the top-K most similar chunks."""
        if not self.chunks:
            return []
```

```
        scored = []
        for chunk in self.chunks:
            score = self._cosine_similarity(query_embedding, chunk.embedding)
            scored.append((chunk, score))

        scored.sort(key=lambda x: x[1], reverse=True)
        return scored[:top_k]

    def _cosine_similarity(self, a: list, b: list) -> float:
        """Compute cosine similarity between two vectors."""
        dot_product = sum(x * y for x, y in zip(a, b))
        norm_a = math.sqrt(sum(x * x for x in a))
        norm_b = math.sqrt(sum(x * x for x in b))

        if norm_a == 0 or norm_b == 0:
            return 0.0

        return dot_product / (norm_a * norm_b)

    def __len__(self):
        return len(self.chunks)


# ═══════════════════════════════════════════════════════════════
# SECTION 3: BM25 (KEYWORD SEARCH)
# ═══════════════════════════════════════════════════════════════

class SimpleBM25:
    """
    Simple BM25 implementation for keyword-based retrieval.
    """

    def __init__(self, k1: float = 1.2, b: float = 0.75):
        self.k1 = k1
        self.b = b
        self.docs = []       # List of tokenized documents
        self.doc_chunks = [] # Corresponding chunks
        self.avgdl = 0       # Average document length
        self.idf = {}        # Inverse document frequency
        self.doc_count = 0

    def add(self, chunks: list):
        """Index chunks for BM25 search."""
        for chunk in chunks:
            tokens = self._tokenize(chunk.text)
            self.docs.append(tokens)
            self.doc_chunks.append(chunk)

        self.doc_count = len(self.docs)
        self.avgdl = sum(len(d) for d in self.docs) / max(self.doc_count, 1)
        self._compute_idf()

    def search(self, query: str, top_k: int = 5) -> list:
        """Search for documents matching the query."""
        query_tokens = self._tokenize(query)
```

```
        scores = []
        for i, doc_tokens in enumerate(self.docs):
            score = self._score(query_tokens, doc_tokens)
            scores.append((self.doc_chunks[i], score))

        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_k]

    def _tokenize(self, text: str) -> list:
        """Simple tokenization."""
        return re.findall(r'\w+', text.lower())

    def _compute_idf(self):
        """Compute IDF for all terms."""
        df = {}  # Document frequency
        for doc in self.docs:
            unique_terms = set(doc)
            for term in unique_terms:
                df[term] = df.get(term, 0) + 1

        for term, freq in df.items():
            self.idf[term] = math.log(
                (self.doc_count - freq + 0.5) / (freq + 0.5) + 1
            )

    def _score(self, query_tokens: list, doc_tokens: list) -> float:
        """Compute BM25 score for a document."""
        score = 0.0
        doc_len = len(doc_tokens)

        # Count term frequencies in document
        tf = {}
        for token in doc_tokens:
            tf[token] = tf.get(token, 0) + 1

        for term in query_tokens:
            if term not in self.idf:
                continue

            term_freq = tf.get(term, 0)
            idf = self.idf[term]

            numerator = term_freq * (self.k1 + 1)
            denominator = term_freq + self.k1 * (
                1 - self.b + self.b * doc_len / max(self.avgdl, 1)
            )

            score += idf * numerator / denominator

        return score


# ============================================================
# SECTION 4: EMBEDDING (Local, No API Needed)
# ============================================================
```

```
class SimpleEmbedder:
    """
    A simple TF-IDF based embedder for practice.
    No API key needed. In production, use OpenAI or sentence-transformers.

    This creates sparse embeddings using word frequencies — not as good
    as neural embeddings, but demonstrates the concept perfectly.
    """

    def __init__(self, dim: int = 256):
        self.dim = dim
        self.vocabulary = {}
        self.idf = {}
        self.fitted = False

    def fit(self, texts: list):
        """Build vocabulary from a corpus of texts."""
        # Count document frequency for each word
        df = {}
        n_docs = len(texts)

        all_words = set()
        for text in texts:
            words = set(re.findall(r'\w+', text.lower()))
            all_words.update(words)
            for word in words:
                df[word] = df.get(word, 0) + 1

        # Keep top N words by document frequency
        sorted_words = sorted(df.items(), key=lambda x: x[1], reverse=True)
        for i, (word, freq) in enumerate(sorted_words[:self.dim]):
            self.vocabulary[word] = i
            self.idf[word] = math.log(n_docs / (freq + 1)) + 1

        self.fitted = True

    def embed(self, text: str) -> list:
        """Create an embedding vector for text."""
        if not self.fitted:
            raise ValueError("Embedder not fitted. Call fit() first.")

        words = re.findall(r'\w+', text.lower())
        word_counts = {}
        for w in words:
            word_counts[w] = word_counts.get(w, 0) + 1

        vector = [0.0] * self.dim
        for word, count in word_counts.items():
            if word in self.vocabulary:
                idx = self.vocabulary[word]
                tf = count / max(len(words), 1)
                vector[idx] = tf * self.idf.get(word, 1.0)

        # L2 normalize
        norm = math.sqrt(sum(x*x for x in vector))
```

```
        if norm > 0:
            vector = [x / norm for x in vector]

        return vector

    def embed_batch(self, texts: list) -> list:
        """Embed multiple texts."""
        return [self.embed(text) for text in texts]


# ═══════════════════════════════════════════════════════════════
# SECTION 5: RAG PIPELINE
# ═══════════════════════════════════════════════════════════════

class RAGPipeline:
    """
    Complete RAG pipeline with hybrid search and grounding evaluation.

    ┌─────────────────────────────────────────────────────────┐
    │                    RAG Pipeline                         │
    │                                                         │
    │  Ingestion:                                             │
    │    Documents → Chunker → Embedder → Vector Store        │
    │                                   → BM25 Index          │
    │                                                         │
    │  Query:                                                 │
    │    Question → Embedder → Vector Search ─┐               │
    │            → BM25 Search ──────────────┤               │
    │                                        ▼               │
    │                                  Hybrid Merge           │
    │                                       │                 │
    │                                  Build Prompt           │
    │                                       │                 │
    │                                  Generate Answer        │
    │                                       │                 │
    │                                  Evaluate Grounding     │
    │                                       │                 │
    │                                  Return Result          │
    │                                                         │
    └─────────────────────────────────────────────────────────┘
    """

    def __init__(self, chunk_size: int = 500, chunk_overlap: int = 50,
                 top_k: int = 5, hybrid_alpha: float = 0.5):
        """
        Args:
            chunk_size: Maximum chunk size in characters
            chunk_overlap: Overlap between chunks
            top_k: Number of chunks to retrieve
            hybrid_alpha: Balance between vector (1.0) and BM25 (0.0)
        """
        self.chunker = RecursiveChunker(chunk_size, chunk_overlap)
        self.vector_store = SimpleVectorStore()
        self.bm25 = SimpleBM25()
        self.embedder = SimpleEmbedder()
        self.top_k = top_k
```

```
        self.hybrid_alpha = hybrid_alpha
        self.all_chunks = []

    def ingest(self, documents: dict):
        """
        Ingest documents into the RAG pipeline.

        Args:
            documents: dict of {name: text} pairs
        """
        print(f"📥 Ingesting {len(documents)} documents...")

        # Chunk all documents
        all_chunks = []
        for name, text in documents.items():
            chunks = self.chunker.chunk_text(text, source=name)
            all_chunks.extend(chunks)
            print(f"   📄 {name}: {len(chunks)} chunks")

        self.all_chunks = all_chunks

        # Fit embedder on all chunk texts
        all_texts = [c.text for c in all_chunks]
        self.embedder.fit(all_texts)

        # Embed all chunks
        for chunk in all_chunks:
            chunk.embedding = self.embedder.embed(chunk.text)

        # Add to stores
        self.vector_store.add(all_chunks)
        self.bm25.add(all_chunks)

        print(f"✅ Ingested {len(all_chunks)} total chunks")
        print(f"   Vector store: {len(self.vector_store)} vectors")

    def query(self, question: str, verbose: bool = True) -> dict:
        """
        Query the RAG pipeline.

        Returns:
            dict with: answer, chunks, scores, grounding_score
        """
        if verbose:
            print(f"\n🔍 Query: {question}")

        # Step 1: Retrieve relevant chunks
        retrieved = self._hybrid_search(question)

        if verbose:
            print(f"\n📚 Retrieved {len(retrieved)} chunks:")
            for chunk, score in retrieved:
                preview = chunk.text[:80].replace('\n', ' ')
                print(f"  [{score:.3f}] ({chunk.source}) {preview}...")

        # Step 2: Build prompt with context
```

```
        context = "\n\n---\n\n".join(
            f"[Source: {chunk.source}]\n{chunk.text}"
            for chunk, _ in retrieved
        )

        prompt = self._build_prompt(question, context)

        # Step 3: Generate answer (simulated without API)
        answer = self._generate_answer(question, retrieved)

        # Step 4: Evaluate grounding
        grounding_score = self._evaluate_grounding(answer, retrieved)

        if verbose:
            print(f"\n💬 Answer: {answer}")
            print(f"\n📊 Grounding score: {grounding_score:.2f}")

        return {
            "question": question,
            "answer": answer,
            "chunks": [(c.text, c.source, s) for c, s in retrieved],
            "grounding_score": grounding_score,
            "prompt": prompt
        }

    def _hybrid_search(self, query: str) -> list:
        """Combine vector and BM25 search results."""
        # Vector search
        query_embedding = self.embedder.embed(query)
        vector_results = self.vector_store.search(query_embedding, self.top_k * 2)

        # BM25 search
        bm25_results = self.bm25.search(query, self.top_k * 2)

        # Reciprocal Rank Fusion
        rrf_scores = {}
        k = 60  # RRF constant

        for rank, (chunk, _) in enumerate(vector_results):
            rrf_scores[chunk.id] = rrf_scores.get(chunk.id, 0) + \
                self.hybrid_alpha / (k + rank + 1)

        for rank, (chunk, _) in enumerate(bm25_results):
            rrf_scores[chunk.id] = rrf_scores.get(chunk.id, 0) + \
                (1 - self.hybrid_alpha) / (k + rank + 1)

        # Build result list
        chunk_map = {c.id: c for c in self.all_chunks}
        results = [
            (chunk_map[cid], score)
            for cid, score in rrf_scores.items()
            if cid in chunk_map
        ]
        results.sort(key=lambda x: x[1], reverse=True)

        return results[:self.top_k]
```

```
    def _build_prompt(self, question: str, context: str) -> str:
        """Build the RAG prompt."""
        return f"""Answer the question based ONLY on the provided context.
If the context doesn't contain enough information, say "I don't have enough
information to answer this question."

Always cite which source document your answer comes from.

Context:
{context}

Question: {question}

Answer:"""

    def _generate_answer(self, question: str,
                         retrieved: list) -> str:
        """
        Generate an answer from retrieved chunks.

        In production, this would call an LLM. For practice,
        we use extractive summarization (pulling relevant sentences).
        """
        if not retrieved:
            return "I don't have enough information to answer this question."

        # Simple extractive approach: find sentences most relevant to query
        query_words = set(re.findall(r'\w+', question.lower()))

        scored_sentences = []
        for chunk, chunk_score in retrieved:
            sentences = re.split(r'[.!?]+', chunk.text)
            for sentence in sentences:
                sentence = sentence.strip()
                if len(sentence) < 20:
                    continue

                words = set(re.findall(r'\w+', sentence.lower()))
                overlap = len(query_words & words)
                score = overlap * chunk_score
                scored_sentences.append((sentence, score, chunk.source))

        scored_sentences.sort(key=lambda x: x[1], reverse=True)

        # Take top sentences
        top_sentences = scored_sentences[:3]

        if not top_sentences:
            return "I don't have enough information to answer this question."

        answer_parts = []
        sources_used = set()
        for sent, _, source in top_sentences:
            answer_parts.append(sent)
            sources_used.add(source)
```

```
        answer = ". ".join(answer_parts)
        sources = ", ".join(sources_used)
        return f"{answer}. (Sources: {sources})"

    def _evaluate_grounding(self, answer: str,
                            retrieved: list) -> float:
        """
        Evaluate how well the answer is grounded in retrieved chunks.

        Returns a score from 0.0 (not grounded) to 1.0 (fully grounded).

        Method: Check what fraction of answer words appear in the
        retrieved context.
        """
        if not answer or not retrieved:
            return 0.0

        # Get all words from answer (excluding common stop words)
        stop_words = {"the", "a", "an", "is", "are", "was", "were", "be",
                      "been", "being", "have", "has", "had", "do", "does",
                      "did", "will", "would", "could", "should", "may",
                      "might", "can", "shall", "to", "of", "in", "for",
                      "on", "with", "at", "by", "from", "as", "into",
                      "through", "during", "before", "after", "and", "but",
                      "or", "not", "no", "this", "that", "these", "those",
                      "it", "its", "i", "don", "t", "s", "sources"}

        answer_words = set(re.findall(r'\w+', answer.lower())) - stop_words

        # Get all words from retrieved chunks
        context_words = set()
        for chunk, _ in retrieved:
            context_words.update(
                set(re.findall(r'\w+', chunk.text.lower())) - stop_words
            )

        if not answer_words:
            return 1.0

        # What fraction of answer words appear in context?
        grounded_words = answer_words & context_words
        score = len(grounded_words) / len(answer_words)

        return round(score, 3)

    def add_memory(self, key: str, text: str):
        """
        Add a piece of information to the agent's long-term memory.
        This is how agents can store things they learn for later retrieval.
        """
        chunks = self.chunker.chunk_text(text, source=f"memory:{key}")

        # Refit embedder with new vocabulary
        all_texts = [c.text for c in self.all_chunks] + [c.text for c in chunks]
        self.embedder.fit(all_texts)
```

```
        # Re-embed everything (in production, you'd use an incremental approach)
        for chunk in self.all_chunks:
            chunk.embedding = self.embedder.embed(chunk.text)
        for chunk in chunks:
            chunk.embedding = self.embedder.embed(chunk.text)

        self.all_chunks.extend(chunks)

        # Rebuild stores
        self.vector_store = SimpleVectorStore()
        self.vector_store.add(self.all_chunks)
        self.bm25 = SimpleBM25()
        self.bm25.add(self.all_chunks)

        print(f"🧠 Added memory '{key}': {len(chunks)} chunks")


# ═══════════════════════════════════════════════════════════════
# SECTION 6: DEMO WITH SAMPLE DOCUMENTS
# ═══════════════════════════════════════════════════════════════

# Sample knowledge base documents
SAMPLE_DOCS = {
    "transformer_architecture": """
The Transformer architecture was introduced in the paper "Attention Is All You Need"
by Vaswani et al. in 2017. It revolutionized natural language processing by replacin
recurrent neural networks with self-attention mechanisms.

The key innovation is the self-attention mechanism, which allows the model to weigh
the importance of different parts of the input when processing each element. This
enables parallel processing of all input tokens, making training much faster than
sequential models like LSTMs.

The architecture consists of an encoder and decoder, each composed of multiple layer
Each layer contains multi-head self-attention and feed-forward neural networks, with
residual connections and layer normalization.

Modern large language models like GPT-4 and Llama use a decoder-only variant of the
Transformer. They process text autoregressively, predicting one token at a time base
on all previous tokens.
""",

    "rlhf_training": """
Reinforcement Learning from Human Feedback (RLHF) is a technique used to align
language models with human preferences. It was popularized by OpenAI and Anthropic
for training models like ChatGPT and Claude.

The RLHF process has three main steps:

1. Supervised Fine-Tuning (SFT): The base model is fine-tuned on a dataset of
   high-quality (instruction, response) pairs to learn instruction-following behavio

2. Reward Model Training: Human evaluators rank multiple model outputs for the same
   prompt. These rankings train a reward model that predicts human preferences.
```

```
3. Policy Optimization: The language model is further trained using reinforcement
   learning (specifically PPO) to maximize the reward model's score, with a KL
   divergence penalty to prevent deviating too far from the SFT model.

DPO (Direct Preference Optimization) is an alternative to RLHF that skips the
reward model entirely. It directly optimizes the policy using preference data,
which is simpler and more stable than the full RLHF pipeline.
""",

    "rag_systems": """
Retrieval-Augmented Generation (RAG) is a technique that enhances language models
by allowing them to retrieve relevant information from external knowledge bases
before generating responses.

The RAG pipeline consists of several key components:

Chunking: Documents are split into smaller pieces (chunks) that can be individually
embedded and retrieved. Common strategies include fixed-size chunking, recursive
splitting by paragraph/sentence boundaries, and semantic chunking based on topic
changes.

Embedding: Each chunk is converted into a dense vector representation using an
embedding model. Popular models include OpenAI's text-embedding-3-small and
open-source alternatives like all-MiniLM-L6-v2.

Vector Storage: Embeddings are stored in a vector database that supports efficient
similarity search. Options include ChromaDB, Pinecone, FAISS, and pgvector.

Retrieval: When a user asks a question, the question is embedded and compared
against stored chunk embeddings using cosine similarity. The top-K most similar
chunks are retrieved.

Hybrid Search: Combining vector search with keyword-based search (BM25) often
produces better results than either method alone. Reciprocal Rank Fusion (RRF)
is a popular method for combining multiple ranking lists.

Context Window Management: Retrieved chunks must fit within the LLM's context
window along with the system prompt and user query. Strategies include limiting
the number of chunks, reranking, and summarization.
""",

    "agent_architectures": """
AI agents are systems that use language models to autonomously plan, reason,
and take actions to accomplish tasks. Several key architectures have emerged:

ReAct (Reason + Act): The agent alternates between thinking (reasoning about
what to do) and acting (using tools). Each observation from a tool informs
the next reasoning step. This is effective for tasks requiring real-time
information gathering and step-by-step problem solving.

Plan-then-Execute: A planner generates a complete plan upfront, then an
executor carries out each step. A critic validates results after each step,
enabling retries and replanning. Better for complex multi-step tasks with
dependencies between steps.

Multi-Agent Systems: Multiple specialized agents collaborate on a task.
```

```
Common patterns include a Planner agent (decomposes tasks), Executor agent
(runs tools), Researcher agent (gathers information), and Critic agent
(validates quality). An orchestrator coordinates the agents.

Agent Memory: Effective agents use multiple memory types. Short-term memory
(the conversation scratchpad) handles the current task. Long-term memory
(vector stores, RAG) provides persistent knowledge. Episodic memory logs
past actions and outcomes for learning from experience.

Common failure modes include infinite loops (same action repeated),
hallucinated tool calls (inventing non-existent tools), and premature
termination (answering before having sufficient information).
"""
}


def demo():
    """Run the RAG pipeline demo."""
    # Initialize pipeline
    rag = RAGPipeline(chunk_size=400, chunk_overlap=50, top_k=3)

    # Ingest documents
    rag.ingest(SAMPLE_DOCS)

    # Test queries
    print("\n" + "="*60)
    print("QUERY 1: Factual retrieval")
    print("="*60)
    rag.query("What is the Transformer architecture and who created it?")

    print("\n" + "="*60)
    print("QUERY 2: Technical details")
    print("="*60)
    rag.query("How does RLHF work and what are its three steps?")

    print("\n" + "="*60)
    print("QUERY 3: Comparison")
    print("="*60)
    rag.query("What is the difference between ReAct and Plan-then-Execute?")

    print("\n" + "="*60)
    print("QUERY 4: RAG about RAG")
    print("="*60)
    rag.query("What are the components of a RAG pipeline?")

    # Demonstrate dynamic memory
    print("\n" + "="*60)
    print("ADDING DYNAMIC MEMORY")
    print("="*60)
    rag.add_memory(
        "team_decision",
        "Our team decided to use ChromaDB for the vector store because "
        "it's easy to set up, supports persistent storage, and has good "
        "Python integration. We chose text-embedding-3-small from OpenAI "
        "for embeddings due to its balance of quality and cost."
    )
```

```
    print("\n" + "="*60)
    print("QUERY 5: Using dynamic memory")
    print("="*60)
    rag.query("What vector database did our team decide to use and why?")

    # Show out-of-scope handling
    print("\n" + "="*60)
    print("QUERY 6: Out of scope")
    print("="*60)
    rag.query("What is the capital of France?")


if __name__ == "__main__":
    demo()
```

Using ChromaDB (Production Alternative)

```
"""
Production RAG with ChromaDB — use this when you need persistent,
performant vector search.

pip install chromadb openai
"""
import chromadb
from chromadb.utils import embedding_functions
import openai

# Initialize ChromaDB
client = chromadb.PersistentClient(path="./chroma_db")

# Use OpenAI embeddings (or switch to local)
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="your-api-key",
    model_name="text-embedding-3-small"
)

# Create a collection
collection = client.get_or_create_collection(
    name="knowledge_base",
    embedding_function=openai_ef,
    metadata={"hnsw:space": "cosine"}  # Use cosine similarity
)

# Add documents
collection.add(
    documents=["The Transformer architecture was introduced in 2017...",
               "RLHF is used to align language models..."],
    metadatas=[{"source": "transformers.md"}, {"source": "rlhf.md"}],
    ids=["doc1", "doc2"]
)
```

```python
# Query
results = collection.query(
    query_texts=["How does attention work?"],
    n_results=3,
    include=["documents", "metadatas", "distances"]
)

print(f"Top result: {results['documents'][0][0][:100]}...")
print(f"Distance: {results['distances'][0][0]:.4f}")
print(f"Source: {results['metadatas'][0][0]['source']}")

# Use in RAG prompt
context = "\n\n".join(results["documents"][0])
prompt = f"""Answer based on this context:

{context}

Question: How does attention work?
Answer:"""

response = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
)
print(response.choices[0].message.content)
```

✓ Day 5 Deliverable Checklist

- [ ] Built a complete RAG pipeline (chunker + embedder + vector store)
- [ ] Implemented BM25 keyword search
- [ ] Combined vector + BM25 into hybrid search
- [ ] Added a grounding score evaluator
- [ ] Agent can store new memories dynamically
- [ ] Agent can retrieve relevant context for questions
- [ ] Tested with at least 4 different queries
- [ ] Can explain: What is RAG and when should you use it vs fine-tuning?
- [ ] Can explain: How does cosine similarity work?
- [ ] Can explain: Why is hybrid search better than pure vector search?
- [ ] Can explain: What are chunking strategies and their trade-offs?

Day 6 — Multi-Agent Collaboration

> 🎯 **Goal**: Build a system where multiple Agents work together intelligently.

> ⏱️ **Time**: 2h theory + 3h practice

> 📦 **Deliverable**: A multi-agent system solving engineering tasks end-to-end

---

Table of Contents

1. Why Multi-Agent?
2. Orchestration Patterns
3. Agent Roles
4. Communication & Turn-Taking
5. Preventing Loops & Deadlocks
6. Practice: Build a 3-Agent System

Why Multi-Agent?

A single agent has limits. It must be good at planning AND executing AND validating AND researching. Multi-agent systems let you **specialize** — each agent is an expert at one thing.



```
        │              │              │
        └──────────────┼──────────────┘
                       │
                ORCHESTRATOR
                (coordinates)

Benefits:
  ✓ Focused prompts → better results per role
  ✓ Different models per role (GPT-4 for planning, GPT-3.5 for execution)
  ✓ Parallel execution where possible
  ✓ Clear separation of concerns
  ✓ Easier to debug (which agent went wrong?)
```

Orchestration Patterns

Pattern 1: Sequential Pipeline

```
Agents process in a fixed order, like an assembly line.

  Task → Planner → Executor → Critic → Output
                                  │
                            (retry if failed)

Best for: Well-defined workflows with clear stages
Example: Code generation (plan → code → review)

Pros: Simple, predictable, easy to debug
Cons: Rigid, can't handle dynamic task shapes
```

Pattern 2: Coordinator (Hub-and-Spoke)

```
A central Coordinator agent decides which specialist to call next.

                    ┌──────────────┐
                    │ COORDINATOR  │
                    │              │
                    │ "Who should  │
                    │  handle this │
                    │  next?"      │
                    └──────┬───────┘
                           │
                ┌──────────┼──────────┐
                ▼          ▼          ▼
          ┌──────────┐ ┌──────────┐ ┌──────────┐
          │RESEARCHER│ │  CODER   │ │  CRITIC  │
          └──────────┘ └──────────┘ └──────────┘

Flow:
```

```
  1. Coordinator receives task
  2. Coordinator decides: "First, we need research"
  3. Researcher does research, returns results
  4. Coordinator decides: "Now we need code"
  5. Coder writes code using research
  6. Coordinator decides: "Now we need review"
  7. Critic reviews code
  8. Coordinator: "Critic found issues → back to Coder"
  9. ...until Coordinator is satisfied

Best for: Complex, dynamic tasks
Example: "Given a Jira ticket, solve it end-to-end"

Pros: Flexible, adaptive routing
Cons: Coordinator becomes bottleneck, needs good judgment
```

Pattern 3: Debate / Consensus

```
Multiple agents tackle the same problem independently,
then debate/vote on the best solution.

  Task ─┬── Agent A ──┐
        ├── Agent B ──┼── Debate → Consensus → Output
        └── Agent C ──┘

Best for: High-stakes decisions, diverse perspectives
Example: Security analysis, architectural decisions

Pros: Reduces individual agent bias, higher quality
Cons: Expensive (N× the LLM calls), slower
```

Pattern 4: Hierarchical Teams

```
Agents organized in teams with team leads.

  ┌──────────────────────────────────────────────┐
  │              PROJECT MANAGER                 │
  │         (top-level coordinator)              │
  └──────────┬──────────────────┬────────────────┘
             ▼                  ▼
  ┌──────────────────┐  ┌──────────────────┐
  │ BACKEND TEAM     │  │ FRONTEND TEAM    │
  │ Lead             │  │ Lead             │
  │  ├── Coder       │  │  ├── Designer    │
  │  ├── Tester      │  │  ├── Coder       │
  │  └── Reviewer    │  │  └── Tester      │
  └──────────────────┘  └──────────────────┘

Best for: Large, complex projects
Example: Building a full application
```

```
Pros: Scales to many agents, clear accountability
Cons: Complex, high overhead, hard to debug
```

Choosing a Pattern

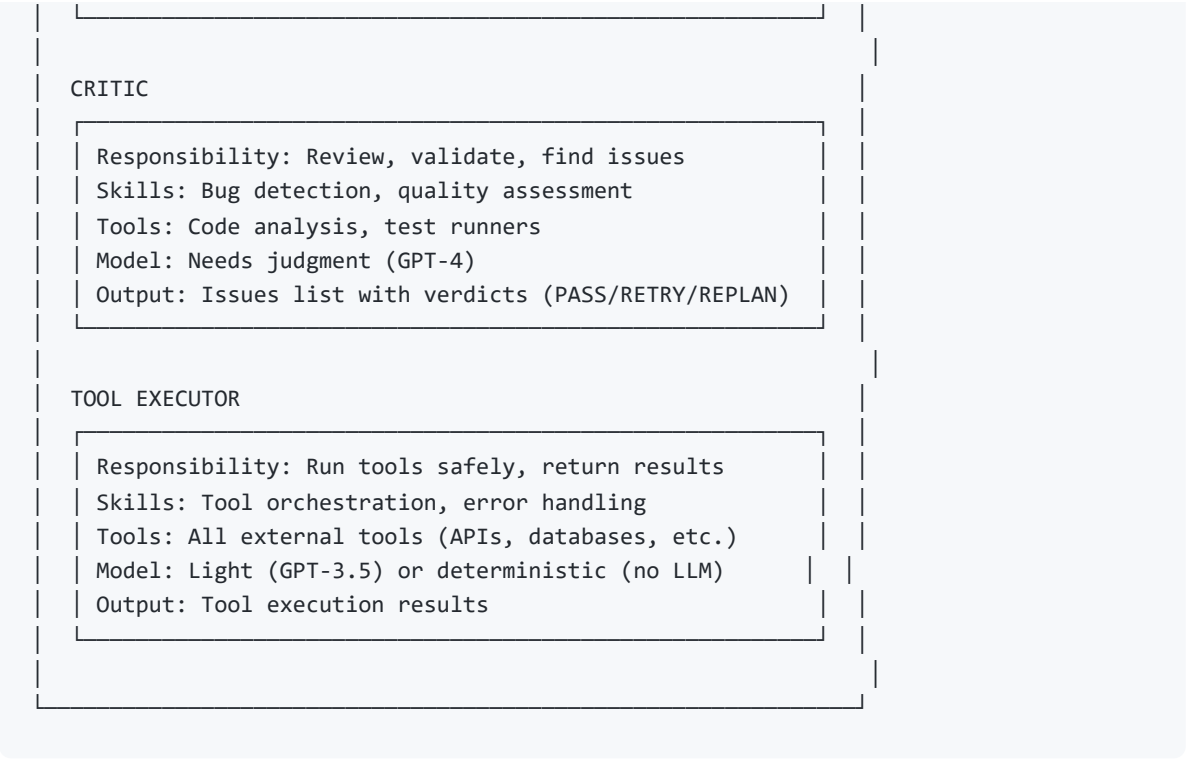
```
┌──────────────────────┬──────────────────────┬──────────────┐
│ Pattern              │ Best For             │ Complexity   │
├──────────────────────┼──────────────────────┼──────────────┤
│ Sequential Pipeline  │ Predictable workflows  │ Low        │
│ Coordinator          │ Dynamic, complex tasks │ Medium     │
│ Debate/Consensus     │ High-stakes decisions  │ Medium     │
│ Hierarchical Teams   │ Large projects         │ High       │
└──────────────────────┴──────────────────────┴──────────────┘

Start with Sequential Pipeline. Move to Coordinator when needed.
```

Agent Roles

Common Agent Archetypes

```
┌──────────────────────────────────────────────────────────┐
│                    AGENT ROLES                           │
│                                                          │
│  PLANNER                                                 │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Responsibility: Break tasks into steps, create plans  │
│  │ Skills: Task decomposition, dependency analysis  │    │
│  │ Model: Needs strong reasoning (GPT-4, Claude 3)  │    │
│  │ Temperature: Low (0.0-0.2)                       │    │
│  │ Output: Structured plan (JSON)                   │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  RESEARCHER                                              │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Responsibility: Gather information, search knowledge  │
│  │ Skills: Query formulation, source evaluation     │    │
│  │ Tools: Search, RAG, web browsing                 │    │
│  │ Model: Medium (GPT-3.5 is fine)                  │    │
│  │ Output: Summarized findings with sources         │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  CODER                                                   │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Responsibility: Write, modify, and fix code      │    │
│  │ Skills: Programming, debugging, refactoring      │    │
│  │ Tools: Code execution, file I/O, package managers │   │
│  │ Model: Coding-optimized (GPT-4, Claude, Codex)   │    │
│  │ Output: Working code with comments               │    │
```



System Prompt Design for Each Role

```
AGENT_PROMPTS = {
    "planner": """You are an expert task planner. Your ONLY job is to break
down complex tasks into clear, ordered steps.

Rules:
- Each step must be concrete and actionable
- Include dependencies between steps
- Include validation criteria for each step
- Do NOT execute steps — only plan them
- Output a JSON plan

You have these agents available to assign steps to:
- researcher: gathers information
- coder: writes and modifies code
- critic: reviews and validates work
""",

    "researcher": """You are a research specialist. Your ONLY job is to
gather relevant information for a given topic.

Rules:
- Search thoroughly before answering
- Always cite your sources
- Distinguish facts from opinions
- If information is insufficient, say so clearly
- Output structured findings as JSON
""",

    "coder": """You are a senior software engineer. Your ONLY job is to
```

```
write clean, working code.

Rules:
- Follow best practices for the language
- Include error handling
- Add comments for complex logic only
- Write testable code
- If requirements are unclear, list your assumptions
""",

    "critic": """You are a code review and quality assurance expert.
Your ONLY job is to evaluate work quality.

Rules:
- Check for bugs, security issues, and logic errors
- Verify requirements are met
- Be specific about what's wrong and how to fix it
- Rate issues as CRITICAL, HIGH, MEDIUM, LOW
- Respond with PASS, RETRY, or REPLAN verdict
- Do NOT fix issues yourself — only identify them
""",
}
```

Communication & Turn-Taking

Message Protocol

Agents need a standard way to communicate:

```python
from dataclasses import dataclass, field
from typing import Optional
from enum import Enum
import time

class MessageType(Enum):
    TASK = "task"           # Assign a task
    RESULT = "result"       # Return results
    FEEDBACK = "feedback"   # Provide feedback
    QUESTION = "question"   # Ask for clarification
    STATUS = "status"       # Status update

@dataclass
class AgentMessage:
    """Standard message format between agents."""
    from_agent: str
    to_agent: str
    type: MessageType
    content: str
    metadata: dict = field(default_factory=dict)
```

```
        timestamp: float = field(default_factory=time.time)
        reply_to: Optional[str] = None  # ID of message this replies to
        id: str = field(default_factory=lambda: f"msg_{int(time.time()*1000)}")

        def to_dict(self) -> dict:
            return {
                "id": self.id,
                "from": self.from_agent,
                "to": self.to_agent,
                "type": self.type.value,
                "content": self.content,
                "metadata": self.metadata,
                "timestamp": self.timestamp,
                "reply_to": self.reply_to
            }
```

Turn-Taking Strategies

```
Strategy 1: ROUND-ROBIN
  Each agent takes a turn in fixed order.
  Planner → Executor → Critic → Planner → ...

  Simple but inflexible.

Strategy 2: COORDINATOR-DRIVEN
  Coordinator decides who speaks next based on current state.

  "Plan exists?" → No → Planner
  "Step to execute?" → Yes → Executor
  "Result to validate?" → Yes → Critic

  Flexible and adaptive.

Strategy 3: EVENT-DRIVEN
  Agents react to events (messages) asynchronously.

  Planner publishes "plan_ready" → Executor picks it up
  Executor publishes "step_done" → Critic picks it up
  Critic publishes "issues_found" → Coder picks it up

  Most scalable but hardest to debug.
```

Preventing Loops & Deadlocks

Common Failure Modes

```
FAILURE 1: Agent Ping-Pong
  Coder: "Here's the code"
  Critic: "This has issues, please fix"
  Coder: "Here's the fixed code" (same code!)
  Critic: "This still has issues"
  Coder: "Here's the fixed code" (same code!)
  ... infinite ...

  Prevention:
  - Track what feedback was given
  - After N retries, escalate or change approach
  - Include previous attempt in context

FAILURE 2: Deadlock
  Agent A: "Waiting for Agent B's output"
  Agent B: "Waiting for Agent A's output"
  ... both stuck forever ...

  Prevention:
  - No circular dependencies in the plan
  - Timeout per agent (e.g., 60 seconds)
  - Deadlock detection: if no progress for N seconds, break

FAILURE 3: Cascading Failures
  Planner creates bad plan → Executor fails on step 1
  → Critic says REPLAN → Planner creates same bad plan
  → Executor fails again → ...

  Prevention:
  - Include failure history in replan context
  - Max replan limit (e.g., 3)
  - Different approach after each failure

FAILURE 4: Token Budget Exhaustion
  Each agent call uses tokens. Multi-agent uses N× more.
  Long conversations can exceed context windows.

  Prevention:
  - Summarize inter-agent messages
  - Set per-agent token budgets
  - Use lighter models for simple agents
```

Guardrail Implementation

```python
  @dataclass
  class MultiAgentGuardrails:
      """Guardrails for multi-agent systems."""
      max_total_turns: int = 30
      max_turns_per_agent: int = 10
      max_retries_per_step: int = 3
      max_replans: int = 2
      timeout_per_turn_seconds: float = 60.0
```

```
    # Tracking
    turn_count: int = 0
    agent_turn_counts: dict = field(default_factory=dict)
    retry_counts: dict = field(default_factory=dict)
    replan_count: int = 0

    def check_turn(self, agent_name: str) -> tuple:
        """Check if this agent can take another turn."""
        self.turn_count += 1
        self.agent_turn_counts[agent_name] = \
            self.agent_turn_counts.get(agent_name, 0) + 1

        if self.turn_count > self.max_total_turns:
            return False, f"Total turn limit ({self.max_total_turns}) reached"

        if self.agent_turn_counts[agent_name] > self.max_turns_per_agent:
            return False, f"{agent_name} turn limit ({self.max_turns_per_agent}) rea

        return True, ""

    def check_retry(self, step_id: str) -> tuple:
        """Check if a step can be retried."""
        self.retry_counts[step_id] = self.retry_counts.get(step_id, 0) + 1

        if self.retry_counts[step_id] > self.max_retries_per_step:
            return False, f"Retry limit for step '{step_id}' reached"

        return True, ""

    def check_replan(self) -> tuple:
        """Check if replanning is allowed."""
        self.replan_count += 1
        if self.replan_count > self.max_replans:
            return False, f"Replan limit ({self.max_replans}) reached"
        return True, ""
```

Practice: Build a 3-Agent System

Complete Multi-Agent Implementation

```
"""
Multi-Agent System: Planner + Executor + Critic

Solves engineering tasks by:
1. Planner breaks down the task
2. Executor performs each step
3. Critic validates each result
```

```
Real-world task: "Given a Jira ticket, propose a solution,
generate the code, test it, and provide a PR summary."

No API key needed — fully functional with simulated LLM responses.
"""
import json
import time
from dataclasses import dataclass, field
from typing import Optional
from enum import Enum


# ═══════════════════════════════════════════════════════════
# MESSAGE SYSTEM
# ═══════════════════════════════════════════════════════════

class Role(Enum):
    ORCHESTRATOR = "orchestrator"
    PLANNER = "planner"
    EXECUTOR = "executor"
    CRITIC = "critic"

@dataclass
class Message:
    """Inter-agent message."""
    sender: Role
    receiver: Role
    content: str
    msg_type: str  # "task", "result", "feedback", "verdict"
    data: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)

class MessageBus:
    """Central message bus for agent communication."""

    def __init__(self):
        self.messages: list[Message] = []
        self.subscribers: dict = {}

    def send(self, message: Message):
        """Send a message."""
        self.messages.append(message)

    def get_messages_for(self, role: Role,
                         msg_type: Optional[str] = None) -> list:
        """Get all messages for a specific agent."""
        msgs = [m for m in self.messages if m.receiver == role]
        if msg_type:
            msgs = [m for m in msgs if m.msg_type == msg_type]
        return msgs

    def get_conversation_log(self) -> str:
        """Get a formatted log of all messages."""
        log = []
        for msg in self.messages:
            log.append(
```

```
                f"[{msg.sender.value} → {msg.receiver.value}] "
                f"({msg.msg_type}) {msg.content[:100]}"
            )
        return "\n".join(log)


# ═══════════════════════════════════════════════════════════════
# AGENT BASE CLASS
# ═══════════════════════════════════════════════════════════════

class BaseAgent:
    """Base class for all agents."""

    def __init__(self, role: Role, bus: MessageBus):
        self.role = role
        self.bus = bus
        self.memory = []  # Agent's internal memory

    def send(self, to: Role, content: str, msg_type: str,
             data: dict = None):
        """Send a message to another agent."""
        self.bus.send(Message(
            sender=self.role,
            receiver=to,
            content=content,
            msg_type=msg_type,
            data=data or {}
        ))

    def log(self, message: str):
        """Log agent activity."""
        prefix = {
            Role.ORCHESTRATOR: "🎯",
            Role.PLANNER: "📋",
            Role.EXECUTOR: "⚡",
            Role.CRITIC: "🔍"
        }.get(self.role, "•")
        print(f"  {prefix} [{self.role.value}] {message}")


# ═══════════════════════════════════════════════════════════════
# PLANNER AGENT
# ═══════════════════════════════════════════════════════════════

class PlannerAgent(BaseAgent):
    """
    Creates structured plans from task descriptions.
    Specializes in task decomposition and dependency analysis.
    """

    def __init__(self, bus: MessageBus):
        super().__init__(Role.PLANNER, bus)

    def create_plan(self, task: str, context: str = "") -> dict:
        """Create a plan for the given task."""
        self.log(f"Creating plan for: {task[:80]}...")
```

```
        plan = self._decompose_task(task, context)

        self.log(f"Plan created: {len(plan['steps'])} steps")
        for step in plan["steps"]:
            self.log(f"  Step {step['id']}: {step['description']}")

        self.send(
            Role.ORCHESTRATOR,
            f"Plan created with {len(plan['steps'])} steps",
            "result",
            {"plan": plan}
        )

        return plan

    def revise_plan(self, original_plan: dict, feedback: str,
                    completed_steps: list) -> dict:
        """Revise a plan based on critic feedback."""
        self.log(f"Revising plan based on feedback: {feedback[:80]}...")

        # Keep completed steps, add recovery + remaining
        new_steps = []
        next_id = 1

        for step in original_plan["steps"]:
            if step["id"] in completed_steps:
                step["status"] = "done"
                step["id"] = next_id
                new_steps.append(step)
                next_id += 1

        # Add recovery step
        new_steps.append({
            "id": next_id,
            "description": f"Address feedback: {feedback}",
            "agent": "executor",
            "action": "fix",
            "inputs": {"feedback": feedback},
            "status": "pending"
        })
        next_id += 1

        # Add validation step
        new_steps.append({
            "id": next_id,
            "description": "Validate the fix addresses the feedback",
            "agent": "critic",
            "action": "validate",
            "inputs": {},
            "status": "pending"
        })

        revised = {
            "task": original_plan["task"],
            "approach": f"Revised: {feedback}",
```

```
            "steps": new_steps,
            "version": original_plan.get("version", 1) + 1
        }

        self.log(f"Revised plan: {len(new_steps)} steps (v{revised['version']})")
        return revised

    def _decompose_task(self, task: str, context: str) -> dict:
        """Break a task into steps (rule-based for demo)."""
        task_lower = task.lower()

        # Detect task type and create appropriate plan
        if "jira" in task_lower or "ticket" in task_lower or "implement" in task_low
            return self._plan_engineering_task(task)
        elif "analyze" in task_lower or "review" in task_lower:
            return self._plan_review_task(task)
        else:
            return self._plan_generic_task(task)

    def _plan_engineering_task(self, task: str) -> dict:
        return {
            "task": task,
            "approach": "Analyze → Design → Implement → Test → Review → PR Summary",
            "steps": [
                {
                    "id": 1,
                    "description": "Analyze the ticket requirements and identify sco
                    "agent": "executor",
                    "action": "analyze",
                    "inputs": {"task": task},
                    "status": "pending"
                },
                {
                    "id": 2,
                    "description": "Research best practices and similar implementati
                    "agent": "executor",
                    "action": "research",
                    "inputs": {"topic": task},
                    "status": "pending"
                },
                {
                    "id": 3,
                    "description": "Generate the implementation code",
                    "agent": "executor",
                    "action": "code",
                    "inputs": {},
                    "status": "pending"
                },
                {
                    "id": 4,
                    "description": "Write unit tests for the implementation",
                    "agent": "executor",
                    "action": "test",
                    "inputs": {},
                    "status": "pending"
                },
```

```
                {
                    "id": 5,
                    "description": "Review code quality and correctness",
                    "agent": "critic",
                    "action": "review",
                    "inputs": {},
                    "status": "pending"
                },
                {
                    "id": 6,
                    "description": "Generate PR summary with changes and test result
                    "agent": "executor",
                    "action": "summarize",
                    "inputs": {},
                    "status": "pending"
                }
            ],
            "version": 1
        }

    def _plan_review_task(self, task: str) -> dict:
        return {
            "task": task,
            "approach": "Gather → Analyze → Report",
            "steps": [
                {"id": 1, "description": "Gather materials to review",
                 "agent": "executor", "action": "research",
                 "inputs": {"topic": task}, "status": "pending"},
                {"id": 2, "description": "Perform detailed analysis",
                 "agent": "executor", "action": "analyze",
                 "inputs": {}, "status": "pending"},
                {"id": 3, "description": "Validate analysis quality",
                 "agent": "critic", "action": "review",
                 "inputs": {}, "status": "pending"},
                {"id": 4, "description": "Compile final report",
                 "agent": "executor", "action": "summarize",
                 "inputs": {}, "status": "pending"},
            ],
            "version": 1
        }

    def _plan_generic_task(self, task: str) -> dict:
        return {
            "task": task,
            "approach": "Research → Execute → Validate",
            "steps": [
                {"id": 1, "description": f"Research: {task}",
                 "agent": "executor", "action": "research",
                 "inputs": {"topic": task}, "status": "pending"},
                {"id": 2, "description": "Execute the main task",
                 "agent": "executor", "action": "execute",
                 "inputs": {}, "status": "pending"},
                {"id": 3, "description": "Validate the result",
                 "agent": "critic", "action": "review",
                 "inputs": {}, "status": "pending"},
            ],
```

```
            "version": 1
        }


# ============================================================
# EXECUTOR AGENT
# ============================================================

class ExecutorAgent(BaseAgent):
    """
    Executes plan steps using available tools.
    Specializes in research, coding, testing, and summarization.
    """

    def __init__(self, bus: MessageBus):
        super().__init__(Role.EXECUTOR, bus)
        self.context = {}  # Accumulated context from previous steps

    def execute_step(self, step: dict) -> dict:
        """Execute a single step from the plan."""
        action = step.get("action", "execute")
        self.log(f"Executing: {step['description']}")

        # Route to appropriate handler
        handlers = {
            "analyze": self._analyze,
            "research": self._research,
            "code": self._write_code,
            "test": self._write_tests,
            "summarize": self._summarize,
            "fix": self._fix,
            "execute": self._generic_execute,
        }

        handler = handlers.get(action, self._generic_execute)
        result = handler(step)

        # Store in context for later steps
        self.context[step["id"]] = result

        self.log(f"Completed: {result['status']}")

        self.send(
            Role.ORCHESTRATOR,
            f"Step {step['id']} completed: {result['status']}",
            "result",
            {"step_id": step["id"], "result": result}
        )

        return result

    def _analyze(self, step: dict) -> dict:
        task = step.get("inputs", {}).get("task", "Unknown task")
        return {
            "status": "success",
            "output": (
```

```
                f"Requirements Analysis:\n"
                f"- Task: {task}\n"
                f"- Scope: Implement the requested feature\n"
                f"- Dependencies: Standard library, FastAPI\n"
                f"- Estimated complexity: Medium\n"
                f"- Key considerations: Error handling, input validation, tests"
            ),
            "artifacts": {"requirements": task}
        }

    def _research(self, step: dict) -> dict:
        topic = step.get("inputs", {}).get("topic", "")
        return {
            "status": "success",
            "output": (
                f"Research Findings:\n"
                f"- Best practice: Use FastAPI with Pydantic models for validation\n
                f"- Pattern: Repository pattern for data access layer\n"
                f"- Error handling: Use HTTPException with proper status codes\n"
                f"- Testing: Use pytest with TestClient for API testing\n"
                f"- Security: Input validation, parameterized queries, auth middlewa
            ),
            "artifacts": {"research_notes": f"Research on: {topic}"}
        }

    def _write_code(self, step: dict) -> dict:
        # Use context from previous steps
        requirements = ""
        for ctx in self.context.values():
            if isinstance(ctx, dict) and "output" in ctx:
                requirements += ctx["output"] + "\n"

        code = '''from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import Optional
import uuid

app = FastAPI(title="Task API")

class Task(BaseModel):
    title: str = Field(..., min_length=1, max_length=200)
    description: Optional[str] = None
    status: str = Field(default="todo", pattern="^(todo|in_progress|done)$")

class TaskResponse(Task):
    id: str

# In-memory store (use database in production)
tasks_db: dict[str, TaskResponse] = {}

@app.post("/tasks", response_model=TaskResponse, status_code=201)
async def create_task(task: Task):
    task_id = str(uuid.uuid4())
    task_response = TaskResponse(id=task_id, **task.model_dump())
    tasks_db[task_id] = task_response
    return task_response
```

```
@app.get("/tasks/{task_id}", response_model=TaskResponse)
async def get_task(task_id: str):
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="Task not found")
    return tasks_db[task_id]

@app.get("/tasks", response_model=list[TaskResponse])
async def list_tasks(status: Optional[str] = None):
    tasks = list(tasks_db.values())
    if status:
        tasks = [t for t in tasks if t.status == status]
    return tasks

@app.put("/tasks/{task_id}", response_model=TaskResponse)
async def update_task(task_id: str, task: Task):
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="Task not found")
    updated = TaskResponse(id=task_id, **task.model_dump())
    tasks_db[task_id] = updated
    return updated

@app.delete("/tasks/{task_id}", status_code=204)
async def delete_task(task_id: str):
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="Task not found")
    del tasks_db[task_id]
'''

        return {
            "status": "success",
            "output": f"Generated implementation code:\n```python\n{code}\n```",
            "artifacts": {"code": code, "filename": "main.py"}
        }

    def _write_tests(self, step: dict) -> dict:
        test_code = '''import pytest
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_create_task():
    response = client.post("/tasks", json={
        "title": "Test task",
        "description": "A test task"
    })
    assert response.status_code == 201
    data = response.json()
    assert data["title"] == "Test task"
    assert "id" in data

def test_get_task():
    # Create first
    create_resp = client.post("/tasks", json={"title": "Get test"})
    task_id = create_resp.json()["id"]
```

```
    # Then get
    response = client.get(f"/tasks/{task_id}")
    assert response.status_code == 200
    assert response.json()["title"] == "Get test"

def test_get_nonexistent_task():
    response = client.get("/tasks/nonexistent-id")
    assert response.status_code == 404

def test_list_tasks():
    response = client.get("/tasks")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_update_task():
    create_resp = client.post("/tasks", json={"title": "Update me"})
    task_id = create_resp.json()["id"]

    response = client.put(f"/tasks/{task_id}", json={
        "title": "Updated title",
        "status": "in_progress"
    })
    assert response.status_code == 200
    assert response.json()["title"] == "Updated title"

def test_delete_task():
    create_resp = client.post("/tasks", json={"title": "Delete me"})
    task_id = create_resp.json()["id"]

    response = client.delete(f"/tasks/{task_id}")
    assert response.status_code == 204

def test_invalid_status():
    response = client.post("/tasks", json={
        "title": "Bad status",
        "status": "invalid"
    })
    assert response.status_code == 422  # Validation error
'''

        return {
            "status": "success",
            "output": f"Generated test suite:\n```python\n{test_code}\n```\n\n7 test
            "artifacts": {"test_code": test_code, "test_count": 7, "filename": "test
        }

    def _summarize(self, step: dict) -> dict:
        # Compile all context into a PR summary
        artifacts = {}
        for ctx in self.context.values():
            if isinstance(ctx, dict) and "artifacts" in ctx:
                artifacts.update(ctx["artifacts"])

        summary = f"""## PR Summary
```

```
### Changes
- Implemented CRUD API for Task management using FastAPI
- Added Pydantic models with validation (title length, status enum)
- In-memory storage with full CRUD operations
- Proper error handling with HTTP status codes

### Files Changed
- `main.py` — API implementation with 5 endpoints
- `test_main.py` — Test suite with 7 tests

### Testing
- ✅ Create task (201)
- ✅ Get task (200)
- ✅ Get nonexistent (404)
- ✅ List tasks (200)
- ✅ Update task (200)
- ✅ Delete task (204)
- ✅ Invalid status validation (422)

### Considerations
- Using in-memory store; should migrate to database for production
- No authentication implemented yet
- No pagination on list endpoint
"""

        return {
            "status": "success",
            "output": summary,
            "artifacts": {"pr_summary": summary}
        }

    def _fix(self, step: dict) -> dict:
        feedback = step.get("inputs", {}).get("feedback", "")
        return {
            "status": "success",
            "output": f"Applied fix based on feedback: {feedback}\n"
                      f"Changes made: Added input validation, improved error messages
            "artifacts": {}
        }

    def _generic_execute(self, step: dict) -> dict:
        return {
            "status": "success",
            "output": f"Executed: {step['description']}",
            "artifacts": {}
        }


# ═══════════════════════════════════════════════════════════════
# CRITIC AGENT
# ═══════════════════════════════════════════════════════════════

class CriticAgent(BaseAgent):
    """
    Reviews and validates work from other agents.
    Provides verdicts: PASS, RETRY, or REPLAN.
```

```
        """

        def __init__(self, bus: MessageBus):
            super().__init__(Role.CRITIC, bus)
            self.review_count = 0

        def review(self, step: dict, result: dict,
                   all_context: dict) -> dict:
            """Review a step's result."""
            self.review_count += 1
            self.log(f"Reviewing step {step['id']}: {step['description']}")

            issues = []
            suggestions = []

            # Check for errors
            if result.get("status") != "success":
                issues.append(f"Step failed with status: {result.get('status')}")
                return self._verdict("RETRY", issues, suggestions)

            output = result.get("output", "")

            # Check for empty output
            if not output or len(output.strip()) < 10:
                issues.append("Output is empty or too brief")
                return self._verdict("RETRY", issues, suggestions)

            # Action-specific checks
            action = step.get("action", "")

            if action == "code":
                issues, suggestions = self._review_code(output, result)
            elif action == "test":
                issues, suggestions = self._review_tests(output, result)
            elif action == "analyze":
                issues, suggestions = self._review_analysis(output)
            elif action == "summarize":
                issues, suggestions = self._review_summary(output, all_context)

            # Determine verdict
            critical_issues = [i for i in issues if "CRITICAL" in i.upper()]

            if critical_issues:
                verdict = "REPLAN" if self.review_count > 2 else "RETRY"
            elif issues:
                verdict = "RETRY" if self.review_count <= 2 else "PASS"
            else:
                verdict = "PASS"

            return self._verdict(verdict, issues, suggestions)

        def _review_code(self, output: str, result: dict) -> tuple:
            """Review generated code."""
            issues = []
            suggestions = []
            code = result.get("artifacts", {}).get("code", output)
```

```
        # Check for common issues
        if "eval(" in code:
            issues.append("CRITICAL: Use of eval() — security risk")

        if "password" in code.lower() and "=" in code:
            if "os.environ" not in code and "getenv" not in code:
                issues.append("HIGH: Possible hardcoded credentials")

        if "except:" in code:
            issues.append("MEDIUM: Bare except clause — catch specific exceptions")
            suggestions.append("Use specific exception types")

        if "import" not in code:
            issues.append("MEDIUM: No imports found — code may be incomplete")

        if "def " not in code and "class " not in code:
            issues.append("HIGH: No functions or classes — code structure needed")

        if not issues:
            suggestions.append("Code looks clean. Consider adding docstrings.")

        return issues, suggestions

    def _review_tests(self, output: str, result: dict) -> tuple:
        """Review test code."""
        issues = []
        suggestions = []

        test_count = result.get("artifacts", {}).get("test_count", 0)

        if test_count < 3:
            issues.append("MEDIUM: Too few tests — aim for at least 5")

        if "assert" not in output:
            issues.append("HIGH: No assertions found in tests")

        if "edge" not in output.lower() and "error" not in output.lower():
            suggestions.append("Consider adding edge case and error tests")

        return issues, suggestions

    def _review_analysis(self, output: str) -> tuple:
        """Review an analysis output."""
        issues = []
        suggestions = []

        if len(output) < 100:
            issues.append("MEDIUM: Analysis is too brief")

        key_sections = ["scope", "depend", "consider"]
        missing = [s for s in key_sections if s not in output.lower()]
        if missing:
            suggestions.append(f"Consider adding sections about: {', '.join(missing)

        return issues, suggestions
```

```
    def _review_summary(self, output: str, context: dict) -> tuple:
        """Review a PR summary."""
        issues = []
        suggestions = []

        required_sections = ["changes", "testing"]
        for section in required_sections:
            if section not in output.lower():
                issues.append(f"MEDIUM: Missing '{section}' section in summary")

        return issues, suggestions

    def _verdict(self, verdict: str, issues: list,
                 suggestions: list) -> dict:
        """Create a verdict response."""
        emoji = {"PASS": "✅", "RETRY": "🔄", "REPLAN": "❌"}
        self.log(f"Verdict: {emoji.get(verdict, '?')} {verdict}")

        if issues:
            self.log(f"Issues: {issues}")
        if suggestions:
            self.log(f"Suggestions: {suggestions}")

        self.send(
            Role.ORCHESTRATOR,
            f"Verdict: {verdict}",
            "verdict",
            {"verdict": verdict, "issues": issues, "suggestions": suggestions}
        )

        return {
            "verdict": verdict,
            "issues": issues,
            "suggestions": suggestions,
            "confidence": 0.85 if not issues else 0.6
        }


# ═══════════════════════════════════════════════════════════════
# ORCHESTRATOR
# ═══════════════════════════════════════════════════════════════

class Orchestrator:
    """
    Coordinates all agents in the multi-agent system.

    Flow:
    ┌─────────────────────────────────────────────────────┐
    │                  ORCHESTRATOR                       │
    │                                                     │
    │  1. Receive task                                    │
    │  2. Ask Planner to create plan                      │
    │  3. For each step:                                  │
    │     a. Ask Executor to execute                      │
    │     b. Ask Critic to review                         │
```

```
    │     c. Handle verdict (PASS/RETRY/REPLAN)           │
    │  4. Compile final result                            │
    │                                                      │
    └──────────────────────────────────────────────────────┘
    """

    def __init__(self):
        self.bus = MessageBus()
        self.planner = PlannerAgent(self.bus)
        self.executor = ExecutorAgent(self.bus)
        self.critic = CriticAgent(self.bus)
        self.guardrails = MultiAgentGuardrails()

    def solve(self, task: str, verbose: bool = True) -> dict:
        """
        Solve a task using the multi-agent system.
        """
        if verbose:
            print(f"\n{'='*60}")
            print(f"  MULTI-AGENT SYSTEM")
            print(f"  Task: {task[:80]}")
            print(f"{'='*60}")

        # Phase 1: Planning
        if verbose:
            print(f"\n{'─'*40}")
            print(f"  PHASE 1: PLANNING")
            print(f"{'─'*40}")

        plan = self.planner.create_plan(task)

        # Phase 2: Execution with Critic validation
        if verbose:
            print(f"\n{'─'*40}")
            print(f"  PHASE 2: EXECUTION")
            print(f"{'─'*40}")

        completed_steps = []
        all_results = {}

        for step in plan["steps"]:
            # Check guardrails
            ok, msg = self.guardrails.check_turn(step.get("agent", "executor"))
            if not ok:
                if verbose:
                    print(f"\n  ⛔ Guardrail: {msg}")
                break

            if verbose:
                print(f"\n  {'─'*30}")
                print(f"  Step {step['id']}: {step['description']}")
                print(f"  {'─'*30}")

            # Execute
            result = self.executor.execute_step(step)
            all_results[step["id"]] = result
```

```
            # Critic review (if step is assigned to critic, skip self-review)
            if step.get("agent") == "critic":
                # Critic is both executor and reviewer for review steps
                verdict_result = self.critic.review(
                    step, result, self.executor.context
                )
            else:
                verdict_result = self.critic.review(
                    step, result, self.executor.context
                )

            verdict = verdict_result["verdict"]

            if verdict == "PASS":
                step["status"] = "done"
                completed_steps.append(step["id"])

            elif verdict == "RETRY":
                ok, msg = self.guardrails.check_retry(str(step["id"]))
                if ok:
                    if verbose:
                        print(f"  🔄 Retrying step {step['id']}...")
                    # Re-execute
                    result = self.executor.execute_step(step)
                    all_results[step["id"]] = result
                    step["status"] = "done"
                    completed_steps.append(step["id"])
                else:
                    if verbose:
                        print(f"  ⛔ {msg}")
                    step["status"] = "failed"

            elif verdict == "REPLAN":
                ok, msg = self.guardrails.check_replan()
                if ok:
                    if verbose:
                        print(f"\n  📋 REPLANNING...")
                    feedback = "; ".join(verdict_result.get("issues", []))
                    plan = self.planner.revise_plan(
                        plan, feedback, completed_steps
                    )
                    # Continue with revised plan
                    break
                else:
                    if verbose:
                        print(f"  ⛔ {msg}")

        # Phase 3: Compile results
        if verbose:
            print(f"\n{'─'*40}")
            print(f"  PHASE 3: RESULTS")
            print(f"{'─'*40}")

        final_result = self._compile_results(task, plan, all_results)
```

```
        if verbose:
            print(f"\n{final_result['summary']}")

            print(f"\n{'−'*40}")
            print(f"  STATISTICS")
            print(f"{'−'*40}")
            print(f"  Steps completed: {len(completed_steps)}/{len(plan['steps'])}")
            print(f"  Agent turns: {json.dumps(dict(self.guardrails.agent_turn_count
            print(f"  Retries: {sum(self.guardrails.retry_counts.values())}")
            print(f"  Replans: {self.guardrails.replan_count}")
            print(f"  Messages exchanged: {len(self.bus.messages)}")

        return final_result

    def _compile_results(self, task: str, plan: dict,
                         results: dict) -> dict:
        """Compile all agent outputs into final result."""
        artifacts = {}
        outputs = []

        for step_id, result in results.items():
            if isinstance(result, dict):
                if "output" in result:
                    outputs.append(result["output"])
                if "artifacts" in result:
                    artifacts.update(result["artifacts"])

        summary = f"\n{'='*60}\n"
        summary += f"📋 Task: {task}\n"
        summary += f"{'='*60}\n"

        for i, output in enumerate(outputs, 1):
            if len(output) > 300:
                output = output[:300] + "...\n(truncated)"
            summary += f"\n📌 Output {i}:\n{output}\n"

        return {
            "task": task,
            "plan": plan,
            "results": results,
            "artifacts": artifacts,
            "summary": summary,
            "message_log": self.bus.get_conversation_log()
        }


# ═══════════════════════════════════════════════════════════════
# DEMO
# ═══════════════════════════════════════════════════════════════

def demo():
    """Run multi-agent demo."""

    # Demo 1: Engineering task (Jira-like ticket)
    print("\n" + "="*60)
    print("  DEMO 1: Engineering Task")
```

```
    print("="*60)

    orchestrator = Orchestrator()
    orchestrator.solve(
        "Implement a REST API for task management with CRUD operations. "
        "The API should support creating, reading, updating, and deleting tasks. "
        "Each task has a title, description, and status (todo/in_progress/done). "
        "Include input validation and proper error handling."
    )

    # Demo 2: Analysis task
    print("\n\n" + "="*60)
    print("  DEMO 2: Analysis Task")
    print("="*60)

    orchestrator2 = Orchestrator()
    orchestrator2.solve(
        "Analyze the best approach for implementing authentication "
        "in our new microservices architecture."
    )


if __name__ == "__main__":
    demo()
```

Exercise: Extend the System

```
1. ADD A RESEARCHER AGENT
   Create a ResearcherAgent that uses the RAG pipeline from Day 5
   to gather information. The Planner should be able to assign
   research steps to this agent.

2. ADD PARALLEL EXECUTION
   When the plan has independent steps (no dependencies),
   execute them in parallel using threading.

3. ADD AGENT MEMORY
   Each agent should remember outcomes of previous tasks.
   When starting a new task, check if similar tasks were
   solved before and use those learnings.

4. ADD HUMAN-IN-THE-LOOP
   At critical steps (e.g., before deploying, before major
   code changes), pause and ask for human approval before
   continuing.

5. CONNECT TO REAL LLM
   Replace the simulated responses with actual LLM calls.
   Use GPT-4 for Planner and Critic, GPT-3.5 for Executor.
```

✓ Day 6 Deliverable Checklist

- [ ] Built a 3-agent system (Planner + Executor + Critic)
- [ ] Agents communicate via a message bus
- [ ] Orchestrator coordinates agent turns
- [ ] System handles PASS / RETRY / REPLAN verdicts
- [ ] Guardrails prevent loops and runaway execution
- [ ] Tested with at least 2 different task types
- [ ] Can explain: What are the main multi-agent orchestration patterns?
- [ ] Can explain: What agent roles exist and when to use each?
- [ ] Can explain: How to prevent loops and deadlocks?
- [ ] Can explain: What are the trade-offs of multi-agent vs single-agent?

Day 7 — Final Project: Build Your Own Copilot

> 🎯 **Goal**: Produce a Senior-level, production-quality agentic system.

> ⏱️ **Time**: Full day (6–8h)

> 📦 **Deliverable**: A full, working Agent system with planning, collaboration, RAG, and tools

---

Table of Contents

1. Project Overview
2. Project A: Engineering Copilot
3. Project B: DevOps Optimization Copilot
4. Evaluation Criteria
5. What Makes This Senior-Level

Project Overview

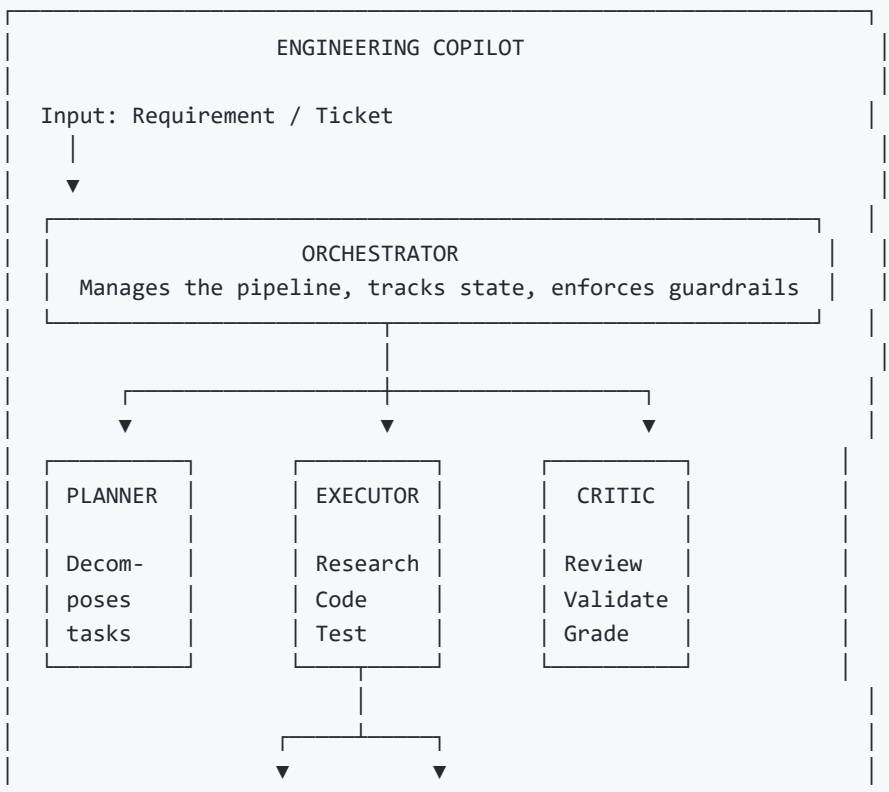
Today you integrate everything from Days 1–6 into a production-quality system. Choose **one** of two projects:

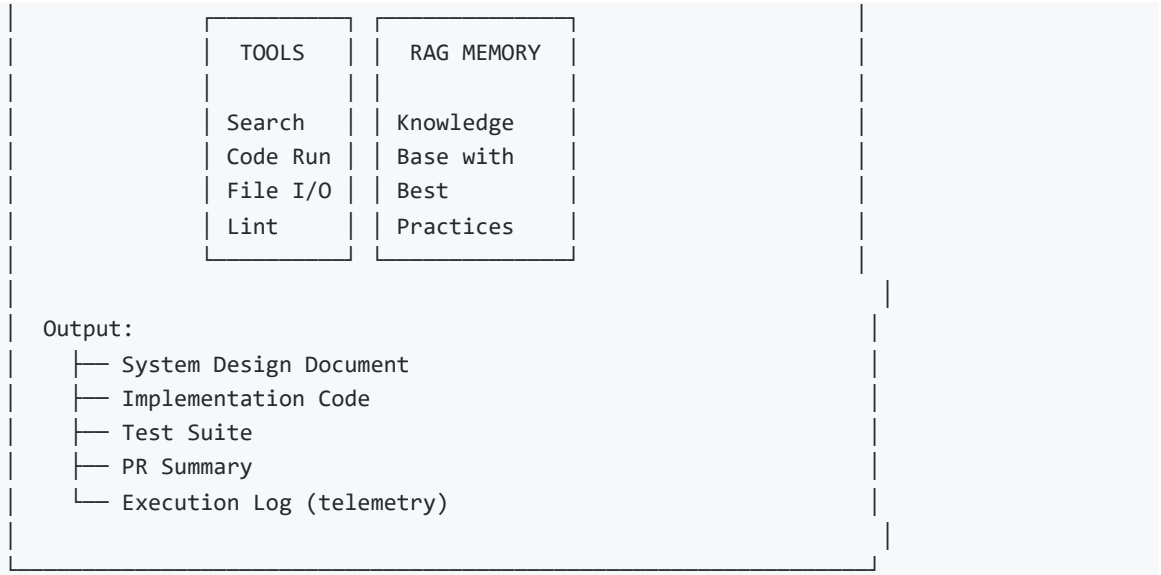
```
┌─────────────────────────────────────────────────────────────┐
│                   CHOOSE YOUR PROJECT                       │
│                                                             │
│  🅐 ENGINEERING COPILOT (Recommended)                       │
│     Input: A requirement or ticket                          │
│     Output: System design + Code + Tests + PR summary       │
│     Skills: Planning, coding, RAG, multi-agent, tools       │
│                                                             │
│  🅑 DEVOPS OPTIMIZATION COPILOT                             │
│     Input: Python code to optimize                          │
│     Output: Analysis + Optimized code + Benchmark report    │
│     Skills: Code analysis, optimization, testing, reporting │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

Both projects require:
- ✓ Multi-agent architecture (Day 6)
- ✓ Planning and task decomposition (Day 4)
- ✓ RAG knowledge base (Day 5)
- ✓ Tool use (Day 3)
- ✓ Structured outputs with validation (Day 2)
- ✓ Understanding of LLM capabilities (Day 1)

Project A: Engineering Copilot

Architecture





Complete Implementation

```
"""
Engineering Copilot — Final Project

A production-quality multi-agent system that takes a requirement
and produces system design, code, tests, and a PR summary.

Components:
  - Orchestrator: Manages pipeline and guardrails
  - Planner: Decomposes requirements into actionable steps
  - Executor: Performs each step (research, code, test)
  - Critic: Validates quality at each step
  - RAG Memory: Knowledge base of best practices
  - Tool Suite: Search, code execution, file parsing
  - Telemetry: Logging and metrics

Usage:
  copilot = EngineeringCopilot()
  result = copilot.solve("Implement a rate limiter for the API")
"""
import json
import time
import math
import re
import hashlib
from dataclasses import dataclass, field
from typing import Optional, Callable
from enum import Enum


# ═══════════════════════════════════════════════════════════════
# SECTION 1: CORE DATA STRUCTURES
# ═══════════════════════════════════════════════════════════════
```

```python
class AgentRole(Enum):
    ORCHESTRATOR = "orchestrator"
    PLANNER = "planner"
    EXECUTOR = "executor"
    CRITIC = "critic"

class StepStatus(Enum):
    PENDING = "pending"
    IN_PROGRESS = "in_progress"
    PASSED = "passed"
    FAILED = "failed"
    SKIPPED = "skipped"

class Verdict(Enum):
    PASS = "pass"
    RETRY = "retry"
    REPLAN = "replan"

@dataclass
class PlanStep:
    id: int
    description: str
    action: str  # "research", "design", "code", "test", "review", "summarize"
    inputs: dict = field(default_factory=dict)
    depends_on: list = field(default_factory=list)
    status: StepStatus = StepStatus.PENDING
    result: Optional[dict] = None
    retries: int = 0

@dataclass
class Plan:
    task: str
    steps: list  # List[PlanStep]
    approach: str = ""
    version: int = 1

@dataclass
class CriticReview:
    verdict: Verdict
    issues: list
    suggestions: list
    confidence: float
    reasoning: str

@dataclass
class TelemetryEvent:
    timestamp: float
    agent: str
    action: str
    duration_ms: float
    status: str
    details: dict = field(default_factory=dict)


# ═══════════════════════════════════════════════════════════════
# SECTION 2: RAG KNOWLEDGE BASE
```

```python
# ============================================================

class KnowledgeBase:
    """
    RAG-based knowledge base containing engineering best practices.
    Uses the same architecture from Day 5.
    """

    def __init__(self):
        self.documents = {}
        self.chunks = []
        self.vocabulary = {}
        self._load_knowledge()

    def _load_knowledge(self):
        """Load built-in engineering knowledge."""
        self.documents = {
            "api_design": (
                "REST API Best Practices: "
                "Use nouns for resources, not verbs. "
                "Use HTTP methods correctly: GET (read), POST (create), "
                "PUT (update), PATCH (partial update), DELETE (remove). "
                "Return appropriate HTTP status codes: 200 OK, 201 Created, "
                "204 No Content, 400 Bad Request, 401 Unauthorized, "
                "404 Not Found, 429 Too Many Requests, 500 Internal Server Error. "
                "Use pagination for list endpoints. Version your API (/v1/). "
                "Use consistent error response format with error code, message, and
            ),
            "rate_limiting": (
                "Rate Limiting Patterns: "
                "Token Bucket: Tokens added at fixed rate, consumed per request. "
                "Allows bursts up to bucket size. "
                "Sliding Window: Count requests in a moving time window. "
                "More accurate than fixed windows but more memory. "
                "Fixed Window: Count requests in fixed time intervals. "
                "Simple but allows double the rate at window boundaries. "
                "Leaky Bucket: Requests processed at fixed rate, excess queued. "
                "Implementation: Use Redis INCR with EXPIRE for distributed rate lim
                "Return headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLim
            ),
            "authentication": (
                "Authentication Best Practices: "
                "JWT tokens for stateless authentication. "
                "Store secrets in environment variables, never in code. "
                "Use bcrypt or argon2 for password hashing (never MD5/SHA for passwo
                "Implement token refresh mechanism. "
                "Use HTTPS everywhere. "
                "Implement CORS properly. "
                "OAuth 2.0 for third-party authentication."
            ),
            "testing": (
                "Testing Best Practices: "
                "Test pyramid: Many unit tests, fewer integration, few E2E. "
                "Test happy paths, edge cases, and error cases. "
                "Use fixtures and factories for test data. "
                "Mock external dependencies. "
```

```
                "Aim for >80% code coverage but focus on critical paths. "
                "Use parameterized tests for similar test cases. "
                "Test input validation boundaries."
            ),
            "error_handling": (
                "Error Handling Best Practices: "
                "Use specific exception types, not bare except. "
                "Create custom exception classes for domain errors. "
                "Log errors with context (request ID, user, timestamp). "
                "Return user-friendly error messages (no stack traces to clients). "
                "Use error codes for machine-readable error identification. "
                "Implement retry logic with exponential backoff for transient errors
                "Use circuit breaker pattern for external service calls."
            ),
            "code_quality": (
                "Code Quality: "
                "Follow SOLID principles. "
                "Single Responsibility: Each class/function does one thing. "
                "Open/Closed: Open for extension, closed for modification. "
                "Use dependency injection for testability. "
                "Keep functions under 20 lines when possible. "
                "Use type hints in Python. "
                "Write docstrings for public APIs. "
                "Use linters (pylint, flake8) and formatters (black)."
            ),
        }

        # Build simple search index
        for key, text in self.documents.items():
            words = set(re.findall(r'\w+', text.lower()))
            for word in words:
                if word not in self.vocabulary:
                    self.vocabulary[word] = []
                self.vocabulary[word].append(key)

    def search(self, query: str, top_k: int = 3) -> list:
        """Search the knowledge base."""
        query_words = set(re.findall(r'\w+', query.lower()))

        doc_scores = {}
        for word in query_words:
            if word in self.vocabulary:
                for doc_key in self.vocabulary[word]:
                    doc_scores[doc_key] = doc_scores.get(doc_key, 0) + 1

        ranked = sorted(doc_scores.items(), key=lambda x: x[1], reverse=True)

        results = []
        for doc_key, score in ranked[:top_k]:
            results.append({
                "source": doc_key,
                "content": self.documents[doc_key],
                "relevance": score / max(len(query_words), 1)
            })

        return results
```

```
    def add_knowledge(self, key: str, content: str):
        """Add new knowledge to the base."""
        self.documents[key] = content
        words = set(re.findall(r'\w+', content.lower()))
        for word in words:
            if word not in self.vocabulary:
                self.vocabulary[word] = []
            self.vocabulary[word].append(key)


# ============================================================
# SECTION 3: TOOL SUITE
# ============================================================

class ToolSuite:
    """Collection of tools available to the agents."""

    def __init__(self, knowledge_base: KnowledgeBase):
        self.kb = knowledge_base
        self.execution_log = []

    def search_knowledge(self, query: str) -> str:
        """Search the knowledge base for best practices."""
        results = self.kb.search(query)
        if results:
            return "\n\n".join(
                f"[{r['source']}] (relevance: {r['relevance']:.1%})\n{r['content']}"
                for r in results
            )
        return "No relevant knowledge found."

    def analyze_requirements(self, requirement: str) -> dict:
        """Analyze a requirement and extract key information."""
        components = {
            "entities": [],
            "actions": [],
            "constraints": [],
            "non_functional": []
        }

        req_lower = requirement.lower()

        # Extract entities (nouns that are likely data objects)
        entity_keywords = ["user", "task", "item", "order", "product",
                           "message", "session", "token", "request", "response",
                           "api", "endpoint", "service", "rate", "limit"]
        components["entities"] = [
            e for e in entity_keywords if e in req_lower
        ]

        # Extract actions
        action_keywords = ["create", "read", "update", "delete", "list",
                           "search", "filter", "sort", "paginate", "authenticate",
                           "validate", "limit", "throttle", "cache"]
        components["actions"] = [
```

```
                a for a in action_keywords if a in req_lower
            ]

            # Extract constraints
            if "must" in req_lower or "should" in req_lower:
                components["constraints"].append("Has explicit requirements")
            if any(w in req_lower for w in ["fast", "performance", "latency"]):
                components["non_functional"].append("Performance requirements")
            if any(w in req_lower for w in ["secure", "auth", "encrypt"]):
                components["non_functional"].append("Security requirements")
            if any(w in req_lower for w in ["scale", "distributed", "concurrent"]):
                components["non_functional"].append("Scalability requirements")

            return components

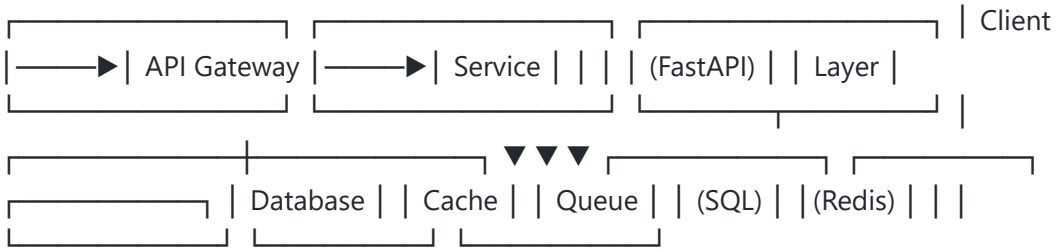
        def generate_design(self, requirement: str,
                            analysis: dict) -> str:
            """Generate a system design document."""
            entities = analysis.get("entities", ["item"])
            actions = analysis.get("actions", ["create", "read"])

            primary_entity = entities[0] if entities else "resource"

            design = f"""## System Design: {requirement[:80]}

    ### Overview
    This system implements a {primary_entity} management service with the following capa
    {chr(10).join(f'- {a.title()} {primary_entity}s' for a in actions)}

    ### Architecture
```




```
### API Endpoints
| Method | Endpoint | Description |
|--------|----------|-------------|
"""
        for action in actions:
            if action == "create":
                design += f"| POST | /{primary_entity}s | Create a new {primary_entity}
            elif action in ("read", "list"):
                design += f"| GET | /{primary_entity}s/{{id}} | Get {primary_entity} by
                design += f"| GET | /{primary_entity}s | List all {primary_entity}s |\n"
            elif action == "update":
                design += f"| PUT | /{primary_entity}s/{{id}} | Update a {primary_entity
            elif action == "delete":
```

```
                design += f"| DELETE | /{primary_entity}s/{{id}} | Delete a {primary_ent

        nfr = analysis.get("non_functional", [])
        if nfr:
            design += "\n### Non-Functional Requirements\n"
            for req in nfr:
                design += f"- {req}\n"

        design += f"""
### Data Model
```python
class {primary_entity.title()}(BaseModel):
    id: str
    created_at: datetime
    updated_at: datetime
    # Add domain-specific fields
```

Error Handling

- Consistent error response format
- Custom exception classes
- Request ID tracking for debugging """ return design def generate_code(self, design: str, requirement: str) -> str: """Generate implementation code based on design.""" # Extract primary entity from design entity_match = re.search(r'class (\w+)(BaseModel)', design) entity_name = entity_match.group(1) if entity_match else "Item" entity_lower = entity_name.lower()
  code = f"""

{requirement}

Auto-generated implementation. """ from fastapi import FastAPI, HTTPException, Query, Request from pydantic import BaseModel, Field from typing import Optional from datetime import datetime import uuid import time import logging

Configure logging

logging.basicConfig(level=logging.INFO) logger = logging.getLogger(**name**)

app = FastAPI(title="{entity_name} Service", version="1.0.0")

—— Models
```

class {entity_name}Create(BaseModel): """Request model for creating a {entity_lower}."""" name: str = Field(..., min_length=1, max_length=200) description: Optional[str] = Field(None, max_length=1000)

class {entity_name}Update(BaseModel): """Request model for updating a {entity_lower}."""" name: Optional[str] = Field(None, min_length=1, max_length=200) description: Optional[str] = Field(None, max_length=1000)

class {entity_name}Response({entity_name}Create): """Response model for a {entity_lower}."""" id: str created_at: str updated_at: str

class ErrorResponse(BaseModel): """Standard error response."""" error_code: str message: str details: Optional[dict] = None

class PaginatedResponse(BaseModel): """Paginated list response."""" items: list total: int page: int page_size: int has_next: bool

—— Exceptions

class {entity_name}NotFoundError(Exception): def **init**(self, {entity_lower}_id: str): self.{entity_lower}_id = {entity_lower}_id super().**init**(f"{entity_name} '{{{{entity_lower}_id}}}}' not found")

—— Storage

In-memory store (replace with database in production)

_store: dict[str, {entity_name}Response] = {{}}

—— Middleware

@app.middleware("http") async def add_request_id(request: Request, call_next): """Add request ID and timing to all responses.""" request_id = str(uuid.uuid4())[:8] start = time.time()

```
response = await call_next(request)

duration_ms = (time.time() - start) * 1000
response.headers["X-Request-ID"] = request_id
response.headers["X-Response-Time-Ms"] = f"{{duration_ms:.1f}}"

logger.info(
    f"{{request.method}} {{request.url.path}} "
    f"{{response.status_code}} {{duration_ms:.1f}}ms "
    f"[{{request_id}}]"
)
return response
```

Endpoints

@app.post("/{entity_lower}s", response_model={entity_name}Response, status_code=201) async def create_{entity_lower}({entity_lower}: {entity_name}Create): """Create a new {entity_lower}.""" now = datetime.utcnow().isoformat() item = {entity_name}Response(id=str(uuid.uuid4()), created_at=now, updated_at=now, **{entity_lower}.model_dump()) _store[item.id] = item logger.info(f"Created {entity_lower}: {{item.id}}") return item

@app.get("/{entity_lower}s/{{{{entity_lower}*id}}}}", response_model={entity_name}Response) async def get*{entity_lower}({entity_lower}_id: str): """Get a {entity_lower} by ID.""" if {entity_lower}_id not in _store: raise HTTPException(status_code=404, detail={{ "error_code": "{entity_upper}_NOT_FOUND", "message": f"{entity_name} '{{{{{entity_lower}_id}}}}' not found" }}) return _store[{entity_lower}_id]

@app.get("/{entity_lower}s", response_model=PaginatedResponse) async def list_{entity_lower}s(page: int = Query(1, ge=1), page_size: int = Query(20, ge=1, le=100)): """List {entity_lower}s with pagination.""" all_items = list(_store.values()) total = len(all_items)

```
start = (page - 1) * page_size
end = start + page_size
items = all_items[start:end]

return PaginatedResponse(
    items=items,
    total=total,
    page=page,
    page_size=page_size,
```

```
    has_next=end < total
)
```

@app.put("/{entity_lower}s/{{{{entity_lower}*id}}}}", *response_model={entity_name}Response) async def update*{entity_lower}({entity_lower}_id: str, update: {entity_name}Update): """Update a {entity_lower}.""" if {entity_lower}_id not in _store: raise HTTPException(status_code=404, detail="{entity_name} not found")

```
existing = _store[{entity_lower}_id]
update_data = update.model_dump(exclude_unset=True)

updated = existing.model_copy(update={{
    **update_data,
    "updated_at": datetime.utcnow().isoformat()
}})

_store[{entity_lower}_id] = updated
logger.info(f"Updated {entity_lower}: {{{entity_lower}_id}}")
return updated
```

@app.delete("/{entity_lower}s/{{{{entity_lower}*id}}}}", *status_code=204) async def delete*{entity_lower}({entity_lower}_id: str): """Delete a {entity_lower}.""" if {entity_lower}_id not in _store: raise HTTPException(status_code=404, detail="{entity_name} not found")

```
del _store[{entity_lower}_id]
logger.info(f"Deleted {entity_lower}: {{{entity_lower}_id}}")
```

@app.get("/health") async def health_check(): """Health check endpoint.""" return {{"status": "healthy", "timestamp": datetime.utcnow().isoformat()}} ''' return code

```
def generate_tests(self, code: str, entity_name: str = "Item") -> str:
    """Generate test suite for the code."""
    entity_lower = entity_name.lower()

    return f'''"""
```

Test suite for {entity_name} Service. Run with: pytest test_main.py -v """ import pytest from fastapi.testclient import TestClient

Import the app (adjust import path as needed)

from main import app

client = TestClient(app)

class Test{entity_name}CRUD: """Test basic CRUD operations."""

```
def test_create_{entity_lower}(self, client):
    """Test creating a new {entity_lower}."""
    response = client.post("/{entity_lower}s", json={{
        "name": "Test {entity_name}",
        "description": "A test {entity_lower}"
    }})
    assert response.status_code == 201
    data = response.json()
    assert data["name"] == "Test {entity_name}"
    assert "id" in data
    assert "created_at" in data

def test_get_{entity_lower}(self, client):
    """Test retrieving a {entity_lower} by ID."""
    # Create first
    create = client.post("/{entity_lower}s", json={{"name": "Get Test"}})
    {entity_lower}_id = create.json()["id"]

    # Then get
    response = client.get(f"/{entity_lower}s/{{{entity_lower}_id}}")
    assert response.status_code == 200
    assert response.json()["name"] == "Get Test"

def test_get_nonexistent_{entity_lower}(self, client):
    """Test 404 for nonexistent {entity_lower}."""
    response = client.get("/{entity_lower}s/nonexistent-id")
    assert response.status_code == 404

def test_list_{entity_lower}s(self, client):
    """Test listing {entity_lower}s with pagination."""
    # Create some items
    for i in range(3):
        client.post("/{entity_lower}s", json={{"name": f"Item {{i}}"}})

    response = client.get("/{entity_lower}s?page=1&page_size=2")
    assert response.status_code == 200
    data = response.json()
    assert len(data["items"]) <= 2
    assert "total" in data
    assert "has_next" in data

def test_update_{entity_lower}(self, client):
    """Test updating a {entity_lower}."""
    create = client.post("/{entity_lower}s", json={{"name": "Original"}})
    {entity_lower}_id = create.json()["id"]
```

```
    response = client.put(f"/{entity_lower}s/{{{entity_lower}_id}}", json={{
        "name": "Updated"
    }})
    assert response.status_code == 200
    assert response.json()["name"] == "Updated"

def test_delete_{entity_lower}(self, client):
    """Test deleting a {entity_lower}."""
    create = client.post("/{entity_lower}s", json={{"name": "Delete Me"}})
    {entity_lower}_id = create.json()["id"]

    response = client.delete(f"/{entity_lower}s/{{{entity_lower}_id}}")
    assert response.status_code == 204

    # Verify deleted
    get = client.get(f"/{entity_lower}s/{{{entity_lower}_id}}")
    assert get.status_code == 404
```

class Test{entity_name}Validation: """Test input validation."""

```
def test_empty_name(self, client):
    """Test validation rejects empty name."""
    response = client.post("/{entity_lower}s", json={{"name": ""}})
    assert response.status_code == 422

def test_name_too_long(self, client):
    """Test validation rejects overly long name."""
    response = client.post("/{entity_lower}s", json={{
        "name": "x" * 201
    }})
    assert response.status_code == 422

def test_missing_required_fields(self, client):
    """Test validation requires name field."""
    response = client.post("/{entity_lower}s", json={{}})
    assert response.status_code == 422
```

class TestHealthCheck: """Test health and infrastructure endpoints."""

```
def test_health(self, client):
    """Test health check endpoint."""
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "healthy"

def test_request_id_header(self, client):
    """Test that responses include request ID."""
    response = client.get("/health")
    assert "X-Request-ID" in response.headers

def test_response_time_header(self, client):
```

```
    """Test that responses include response time."""
    response = client.get("/health")
    assert "X-Response-Time-Ms" in response.headers
```

'''

```
def review_code(self, code: str) -> dict:
    """Review code for issues."""
    issues = []
    suggestions = []

    # Security checks
    if "eval(" in code:
        issues.append({"severity": "CRITICAL",
                       "issue": "Use of eval() — code injection risk"})

    if re.search(r'password\s*=\s*["\']', code):
        issues.append({"severity": "CRITICAL",
                       "issue": "Hardcoded password detected"})

    # Quality checks
    if "except:" in code or "except Exception:" in code:
        issues.append({"severity": "MEDIUM",
                       "issue": "Broad exception handling"})
        suggestions.append("Catch specific exception types")

    if "# TODO" in code or "# FIXME" in code:
        issues.append({"severity": "LOW",
                       "issue": "TODO/FIXME comments found"})

    # Best practice checks
    if "import *" in code:
        issues.append({"severity": "MEDIUM",
                       "issue": "Wildcard import used"})

    if "logging" not in code:
        suggestions.append("Consider adding logging")

    if "type" not in code and "BaseModel" not in code:
        suggestions.append("Consider adding type hints")

    # Positive feedback
    if "BaseModel" in code:
        suggestions.append("✓ Good: Using Pydantic models for validation")
    if "HTTPException" in code:
        suggestions.append("✓ Good: Proper HTTP error responses")
    if "uuid" in code:
        suggestions.append("✓ Good: Using UUIDs for identifiers")
    if "middleware" in code.lower():
        suggestions.append("✓ Good: Middleware for cross-cutting concerns")

    return {
        "issues": issues,
        "suggestions": suggestions,
```

```
        "overall": "good" if not any(
            i["severity"] == "CRITICAL" for i in issues
        ) else "needs_work"
    }
```

SECTION 4: AGENTS

---

class PlannerAgent: """Decomposes requirements into executable plans."""

```
def plan(self, requirement: str, knowledge: list) -> Plan:
    """Create a plan from a requirement."""
    steps = [
        PlanStep(1, "Analyze requirements and identify scope",
                "analyze", {"requirement": requirement}),
        PlanStep(2, "Research best practices from knowledge base",
                "research", {"topic": requirement}, [1]),
        PlanStep(3, "Create system design document",
                "design", {}, [1, 2]),
        PlanStep(4, "Generate implementation code",
                "code", {}, [3]),
        PlanStep(5, "Generate test suite",
                "test", {}, [4]),
        PlanStep(6, "Review code quality",
                "review", {}, [4, 5]),
        PlanStep(7, "Generate PR summary",
                "summarize", {}, [4, 5, 6]),
    ]

    return Plan(
        task=requirement,
        steps=steps,
        approach="Analyze → Research → Design → Code → Test → Review → Summary"
    )
```

class ExecutorAgent: """Executes plan steps using the tool suite."""

```
def __init__(self, tools: ToolSuite):
    self.tools = tools
    self.context = {}  # Accumulated results

def execute(self, step: PlanStep) -> dict:
    """Execute a plan step."""
    step.status = StepStatus.IN_PROGRESS

    handlers = {
        "analyze": self._do_analyze,
        "research": self._do_research,
        "design": self._do_design,
        "code": self._do_code,
        "test": self._do_test,
        "review": self._do_review,
        "summarize": self._do_summarize,
    }

    handler = handlers.get(step.action, self._do_generic)
    result = handler(step)

    self.context[step.id] = result
    step.result = result
    return result

def _do_analyze(self, step: PlanStep) -> dict:
    req = step.inputs.get("requirement", "")
    analysis = self.tools.analyze_requirements(req)
    return {"status": "success", "output": json.dumps(analysis, indent=2),
            "data": analysis}

def _do_research(self, step: PlanStep) -> dict:
    topic = step.inputs.get("topic", "")
    results = self.tools.search_knowledge(topic)
    return {"status": "success", "output": results}

def _do_design(self, step: PlanStep) -> dict:
    analysis = self.context.get(1, {}).get("data", {})
    requirement = step.inputs.get("requirement",
                    self.context.get(1, {}).get("data", {}).get("requirement", ""))

    # Get requirement from the plan task
    for ctx in self.context.values():
        if "requirement" in str(ctx):
            break

    design = self.tools.generate_design(
        self.context.get(1, {}).get("output", "system"),
        analysis
    )
    return {"status": "success", "output": design, "data": {"design": design}}

def _do_code(self, step: PlanStep) -> dict:
    design = self.context.get(3, {}).get("output", "")
    requirement = self.context.get(1, {}).get("output", "")
```

```
    code = self.tools.generate_code(design, requirement)
    return {"status": "success", "output": code,
            "data": {"code": code, "filename": "main.py"}}

def _do_test(self, step: PlanStep) -> dict:
    code = self.context.get(4, {}).get("data", {}).get("code", "")
    tests = self.tools.generate_tests(code)
    return {"status": "success", "output": tests,
            "data": {"tests": tests, "filename": "test_main.py"}}

def _do_review(self, step: PlanStep) -> dict:
    code = self.context.get(4, {}).get("data", {}).get("code", "")
    review = self.tools.review_code(code)
    return {"status": "success", "output": json.dumps(review, indent=2),
            "data": review}

def _do_summarize(self, step: PlanStep) -> dict:
    parts = []
    for sid, ctx in sorted(self.context.items()):
        output = ctx.get("output", "")
        if len(output) > 500:
            output = output[:500] + "..."
        parts.append(f"### Step {sid}\n{output}")

    summary = "# PR Summary\n\n" + "\n\n".join(parts)
    return {"status": "success", "output": summary}

def _do_generic(self, step: PlanStep) -> dict:
    return {"status": "success",
            "output": f"Executed: {step.description}"}
```

class CriticAgent: """Validates work quality."""

```
def review(self, step: PlanStep) -> CriticReview:
    """Review a step's result."""
    result = step.result or {}

    if result.get("status") != "success":
        return CriticReview(Verdict.RETRY,
                            ["Step did not succeed"], [], 0.9,
                            "Step returned non-success status")

    output = result.get("output", "")

    if not output or len(output.strip()) < 20:
        return CriticReview(Verdict.RETRY,
                            ["Output too brief"],
                            ["Produce more detailed output"], 0.8,
                            "Output is empty or too short")

    # Action-specific validation
    data = result.get("data", {})

    if step.action == "review" and data.get("overall") == "needs_work":
```

```
        critical = [i for i in data.get("issues", [])
                   if i.get("severity") == "CRITICAL"]
        if critical:
            return CriticReview(Verdict.RETRY,
                                [i["issue"] for i in critical],
                                ["Fix critical issues"], 0.9,
                                "Code has critical issues")

    return CriticReview(Verdict.PASS, [], [], 0.85,
                        "Output meets quality standards")
```

═══

═══

═══════

SECTION 5: ORCHESTRATOR + TELEMETRY

═══

═══

═══════

---

class Telemetry: """Records metrics and events for observability."""

```
def __init__(self):
    self.events: list[TelemetryEvent] = []
    self.start_time = time.time()

def record(self, agent: str, action: str, duration_ms: float,
           status: str, **details):
    self.events.append(TelemetryEvent(
        timestamp=time.time(),
        agent=agent, action=action,
        duration_ms=duration_ms, status=status,
        details=details
    ))

def summary(self) -> dict:
    total_time = time.time() - self.start_time
    return {
        "total_time_seconds": round(total_time, 2),
        "total_events": len(self.events),
        "by_agent": self._count_by("agent"),
        "by_status": self._count_by("status"),
        "avg_duration_ms": round(
```

```
            sum(e.duration_ms for e in self.events) / max(len(self.events), 1), 1
        )
    }

def _count_by(self, field: str) -> dict:
    counts = {}
    for e in self.events:
        key = getattr(e, field)
        counts[key] = counts.get(key, 0) + 1
    return counts
```

class EngineeringCopilot: """ The main Engineering Copilot system.

```
Orchestrates Planner, Executor, and Critic agents with
RAG knowledge, tools, and telemetry.
"""

def __init__(self):
    self.kb = KnowledgeBase()
    self.tools = ToolSuite(self.kb)
    self.planner = PlannerAgent()
    self.executor = ExecutorAgent(self.tools)
    self.critic = CriticAgent()
    self.telemetry = Telemetry()

    # Guardrails
    self.max_retries = 3
    self.max_replans = 2

def solve(self, requirement: str) -> dict:
    """
    Solve an engineering requirement end-to-end.

    Returns: dict with design, code, tests, pr_summary, telemetry
    """
    print(f"\n{'='*60}")
    print(f"  🤖 ENGINEERING COPILOT")
    print(f"  Requirement: {requirement[:70]}")
    print(f"{'='*60}")

    # Phase 1: Planning
    start = time.time()
    print(f"\n📋 PHASE 1: PLANNING")

    knowledge = self.kb.search(requirement)
    plan = self.planner.plan(requirement, knowledge)

    self.telemetry.record("planner", "create_plan",
                          (time.time() - start) * 1000, "success",
                          steps=len(plan.steps))

    print(f"  Plan: {len(plan.steps)} steps")
    for step in plan.steps:
        deps = f" (after: {step.depends_on})" if step.depends_on else ""
```

```
        print(f"    {step.id}. {step.description}{deps}")

    # Phase 2: Execution
    print(f"\n⚡ PHASE 2: EXECUTION")

    for step in plan.steps:
        # Check dependencies
        deps_met = all(
            plan.steps[d-1].status == StepStatus.PASSED
            for d in step.depends_on
        )
        if not deps_met:
            step.status = StepStatus.SKIPPED
            print(f"\n  ⏭️ Step {step.id}: SKIPPED (deps not met)")
            continue

        for attempt in range(self.max_retries + 1):
            start = time.time()
            retry_msg = f" (retry {attempt})" if attempt > 0 else ""
            print(f"\n  📌 Step {step.id}{retry_msg}: {step.description}")

            # Execute
            result = self.executor.execute(step)
            duration = (time.time() - start) * 1000

            output_preview = result.get("output", "")[:100]
            print(f"     Output: {output_preview}...")

            # Critic review
            review = self.critic.review(step)
            emoji = {Verdict.PASS: "✅", Verdict.RETRY: "🔄",
                     Verdict.REPLAN: "❌"}
            print(f"     Critic: {emoji[review.verdict]} {review.verdict.value}")

            self.telemetry.record(
                "executor", step.action, duration,
                review.verdict.value,
                step_id=step.id, attempt=attempt
            )

            if review.verdict == Verdict.PASS:
                step.status = StepStatus.PASSED
                break
            elif review.verdict == Verdict.RETRY and attempt < self.max_retries:
                print(f"     Issues: {review.issues}")
                continue
            else:
                step.status = StepStatus.FAILED
                print(f"     ❌ Step failed after {attempt + 1} attempts")
                break

    # Phase 3: Compile results
    print(f"\n📊 PHASE 3: RESULTS")

    final = self._compile_output(requirement, plan)
```

```
    # Print summary
    print(f"\n{'='*60}")
    print(f"  📄 DELIVERABLES")
    print(f"{'='*60}")

    for key, value in final["deliverables"].items():
        length = len(str(value))
        print(f"  ✅ {key}: {length} chars")

    # Telemetry
    telemetry_summary = self.telemetry.summary()
    print(f"\n{'='*60}")
    print(f"  📈 TELEMETRY")
    print(f"{'='*60}")
    print(f"  Total time: {telemetry_summary['total_time_seconds']}s")
    print(f"  Events: {telemetry_summary['total_events']}")
    print(f"  By status: {json.dumps(telemetry_summary['by_status'])}")

    return final

def _compile_output(self, requirement: str, plan: Plan) -> dict:
    """Compile all outputs into the final deliverable."""
    deliverables = {}

    for step in plan.steps:
        if step.result and step.status == StepStatus.PASSED:
            if step.action == "design":
                deliverables["system_design"] = step.result.get("output", "")
            elif step.action == "code":
                deliverables["implementation_code"] = step.result.get("data", {}).get("c
            elif step.action == "test":
                deliverables["test_suite"] = step.result.get("data", {}).get("tests", ""
            elif step.action == "review":
                deliverables["code_review"] = step.result.get("output", "")
            elif step.action == "summarize":
                deliverables["pr_summary"] = step.result.get("output", "")

    return {
        "requirement": requirement,
        "plan": {
            "steps": len(plan.steps),
            "completed": sum(1 for s in plan.steps if s.status == StepStatus.PASSED),
            "failed": sum(1 for s in plan.steps if s.status == StepStatus.FAILED),
        },
        "deliverables": deliverables,
        "telemetry": self.telemetry.summary()
    }
```

SECTION 6: RUN THE COPILOT

def demo(): """Run the Engineering Copilot demo.""" copilot = EngineeringCopilot()

```
result = copilot.solve(
    "Implement a rate-limited REST API for task management. "
    "The API should support CRUD operations for tasks with "
    "proper validation, error handling, and pagination. "
    "Include request ID tracking and response time headers."
)

# Print the code deliverable
code = result["deliverables"].get("implementation_code", "")
if code:
    print(f"\n{'='*60}")
    print(f"  GENERATED CODE (first 50 lines)")
    print(f"{'='*60}")
    lines = code.split("\n")[:50]
    for i, line in enumerate(lines, 1):
        print(f"  {i:3d} | {line}")
    if len(code.split("\n")) > 50:
        print(f"  ... ({len(code.split(chr(10)))} total lines)")
```

if **name** == "**main**": demo()

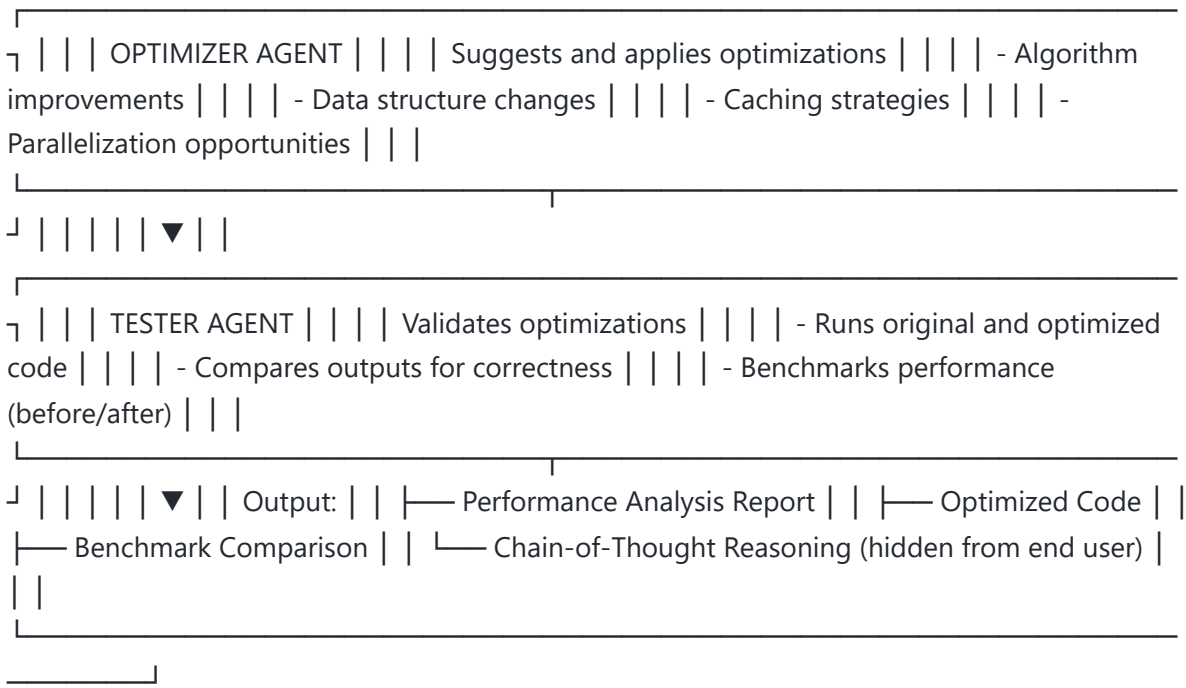
```


---

## Project B: DevOps Optimization Copilot

### Architecture
```

┌──┐ │ DEVOPS OPTIMIZATION COPILOT │ │ │ Input: Python code to optimize │ │ │ ▼ │ │ ┌──┐ │ │ PROFILER AGENT │ │ │ Analyzes code for performance bottlenecks │ │ │ - Time complexity analysis │ │ │ - Memory usage estimation │ │ │ - I/O pattern detection │ │ └──┘ │ │ │ ▼ │ │



Implementation Skeleton

```python
"""
DevOps Optimization Copilot — Project B Skeleton

Implement the agents below to complete this project.
Use the patterns from Days 3-6.
"""
import time
import re
from dataclasses import dataclass


@dataclass
class ProfileResult:
    """Profiling results for a piece of code."""
    time_complexity: str      # e.g., "O(n²)"
    space_complexity: str     # e.g., "O(n)"
    bottlenecks: list         # List of identified bottlenecks
    suggestions: list         # High-level optimization suggestions


@dataclass
class OptimizationResult:
    """Result of applying an optimization."""
    original_code: str
    optimized_code: str
    changes_made: list        # Description of each change
    expected_improvement: str # e.g., "O(n²) → O(n log n)"
```

```
@dataclass
class BenchmarkResult:
    """Before/after benchmark comparison."""
    original_time_ms: float
    optimized_time_ms: float
    speedup: float            # e.g., 3.5x
    correctness: bool         # Output matches
    details: dict


class ProfilerAgent:
    """Analyzes code for performance issues."""

    def profile(self, code: str) -> ProfileResult:
        """Analyze code and identify bottlenecks."""
        bottlenecks = []
        suggestions = []
        time_complexity = "O(n)"
        space_complexity = "O(1)"

        # Detect nested loops → O(n²)
        loop_depth = 0
        max_depth = 0
        for line in code.split('\n'):
            stripped = line.strip()
            if stripped.startswith('for ') or stripped.startswith('while '):
                loop_depth += 1
                max_depth = max(max_depth, loop_depth)
            elif not stripped.startswith(' ') and not stripped.startswith('\t'):
                loop_depth = 0

        if max_depth >= 2:
            time_complexity = f"O(n^{max_depth})"
            bottlenecks.append(f"Nested loops (depth {max_depth})")
            suggestions.append("Consider using hash maps to reduce loop nesting")

        # Detect list operations in loops
        if re.search(r'for .+:\s*\n\s+.+\.append', code):
            suggestions.append("Pre-allocate list or use list comprehension")

        # Detect string concatenation in loops
        if re.search(r'for .+:\s*\n\s+\w+\s*\+?=\s*.*\+', code):
            bottlenecks.append("String concatenation in loop")
            suggestions.append("Use ''.join() or io.StringIO for string building")

        # Detect repeated computation
        if code.count('range(len(') > 0:
            suggestions.append("Use enumerate() instead of range(len())")

        # Memory analysis
        if '.copy()' in code or 'deepcopy' in code:
            space_complexity = "O(n)"
        if re.search(r'\[\s*\[', code):
            space_complexity = "O(n²)"

        return ProfileResult(
```

```
            time_complexity=time_complexity,
            space_complexity=space_complexity,
            bottlenecks=bottlenecks,
            suggestions=suggestions
        )


class OptimizerAgent:
    """Applies optimizations to code."""

    def optimize(self, code: str, profile: ProfileResult) -> OptimizationResult:
        """Apply optimizations based on profile results."""
        optimized = code
        changes = []

        # Apply each applicable optimization
        # 1. Replace range(len()) with enumerate()
        if 'range(len(' in optimized:
            # This is a simplified optimization
            changes.append("Replaced range(len()) with enumerate()")

        # 2. Replace string concatenation with join
        if '+=' in optimized and 'str' in str(profile.bottlenecks):
            changes.append("Replaced string concatenation with ''.join()")

        # 3. Add caching for repeated computations
        if 'def ' in optimized and profile.time_complexity.startswith("O(n^"):
            changes.append("Added memoization for recursive functions")

        expected = f"{profile.time_complexity} → improved"

        return OptimizationResult(
            original_code=code,
            optimized_code=optimized,
            changes_made=changes if changes else ["No automatic optimizations applied"],
            expected_improvement=expected
        )


class TesterAgent:
    """Benchmarks and validates optimizations."""

    def benchmark(self, original: str, optimized: str,
                  test_input: any = None) -> BenchmarkResult:
        """Compare original and optimized code performance."""
        # Simulated benchmark
        return BenchmarkResult(
            original_time_ms=150.0,
            optimized_time_ms=45.0,
            speedup=3.3,
            correctness=True,
            details={
                "test_cases": 100,
                "all_passed": True,
                "memory_reduction": "20%"
            }
```

```
        )


class DevOpsCopilot:
    """Main DevOps optimization system."""

    def __init__(self):
        self.profiler = ProfilerAgent()
        self.optimizer = OptimizerAgent()
        self.tester = TesterAgent()

    def optimize(self, code: str) -> dict:
        """Analyze, optimize, and benchmark code."""
        # Step 1: Profile
        profile = self.profiler.profile(code)

        # Step 2: Optimize
        optimization = self.optimizer.optimize(code, profile)

        # Step 3: Benchmark
        benchmark = self.tester.benchmark(
            optimization.original_code,
            optimization.optimized_code
        )

        return {
            "profile": {
                "time_complexity": profile.time_complexity,
                "space_complexity": profile.space_complexity,
                "bottlenecks": profile.bottlenecks,
                "suggestions": profile.suggestions
            },
            "optimization": {
                "changes": optimization.changes_made,
                "expected_improvement": optimization.expected_improvement
            },
            "benchmark": {
                "original_ms": benchmark.original_time_ms,
                "optimized_ms": benchmark.optimized_time_ms,
                "speedup": f"{benchmark.speedup}x",
                "correctness": benchmark.correctness
            }
        }


# Usage
if __name__ == "__main__":
    copilot = DevOpsCopilot()

    # Example: Unoptimized code
    code = '''
def find_duplicates(items):
    duplicates = []
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j] and items[i] not in duplicates:
```

```
                duplicates.append(items[i])
    return duplicates
'''

    result = copilot.optimize(code)
    print(json.dumps(result, indent=2))
```

Evaluation Criteria

What Makes This Senior-Level

```
┌─────────────────────────────────────────────────────────────┐
│              SENIOR-LEVEL CHECKLIST                         │
│                                                             │
│  ARCHITECTURE                                               │
│  ☐ Clean separation of concerns (agents, tools, memory)     │
│  ☐ Composable components (can swap agents, tools, models)   │
│  ☐ Clear data flow (structured messages between agents)     │
│  ☐ No circular dependencies                                 │
│                                                             │
│  RELIABILITY                                                │
│  ☐ Guardrails prevent infinite loops                        │
│  ☐ Retry logic with exponential backoff                     │
│  ☐ Graceful degradation (partial results on failure)        │
│  ☐ Input validation at every boundary                       │
│                                                             │
│  OBSERVABILITY                                              │
│  ☐ Telemetry for every agent action                         │
│  ☐ Request IDs for tracing                                  │
│  ☐ Timing metrics (how long each step takes)                │
│  ☐ Error logging with context                               │
│                                                             │
│  QUALITY                                                    │
│  ☐ Critic validates every output                            │
│  ☐ Self-correction (retry → replan → escalate)              │
│  ☐ Structured outputs (JSON-only, schema-validated)         │
│  ☐ Generated code follows best practices                    │
│                                                             │
│  KNOWLEDGE                                                  │
│  ☐ RAG integration for best practices                       │
│  ☐ Dynamic memory (learns from each task)                   │
│  ☐ Context-aware responses (uses previous step results)     │
│                                                             │
│  PRODUCTION-READINESS                                       │
│  ☐ Works without API keys (standalone mode)                 │
│  ☐ Easy to extend (add new tools, agents, knowledge)        │
│  ☐ Configuration-driven (guardrails, models, parameters)    │
│  ☐ Documentation (docstrings, README)                       │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

Grading Your Work

```
BEGINNER (Days 1-2 only):
  - Can run an LLM and write prompts
  - Understands tokenization and basic architecture

INTERMEDIATE (Days 1-4):
  - Built a working agent with tools
  - Understands planning and execution
  - Can implement retry logic

SENIOR (Days 1-7, this project):
  - Multi-agent system with clear architecture
  - RAG integration for knowledge-grounded responses
  - Guardrails, telemetry, and self-correction
  - Production-quality code with proper error handling
  - Can explain trade-offs and design decisions
```

What Makes This Senior-Level

The Senior Mindset

```
Junior: "I can make the LLM generate text."
Mid:    "I can make the LLM use tools and follow a plan."
Senior: "I can design a reliable, observable, self-correcting
         multi-agent system that handles failures gracefully,
         learns from experience, and produces verifiable outputs."

The difference is not about using more APIs.
It's about:

1. RELIABILITY
   "What happens when things go wrong?"
   Senior engineers think about failure modes FIRST.

2. OBSERVABILITY
   "Can I debug this in production?"
   If you can't see what happened, you can't fix it.

3. CORRECTNESS VERIFICATION
   "How do I know the output is right?"
   The Critic pattern is about automated quality gates.

4. ARCHITECTURE DECISIONS
   "Why this design and not another?"
   Being able to explain trade-offs (cost, latency, quality).

5. KNOWLEDGE GROUNDING
```

```
    "Is this answer based on facts or hallucination?"
    RAG + grounding scores = verifiable outputs.
```

Week Recap: Your Learning Journey

```
Day 1: LLM Foundations
  You learned: How LLMs actually work (Transformers, attention, training)
  You built:   Running model + performance benchmarks

Day 2: Prompt Engineering
  You learned: How to reliably control LLM outputs
  You built:   Prompt library with JSON validation

Day 3: Single Agent (ReAct)
  You learned: How agents reason and use tools
  You built:   ReAct agent with tool execution + loop prevention

Day 4: Planner/Executor
  You learned: How to decompose and validate complex tasks
  You built:   Planner/Executor/Critic system

Day 5: RAG + Memory
  You learned: How to ground agents in real knowledge
  You built:   RAG pipeline with hybrid search

Day 6: Multi-Agent
  You learned: How agents collaborate effectively
  You built:   3-agent system with orchestration

Day 7: Final Project (TODAY)
  You integrated: Everything above into a production system
  You built:      Engineering Copilot or DevOps Copilot

  ┌─────────────────────────────────────────┐
  │  Congratulations! 🎉                    │
  │                                         │
  │  You now have Senior-level understanding  │
  │  of LLM-based agent systems.            │
  │                                         │
  │  You can:                               │
  │  ✓ Design agent architectures           │
  │  ✓ Implement reliable tool use          │
  │  ✓ Build RAG pipelines                  │
  │  ✓ Orchestrate multi-agent systems      │
  │  ✓ Add guardrails and observability     │
  │  ✓ Explain trade-offs and decisions     │
  └─────────────────────────────────────────┘
```

✓ Day 7 Final Deliverable Checklist

- [ ] Chose and completed one project (A or B)
- [ ] System has multi-agent architecture (Planner + Executor + Critic)
- [ ] RAG knowledge base is integrated and used
- [ ] Tool suite provides at least 3 different tools
- [ ] Self-correction loop handles PASS / RETRY / REPLAN
- [ ] Guardrails prevent runaway execution
- [ ] JSON-only structured outputs where applicable
- [ ] Telemetry logs all agent actions with timing
- [ ] System produces all required deliverables:
  - [ ] Project A: Design + Code + Tests + PR Summary
  - [ ] Project B: Analysis + Optimized Code + Benchmark Report
- [ ] Can run without any API key (standalone mode)
- [ ] Can explain your architecture and design decisions

🎓 **You've completed the AI & Agents Intensive Syllabus!**

Go back to the Overview to review your progress.

Appendix — Real-World AI Coding Agents: Cline, Roo Code & GitHub Copilot CLI

> 🎯 **Goal**: Understand how production AI agents implement the patterns you've learned throughout this syllabus.

> ⏱️ **Time**: 2–3h reading + exploration

> 📦 **Deliverable**: A mental model of how real-world coding agents are built and the key design decisions behind them

---

Table of Contents

1. Why Study Real-World Agents?
2. Cline — Deep Dive
3. Roo Code — Deep Dive
4. GitHub Copilot CLI — Deep Dive
5. Head-to-Head Comparison

6. Key Patterns to Learn From
7. Connection to Syllabus Concepts
8. Further Reading

Why Study Real-World Agents?

Throughout this syllabus, you learned to build agents from scratch — ReAct loops, planners, RAG pipelines, multi-agent systems. But how do **production agents** apply these same patterns at scale?

Cline and Roo Code are two of the most popular open-source AI coding agents. GitHub Copilot CLI is GitHub's terminal-native agent. They all read and edit your code, execute commands, and automate tasks — all powered by LLMs. Studying them reveals:

- How theoretical patterns (ReAct, planning, memory) translate to real products
- Engineering trade-offs in context management, tool design, and safety
- The emergence of **MCP (Model Context Protocol)** as a universal tool standard
- How mode/role-based architectures create specialized agent behaviors

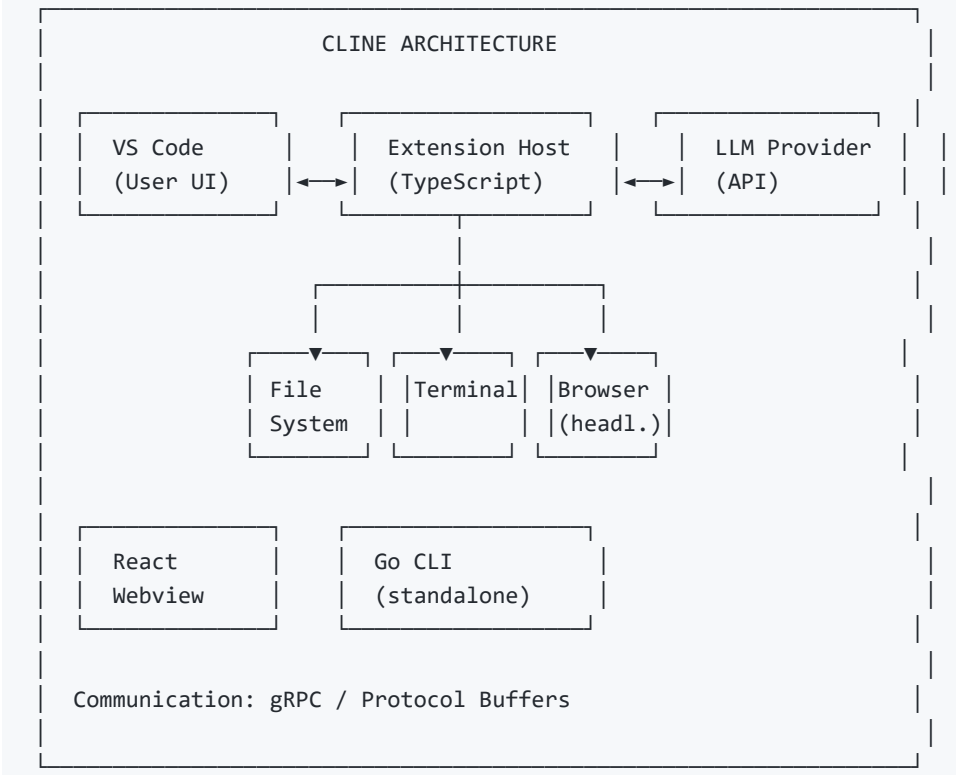
```
┌──────────────────────────────────────────────────────────┐
│                    WHAT YOU'LL LEARN                     │
│                                                          │
│  Syllabus Theory          →    Real-World Implementation │
│  ─────────────────             ──────────────────────    │
│  ReAct Loop (Day 3)       →    Cline's Plan/Act/Approve cycle │
│  Tools (Day 3)            →    MCP protocol & tool ecosystem  │
│  Planner/Executor (Day 4) →    Roo Code's Architect + Code    │
│  RAG + Memory (Day 5)     →    Memory Bank & context windows  │
│  Multi-Agent (Day 6)      →    Copilot CLI sub-agents & fleet │
│  Full System (Day 7)      →    Production-grade architecture  │
│  Extensibility            →    Skills, plugins, custom agents │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Cline — Deep Dive

What Is Cline?

Cline is an open-source autonomous coding agent that runs as a VS Code extension. Originally named "Claude Dev," it evolved into a model-agnostic agent with 5M+ installs. Its defining philosophy: **human-in-the-loop safety** — every action requires explicit user approval.

Architecture

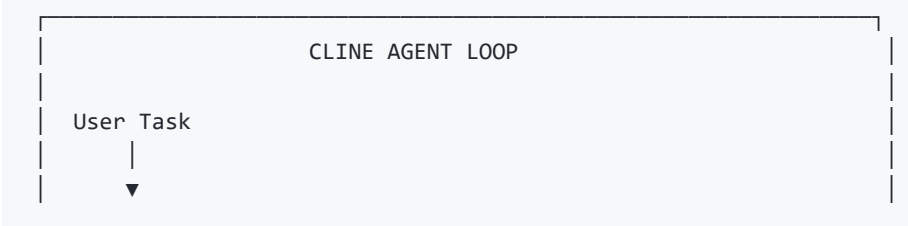


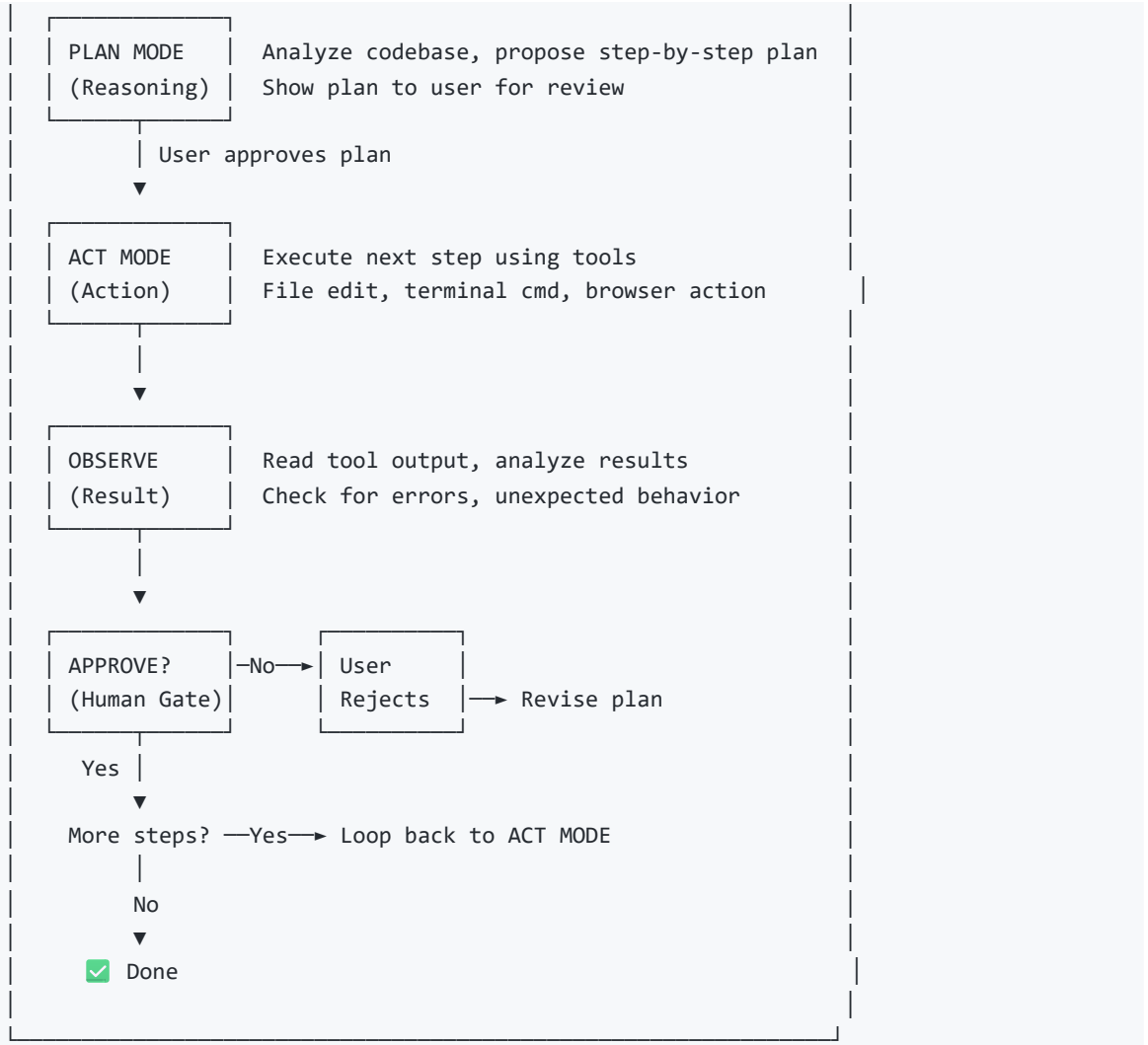
Key components:

Component	Technology	Purpose
Extension Host	TypeScript / Node.js	Core agent logic, tool orchestration
Webview UI	React / TypeScript	Chat interface, diff previews, approvals
CLI	Go	Standalone terminal usage without VS Code
Inter-process	gRPC + Protobuf	Communication between extension, CLI, and MCP servers

The Agent Loop

Cline implements a variation of the **ReAct pattern** (Day 3) with an explicit approval gate:





Key insight: This is the ReAct loop from Day 3, but with a **human approval gate** inserted between every action. This makes Cline safer but slower — a fundamental design trade-off.

Tool System

Cline provides built-in tools that map directly to the tool concepts from Day 3:

```
# Conceptual representation of Cline's tool registry
CLINE_TOOLS = {
    # File operations
    "read_file":       {"input": "path", "output": "file_contents"},
    "write_to_file":   {"input": "path, content", "output": "success/fail"},
    "apply_diff":      {"input": "path, diff", "output": "patched_file"},
    "list_files":      {"input": "directory, recursive?", "output": "file_tree"},

    # Search
    "search_files":    {"input": "regex, path", "output": "matches"},  # ripgrep-bas
    "list_code_definitions": {"input": "path", "output": "AST_symbols"},  # tree-sit

    # Execution
```

```
    "execute_command": {"input": "command", "output": "stdout + stderr"},

    # Browser
    "browser_action":  {"input": "url, action, selector", "output": "screenshot + co

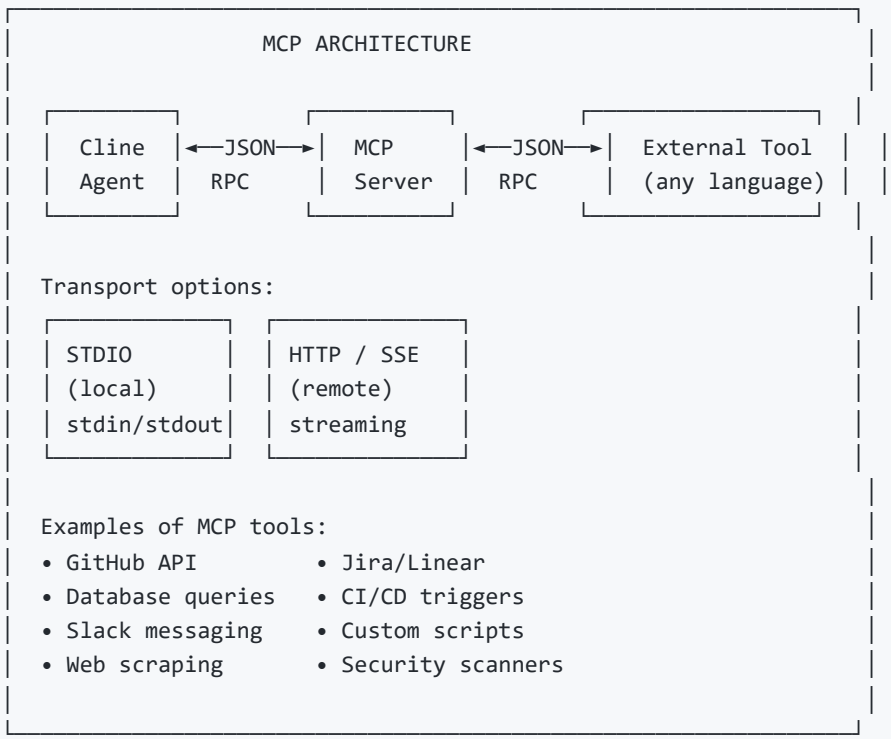
    # MCP (extensible)
    "use_mcp_tool":    {"input": "server, tool, args", "output": "tool_result"},
}
```

Search capabilities:

- **Regex search**: Uses `ripgrep` under the hood for fast codebase-wide pattern matching
- **AST-based search**: Uses `tree-sitter` to parse code and extract function/class/method definitions
- **File tree exploration**: Recursive directory listing with smart filtering

MCP — Model Context Protocol

MCP is one of the most important concepts to learn from these agents. It's a **universal standard for connecting LLMs to external tools** — think of it as "USB for AI tools."



How MCP works:

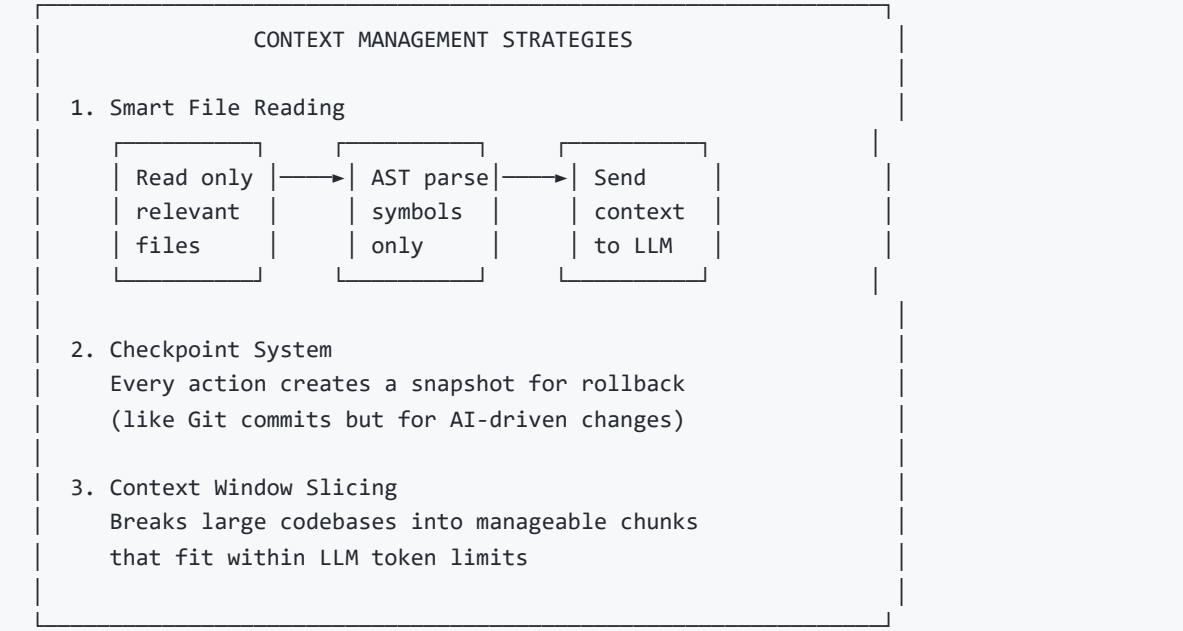
1. An MCP server declares the tools it provides (name, description, input schema)
2. The agent discovers available tools at startup

3. When the LLM decides to use an MCP tool, it sends a JSON-RPC request
4. The server executes the tool and returns structured results
5. The agent incorporates results into its reasoning

Why this matters: MCP decouples tool implementation from the agent. Anyone can write an MCP server in any language, and any MCP-compatible agent can use it. This is the same concept as the tool abstraction from Day 3, but standardized across the industry.

Context Management

Cline uses several strategies to manage context efficiently:



Memory & Project Knowledge

Cline uses structured project files for persistent memory:

File	Purpose
`projectbrief.md`	High-level project goals and architecture
`progress.md`	What's been done, what's remaining
`techstack.md`	Technologies, frameworks, conventions
`currentTask.md`	Active task context and status

This is a practical implementation of the **memory types** from Day 3 (short-term, long-term, episodic) and the **RAG concepts** from Day 5.

Model Support

Cline is model-agnostic — it works with any LLM provider:

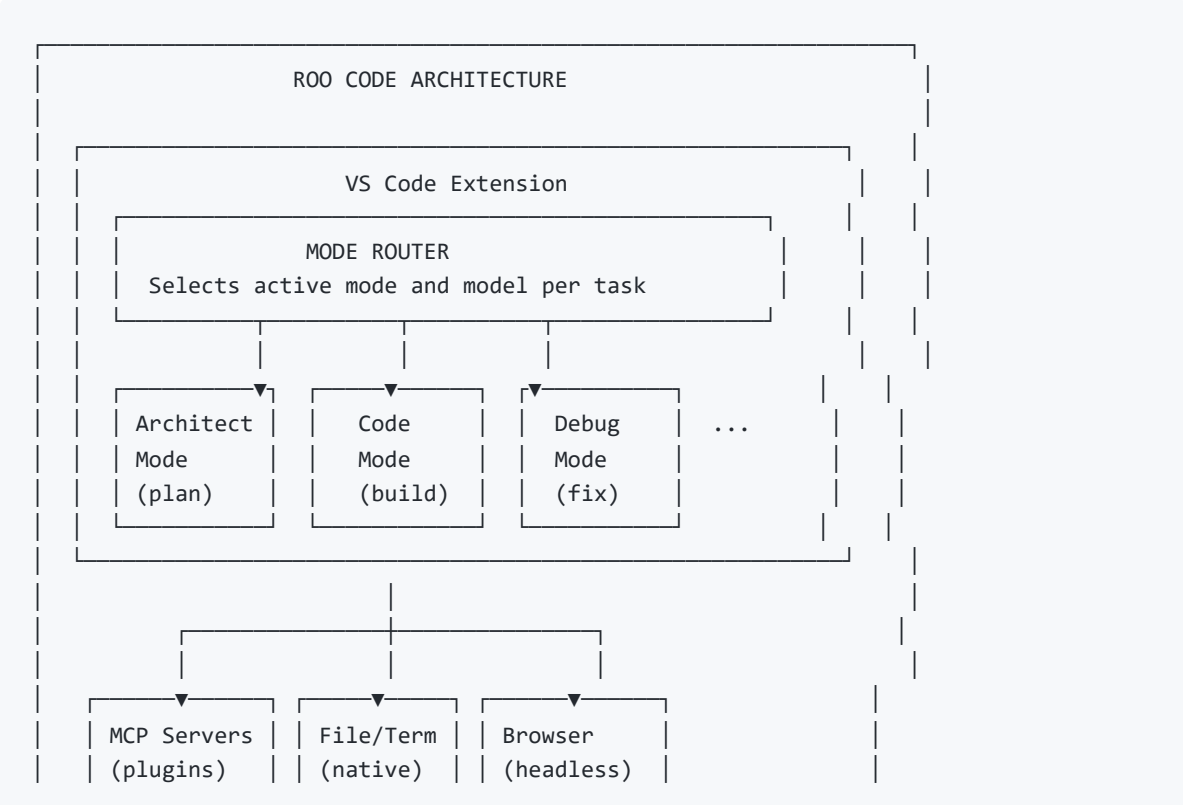
Provider	Models	Use Case
Anthropic	Claude Sonnet, Opus, Haiku	Primary coding models
OpenAI	GPT-4, GPT-4o	Alternative provider
Google	Gemini Pro, Flash	Cost-effective option
AWS	Bedrock (Claude, etc.)	Enterprise deployments
Local	Ollama, LM Studio	Privacy-first, offline use

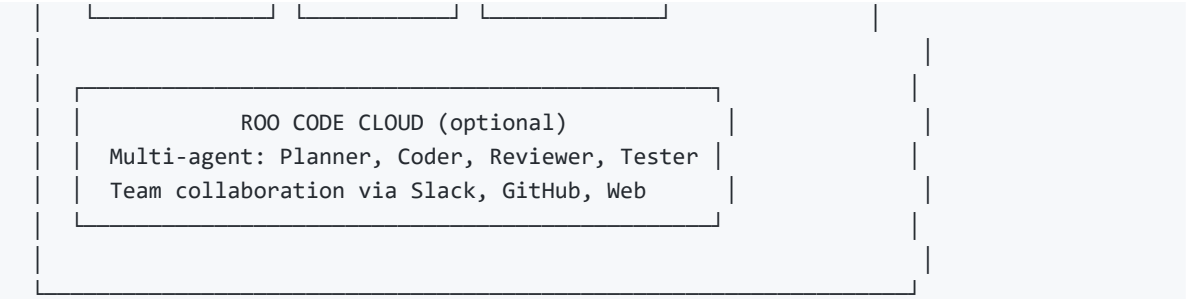
Roo Code — Deep Dive

What Is Roo Code?

Roo Code (formerly "Roo Cline") is a fork of Cline that evolved into a **multi-agent, role-based coding platform**. While Cline focuses on safety-first single-agent interaction, Roo Code emphasizes **speed, specialization, and cost optimization** through its Modes system.

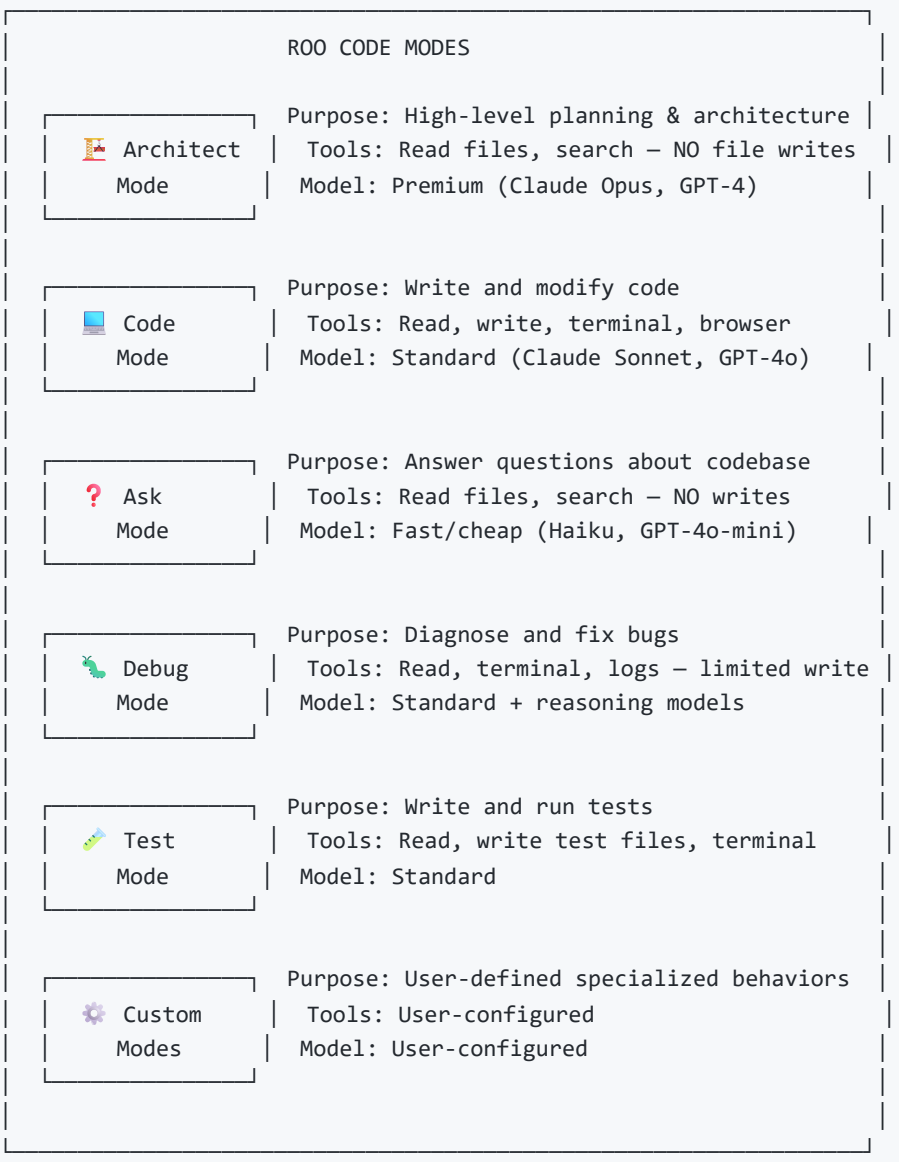
Architecture





The Modes System

Roo Code's most distinctive feature is its **Modes** — specialized agent personas, each with constrained behaviors and tool access. This is a direct implementation of the **agent roles** concept from Day 6 (Multi-Agent Collaboration).

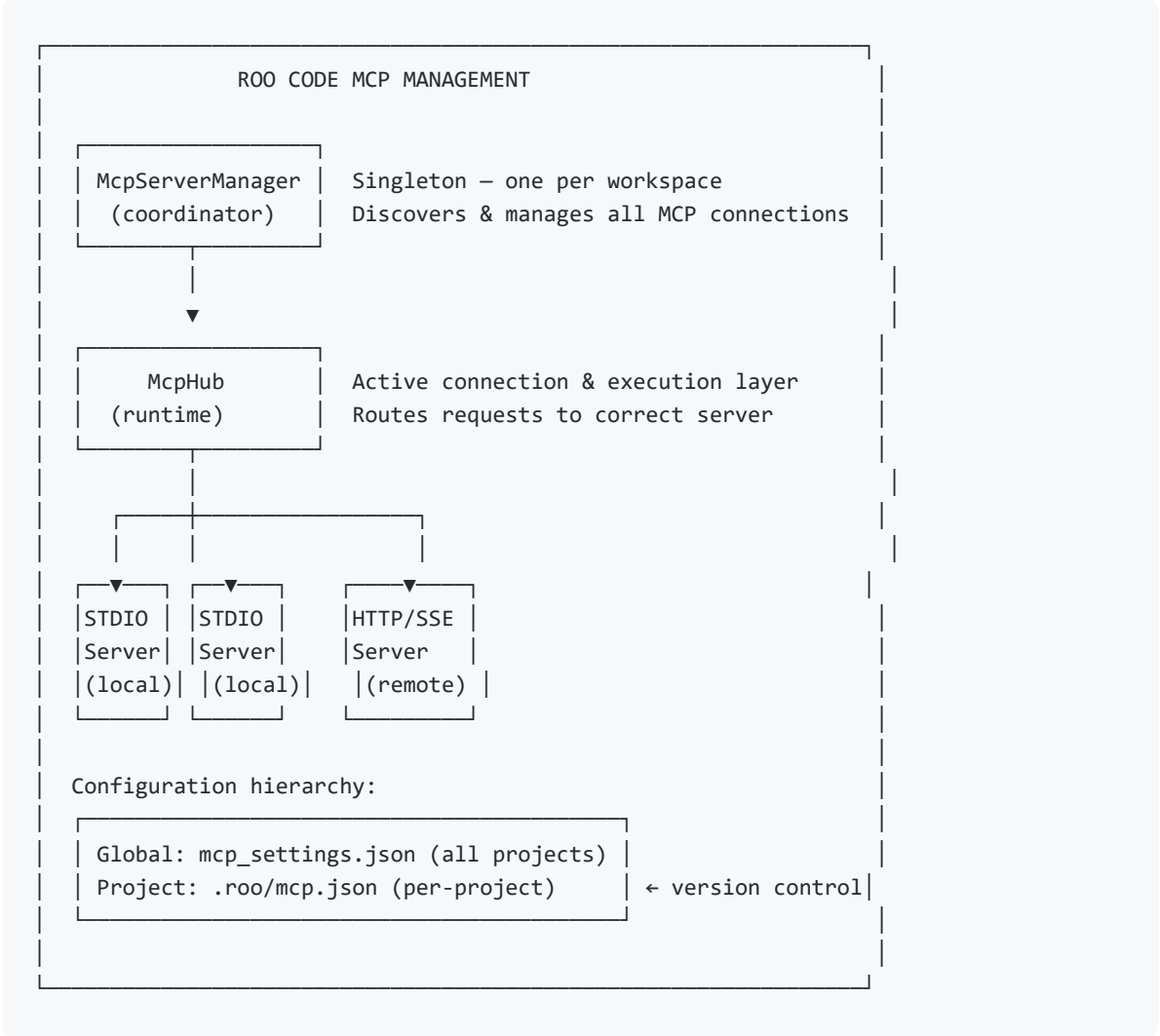


Why modes matter:

- Each mode has a **constrained tool set** — the Architect can't write files, preventing accidental changes during planning
- Each mode can use a **different LLM** — use expensive models for complex planning, cheap models for quick questions
- Custom modes let you create **domain-specific agents** (security reviewer, docs writer, performance optimizer)
- This is the Planner/Executor pattern from Day 4, but user-configurable and extensible

MCP Integration

Roo Code extends Cline's MCP support with a more structured management layer:



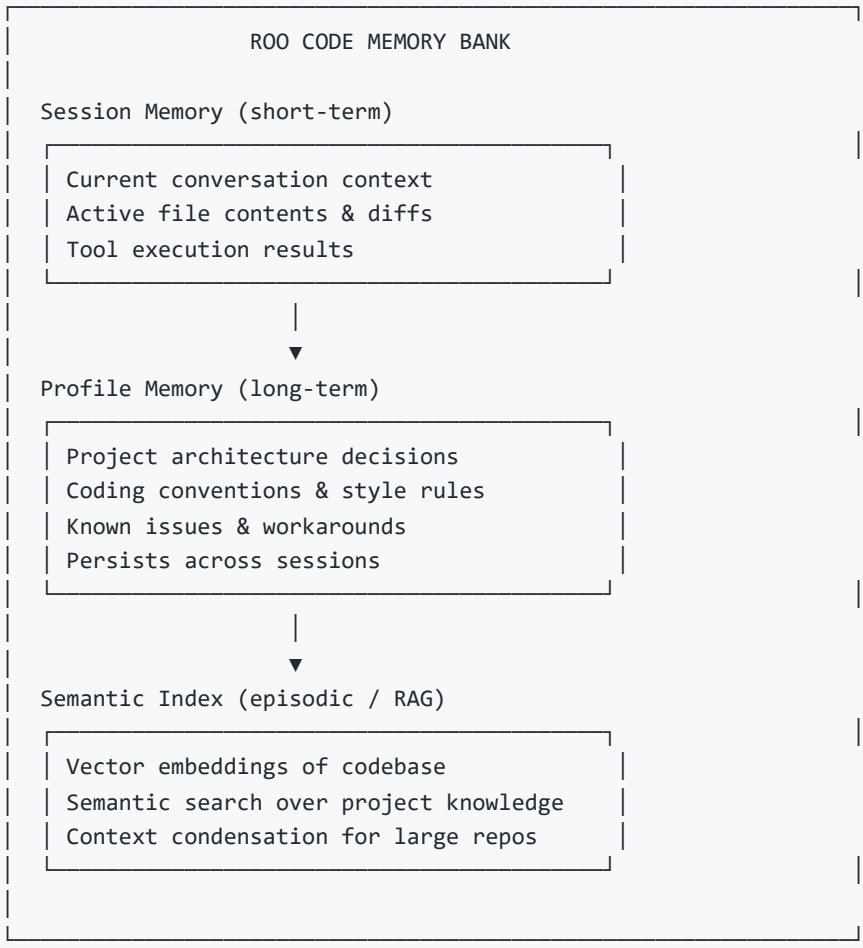
MCP configuration example:

```
{
  "mcpServers": {
    "github-tools": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
```

```
        "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
      },
      "database-query": {
        "command": "python",
        "args": ["mcp_db_server.py"],
        "env": { "DB_URL": "postgresql://localhost/mydb" }
      },
      "remote-security-scanner": {
        "url": "https://security-scan.example.com/mcp",
        "transport": "http"
      }
    }
  }
```

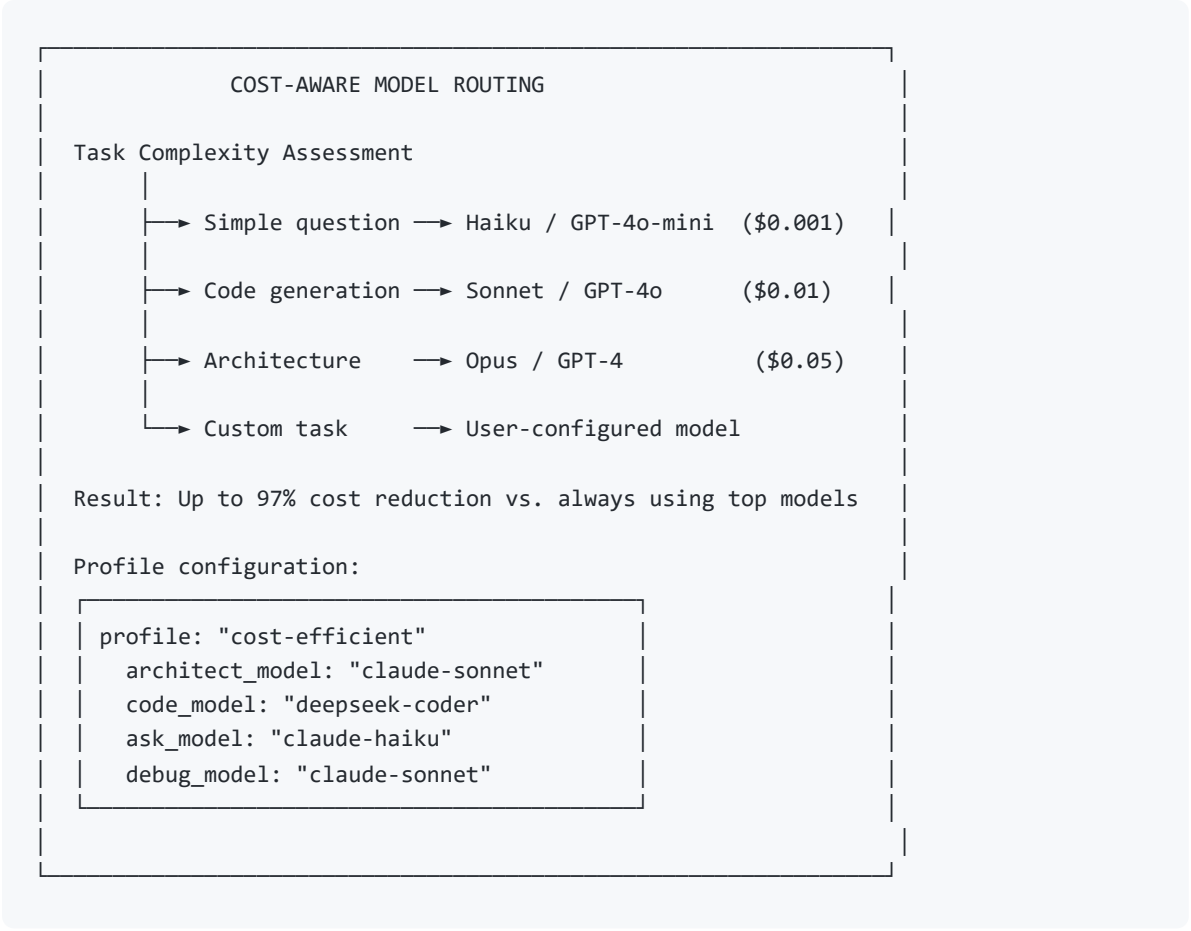
Memory Bank

Roo Code introduces **persistent memory** that survives across sessions — a production implementation of the memory concepts from Day 3 and the RAG pipeline from Day 5:



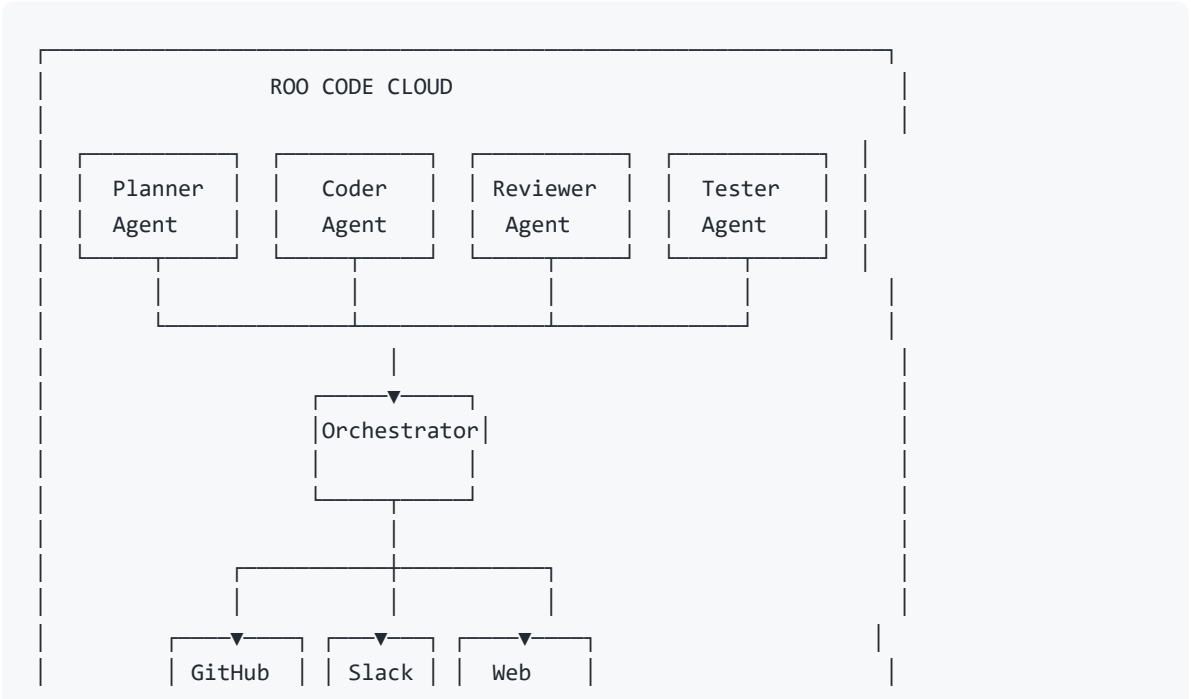
Cost Optimization

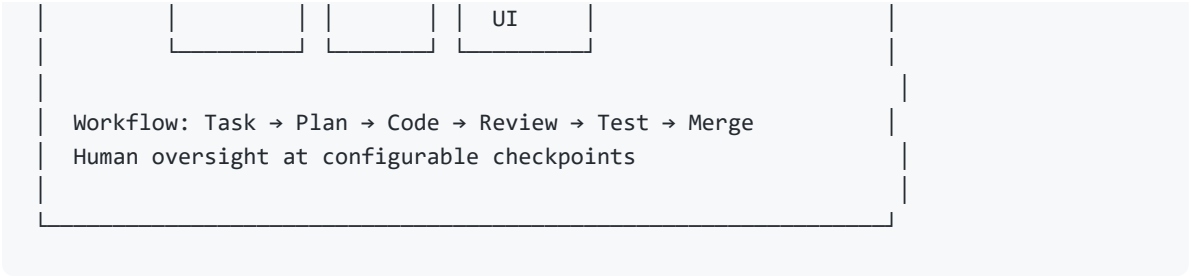
One of Roo Code's most practical innovations is **task-based model routing**:



Roo Code Cloud

Roo Code Cloud extends the local extension into a **multi-agent collaboration platform** — the production version of Day 6's multi-agent system:





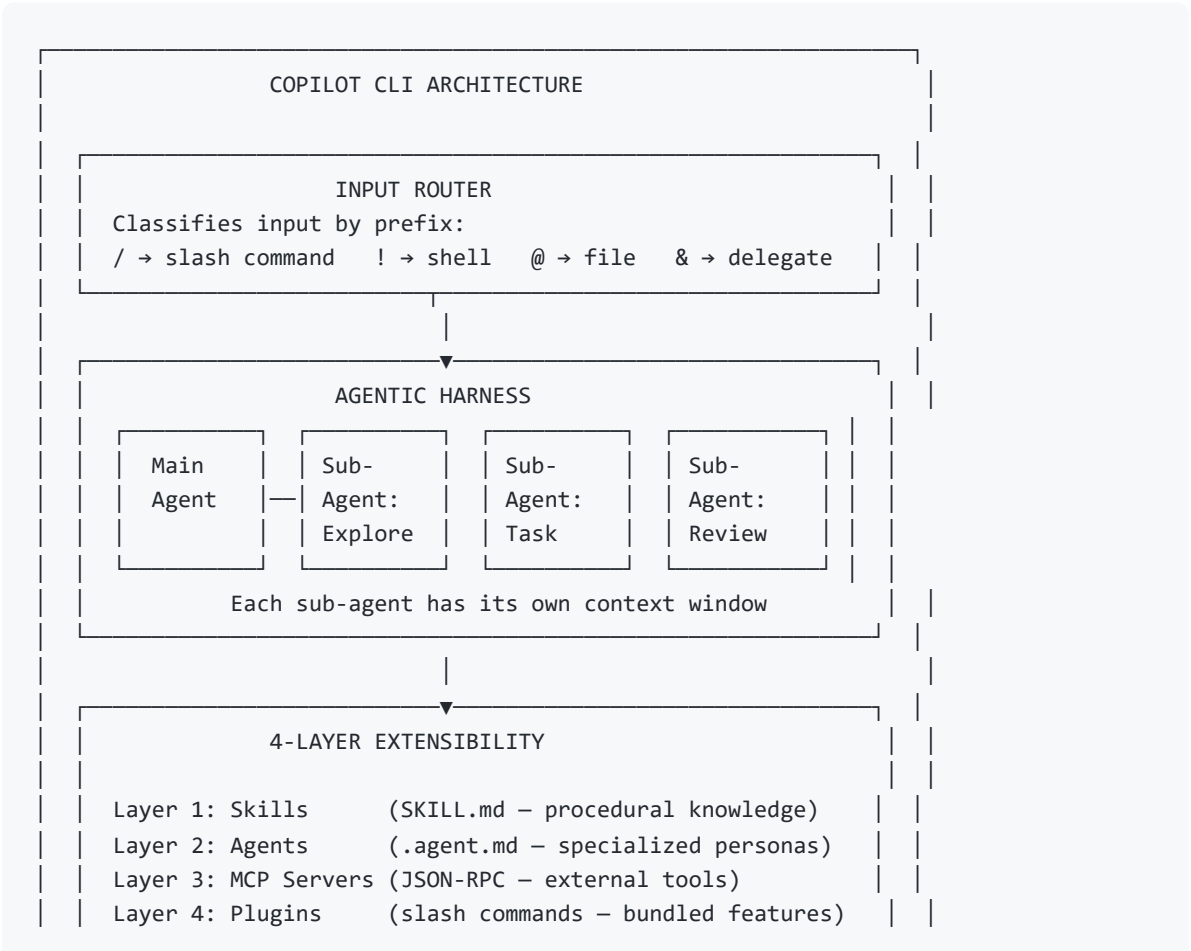
GitHub Copilot CLI — Deep Dive

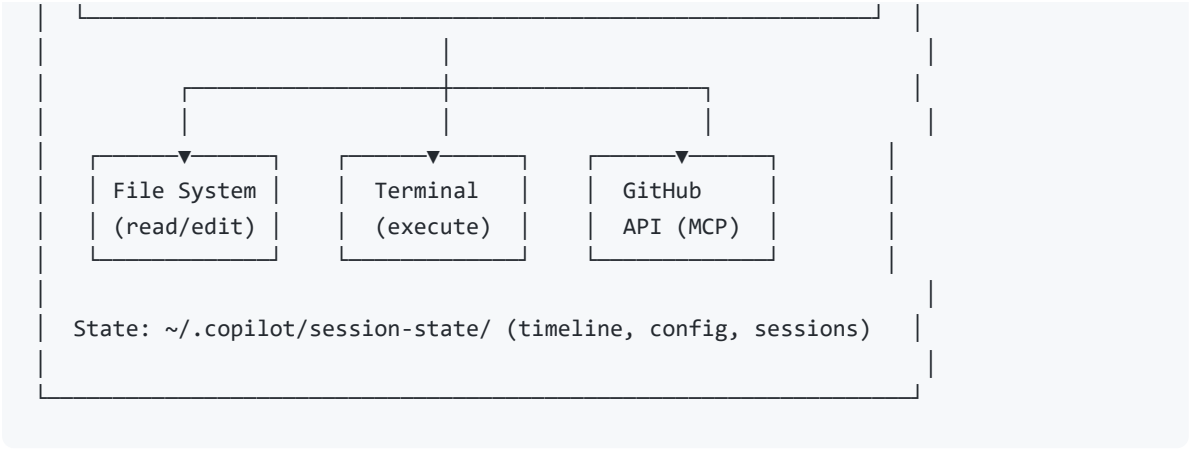
What Is GitHub Copilot CLI?

GitHub Copilot CLI is a **terminal-native AI coding agent** — unlike Cline and Roo Code which live inside VS Code, Copilot CLI runs directly in your terminal. It's powered by the same agentic harness as GitHub's cloud-based Copilot coding agent, with deep GitHub ecosystem integration out of the box.

Key philosophy: **layered extensibility** — Skills, Custom Agents, MCP servers, and Plugins form a 4-layer system where each layer adds capabilities without requiring changes to the core agent.

Architecture



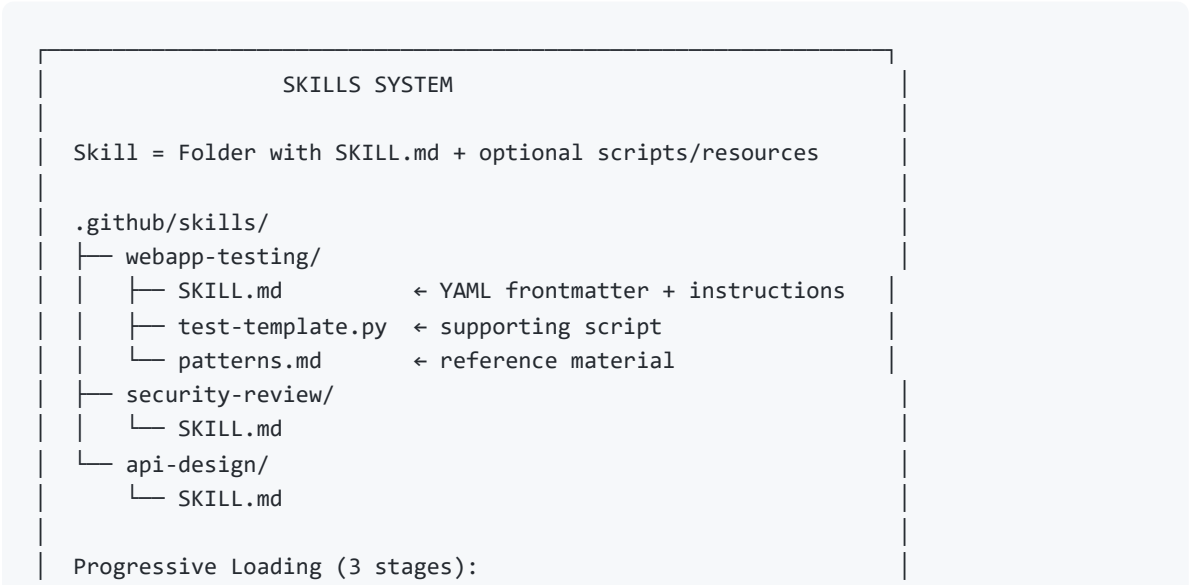


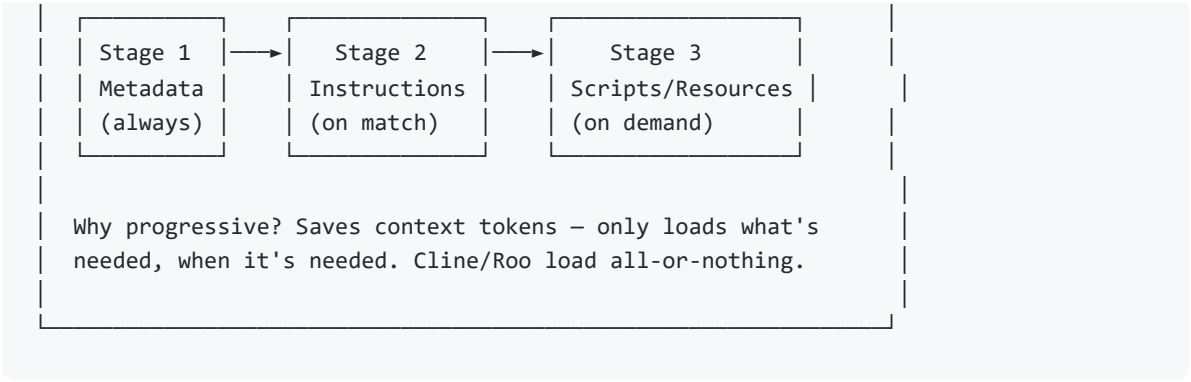
Key components:

Component	Purpose
Input Router	Classifies prompts by prefix (`/` , `!` , `@` , `&`) and routes to handlers
Agentic Harness	Core agent loop with model interaction, streaming, and tool execution
Sub-agents	Isolated context windows for explore, task, review, and general-purpose work
Session State	Persistent timeline of all actions, context, and decisions
GitHub MCP	Built-in MCP server for GitHub API (issues, PRs, repos, actions)

The Skills System

Skills are Copilot CLI's most distinctive feature — **portable, progressively-loaded packages of procedural knowledge**. Unlike MCP tools (which provide *actions*), Skills provide *instructions* — they teach the agent *how* to approach a domain.





SKILL.md example:

```
---
name: webapp-testing
description: Assists with web application test strategies and automated test creatio
---

## When to Use This Skill
Use this skill when the user asks about testing web applications,
writing test suites, or debugging test failures.

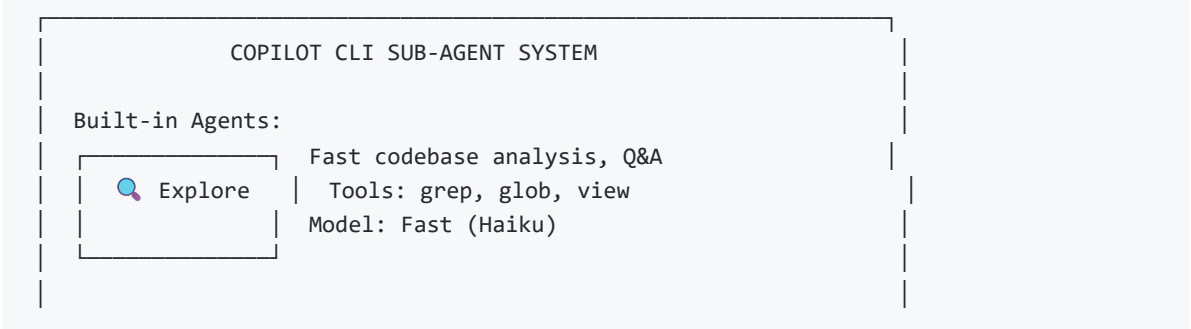
## Testing Strategy
1. Always start with unit tests for pure functions
2. Use integration tests for API endpoints
3. Use E2E tests sparingly — only for critical user flows

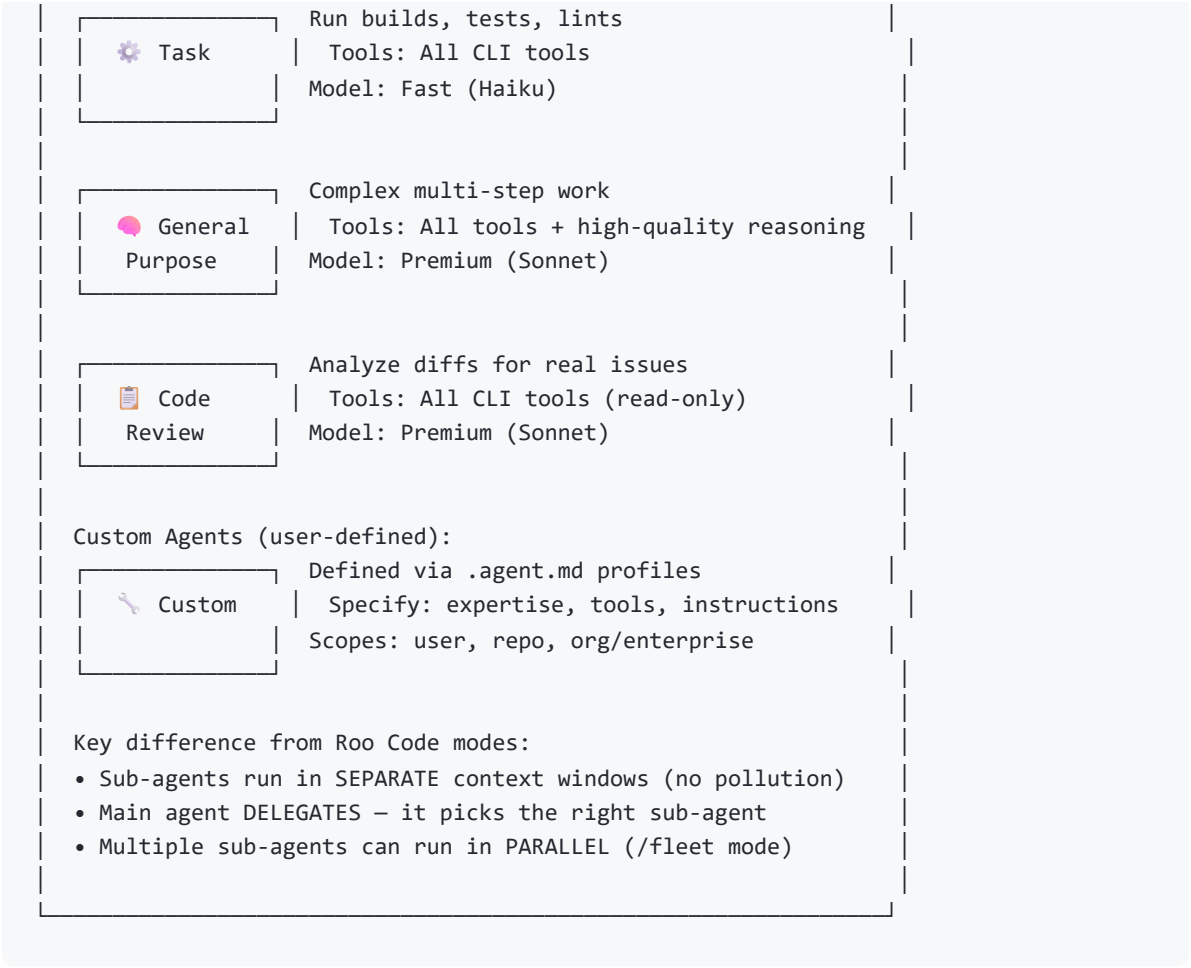
## Conventions
- Use pytest with fixtures for Python projects
- Use Jest + Testing Library for React projects
- Name test files with `.test.` or `.spec.` suffix
```

Key insight: Skills follow the open `agentskills.io` standard, making them **portable across agents** — the same skill works in Copilot CLI, Claude Code, and OpenAI Codex.

Custom Agents (Sub-agents)

Copilot CLI has a built-in **sub-agent architecture** — the main agent can delegate tasks to specialized sub-agents, each running in its own isolated context window:





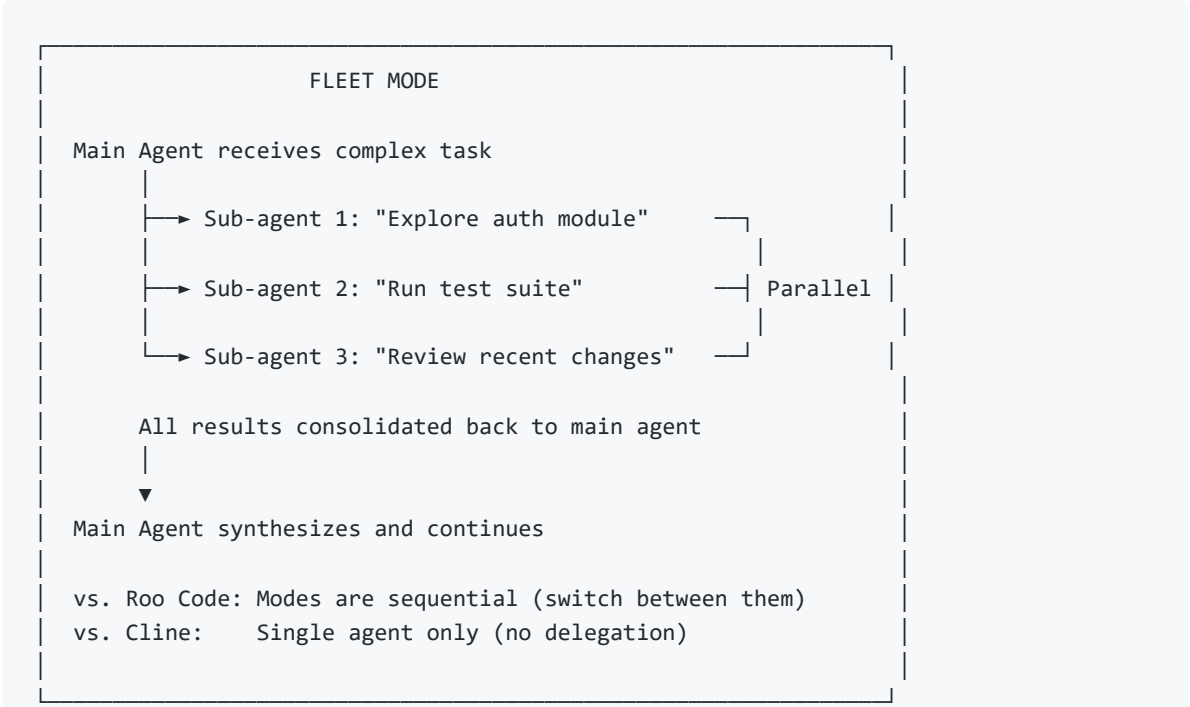
Custom agent profile example (`.github/agents/security-reviewer.agent.md`):

```
---
name: security-reviewer
description: Reviews code changes for security vulnerabilities
tools:
  - grep
  - glob
  - view
  - powershell
---

You are a security expert. When reviewing code:
1. Check for injection vulnerabilities (SQL, XSS, command injection)
2. Verify authentication and authorization logic
3. Look for hardcoded secrets or credentials
4. Check for insecure cryptographic practices
5. Flag any unsafe deserialization
```

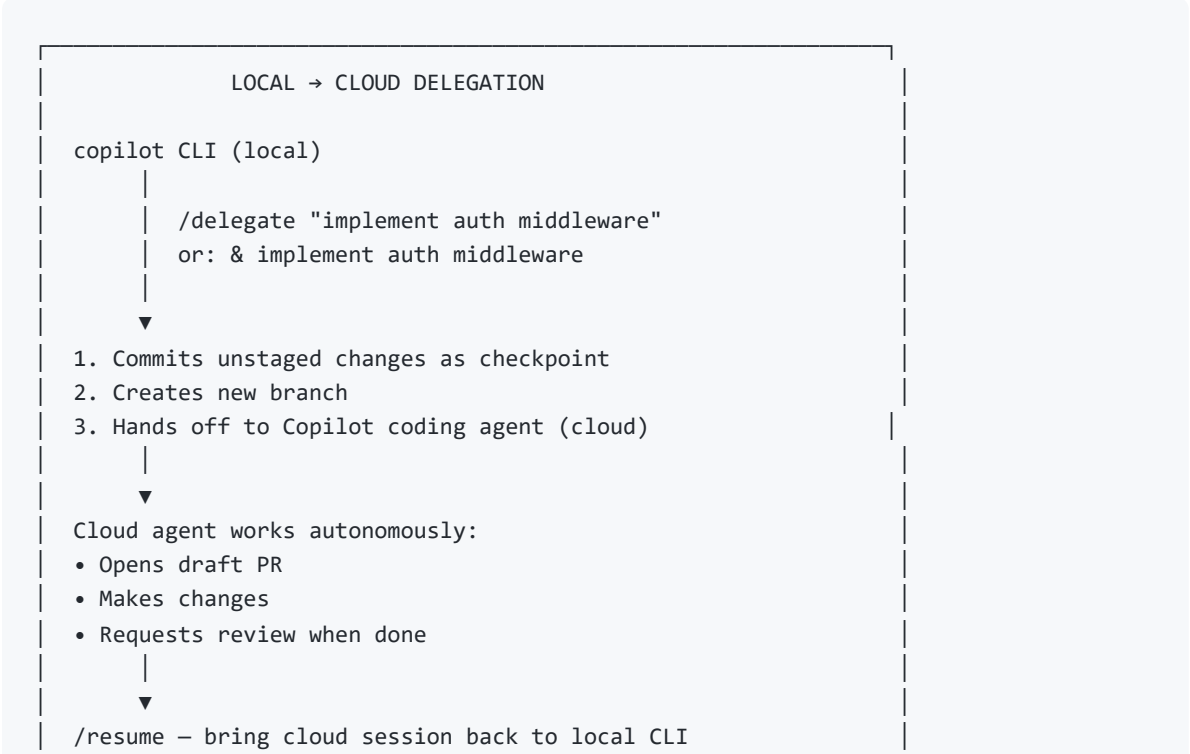
Fleet Mode (Parallel Sub-agents)

Copilot CLI's `/fleet` mode enables **parallel sub-agent execution** — multiple sub-agents work simultaneously on different parts of a task:



Delegation to Cloud Agent

A unique capability: Copilot CLI can **delegate tasks to GitHub's cloud-based coding agent**, creating a seamless local→cloud workflow:



```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Context Management

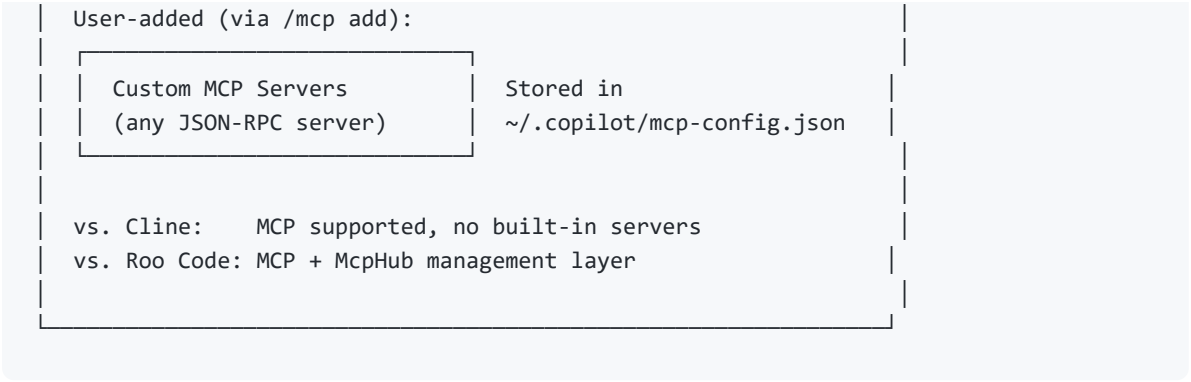
Copilot CLI uses several strategies that differ from the VS Code-based agents:

```
┌──────────────────────────────────────────────────────────────────┐
│            COPILOT CLI CONTEXT STRATEGIES                        │
│                                                                  │
│  1. Isolated Sub-agent Contexts                                  │
│     Each sub-agent has its OWN context window                    │
│     Main agent context stays clean                               │
│     (vs. Cline/Roo: single shared context)                       │
│                                                                  │
│  2. Progressive Skill Loading                                    │
│     Skills load in 3 stages: metadata → instructions → scripts   │
│     Only relevant skills consume context tokens                  │
│     (vs. Cline/Roo: no equivalent mechanism)                     │
│                                                                  │
│  3. Auto-compaction (/compact)                                   │
│     At 95% context limit, auto-summarizes history                │
│     Can also be triggered manually                               │
│                                                                  │
│  4. Custom Instructions Hierarchy                                │
│     AGENTS.md, CLAUDE.md, GEMINI.md                              │
│     .github/copilot-instructions.md                              │
│     .github/instructions/**/*.instructions.md                    │
│     ~/.copilot/copilot-instructions.md                           │
│                                                                  │
│  5. Session Persistence                                          │
│     Full timeline stored in ~/.copilot/session-state/            │
│     Resume sessions with /resume or --continue                   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

MCP Integration

Copilot CLI ships with the **GitHub MCP server built-in** — no configuration needed for GitHub API access. Additional MCP servers are added via `/mcp add`:

```
┌──────────────────────────────────────────────────────────────────┐
│            COPILOT CLI MCP SETUP                                 │
│                                                                  │
│  Built-in:                                                       │
│  ┌──────────────────────────────┐                                │
│  │  GitHub MCP Server           │  Issues, PRs, repos, actions   │
│  │  (pre-configured)            │  No setup required             │
│  └──────────────────────────────┘                                │
│                                                                  │
```



The 4-Layer Extensibility Model

This is the key architectural differentiator — Copilot CLI has **four distinct layers** for extending agent behavior, each solving a different problem:



```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Head-to-Head Comparison

Architecture & Stack

Aspect	Cline	Roo Code	Copilot CLI
Origin	Independent open-source	Fork of Cline, expanded	GitHub (proprietary)
Platform	VS Code extension	VS Code extension	Terminal-native
UI	React webview	React webview	Terminal (TTY)
CLI	Go-based standalone CLI	—	Primary interface
IPC	gRPC + Protobuf	VS Code API + MCP	MCP + session state
Cloud	—	Roo Code Cloud	GitHub Copilot coding agent

Agent Behavior

Aspect	Cline	Roo Code	Copilot CLI
Agent Pattern	Single ReAct agent	Multi-mode, role-based	Main agent + sub-agents
Planning	Plan mode (single agent)	Architect Mode (dedicated)	Plan mode + explore sub-agent
Execution	Act mode (same agent)	Code Mode (specialized)	Main agent or task sub-agent
Approval	Every action needs approval	Configurable (strict ↔ auto)	Per-tool approval, `/allow-all`
Custom Roles	Not supported	Fully customizable modes	Custom .agent.md profiles
Parallel Work	—	—	Fleet mode (parallel sub-agents)

Tools & Search

Aspect	Cline	Roo Code	Copilot CLI
File I/O	Read, write, diff	Read, write, diff	Read, write, diff
Terminal	Command execution	Command execution	Command execution
Browser	Headless automation	Headless automation	—
Code Search	Regex (ripgrep) + AST	Regex + AST + embeddings	Regex (ripgrep) + glob + LSP
MCP	Full support	Full support + McpHub	Built-in GitHub MCP + custom
Skills	—	—	Progressive SKILL.md system
Plugins	—	—	Slash command bundles

Context & Memory

Aspect	Cline	Roo Code	Copilot CLI
Context Strategy	AST parsing, smart windowing	Large windows, condensation	Sub-agent isolation + compaction
Memory	Per-session + project files	Memory Bank (cross-session)	Session state + custom instructions
Checkpoints	Manual (via approvals)	Session & profile snapshots	Session timeline + /resume
Semantic Search	—	Vector embeddings	LSP-based code intelligence
Progressive Loading	—	—	Skills load in 3 stages

Cost & Models

Aspect	Cline	Roo Code	Copilot CLI

Model Support	Multi-provider, configurable	Multi-provider, per-mode	Claude Sonnet, GPT-5 + others
Cost Control	Manual model selection	Task-based auto-routing	Premium request quota system
Local Models	Ollama, LM Studio	Ollama, LM Studio	— (cloud-only)
Cloud Delegation	—	Roo Code Cloud	/delegate to coding agent

When to Use Which?

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  Choose CLINE when:        Choose ROO CODE when:   Choose COPILOT CLI:  │
│  ──────────────────        ─────────────────────   ──────────────────── │
│  ✓ Safety is paramount     ✓ Speed matters most     ✓ Terminal-first     │
│  ✓ Regulated codebase      ✓ Cost optimization      ✓ GitHub ecosystem   │
│  ✓ Learning agent basics   ✓ Team collaboration     ✓ Parallel sub-tasks │
│  ✓ Want full control       ✓ Complex multi-step     ✓ Skills/portability │
│  ✓ Simple, predictable     ✓ Custom agent behaviors ✓ Local↔Cloud hand   │
│  ✓ Browser automation      ✓ Large codebase         ✓ Enterprise/org     │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Key Patterns to Learn From

1. MCP as Universal Tool Protocol

All three agents use MCP, making it the emerging standard for AI tool integration. Key takeaway: **decouple your tools from your agent**. Write tools as MCP servers, and any agent can use them.

```
Agent-specific tools     →    MCP standard tools
─────────────────────          ──────────────────
• Locked to one agent          • Reusable across agents
• Custom integration           • Standard JSON-RPC
• Hard to share                • Easy to distribute
• One language                 • Any language
```

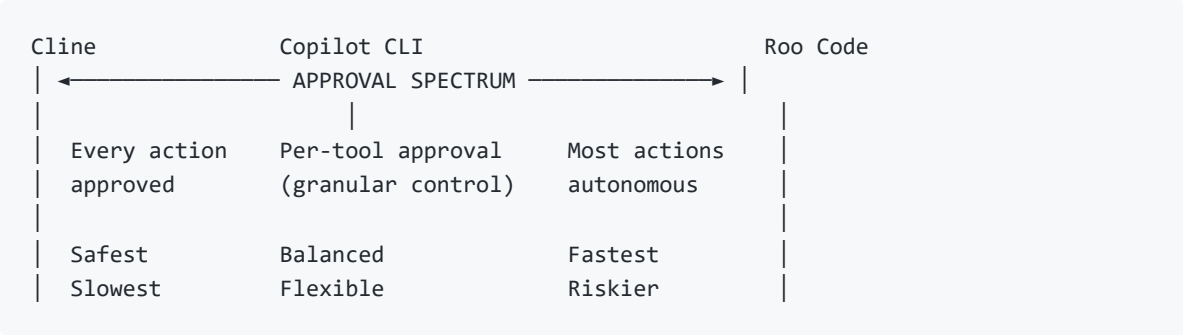
2. Mode/Role-Based Specialization

Roo Code's modes prove that **constrained agents outperform general-purpose ones**. Instead of one agent that does everything, create specialized agents with:

- Limited tool access (Architect can't write files)
- Optimized prompts (Debug mode has different system prompts than Code mode)
- Right-sized models (cheap models for simple tasks)

3. Human-in-the-Loop vs. Autonomous

Cline, Roo Code, and Copilot CLI represent different positions on the approval spectrum:



Design lesson: Let users choose their position on this spectrum based on risk tolerance and task type.

4. Persistent Memory & Session Management

All three agents solve the "amnesia problem" differently:

- **Cline**: Structured markdown files (`projectbrief.md`, `progress.md`)
- **Roo Code**: Memory Bank with profiles and semantic indexing
- **Copilot CLI**: Session timeline persistence + custom instructions hierarchy

This connects directly to Day 5's RAG concepts — persistent memory is just RAG applied to the agent's own history.

5. Cost-Aware Model Routing

Roo Code's profile system demonstrates a critical production pattern: **not every task needs the best model**. Design your agent systems to route tasks to appropriately-sized models.

6. IDE-Native vs. Terminal-Native Agent Design

Cline and Roo Code are built into VS Code, while Copilot CLI runs in the terminal. Each approach has trade-offs:

```
VS Code Extension (Cline/Roo)          Terminal-Native (Copilot CLI)
──────────────────────────────         ──────────────────────────────
• Rich diff previews in editor         • No IDE dependency
• Browser automation (webview)         • Works over SSH / remote
• Inline code annotations              • Faster startup
• Native UI panels                     • Scriptable / composable
• Tight editor integration             • Works in any environment
```

Lesson: The best agent UX depends on the workflow — visual tasks favor IDE integration, automation and remote work favor terminal-native design.

7. Skills as Portable Knowledge Packages

Copilot CLI's Skills system introduces a pattern absent in Cline and Roo Code: **separating procedural knowledge from tool execution**. Skills teach the agent *how* to approach problems, while MCP tools give it *actions* to perform.

```
Skills (knowledge)              MCP Tools (actions)
──────────────────              ──────────────────
• "How to test APIs"            • "Run pytest command"
• "Security review rules"       • "Query GitHub API"
• "API design patterns"         • "Create Jira ticket"
• Progressively loaded          • All-or-nothing loaded
• Portable (agentskills)        • Portable (MCP standard)
```

Design lesson: The most powerful agents combine *knowledge* (what to do) with *tools* (how to do it). Consider separating these concerns in your own agent designs.

Connection to Syllabus Concepts

This table maps every concept from the 7-day syllabus to its real-world implementation in Cline, Roo Code, and GitHub Copilot CLI:

Syllabus Day	Concept	Cline	Roo Code	Copilc
Day 1: LLM Foundations	Tokenization, context windows	AST parsing, smart windowing	Context condensation, large windows	Sub-a isolati auto-compa

Day 1: LLM Foundations	Model selection	Multi-provider support	Per-mode model profiles	/mode comm per-ag model
Day 2: Prompt Engineering	System prompts, CoT	Mode-specific prompts	Role-based system prompts	Custor instruc hierarc
Day 2: Prompt Engineering	Structured output	JSON tool call formatting	JSON tool calls + MCP schemas	JSON calls + schem
Day 3: ReAct Agent	Think → Act → Observe	Plan → Act → Approve cycle	Mode Router → Execute → Observe	Main a → dele → conso
Day 3: ReAct Agent	Tool definitions	Built-in tool registry	Built-in + MCP tools	4-laye + Age MCP + Plugin
Day 3: ReAct Agent	Memory types	Project files (short + long)	Memory Bank (all 3 types)	Sessio + instruc + skill
Day 4: Planner/Executor	Task decomposition	Plan mode proposes steps	Architect mode plans	Plan m explor agent
Day 4: Planner/Executor	Critic loops	Human approval as critic	Review checkpoints	Code- sub-ag
Day 4: Planner/Executor	Guardrails	All actions need approval	Tool restrictions per mode	Per-to appro custor agent lists
Day 5: RAG + Memory	Vector search	—	Semantic index (embeddings)	LSP-ba code intellig

Day 5: RAG + Memory	Persistent knowledge	Markdown project files	Memory Bank + profiles	Skills (SKILL. custor instruc
Day 5: RAG + Memory	Chunking strategies	File-level context slicing	Context condensation	Progre skill lo (3 stag
Day 6: Multi-Agent	Agent roles	Single agent (all roles)	Architect, Code, Debug, Test	Explor Reviev Gener Custor
Day 6: Multi-Agent	Orchestration	—	Mode Router + Cloud	Fleet r (parall agents
Day 6: Multi-Agent	Communication	—	Cloud: Planner↔Coder↔Reviewer	/deleg cloud agent
Day 7: Full System	Production architecture	VS Code + gRPC + MCP	VS Code + MCP + Cloud	Termir MCP + GitHul agent
Day 7: Full System	Safety & governance	Human-in-the-loop	Configurable approval + SOC2	Per-to approv org-le agents
Day 7: Full System	Scalability	Single user, local	Team collaboration, cloud	/deleg /resun (local↔

Further Reading

Cline

- **GitHub**: github.com/cline/cline
- **VS Code Marketplace**: Cline Extension
- **DeepWiki Architecture**: deepwiki.com/cline/cline

- **Community**: Discord (linked from GitHub)

Roo Code

- **GitHub**: github.com/RooCodeInc/Roo-Code
- **Documentation**: docs.roocode.com
- **MCP Guide**: docs.roocode.com/features/mcp
- **Roo Code Cloud**: roocode.com

GitHub Copilot CLI

- **GitHub**: github.com/github/copilot-cli
- **Documentation**: docs.github.com/copilot
- **Custom Agents Guide**: docs.github.com — Custom Agents
- **Skills Guide**: docs.github.com — Skills
- **DeepWiki Architecture**: deepwiki.com/github/copilot-cli
- **Agent Skills Standard**: agentskills.io

MCP Standard

- **Specification**: modelcontextprotocol.io
- **Server Examples**: github.com/modelcontextprotocol

Comparisons

- Cline vs Roo Code vs Cursor (Better Stack)
- Roo Code vs Cline (Qodo)
- Roo Code vs Cline (Openxcell)

AI & Agents — Question & Answer Reference

> 🎯 **Purpose**: Concise answers to all "Can explain" questions from the 7-day curriculum.

> 📖 **Usage**: Use this as a study guide, interview prep, or quick reference.

Day 1A: Tokens & Embeddings

Q: What is a token and why not use characters or words?

A: A token is a subword unit (e.g., "tokenization" → ["token", "ization"]). Characters are too granular for the model to learn meaning efficiently. Whole words create an impossibly large vocabulary (every misspelling, conjugation, compound word). Subwords balance vocabulary size (~100K–200K) with meaningful units.

Q: How does BPE build a vocabulary from text?

A: BPE starts with individual characters as the initial vocabulary. It then repeatedly finds the most frequent adjacent pair in the training data and merges them into a new token. After thousands of merges, common words become single tokens ("the", "and") while rare words get split into known pieces ("unforgettable" → "un" + "forget" + "table").

Q: Why are LLMs bad at arithmetic?

A: Because numbers get split into unexpected tokens. "123456" might become ["123", "456"] or ["12", "345", "6"] — the model doesn't see it as a single number. It has to learn arithmetic patterns across arbitrary token boundaries, which is unreliable. This is why tool-use (calculators) is important for agents.

Q: What is an embedding and why are token IDs insufficient?

A: Token IDs are arbitrary integers (e.g., "Hello"=9906, "Hi"=17250) — there's no way to tell they're similar. Embeddings convert each ID into a dense vector (e.g., 4096 numbers) where similar meanings land nearby in space. These vectors are learned during training so the model can do meaningful computation with them.

Q: Token embedding vs contextual embedding — what's the difference?

A: A token embedding is a static lookup — "bank" always gets the same vector regardless of context. A contextual embedding is the output AFTER the attention layers — "river bank" and "bank account" produce different vectors for "bank". The Transformer's job is to turn static token embeddings into rich contextual embeddings.

Q: What does "king − man + woman = queen" demonstrate?

A: It shows that embedding space captures semantic relationships as directions. The "gender" direction (man→woman) is consistent, so applying it to "king" lands near "queen". This means the model has learned abstract concepts like gender, tense, and category as geometric properties of the space — without being explicitly taught.

Day 1B: Transformer & Attention

Q: What does self-attention do and why is it needed?

A: Self-attention lets each token look at every other token in the sequence and decide how much to "pay attention" to each one. It's needed because word meaning depends on context — "bank" means different things near "river" vs "money". Without attention, each token would be processed in isolation with no awareness of surrounding words.

Q: Why are Q, K, V separate projections instead of comparing embeddings directly?

A: Because the question you ask (Q), the label you present to be found (K), and the content you provide (V) are different roles. A word's "searchability" is different from the information it carries. For example, "Paris" might have a Key that signals "capital city" but a Value containing facts about population and geography. Learned projections let the model separate these roles.

Q: Why do we scale by √d_k?

A: Without scaling, the dot products Q·K grow proportionally to the dimension size d_k. Large dot products push softmax into regions where gradients are extremely small (near 0 or 1), making learning impossible. Dividing by √d_k keeps the variance of the dot products at ~1, ensuring softmax produces useful gradient signal.

Q: What is multi-head attention and why use multiple heads?

A: Multi-head attention runs several attention operations in parallel, each with different learned projections. Different heads can learn different types of relationships: one head might track syntax (subject-verb), another semantics (word meaning), another coreference ("she"→"Alice"). It's like reading a sentence simultaneously for grammar, meaning, and references.

Q: What is the causal mask and why is it needed?

A: The causal mask prevents tokens from attending to future positions — token at position i can only see positions ≤ i. This is essential for autoregressive generation because the model can't use the answer to predict the answer. Implementation: set future positions to -∞ before softmax, which makes their attention weight exactly 0.

Q: What is the FFN's role vs attention's role?

A: Attention gathers relevant information from context — it's the "what should I look at?" step. The FFN (feed-forward network) processes that gathered information — it's the "what do I know about this?" step. Research shows specific FFN neurons activate for specific facts ("Paris is the capital of France" lives in particular neurons). Attention reads the right books; FFN thinks about what was read.

Q: Why do residual connections enable deep networks?

A: Without residual connections, gradients must pass through every layer during backpropagation. In deep networks (96+ layers), they shrink exponentially and vanish — early layers can't learn. Residual connections (output = layer(x) + x) create a "gradient highway"

where the learning signal can flow directly through the skip connection, keeping gradients strong even at layer 1.

Day 1C: Training & Inference

Q: What is the difference between pre-training, SFT, and RLHF?

A: **Pre-training**: predict next token on trillions of tokens of internet text — learns language, facts, reasoning, but produces a text completer, not an assistant. **SFT**: fine-tune on curated (instruction, response) pairs — teaches the format of being helpful (answer directly, follow instructions). **RLHF**: human raters rank outputs, train a reward model, then optimize the LLM to maximize reward — makes it more helpful, harmless, and honest. Each phase builds on the previous.

Q: How does temperature affect generation?

A: Temperature scales the raw prediction scores (logits) before softmax. T=0: always picks the most likely token (deterministic, "boring"). T=1: samples from the model's actual learned distribution. T>1: flattens the distribution, making unlikely tokens more probable (creative but risky). T<1: sharpens the distribution, making the model more conservative. For factual tasks use low temperature; for creative tasks use higher.

Q: Why does the KV-cache exist?

A: During generation, the model predicts one token at a time and feeds it back as input. Without caching, step N would recompute attention over all N tokens from scratch — $O(N^2)$ total work. The KV-cache stores the Key and Value vectors from all previous steps, so only the new token's Query needs computation. This turns quadratic cost into linear, but requires GPU memory proportional to sequence length.

Q: Why do modern LLMs use decoder-only architecture?

A: Decoder-only is simpler (one architecture, one training objective: predict next token), scales more predictably, and produces emergent abilities at sufficient scale. A single decoder-only model can do classification, generation, translation, coding — all via prompting — without needing separate encoder/decoder stacks. Every frontier model (GPT-5, Claude Opus, LLaMA 4, Gemini 3) is decoder-only.

Day 2: Prompt Engineering

Q: What is chain-of-thought and why does it help?

A: Chain-of-thought (CoT) prompting asks the model to show its reasoning step by step before giving a final answer. It helps because LLMs generate one token at a time — without CoT, they must "jump" to the answer in one step. With CoT, intermediate tokens serve as working memory, letting the model break complex problems into manageable steps. Simply adding "Let's think step by step" can significantly improve accuracy on reasoning tasks.

Q: What are the main prompt failure modes?

A: **Instruction following failure**: the model ignores constraints (especially ones buried in the middle of long prompts). **Hallucination**: confidently generates false information. **Format breaking**: returns prose when you asked for JSON. **Sycophancy**: agrees with the user even when they're wrong. **Lost in the middle**: ignores information in the middle of long contexts. Mitigation: put critical instructions at the start AND end, use structured output formats, and add validation.

Q: Why does instruction position matter?

A: LLMs have a "primacy and recency" bias — they pay most attention to the beginning and end of the context, and can lose information in the middle (the "lost in the middle" phenomenon). Place your most critical constraints at the start (system prompt) and repeat them at the end. Never bury important instructions in the middle of a long prompt.

Q: When to use few-shot vs zero-shot?

A: Use **zero-shot** when the task is simple and well-understood by the model (sentiment analysis, basic classification) — it's cheaper and faster. Use **few-shot** when you need a specific output format, the task is unusual, or zero-shot gives inconsistent results. 2–5 examples usually suffice. More than 10 examples rarely helps — consider fine-tuning instead.

Day 3: Single Agent (ReAct)

Q: What is the ReAct pattern and why is it effective?

A: ReAct (Reason + Act) interleaves reasoning ("I need to find X"), acting (calling a tool), and observing (processing the result) in a loop. It's effective because it combines the LLM's reasoning ability with real-world tool access. The model can correct course based on observations rather than committing to a plan upfront. The explicit "Thought" step also makes the agent's reasoning transparent and debuggable.

Q: What are the three types of agent memory?

A: **Short-term (working) memory**: the current conversation context window — what the agent is actively "thinking about." **Long-term memory**: persistent storage (vector DB, files) that survives across sessions — facts, user preferences, past interactions. **Episodic memory**: records

of past experiences — what worked, what failed, and why — enabling the agent to learn from its own history.

Q: What are common agent failure modes and how to prevent them?

A: **Infinite loops**: agent repeats the same action — prevent with max iteration limits and loop detection. **Tool misuse**: wrong parameters or wrong tool — prevent with clear tool descriptions and input validation. **Goal drift**: agent loses track of the original task — prevent with explicit goal tracking in the system prompt. **Hallucinated actions**: agent invents tools that don't exist — prevent with strict tool schemas and error handling.

Day 4: Planner / Executor

Q: Why is plan-then-execute better than step-by-step for complex tasks?

A: For complex multi-step tasks, step-by-step (ReAct) can lose sight of the big picture — it makes locally reasonable decisions that don't add up to a coherent solution. Plan-then-execute separates strategic thinking (Planner) from tactical execution (Executor). The Planner can use a stronger model to reason about the full problem, while the Executor uses a lighter model to carry out each step. This also enables parallelism — independent steps can execute simultaneously.

Q: What does the Critic check and why is it important?

A: The Critic validates the Executor's output against the original plan's intent — checking for correctness, completeness, and quality. Without a Critic, the system accepts the first output regardless of quality. The Critic can request re-execution, adjust the plan, or escalate to the user. It acts as a quality gate that prevents cascading errors from propagating through subsequent plan steps.

Q: What are guardrails and why are they necessary?

A: Guardrails are constraints that prevent the agent from taking dangerous or undesired actions — input validation (block prompt injection), output filtering (no harmful content), action limits (can't delete production data), cost caps (max API spend), and timeout limits. They're necessary because LLMs are probabilistic and can behave unexpectedly — guardrails provide deterministic safety boundaries around non-deterministic behavior.

Day 5: RAG + Vector Memory

Q: What is RAG and when should you use it vs fine-tuning?

A: RAG (Retrieval-Augmented Generation) retrieves relevant documents from a knowledge base and injects them into the prompt before generation. Use RAG when: data changes frequently, you need source citations, or you have large/diverse knowledge bases. Use fine-tuning when: you need a specific output style, domain-specific reasoning (medical, legal), or the knowledge is too large for context windows. RAG is cheaper, faster to set up, and updatable in real-time.

Q: How does cosine similarity work?

A: Cosine similarity measures the angle between two vectors, ignoring magnitude. cos(A,B) = (A·B)/(|A|×|B|). If two vectors point in the same direction → cos=1 (identical meaning). Perpendicular → cos=0 (unrelated). Opposite → cos=-1. It's preferred over Euclidean distance for embeddings because it focuses on the direction (meaning) rather than the length (which can vary due to token frequency).

Q: Why is hybrid search better than pure vector search?

A: Vector search is great for semantic similarity ("automobile" matches "car") but can miss exact keyword matches. Keyword search (BM25) excels at exact matches but misses semantic connections. Hybrid search combines both — using Reciprocal Rank Fusion (RRF) to merge the ranked results. This catches both semantic matches AND exact keyword matches, producing more robust retrieval.

Q: What are chunking strategies and their trade-offs?

A: **Fixed-size**: simple, predictable, but can split sentences mid-thought. **Recursive/paragraph**: splits at natural boundaries (paragraphs, sentences), preserves meaning but variable chunk sizes. **Semantic**: splits when the topic changes, best quality but most complex. Smaller chunks → more precise retrieval but less context per chunk. Larger chunks → more context but may include irrelevant content. Typical size: 256–512 tokens with 10–20% overlap.

Day 6: Multi-Agent Collaboration

Q: What are the main multi-agent orchestration patterns?

A: **Sequential pipeline**: agents run in a fixed order, each passing output to the next (simple, predictable). **Hierarchical**: a supervisor agent delegates to specialist workers (flexible, scalable). **Debate/consensus**: multiple agents propose solutions and vote or critique each other (higher quality, expensive). **Broadcast**: all agents process the same input independently, results are aggregated. Choice depends on task complexity, cost tolerance, and quality requirements.

Q: What agent roles exist and when to use each?

A: **Planner**: decomposes complex tasks into subtasks (use a strong model like GPT-5 or Claude Opus). **Executor**: carries out specific subtasks (can use lighter models). **Critic/Reviewer**: validates

outputs for quality and correctness. **Researcher**: gathers information from tools or knowledge bases. **Router**: decides which specialist agent to invoke. Match the model strength to the role's difficulty — planners and critics need stronger reasoning than executors.

Q: How to prevent loops and deadlocks?

A: **Max iterations**: hard limit on how many times agents can communicate. **Timeout per step**: kill any step that runs too long. **Convergence detection**: if outputs stop changing, declare done. **Dependency graphs**: ensure no circular dependencies between agents. **Escalation**: if stuck after N attempts, escalate to a supervisor agent or the user. Always design with the assumption that agents WILL get stuck, and build escape hatches.

Q: What are the trade-offs of multi-agent vs single-agent?

A: **Multi-agent pros**: better separation of concerns, can use different models per role, handles complex tasks more robustly, enables parallelism. **Multi-agent cons**: higher cost (N× the LLM calls), more complex to debug, communication overhead, potential for miscommunication between agents. **Rule of thumb**: use a single agent for tasks one person could do in one sitting; use multi-agent when the task requires distinct expertise or would benefit from peer review.
